

رمز گشایی از معماری داده

خلاصه فارسی نسخه پیش نویس



۱۴۰۵

حامد سهامی
https://www.linkedin.com/in/hamedsahami/

بخش اول: مبانی

در قسمت اول این کتاب، من پایه‌ای برای تفسیر معماری‌های داده ایجاد خواهم کرد. فصل ۱ با توصیف داده‌های بزرگ شروع می‌شود، در حالی که فصل ۲ یک بررسی کلی از انواع معماری‌های داده و تکامل آن‌ها ارائه می‌دهد. فصل ۳ نشان می‌دهد که چگونه می‌توانید یک جلسه طراحی معماری را برگزار کنید تا به تعیین بهترین معماری داده برای پروژه خود کمک کند.

این بخش از کتاب نقطه شروع خوبی برای درک ارزش داده‌های بزرگ و تاریخچه معماری‌هایی که این داده‌ها را گرفته‌اند ارائه می‌دهد. فصل‌های بعدی سپس به جزئیات می‌پردازند.

فصل اول: کلان داده

تعداد شرکت‌هایی که در دهه ۲۰۲۰ معماری‌های داده را می‌سازند، به شدت افزایش یافته است. این رشد احتمالاً در زمان نزدیک کاهش نخواهد یافت، به طور عمده به دلیل اینکه مقدار بیشتری از هر زمان قبل در دسترس است: از رسانه‌های اجتماعی، دستگاه‌های اینترنت اشیاء (IoT)، برنامه‌های خانگی، و نرم‌افزارهای شخص ثالث، تا نمونه‌های کمی از منابع. بر اساس یک مطالعه BCG ۲۰۲۳، "حجم داده‌های تولید شده تقریباً دو برابر شده از ۲۰۱۸ تا ۲۰۲۱ تا حدود ۸٤ زیتا، نرخ رشدی که انتظار می‌رود ادامه داشته باشد." پژوهشگران "تخمین می‌کنند که حجم داده‌های تولید شده با نرخ رشد سالانه ترکیبی (CAGR) ٪۲۱ از ۲۰۲۱ تا ۲۰۲٤ به ۱٤۹ زیتا رسیده خواهد شد." شرکت‌ها می‌دانند که می‌توانند با جمع‌آوری این داده‌ها و استفاده از آن‌ها برای تحلیل گذشته و حال و پیش‌بینی آینده، میلیون‌ها دلار صرفه‌جویی کنند و درآمد را افزایش دهند - اما برای انجام این کار، به یک راه برای ذخیره‌سازی همه این داده‌ها نیاز دارند.

در سراسر جهان کسب و کار، فشار بر روی ساخت معماری‌های داده به سرعت بیشترین است. این معماری‌ها باید آماده باشند تا هر داده آینده‌ای را که باشد نام یا سرعت یا نوع، مدیریت کنند و دقت آن را حفظ کنند. و ما که با معماری‌های داده کار می‌کنیم، نیاز به درک واضحی از چگونگی کارکرد آن‌ها و گزینه‌های موجود داریم. در اینجا کتاب می‌آید. من به طور مستقیم نتیجه نداشتن درک صحیح از مفاهیم معماری داده را دیده‌ام. یک شرکتی که می‌شناسم معماری داده‌ای را با هزینه ۱۰۰ میلیون دلار در دو سال ساخت، تشخیص داد که معماری از فناوری اشتباه استفاده می‌کند، خیلی سخت برای استفاده است و برای مدیریت برخی انواع داده‌ها به اندازه کافی انعطاف‌پذیر نیست. باید آن را حذف کرد و از ابتدا دوباره شروع کرد. از این اتفاق برای شما جلوگیری کنید!

همه در مورد دریافت اطلاعات صحیح به افراد صحیح در زمان صحیح در فرمت صحیح است. برای انجام این کار، شما نیاز به یک معماری داده دارید تا داده‌ها را بگیرد، ذخیره کند، تبدیل کند و مدل کند (پردازش بزرگ داده) به طوری که بتوان آن‌ها را به طور دقیق و آسان استفاده کرد. شما نیاز به یک معماری دارید که هر کاربر نهایی، حتی یکی با دانش فنی بسیار کم، بتواند داده‌ها را تحلیل کند و گزارشات و داشبوردها را تولید کند، به جای اینکه بر اساس افراد در IT با دانش فنی عمیق برای انجام آن کار برای آن‌ها باور کنید.

فصل ۱ با معرفی بزرگ داده و برخی از ایده‌های اساسی آن شروع می‌شود. سپس می‌پردازم که چگونه شرکت‌ها از داده‌های خود استفاده می‌کنند، با تأکید بر هوش تجاری و چگونگی رشد این استفاده هنگامی که معماری داده شرکت به بلوغ می‌رسد.

کلان داده چیست و چگونه می تواند به شما کمک کند؟

حتی اگر اصطلاح "بزرگ" در بزرگ داده استفاده شود، این فقط در مورد اندازه داده‌ها نیست. همچنین در مورد تمام داده‌ها، بزرگ یا کوچک، درون شرکت شما و تمام داده‌های خارج از شرکت شما که برای شما مفید خواهد بود، است. داده‌ها می‌توانند در هر فرمتی باشند و می‌توانند با هر درجه از منظم بودن جمع‌آوری شوند. بنابراین بهترین راه برای تعریف بزرگ داده، فکر کردن به آن به عنوان تمام داده‌ها، بدون توجه به اندازه (حجم)، سرعت (سرعت) یا نوع (تنوع) آن‌ها است. علاوه بر این معیارها، سه عامل دیگری وجود دارد که می‌توانید برای توصیف داده‌ها استفاده کنید: دقت، تغییر پذیری، و ارزش. با هم، آن‌ها به طور معمول به عنوان "شش بی" بزرگ داده شناخته می‌شوند، همانطور که در شکل ۱-۱ نشان داده شده است.

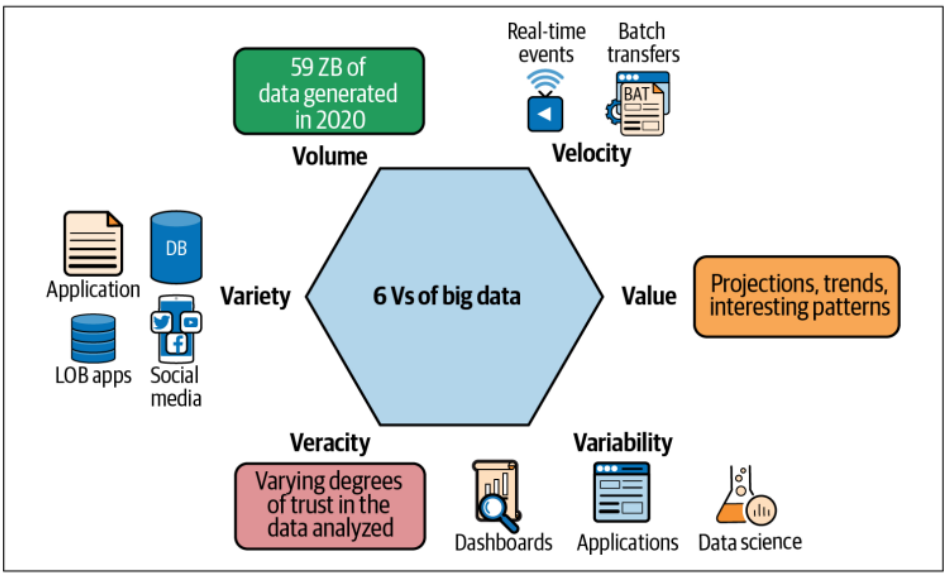


Figure 1-1. The six Vs of big data (source: The Cloud Data Lake *by Rukmani Gopalan [O'Reilly, 2023]).*

بیایید به هر یک از آن‌ها نگاه بیشتری بیندازیم:

حجم

حجم مقدار کلی داده‌های تولید و ذخیره شده است. این می‌تواند از تری‌بایت تا پتابایت داده باشد و می‌تواند از منابع متنوعی مانند رسانه‌های اجتماعی، معاملات تجارت الکترونیکی، آزمایشات علمی، داده‌های حسگر از دستگاه‌های IoT و بسیاری از منابع دیگر باشد. به عنوان مثال، داده‌ها از یک سیستم ورود سفارش ممکن است به مقدار چند تری‌بایت در روز باشد، در حالی که دستگاه‌های IoT می‌توانند هر دقیقه میلیون رویداد را انتشار دهند و هر روز صدها تری‌بایت داده تولید کنند.

تنوع

تنوع به محدوده گسترده منابع و فرمت‌های داده اشاره دارد. این می‌تواند به طور بیشتر به داده‌های ساختاریافته (از پایگاه‌های داده رابطه‌ای)، داده‌های نیمه ساختاریافته (مانند لاگ‌ها و فرمت‌های XML، CSV، و JSON)، داده‌های بدون ساختار (مانند ایمیل‌ها، اسناد، و PDFها)، و داده‌های دودویی (تصاویر، صدا، ویدیو) تقسیم شود. به عنوان مثال، داده‌ها از یک سیستم ورود سفارش داده‌های ساختاریافته خواهد بود زیرا از یک پایگاه داده رابطه‌ای می‌آید، در حالی که داده‌ها از یک دستگاه IoT احتمالاً در فرمت JSON خواهد بود.

سرعت

سرعت به سرعت تولید و پردازش داده اشاره دارد. جمع‌آوری داده به ندرت اغلب به عنوان پردازش دسته‌ای نامیده می‌شود؛ به عنوان مثال، هر شب سفارشات روز جمع‌آوری و پردازش می‌شوند. داده‌ها همچنین می‌توانند بسیار فردانه یا حتی در زمان واقعی جمع‌آوری شوند، به خصوص اگر در سرعت بالایی مانند داده‌های رسانه‌های اجتماعی، دستگاه‌های IoT، و برنامه‌های تلفن همراه تولید شوند.

دقت

دقت در مورد دقت و قابلیت اعتماد داده است. بزرگ داده از یک مجموعه بزرگ از منابع متنوع می‌آید. منابع ناقابل اعتماد یا ناقص می‌توانند کیفیت داده‌ها را آسیب بزنند. به عنوان مثال، اگر داده از یک دستگاه IoT مانند یک دوربین امنیتی خارجی قرار داشته باشد که در جلوی خانه شما قرار دارد و به مسیر راه نگاه می‌کند و یک پیام متنی به شما می‌فرستد وقتی که یک فرد را تشخیص می‌دهد، امکان دارد که عوامل محیطی، مانند آب و هوا، باعث شوند دستگاه به طور غلط یک فرد را تشخیص دهد، داده‌ها را آسیب بزند. بنابراین، داده‌ها باید در زمان دریافت تأیید شوند.

تغییرپذیری

تغییرپذیری به پیوستگی (یا ناپیوستگی) داده در مورد فرمت، کیفیت، و معنا اشاره دارد. پردازش و تحلیل فرمت‌های داده ساختار یافته، نیمه ساختار یافته، و بدون ساختار نیاز به ابزارها و تکنیک‌های مختلف دارد. به عنوان مثال، نوع، فرکانس، و کیفیت داده‌های حسگر از دستگاه‌های IoT می‌توانند به طور قابل توجهی متفاوت باشد. حسگر دما و رطوبت ممکن است داده‌های نقطه‌ای را در فواصل منظم تولید کنند، در حالی که حسگر حرکت ممکن است فقط زمانی داده تولید کند که حرکت را تشخیص دهد.

ارزش

ارزش، بیشترین V، به مفید و مرتبط بودن داده مرتبط است.

شرکت‌ها از بزرگ داده برای به دست آوردن بینش‌ها و اتخاذ تصمیماتی استفاده می‌کنند که می‌توانند منجر به ارزش کسب و کار شوند، مانند افزایش کارایی، صرفه‌جویی در هزینه‌ها، یا سویه‌های درآمد جدید. به عنوان مثال، با تحلیل داده‌های مشتری، سازمان‌ها می‌توانند رفتار، ترجیحات، و نیازهای مشتریان خود را بهتر درک کنند. آن‌ها می‌توانند از این اطلاعات برای توسعه کمپین‌های بازاریابی مورد نظر بهتر، بهبود تجربیات مشتری، و افزایش فروش استفاده کنند.

جمع‌آوری داده‌های بزرگ به شرکت‌ها امکان می‌دهد تا با استفاده از آن‌ها به دنبال درک‌هایی برای تصمیم‌گیری بهتر در کسب و کار بپردازند. تجزیه و تحلیل پیش‌بینی نوعی از تجزیه و تحلیل داده است که شامل استفاده از الگوریتم‌های آماری و یادگیری ماشین برای تجزیه و تحلیل داده‌های تاریخی و پیش‌بینی رویدادها و روندهای آینده است. این امکان را به شرکت‌ها می‌دهد تا به صورت پیشگیرانه عمل کنند، نه فقط به صورت واکنشی.

شما می‌شنوید که بسیاری از شرکت‌ها داده را "نفت جدید" می‌نامند، زیرا در اقتصاد دیجیتال امروز به یک منبع بسیار ارزشمند تبدیل شده است، مشابه نفت در اقتصاد صنعتی. داده به چندین مورد شبیه به نفت است:

- یک ماده خام است که باید استخراج، تصفیه، و پردازش شود تا مفید باشد. در مورد داده، این شامل جمع‌آوری، ذخیره‌سازی، و تجزیه و تحلیل آن است تا بتوان به دنبال درک‌هایی برای تصمیم‌گیری کسب و کار بپردازید.

- بسیار ارزشمند است. شرکت‌هایی که مقادیر زیادی از داده را جمع‌آوری و تجزیه و تحلیل می‌کنند، می‌توانند از آن برای بهبود محصولات و خدمات خود، تصمیم‌گیری بهتر در کسب و کار، و به دست آوردن مزیت رقابتی استفاده کنند.

- می‌تواند به صورت‌های مختلفی استفاده شود. به عنوان مثال، اگر از داده برای آموزش الگوریتم‌های یادگیری ماشین استفاده کنید، می‌توانید سپس از این الگوریتم‌ها برای اتوماسیون وظایف، شناسایی الگوها، و پیش‌بینی استفاده کنید.

- یک منبع قدرتمند با تأثیر تبدیل‌کننده بر جامعه است. استفاده گسترده از نفت به رشد صنایع کمک کرد و فناوری‌های جدید را فراهم کرد، در حالی که داده منجر به پیشرفت‌ها در زمینه‌هایی مانند هوش مصنوعی، یادگیری ماشین، و تجزیه و تحلیل پیش‌بینی شده است.

- به دلیل همه عوامل قبلی، می‌تواند منبعی از قدرت و تأثیر باشد. به عنوان مثال، می‌توانید از داده‌های بزرگ برای تولید گزارشات و داشبوردها استفاده کنید که نشان می‌دهند که فروش در کجا به پشت است و مراحل "پس از وقوع" را برای بهبود آن فروش انجام دهید. همچنین می‌توانید از یادگیری ماشین برای پیش‌بینی کنید که فروش در آینده کجا کاهش خواهد یافت و مراحل پیشگیرانه را برای جلوگیری از این کاهش انجام دهید. این فرایند به نام هوش تجاری (BI) شناخته می‌شود: فرایند جمع‌آوری، تجزیه و تحلیل، و استفاده از داده برای کمک به شرکت‌ها در تصمیم‌گیری بهتر و اطلاعات‌بیشتر.

همانطور که شکل ۱-۲ نشان می‌دهد، من می‌توانم داده را از منابع جدید، مانند دستگاه‌های IoT، لاگ‌های وب، و رسانه‌های اجتماعی، همچنین منابع قدیمی، مانند برنامه‌های کسب و کار، برنامه‌ریزی منابع کسب و کار (ERP)، و مدیریت روابط مشتری (CRM) را جمع‌آوری کنم. این داده می‌تواند در فرمت‌های مختلفی باشد، مانند فایل‌های CSV، فایل‌های JSON، و فایل‌های Parquet. می‌تواند در دسته‌ها یا به صورت پیش‌فرض بارگذاری شود، به عنوان مثال یک بار در ساعت، یا می‌تواند در چندین بار در ثانیه (این به نام جریان زمان واقعی شناخته می‌شود) جریان شود.

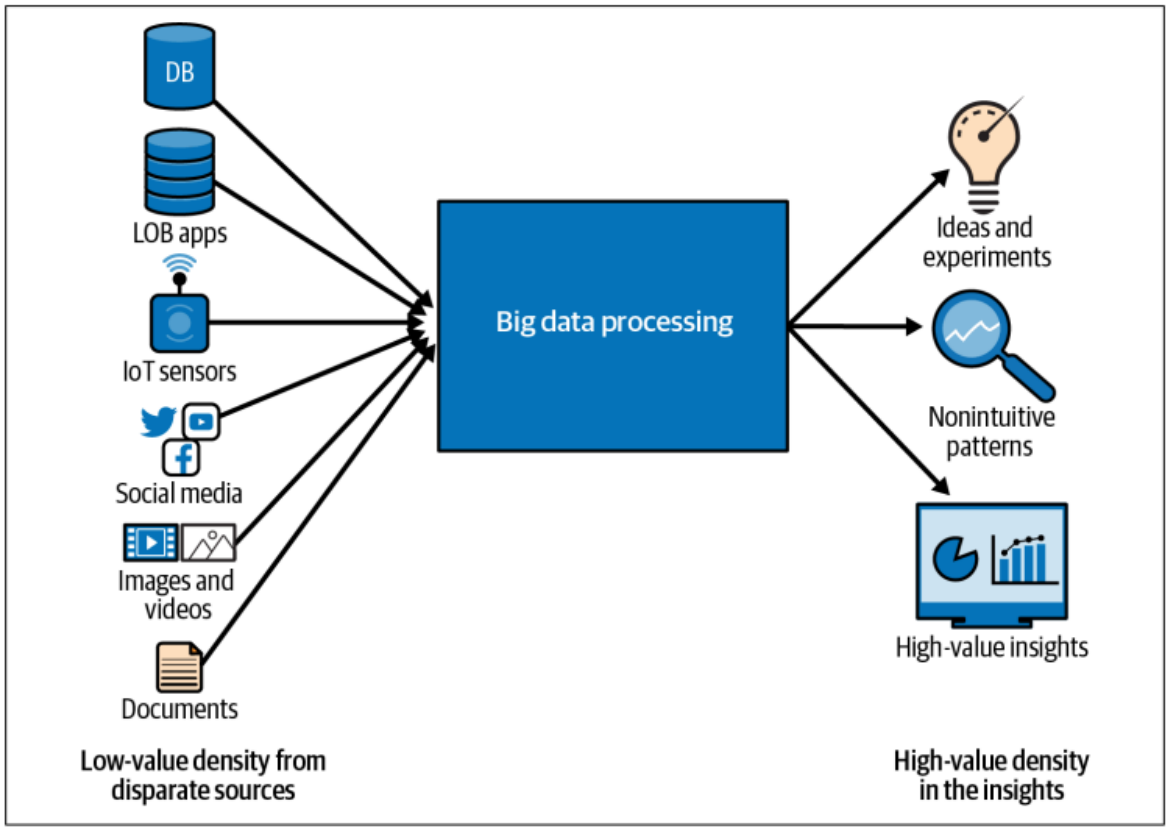


Figure 1-2. Big data processing (source: The Cloud Data Lake *by Rukmani Gopalan [O'Reilly, 2023])*

برای شرکت‌ها مهم است که درک کنند که در چه مرحله‌ای از سفر خود برای استفاده از داده قرار دارند نسبت به شرکت‌های دیگر. این مفهوم به نام بلوگ‌زنی داده شناخته می‌شود و بخش بعدی مراحل سفر بلوگ‌زنی داده را نشان می‌دهد تا بتوانید درک کنید که شرکت شما در چه مرحله‌ای قرار دارد.

بانک معماری داده

بلوغ داده ها

شما ممکن است با استفاده از اصطلاح "تبدیل دیجیتال" در صنعت فناوری اطلاعات با آن آشنا شده باشید، که به معنای این است که چگونه شرکت‌ها فناوری را در سراسر کسب و کار خود جاسازی می‌کنند تا تغییر اساسی در نحوه استخراج ارزش از داده‌ها و در نحوه عملکرد و ارائه ارزش به مشتریان ایجاد کنند. این فرآیند شامل انتقال از فرآیندهای سنتی، دستی یا مبتنی بر کاغذ به فرآیندهای دیجیتال است، با استفاده از قدرت فناوری برای بهبود کارایی، بهره‌وری و نوآوری. یک بخش بزرگ از این تبدیل معمولاً استفاده از داده برای بهبود کسب و کار شرکت است، که می‌تواند به معنای ایجاد پروفایل ۳۶۰ درگیر برای بهبود تجربه مشتری یا استفاده از یادگیری ماشین برای بهبود سرعت و دقت خطوط تولید باشد.

این تبدیل دیجیتال می‌تواند به چهار مرحله تقسیم شود، که به آن‌ها مراحل بلوغ‌زنی داده کسب و کار گفته می‌شود و در شکل ۱-۳ نشان داده شده است. در حالی که این اصطلاح در صنعت فناوری اطلاعات به طور گسترده‌ای استفاده می‌شود، من دیدگاه خاصی در مورد چگونگی ظاهر شدن این مراحل دارم. آن‌ها سطح توسعه و پیشرفت یک سازمان را در مدیریت، استفاده و استخراج ارزش از داده‌های خود توصیف می‌کنند. این مدل راهی برای ارزیابی توانایی‌های مدیریت داده یک سازمان و آمادگی آن برای تحلیل‌های پیشرفته، هوش مصنوعی و سایر اقدامات مبتنی بر داده است. هر مرحله نمایانگر یک قدم به جلو در استفاده از داده برای ارزش کسب و کار و تصمیم‌گیری است. باقیمانده این بخش هر مرحله را توصیف می‌کند.

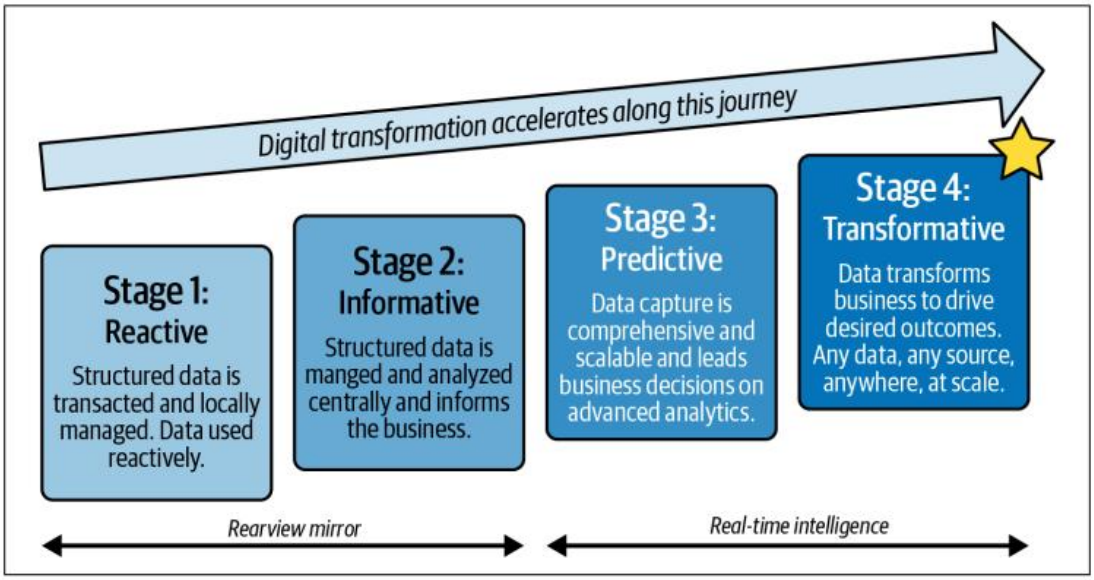


Figure 1-3. Enterprise data maturity stages

مرحله اول:

در مرحله اول، یک شرکت داده‌هایی در سراسر دارد، احتمالاً در یک دسته از اکسل و/یا پایگاه‌های داده دسکتاپ در سیستم‌های فایل مختلف، که در سراسر ایمیل می‌شوند. معماران داده این را یک "اسپرادمارت" می‌نامند (کوتاه برای "مارت داده اکسل"): یک مجموعه غیررسمی، غیرمتمرکز داده‌ها که اغلب در یک سازمانی که از اکسل برای ذخیره، مدیریت و تجزیه و تحلیل داده استفاده می‌کند، یافت می‌شود. افراد یا تیم‌ها معمولاً اسپرادمارت‌ها را به صورت مستقل از سیستم مدیریت داده متمرکز سازمان یا انبار داده رسمی ایجاد و نگهداری می‌کنند. اسپرادمارت‌ها از ناهمگونی داده، کمبود حاکمیت، قابلیت محدود مقیاس‌پذیری، و ناکارآمدی رنج می‌برند (زیرا اغلب منجر به تلاش تکراری زیاد می‌شوند).

مرحله دوم:

شرکت‌ها در مرحله دوم بلوغ‌زنی داده رسیده‌اند وقتی شروع به مرکزی‌سازی داده‌های خود می‌کنند، که تجزیه و تحلیل و گزارش‌سازی را بسیار آسان‌تر می‌کند. مراحل ۱ و ۲ برای گزارش تاریخی یا دیدن الگوها و روندهای گذشته است، بنابراین شکل ۱-۳ آن‌ها را "آینه عقب‌مانده" می‌نامد. در این مراحل، شما به آنچه قبلاً رخ داده است واکنش می‌کنید.

۱ با استفاده از اکسپریمنت "on-prem"، کوتاه برای "on-premises"، به معنای میزبانی و مدیریت زیرساخت فناوری اطلاعات یک سازمان در تمام تجهیزات فیزیکی خود، معمولاً نامیده می‌شود مراکز داده، اشاره می‌کند. این با خدمات مبتنی بر ابر متفاوت است، که در آن‌ها این منابع توسط ارائه‌دهندگان سوم شخصی مانند Amazon Web Services (AWS، Azure)، یا Google Cloud Platform (GCP) در مراکز داده ابری دور میزبانی و مدیریت می‌شوند. من در فصل ۱۶ مزایای انتقال از on-prem به ابر را بحث خواهم کرد، اما برای حال حاضر، بدانید که انتقال از سرورهای on-prem به ابر بخش بزرگی از تبدیل دیجیتال بسیاری از شرکت‌ها است.

در مرحله ۲، راه‌حل ساخته شده برای جمع‌آوری داده معمولاً قابل مقیاس‌پذیری نیست. به طور کلی، اندازه و انواع داده‌هایی که می‌تواند مدیریت کند محدود است، و می‌تواند فقط در فراوانی‌های نادر (مثلاً هر شب) داده را مصرف کند. بیشتر شرکت‌ها در مرحله ۲ هستند، به خصوص اگر زیرساخت آن‌ها هنوز در on-prem است.

مرحله سوم:

در مرحله ۳، شرکت‌ها به ابر منتقل شده‌اند و یک سیستم ساخته‌اند که قادر به مدیریت مقادیر بزرگتر از داده، انواع مختلف از داده، و داده‌هایی که به طور منظم‌تر مصرف می‌شوند (به عنوان مثال به صورت ساعتی یا جریانی) است. آن‌ها همچنین تصمیم‌گیری خود را با ادغام یادگیری ماشین (تحلیل‌های پیشرفته) برای اتخاذ تصمیمات در زمان واقعی بهبود داده‌اند.

برای مثال، در حالی که یک کاربر در یک فروشگاه کتاب آنلاین است، سیستم ممکن است بر اساس خریدهای قبلی کاربر، کتاب‌های اضافی را در صفحه پرداخت پیشنهاد دهد.

مرحله چهارم

در نهایت، در مرحله ٤، شرکت یک راه‌حل ساخته است که قادر به مدیریت هر نوع داده است، بدون محدودیت از اندازه، سرعت یا نوع. این باعث می‌شود که ارائه داده‌های جدید با یک زمان راه‌اندازی کوتاه‌تر آسان باشد زیرا معماری قادر به مدیریت آن است و ظرفیت زیرساخت را برای پشتیبانی از آن دارد. این یک راه‌حل است که به کاربران نهائی غیرفنی اجازه می‌دهد به راحتی گزارش‌ها و داشبوردها را با استفاده از ابزارهای خود ایجاد کنند.

مراحل ۳ و ٤ مورد تمرکز این کتاب هستند. به طور خاص، وقتی کاربران نهایی خود گزارش‌گیری را انجام می‌دهند، این فعالیت به عنوان هوش مصنوعی تجاری Self Service شناخته می‌شود، که موضوع بخش بعدی است.

خدمت/سرویس هوش تجاری بصورت Self-Service

برای سال‌ها، اگر یک کاربر نهایی در یک سازمان نیاز به گزارش یا داشبورد داشت، باید تمام نیازهای خود را جمع‌آوری کند (داده‌های منابع مورد نیاز، به علاوه توصیفی از چگونگی نمایش گزارش یا داشبورد)، فرم درخواست IT را پر کند و منتظر بماند. سپس IT گزارش را ساخت، که شامل استخراج داده‌ها، بارگذاری آن‌ها در انبار داده، ساخت مدل داده و در نهایت ایجاد گزارش با داشبورد بود. کاربر نهایی آن را مرور می‌کند و آن را تأیید می‌کند یا تغییرات را درخواست می‌کند. این اغلب منجر به یک صف طولانی از درخواست‌های IT شد به طوری که IT در نهایت به عنوان یک گلوگاه بزرگ تبدیل شد. برای کاربران نهایی لحظات، روزها، یا حتی ماه‌ها زمان برای استخراج ارزش از داده‌ها طول می‌کشد. این فرآیند اکنون به عنوان "BI سنتی" شناخته می‌شود، زیرا در سال‌های اخیر چیزی بهتر توسعه یافته است: BI Self Service.

هدف هر راه‌حل معماری داده‌ای که ساخته می‌شود، باید برای هر کاربر نهایی، بدون توجه به مهارت‌های فنی آن‌ها، سریع و آسان باشد تا داده‌ها را پرس و جو کنند و گزارش‌ها و داشبوردها را ایجاد کنند. آن‌ها نباید نیاز به دخالت IT برای انجام هر یک از این وظایف داشته باشند - آن‌ها باید تمام کارها را به تنهایی انجام دهند.

این هدف نیاز به کار بیشتر در ابتدا دارد؛ IT باید با تمام کاربران نهایی تماس بگیرد تا بداند که چه داده‌هایی نیاز دارند، سپس با توجه به نیازهای آن‌ها معماری داده را ساخت. اما این کار برای صرفه‌جویی در زمان ایجاد گزارش‌ها ارزش دار خواهد بود. این رویکرد صف و بازگشت با IT را که عموماً درک کمی از داده‌ها دارند، حذف می‌کند. به جای آن، کاربر نهایی، که بهترین دانش را در مورد داده‌ها دارد، به طور مستقیم به داده‌ها دسترسی پیدا می‌کند، آن‌ها را آماده می‌کند، مدل داده را ساخته و گزارش‌ها را ایجاد می‌کند و تأیید می‌کند که گزارش‌ها صحیح هستند. این جریان کار بسیار بیشتر از کارآمدی دارد.

ایجاد این راه‌حل آسان برای مصرف داده منجر به Self Service BI می‌شود. ایجاد گزارش باید به سادگی همان کار باشد که در یک فضای کاری فیلدها را بکشید. کاربران نهایی باید بدانند چگونه داده‌ها را از جداول مختلف به یکدیگر متصل کنند یا نگران باشند که گزارش بیش از حد کند اجرا می‌شود. هنگام ایجاد یک راه‌حل داده، همیشه باید بپرسید:

چقدر آسان خواهد بود برای افراد خود گزارش‌های خود را ساختن

خلاصه

در این فصل، شما یاد گرفتید که چه چیزی داده‌های بزرگ است و چگونه می‌تواند به شما و سازمان شما کمک کند تا تصمیمات تجاری بهتری بگیرید، به خصوص وقتی با یادگیری ماشین ترکیب می‌شود. شما یاد گرفتید که چگونه می‌توانید داده‌های بزرگ را با استفاده از شش وی بیان کنید و یاد گرفتید که چه چیزی به میراسم داده است و چگونه می‌توانید مراحل آن را شناسایی کنید. در نهایت، شما تفاوت بین BI سنتی و Self Service BI را یاد گرفتید، که هدف آن این است که همه بتوانند به سرعت و آسان از داده‌ها برای ایجاد گزارش‌ها و شناسایی بینش‌ها استفاده کنند.

حالا بیایید ایده‌ای از آنچه در فصل‌های بعدی می‌توانید انتظار داشته باشید بدهم. در فصل ۲، من به معنای معماری داده و یک بررسی سطح بالا از تغییرات انواع معماری داده در سال‌ها می‌پردازم. فصل ۳ جایی است که من نشان می‌دهم چگونه یک جلسه طراحی معماری را برگزار کنید تا به شما کمک کند تا بهترین معماری داده را برای استفاده تعیین کنید.

خش دوم، "مفاهیم مشترک معماری داده"، به جزئیات بیشتری در مورد معماری‌های مختلف می‌پردازد. در فصل ٤، من معرفی می‌کنم که چه چیزی یک انبار داده است و چه چیزی نیست، همچنین چرا می‌خواهید یکی را استفاده کنید. من درباره "رویکرد بالا به پایین" صحبت می‌کنم، می‌پرسم که آیا انبار داده رابطه‌ای مرده است یا خیر، و روش‌های پر کردن یک انبار داده را پوشش می‌دهم.

فصل ٥ معرفی می‌کند که چه چیزی یک دریاچه داده است و چرا می‌خواهید یکی را استفاده کنید. همچنین درباره رویکرد پایین به بالا صحبت می‌کند و سپس به طراحی دریاچه داده و زمانی که باید از چندین دریاچه داده استفاده کنید می‌پردازد.فصل ٦ درباره مفاهیم مشترک معماری داده مربوط به ذخیره‌های داده است، از جمله داره‌های داده، ذخیره‌های داده عملیاتی، مدیریت داده‌های اصلی و مجازی‌سازی داده. فصل ۷ مفاهیم مشترک معماری داده مربوط به طراحی را پوشش می‌دهد، از جمله OLTP در مقابل OLAP، داده‌های عملیاتی در مقابل داده‌های تجزیه و تحلیل، SMP در مقابل MPP، معماری Lambda، معماری Kappa و حفظ چندزبانه.

فصل ۸ در مورد مدل‌سازی داده است، از جمله مدل‌سازی رابطه‌ای و بعدی، مناقشه Kimball در مقابل Inmon، مدل داده مشترک، و آرشیوهای داده. و در فصل ۹، شما درباره ورود داده خواهید خواند، با بخش‌هایی در مورد ELT در مقابل ELT، ELT معکوس، پردازش در دسته‌ها در مقابل پردازش واقعی زمان، و حفظ داده.

بخش سوم بر روی معماری‌های داده خاص تمرکز دارد. فصل ۱۰ معماری انبار داده مدرن را توصیف می‌کند و پنج مرحله ساختن آن را می‌پردازد. فصل ۱۱ معماری ناف داده را پوشش می‌دهد و موارد استفاده آن را می‌پردازد. فصل ۱۲ معماری دریاچه‌انبار داده را مرور می‌کند و معاملات با استفاده از یک انبار داده رابطه‌ای را می‌پردازد.

فصل‌های ۱۳ و ١٤ هر دو درباره معماری‌های دریاچه داده هستند - بسیاری از چیزها وجود دارد! فصل ۱۳ بر روی رویکرد غیرمتمرکز دریاچه داده تمرکز دارد و چهار اصل یک دریاچه داده را توصیف می‌کند، و معرفی می‌کند که چه چیزی دامنه‌های داده و محصولات داده هستند.

فصل ١٤ به نگرانی‌ها و چالش‌های ساخت یک دریاچه داده می‌پردازد و برخی از اخلاق رایج دریاچه داده را مواجه می‌کند. این به شما کمک می‌کند تا ببینید آیا آماده برای پذیرش یک دریاچه داده هستید. این با نمایش آینده احتمالی دریاچه داده می‌پایان می‌یابد.

فصل ۱۵ به دلایل موفقیت و شکست پروژه‌ها نگاه می‌کند و سازمان تیمی را که برای ساخت معماری داده نیاز دارید توصیف می‌کند. در نهایت، فصل ١٦ یک بحث درباره منابع باز، مزایای ابر، ارائه‌دهندگان اصلی ابر، بودن چند ابر، و چارچوب‌های نرم‌افزاری است.

حالا من در حال انقلاب در دنیای داده شما هستم. آیا آماده‌اید؟

فصل دوم: انواع معماری های داده

استثنای اهمیت انسانی در سرمایه‌گذاری زمان در طراحی و ساخت معماری داده مناسب است. این نکته را در ابتدای مسیر من به سخت ترین راه‌ها یاد گرفتم. به خاطر وسوسه‌ای که برای شروع ساخت راه‌حل خود داشتم، تصمیمات مهمی درباره طراحی معماری و محصولاتی که باید استفاده کنیم، را نادیده گرفتم. سه ماه بعد از شروع پروژه، متوجه شدم که معماری نمی‌تواند برخی از منابع داده مورد نیاز را پشتیبانی کند. ما به اصل باید پروژه را از ابتدا دوباره شروع کرد و یک معماری دیگر و محصولات متفاوتی را برای آن طراحی کردیم، که باعث هزینه و زمان زیادی از دست رفت. بدون برنامه‌ریزی صحیح، کاربران نهایی ارزشی از راه‌حل شما به دست نمی‌آورند، آن‌ها به دلیل سرریز مهلت‌ها ناراضی هستند، و شرکت شما خطر دارد که از رقیبان خود دور بیاید.

هنگام ساخت یک راه‌حل داده، شما نیاز به یک طرح برنامه‌ریزی شده دارید که بتوانید به آن پیوسته شوید. در اینجا معماری داده به کار می‌آید. معماری داده یک رویکرد و مفهوم معماری سطح بالا را تعریف می‌کند، مجموعه‌ای از فناوری‌ها را که باید استفاده کنید، را مشخص می‌کند و جریان داده‌هایی را که برای ساخت راه‌حل داده شما برای جمع‌آوری داده‌های بزرگ استفاده می‌شود، مشخص می‌کند. تصمیم‌گیری درباره یک معماری داده می‌تواند بسیار چالش‌برانگیز باشد، زیرا هیچ معماری یک‌بار کاربردی وجود ندارد. شما نمی‌توانید از طریق یک کتاب به دنبال یک لیست موجود از رویکردهای معماری با محصولات متناظر بگردید. هیچ نمودار ساده‌ای برای دنبال کردن وجود ندارد با درخت‌های تصمیم که شما را به معماری کامل هدایت می‌کند. رویکرد معماری شما و فناوری‌هایی که استفاده می‌کنید، بسیار بین مشتری به مشتری و بین مورد استفاده به مورد استفاده متفاوت خواهد بود.

انواع اصلی رویکردها و مفاهیم معماری سطح بالا دقیقاً همان چیزی هستند که این کتاب درباره آن است. این یک لیست موجود نیست، اما به شما احساس می‌دهد که همه چه چیزی هستند. با وجود اینکه مفید است که معماری‌های داده را بر اساس ویژگی‌هایشان به انواع تقسیم کنیم، که من در این کتاب این کار را انجام خواهم داد، این همان چیزی نیست که از یک دسته از قالب‌های یک‌بار کاربردی انتخاب کردن. هر معماری داده منحصر به فرد است و نیاز به یک رویکرد سفارشی برای برآورده کردن نیازهای تجاری خاص دارد.

معماری داده به طراحی و سازماندهی کلی داده در یک سیستم اطلاعات اشاره دارد. قالب‌های از پیش تعریف شده برای معماری داده می‌توانند به نظر راهی ساده برای راه‌اندازی یک سیستم جدید به سرعت بیایند. با این حال، قالب‌های معماری داده اغلب نمی‌توانند نیازها و محدودیت‌های خاص سیستمی که به آن‌ها اعمال می‌شوند، را در نظر بگیرند، که می‌تواند منجر به مشکلات در کیفیت داده، عملکرد سیستم و نگهداری شود. علاوه بر این، نیازهای سازمان و سیستم‌های داده احتمالاً با گذشت زمان تغییر می‌کنند، که نیاز به به‌روزرسانی‌ها و تنظیمات در معماری داده دارد. یک قالب استاندارد ممکن است به اندازه کافی انعطاف‌پذیر نباشد تا این تغییرات را پذیرفته باشد، که می‌تواند باعث ناکارآمدی‌ها و محدودیت‌های سیستم شود.

این فصل یک گردش مرور مختصر از انواع اصلی را که کتاب پوشش خواهد داد، ارائه می‌دهد: انبار داده رابطه‌ای، دریاچه داده، انبار داده مدرن، ناف داده، دریاچه‌انبار داده، و دریاچه داده. هر نوع در فصل خود با جزئیات زیادی در بخش‌های بعدی کتاب مطرح خواهد شد.

سیر تکاملی معماری های داده

یک پایگاه داده رابطه‌ای داده‌ها را به صورت ساختاری ذخیره می‌کند، با اینکه روابط بین عناصر داده توسط کلیدها تعریف می‌شوند. داده‌ها معمولاً در جداول سازماندهی می‌شوند، با اینکه هر جدول شامل سطرها و ستون‌ها است. هر سطر نمایانگر یک نمونه از داده است، در حالی که هر ستون یک ویژگی خاص از داده را نمایش می‌دهد.

پایگاه‌های داده رابطه‌ای برای کار با داده‌های ساختاری طراحی شده‌اند و یک چارچوب برای ایجاد، تغییر، و پرس و جوی داده با استفاده از زبان استاندارد شناخته شده به عنوان SQL یا Structured Query Language فراهم می‌کنند. مدل رابطه‌ای اولین بار توسط Edgar F. Codd در سال ۱۹۷۰ پیشنهاد شد و از میانه دهه ۷۰ به بعد، مدل غالب برای سیستم‌های مدیریت پایگاه داده شده است. بیشتر برنامه‌های عملیاتی نیاز دارند تا داده‌ها را به طور دائمی ذخیره کنند و یک پایگاه داده رابطه‌ای ابزار انتخاب شده برای اکثریت است.

در پایگاه‌های داده رابطه‌ای، جایی که یکپارچگی و یکپارچگی داده از اهمیت اصلی برخوردار هستند، داده‌ها معمولاً با یک رویکرد به نام schema-on-write سازماندهی می‌شوند. Schema به ساختار رسمی اشاره دارد که سازماندهی

و روابـط بـین جـداول، فیلـدها، انـواع داده و محـدودیت‌ها را تعریـف می‌کنـد. ایـن بـه عنـوان یـک طـرح بـرای ذخـیره و مـدیریت داده عمـل می‌کنـد و یکپـارچگی، یکپـارچگی و سـازماندهی کارآمـد درون پایگـاه داده را تضـمین می‌کنـد. پایگاه‌هـای داده رابطـه‌ای (و انبارهـای داده رابطـه‌ای) نیـاز بـه کـمی کـار از پـیش دارنـد قبـل از اینکـه داده می‌توانـد در آن‌هـا قـرار بگـیرد. شـما بایـد پایگـاه داده و جـداول، فیلـدها و schema آن را ایجـاد کنیـد، سـپس کـد را بنویسـید تـا داده را بـه پایگـاه داده منتقـل کنیـد. بـا یـک رویکـرد schema، schema-on-write داده تعریـف و اجـرا می‌شـود وقـتی داده نوشـته یـا در پایگاه داده مصـرف می‌شـود. داده بایـد قبـل از اینکـه ذخـیره شـود، بـا شـماره نمادهـا، محـدودیت‌ها و روابـط تعریـف شـده مطابقت داشته باشد.

در مقابـل، در یـک رویکـرد schema، schema-on-read زمـانی اعمـال می‌شـود کـه داده خوانـده یـا دسـترسی پیـدا می‌کنـد، بـه جـای زمـانی کـه نوشـته می‌شـود. داده می‌توانـد بـه سیسـتم ذخیره‌سـازی مصـرف شـود بـدون اینکـه بـه یـک schema سـخت وابسـته باشـد، و سـاختار فقـط زمـانی تعریـف می‌شـود کـه داده پـرس و جـو شـده یـا مصـرف شـده اسـت. ایـن رویکـرد انعطاف‌پـذیری بیشـتری در ذخـیره داده‌هـای غیرسـاختاری یـا نیمـه سـاختاری ارائـه می‌دهـد و معمـولاً در دریاچه‌های داده که در فصل‌های بعدی این کتاب بحث می‌شود، استفاده می‌شود.

در سـطح بـالا، معماری‌هـای داده چـارچوبی را بـرای سـازماندهی و مـدیریت داده‌هـا در گونـه‌ای فـراهم می‌کننـد کـه نیازهـای یـک سـازمان را پشـتیبانی کنـد. ایـن شـامل تعریـف چگـونگی جمع‌آوری، ذخیره‌سـازی، پـردازش و دسـترسی بـه داده‌هـا اسـت، همچنـین حفـظ کیفیـت، امنیـت و حـریم خصـوصی داده‌هـا. در حـالی کـه معماری‌هـای داده می‌تواننـد در اشـکال مخـتلفی باشند، برخی عناصر مشترک عبارتند از:

- **ذخیره‌سـازی داده**: تمـام معماری‌هـای داده بایـد تعیـین کننـد کـه چگونـه داده ذخـیره می‌شـود، از جملـه روش ذخیره‌سـازی فـیزیکی (ماننـد درایوهـای سـخت افـزاری یـا ذخیره‌سـازی ابـری) و سـاختارهای داده اسـتفاده شـده بـرای سازماندهی داده‌ها.

- **پـردازش داده**: معماری‌هـای داده بایـد تعیـین کننـد کـه چگونـه داده پـردازش می‌شـود، از جملـه هرگونـه تبـدیلات یـا محاسباتی که قبل از ذخیره‌سازی یا تجزیه و تحلیل داده انجام می‌شود.

- **دسـترسی بـه داده**: معماری‌هـای داده بایـد مکانیزم‌هـایی را بـرای دسـترسی بـه داده فـراهم کننـد، از جملـه رابط‌هـای کاربری و رابط‌های برنامه‌نویسی که به داده اجازه می‌دهند تا پرس و جو شود و تجزیه و تحلیل شود.

- **امنیـت و حـریم خصـوصی داده**: معماری‌هـای داده بایـد مکانیزم‌هـایی را بـرای تضـمین امنیـت و حـریم خصـوصی داده در بر بگیرند، مانند کنترل‌های دسترسی، رمزگذاری و ماسک کردن داده.

- **حاکمیـت داده**: معماری‌هـای داده بایـد چارچوب‌هـایی را بـرای مـدیریت داده فـراهم کننـد، از جملـه اسـتانداردهای کیفیت، ردیابی ارث‌بری و سیاست‌های حفظ.

در کل، هـدف اصـلی یـک معمـاری داده ایـن اسـت کـه بـه یـک سـازمان امکان مـدیریت و اسـتفاده مـوثر از دارایی‌هـای داده خود را بدهد تا بتواند اهداف تجاری و فرآیندهای تصمیم‌گیری خود را پشتیبانی کند.

جـدول ۲-۱، کـه در آن مقایسـه‌ای از سـطح بـالا از ویژگی‌هـای معماری‌هـای داده‌ای کـه در ایـن کتـاب مطـرح می‌شـوند، ارائـه می‌دهـد، نقطـه شروعی مناسـبی بـرای تعیـین می‌دهـد کـه کـدام معمـاری ممکـن اسـت بهـترین تناسـب بـا مـورد اسـتفاده شما باشد.

Table 2-1. Comparison of data architectures

Characteristic	Relational data warehouse	Data lake	Modern data warehouse	Data fabric	Data lakehouse	Data mesh
Year introduced	1984	2010	2011	2016	2020	2019
Centralized/ decentralized	Centralized	Centralized	Centralized	Centralized	Centralized	Decentralized
Storage type	Relational	Object	Relational and object	Relational and object	Object	Domain-specific
Schema type	Schema-on-write	Schema-on-read	Schema-on-read and schema-on-write	Schema-on-read and schema-on-write	Schema-on-read	Domain-specific
Data security	High	Low to medium	Medium to high	High	Medium	Domain-specific
Data latency	Low	High	Low to high	Low to high	Medium to high	Domain-specific
Time to value	Medium	Low	Low	Low	Low	High
Total cost of solution	High	Low	Medium	Medium to high	Low to medium	High
Supported use cases	Low	Low to medium	Medium	Medium to high	High	High
Difficulty of development	Low	Medium	Medium	Medium	Medium to high	High
Maturity of technology	High	Medium	Medium to high	Medium to high	Medium to high	Low
Company skill set needed	Low	Low to medium	Medium	Medium to high	Medium to high	High

انبار داده رابطه ای

پایگاه‌های داده رابطه‌ای برای چندین دهه اصلی‌ترین روش ذخیره‌سازی داده بودند. اولین سیستم انبار داده رابطه‌ای که در تولید استفاده شد، سیستم Teradata بود که در دانشگاه استنفورد توسط دکتر جک اسمر توسعه یافت و در سال ۱۹۷۹ شرکت Teradata را تأسیس کرد. بانک ولز فارگو اولین سیستم Teradata را در سال ۱۹۸۳ نصب کرد و برای تجزیه و تحلیل داده‌های مالی از آن استفاده کرد.

همانطور که سازمان‌ها شروع به تولید مقادیر بسیار بزرگی از داده کردند، پردازش و تجزیه و تحلیل این داده‌ها بدون تاخیر طولانی بسیار چالش‌برانگیز شد. محدودیت‌های پایگاه‌های داده رابطه‌ای منجر به توسعه انبارهای داده رابطه‌ای شد که در اواخر دهه ۱۹۸۰ به طور گسترده‌تری محبوب شدند، حدود ۱۵ سال پس از آنکه پایگاه‌های داده رابطه‌ای موجود شدند. یک انبار داده رابطه‌ای (RDW) نوع خاصی از پایگاه داده رابطه‌ای است که برای انبارداری داده و برنامه‌های هوش مصنوعی طراحی شده است، با عملکرد پرس و جو بهینه و پشتیبانی از تجزیه و تحلیل داده در مقیاس بزرگ. در حالی که هر دو انبارهای داده رابطه‌ای و پردازش تراکنشی- از مدل رابطه‌ای برای سازماندهی داده استفاده می‌کنند، یک انبار داده رابطه‌ای معمولاً بزرگتر در مقیاس است و برای پرس و جوهای تحلیلی بهینه شده است.

RDW ها هر دو موتور محاسباتی و ذخیره‌سازی دارند. موتور محاسباتی قدرت پردازشی است که برای پرس و جو داده استفاده می‌شود. ذخیره‌سازی مربوط به ذخیره‌سازی رابطه‌ای است که داده‌هایی را که از طریق جداول، سطرها و ستون‌ها ساختار داده شده‌اند، نگهداری می‌کند. قدرت محاسباتی RDW فقط می‌تواند بر روی ذخیره‌سازی رابطه‌ای خود استفاده شود - آن‌ها به هم مرتبط هستند.

برخی از ویژگی‌های مهمترین RDW شامل پشتیبانی از تراکنش (اطمینان از پردازش قابل اعتماد و یکنواخت داده)، رده‌های اعتبارسنجی (نگهداری یک رکورد از تمام فعالیت‌های انجام شده بر روی داده در سیستم) و اجرای طرح (اطمینان از سازماندهی و ساختاردهی داده به شکل پیش‌تعیین شده) است.

در دهه‌های ۱۹۷۰ و ۱۹۸۰، سازمان‌ها از پایگاه‌های داده رابطه‌ای برای برنامه‌های عملیاتی مانند ورود سفارش و مدیریت موجودی استفاده می‌کردند. این برنامه‌ها به عنوان سیستم‌های پردازش تراکنش آنلاین (OLTP) شناخته می‌شوند. سیستم‌های OLTP می‌توانند تغییرات ایجاد، خواندن، به‌روزرسانی و حذف (CRUD) را در یک پایگاه داده ایجاد کنند. عملیات CRUD پایه ساختن و مدیریت داده در معماری‌های داده را تشکیل می‌دهند. آن‌ها برای طراحی و پیاده‌سازی سیستم‌های ذخیره‌سازی داده و رابط‌های کاربری که با داده تعامل دارند، ضروری هستند.

عملیات CRUD نیاز به زمان پاسخگویی سریع از برنامه دارند تا کاربران نهایی ناراضی نشوند که چقدر زمان برای به‌روزرسانی داده طولانی می‌کند. شما می‌توانید پرس و جوها و گزارشات را بر روی یک پایگاه داده رابطه‌ای که توسط یک برنامه عملیاتی در حال استفاده است اجرا کنید، اما این کار منابع زیادی را مصرف می‌کند و می‌تواند با عملیات CRUD دیگری که در همان زمان اجرا می‌شوند، تداخل داشته باشد. این می‌تواند همه چیز را کند کند.

RDW ها به طور جزئی برای حل این مشکل اختراع شدند. داده‌ها از پایگاه داده رابطه‌ای به یک انبار داده کپی می‌شوند و کاربران می‌توانند پرس و جوها و گزارشات را بر روی انبار داده به جای پایگاه داده رابطه‌ای اجرا کنند. به این ترتیب، آن‌ها سیستمی را که پایگاه داده رابطه‌ای را نگهداری می‌کند تحت فشار نمی‌قرارد و برنامه را برای کاربران نهایی کند نمی‌کنند.

RDW ها همچنین داده‌ها را از چندین برنامه متمرکز می‌کنند تا گزارش‌دهی را بهبود بخشند، همانطور که در شکل ۲-۱ نشان داده شده است.

فصل ٤ به طور جزئی‌تر درباره RDW ها صحبت خواهد کرد.

دریاچه داده

جهان داده یک مفهوم جدیدتر است که حدود سال ۲۰۱۰ به وجود آمد. شما می‌توانید جهان داده را به عنوان یک سیستم فایل بزرگتر تفسیر کنید، که به ندرت از سیستم فایل لپ‌تاپ شما متفاوت است. یک جهان داده فقط ذخیره‌سازی است - به طور متفاوت از یک انبار داده رابطه‌ای، به آن هیچ موتور محاسباتی مرتبطی نیست. با خوشحالی، بسیاری از موتورهای محاسباتی می‌توانند با جهان‌های داده کار کنند، بنابراین قدرت محاسباتی معمولاً ارزان‌تر برای یک جهان داده نسبت به یک انبار داده رابطه‌ای است. یک تفاوت دیگر این است که در حالی که RDW ها از ذخیره‌سازی رابطه‌ای استفاده می‌کنند، جهان‌های داده از ذخیره‌سازی اشیاء استفاده می‌کنند، که نیازی به ساختاردهی داده به سطرها و ستون‌ها ندارد.

تکنولوژی ذخیره‌سازی جهان داده با سیستم فایل توزیع‌شده (HDFS Apache Hadoop) شروع شد، که یک تکنولوژی آزاد و منبع باز است که تقریباً به طور منحصر به فرد در محل نصب می‌شود و در اوایل دهه ۲۰۱۰ بسیار محبوب بود. HDFS یک سیستم ذخیره‌سازی توزیع‌شده، قابل انعطاف و مقاوم در برابر خطا است که برای اجرا بر روی سخت‌افزار تجاری طراحی شده است. این یک مؤلفه اصلی از اکوسیستم Apache Hadoop است که در فصل ١٦ بیشتر بحث می‌شود. همانطور که محاسبات ابری ادامه داد، جهان‌های داده در ابر با استفاده از نوع دیگری از ذخیره‌سازی ساخته شدند و بیشتر جهان‌های داده اکنون در ابر وجود دارند.

در مقابل یک انبار داده رابطه‌ای، یک جهان داده schema-on-read است، به این معنی که هیچ کار اولیه‌ای لازم نیست تا داده را در جهان داده قرار دهید: می‌توانید به سادگی مانند کپی کردن فایل‌ها به آن باشد، مانند آنچه با پوشه‌ها در لپ‌تاپ خود انجام می‌دهید. داده‌ها در یک جهان داده در فرمت طبیعی (یا خام) ذخیره می‌شوند، به این معنی که می‌توانند از سیستم منبع خود به جهان داده بدون تبدیل به فرمت دیگری منتقل شوند. به عنوان مثال، اگر داده‌ها را از یک پایگاه داده رابطه‌ای به یک فایل در فرمت CSV خام خروجی دهید، می‌توانید آن را بدون تغییر در یک جهان داده ذخیره کنید. اگر می‌خواهید آن را در یک انبار داده رابطه‌ای ذخیره کنید، باید آن را تبدیل کنید تا به سطرها و ستون‌های یک جدول مناسب شود.

هنگامی که یک فایل داده را به یک جهان داده کپی می‌کنید، ممکن است ساختار (schema) آن با فایل کپی شده شود یا در یک فایل مجزا قرار داشته باشد. بنابراین، شما باید ساختار را با ایجاد آن یا بارگیری آن از فایل مجزا تعریف کنید - این دلیل است که اصطلاح schema-on-read استفاده می‌شود. همانطور که شکل ۲-۲ نشان می‌دهد، داده‌ها از سیستم‌های منبع مانند پایگاه‌های داده برنامه‌های عملیاتی، داده‌های حسگر، و داده‌های رسانه‌های اجتماعی می‌توانند همه در جهان داده قرار بگیرند. این فایل‌ها می‌توانند داده‌ای را نگهداری کنند که ساختاردار است (مانند داده‌های پایگاه داده رابطه‌ای)، نیمه ساختار دار (مانند فایل‌های CSV، لاگ‌ها، XML یا JSON)، یا بدون ساختار (مانند از ایمیل‌ها، اسناد، و PDF ها) را نگهداری کنند. آن‌ها حتی می‌توانند داده‌های دودویی را نگهداری کنند (مانند تصویر، صدا، و ویدیو).

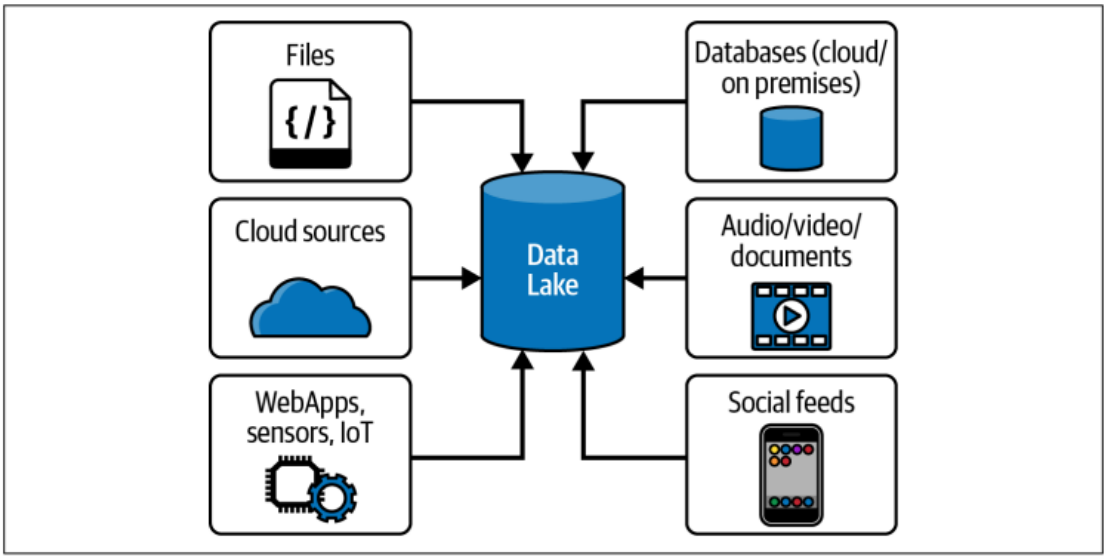


Figure 2-2. Data lake

جهان‌های داده از ابتدا به عنوان راه حلی برای تمام مشکلات مرتبط با انبارهای داده رابطه‌ای شروع شدند، از جمله هزینه بالا، قابلیت مقیاس‌پذیری محدود، عملکرد ضعیف، بار آماده‌سازی داده، و پشتیبانی محدود از انواع پیچیده داده. شرکت‌هایی که Hadoop و جهان‌های داده را فروش می‌دادند، مانند Hortonworks، Cloudera، و MapR، آن‌ها را به شکلی تبلیغ کردند که به نظر می‌رسید که پر از انواع و نقاشی‌های مجیزی هستند که داده‌ها را کپی و تمیز می‌کنند و به کاربران نهایی با آسانی مجیز در دسترس قرار می‌دهند. آن‌ها ادعا کردند که جهان‌های داده می‌توانند کاملاً انبارهای داده رابطه‌ای را جایگزین کنند، در یک رویکرد "یک تکنولوژی برای انجام همه چیز" استفاده کنند.

مشکل این بود که پرس و جو در جهان‌های داده واقعاً آسان نیست: نیاز به مهارت‌های نسبتاً پیشرفته‌ای دارد. IT به کاربران نهایی می‌گفت، "هی، ما همه داده‌هایی که نیاز دارید را به این جهان داده کپی کردیم. فقط برو و یک کتاب چاپی Jupyter باز کن و با استفاده از Hive و Python گزارشات خود را با فایل‌های در این پوشه‌ها ساخته باش." این کاملاً شکست خورد، زیرا بیشتر کاربران نهایی نزدیک به مهارت‌های لازم برای انجام همه این کارها نداشتند.

شرکت‌ها به سختی آموختند که این راه حل‌های پیچیده و سخت برای استفاده واقعاً گران‌تر بودند به دلیل هزینه‌های سخت‌افزار و پشتیبانی، تاخیر در تولید، و کاهش بهره‌وری. علاوه بر این، جهان‌های داده برخی از ویژگی‌هایی که مردم در مورد انبارهای داده دوست داشتند، مانند پشتیبانی از تراکنش، اجرای ساختار (schema enforcement)، و نوارهای ردیابی را نداشتند. این منجر شد به اینکه دو از سه برترین تامین‌کننده جهان داده، Hortonworks و MapR، خارج از کسب و کار شوند.

اما جهان داده ناپدید نشد. به جای آن، هدف آن به یک هدف متفاوت ولی بسیار مفید تغییر کرد: آماده‌سازی و مرحله‌بندی داده. فصل ۵ جهان‌های داده را به طور جزئی بحث خواهد کرد.

انبار داده مدرن

انبارهای داده رابطه‌ای و جهان‌های داده، به تنهایی، معماری‌های ساده هستند. آن‌ها فقط یک تکنولوژی را برای مرکزیت داده‌ها استفاده می‌کنند، با محصولات پشتیبانی کم یا هیچ. وقتی شما از تکنولوژی‌ها و محصولات بیشتری برای پشتیبانی از انبارهای داده رابطه‌ای یا جهان‌های داده استفاده می‌کنید، آن‌ها به معماری‌های مورد بحث در این و فصل‌های بعدی تکامل می‌یابند. جهان‌های داده نتوانستند انبارهای داده رابطه‌ای را جایگزین کنند اما همچنان مزایایی برای مرحله‌بندی و آماده‌سازی داده ارائه دادند. چرا نتوانست مزایای هر دو را داشته باشیم؟

حدود سال ۲۰۱۱، بسیاری از شرکت‌ها شروع به ساخت معماری‌هایی کردند که جهان‌های داده را کنار کنار انبارهای داده رابطه‌ای قرار می‌دادند تا معماری داده‌ای را تشکیل دهند که ما اکنون به آن انبار داده مدرن (MDW) می‌گوییم، نشان داده شده در شکل ۲-۳. مدرن در انبار داده مدرن به استفاده از تکنولوژی‌ها و رویکردهای جدیدتر به انباری داده اشاره دارد.

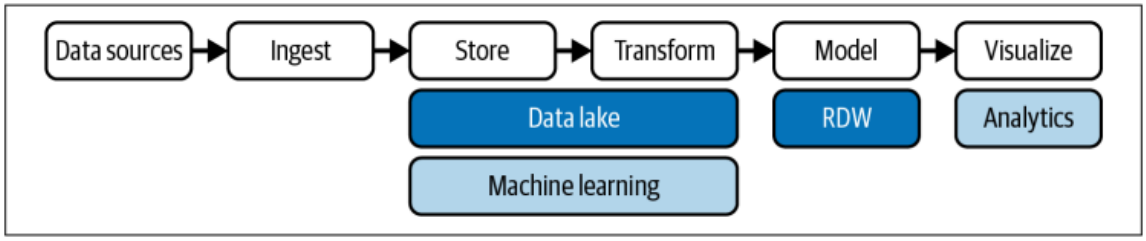


Figure 2-3. Modern data warehouse (MDW)

این یک رویکرد "بهترین هر دو جهان" است: جهان داده برای مرحله‌بندی و آماده‌سازی داده استفاده می‌شود و دانشمندان داده آن را برای ساخت مدل‌های یادگیری ماشین استفاده می‌کنند؛ در حالی که انبار داده برای خدمات، امنیت، و مطابقت استفاده می‌شود و کاربران تجاری با آن پرس و جو و گزارش‌گیری می‌کنند. فصل ۱۰ بسیاری از جزئیات بیشتر در مورد معماری‌های انبار داده مدرن ارائه خواهد داد.

دیتا فابریک

اصطلاح "data lake house" یک ترکیب (blend) از جهان داده و انبار داده است. معماری‌های lake house در حدود سال ۲۰۲۰ محبوب شدند، زمانی که شرکت Data-bricks از این اصطلاح استفاده کرد. مفهوم lake house این است که از انبار داده رابطه‌ای خودداری کنید و فقط یک مخزن، یک جهان داده، را در معماری داده‌ی خود استفاده کنید. همه انواع داده‌ها - ساختار در، نیمه ساختار دار، و بدون ساختار - به جهان داده می‌رسند و همه پرس و جوها و گزارش‌ها از جهان داده انجام می‌شوند.

من می‌دانم که این ایده را می‌خوانید: "منتظر بمانید. شما گفتید که جهان‌های داده هنگام ظهور اولیه آن‌ها این رویکرد را گرفتند و کاملاً شکست خوردا! چه تغییر کرد؟" پاسخ، همانطور که در شکل ۲-۵ نشان داده شده است، یک لایه نرم‌افزاری ذخیره‌سازی تراکنشی- است که بر روی یک جهان داده موجود اجرا می‌شود و آن را بیشتر مانند یک پایگاه داده رابطه‌ای کار می‌کند. گزینه‌های متنافس منبع باز برای این لایه شامل Apache Iceberg، Delta Lake، و Apache Hudi هستند. همه این موارد در فصل ۱۲ با جزئیات بیشتر پوشش داده خواهند شد.

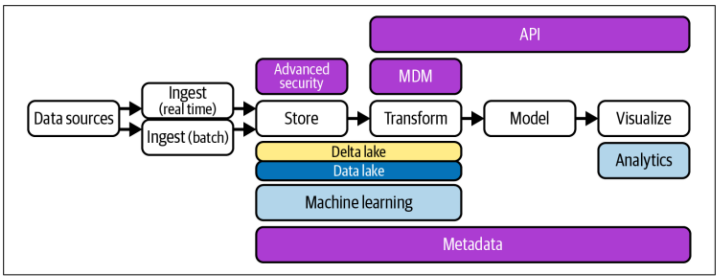


Figure 2-5. Data lakehouse

دیتا مش

اصطلاح "data mesh" برای اولین بار در یک پست وبلاگ در ماه می ۲۰۱۹ توسط Zhamak Deh-Ghani، مؤسس و مدیر عامل Nextdata و نویسنده کتاب "Data Mesh: Delivering Data-Driven Value at Scale"، (O'Reilly ۲۰۲۲) معرفی شد. در دسامبر ۲۰۲۰، Deh-Ghani توضیحات بیشتری در مورد آنچه یک data mesh است ارائه داد و چهار اصل پایه‌ای را مشخص کرد. معماری‌های data mesh از آن زمان به بعد یک موضوع بسیار گرم بوده‌اند، در مقالات وبلاگ، ارائه‌ها، کنفرانس‌ها، و پوشش رسانه‌ای بحث و بحث شده‌اند و حتی در چرخه انتظارات Gartner برای مدیریت داده ظاهر شده است. هر چیزی در مورد معماری data mesh دوست دارید، اما با وجود اینکه این موضوع به شدت مورد توجه قرار گرفته است، فقط برای تعداد کمی از موارد استفاده مناسب است.

انبار داده مدرن، نخ داده، و معماری data lake house همگی شامل مرکزیت داده‌ها هستند: کپی داده‌های عملیاتی به مکان مرکزی که توسط IT مالکیت دارد و که تحت معماری‌ای که IT کنترل می‌کند، که سپس داده‌های تحلیلی (سمت چپ شکل ۲-۶) را ایجاد می‌کند. این رویکرد مرکزیت باعث می‌شود سه چالش اصلی را به همراه داشته باشد: مالکیت داده، کیفیت داده، و مقیاس‌پذیری سازمانی/فنی. هدف از data mesh حل این چالش‌ها است.

در یک data mesh، داده در چندین حوزه در یک شرکت نگهداری می‌شود، مانند تولید، فروش، و تامین‌کنندگان (سمت راست شکل ۲-۶)، که هر یک از آن‌ها تیم کوچک IT خود را دارد که داده‌های خود را نگهداری می‌کند، آن‌ها را تمیز می‌کند، داده‌های تحلیلی را ایجاد می‌کند، و آن‌ها را در دسترس قرار می‌دهد. هر حوزه همچنین دارای ساختار محاسباتی و ذخیره‌سازی خود است. این نتیجه در یک معماری متمرکز می‌شود که داده، افراد، و زیرساخت مقیاس می‌شوند - هر چه تعداد حوزه‌هایی که دارید، هر چه تعداد بیشتری از افراد و زیرساخت دارید.

سیستم می‌تواند با داده‌های بیشتری کنار بیاید و IT دیگر یک گلوگاه نیست.

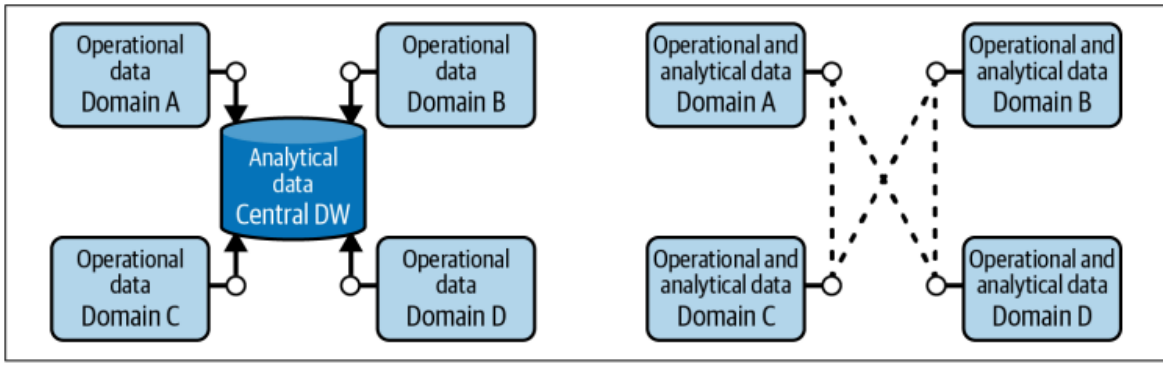


Figure 2-6. Data mesh

مهم است که درک کنید که data mesh یک مفهوم است، نه یک تکنولوژی. هیچ "data mesh" در یک کادر وجود ندارد که می‌توانید آن را خریداری کنید. پیاده‌سازی data mesh شامل یک تغییر بزرگ سازمانی و فرهنگی است که بسیار چند شرکت آماده برای آن هستند. (در واقع، بیشتر شرکت‌ها حتی بزرگ نیستند تا برای معماری data mesh در نظر گرفته شوند: این بسیار یک رویکرد کارآفرینی است.) ساختن آن نیازمند تعیین است که کدام قطعات از تکنولوژی موجود می‌توانید برای آن استفاده مجدد کنید و کدام قطعات را باید ایجاد کنید. هر حوزه می‌تواند تعیین کند که کدام تکنولوژی‌ها را برای ساخت بخش خود از data mesh استفاده می‌کند، که می‌تواند شامل ساخت یک انبار داده مدرن، نخ داده، یا data lake house باشد. در مورد معماری‌های data mesh بسیار بحث وجود دارد، و من در فصل‌های ۱۳ و ۱۴ آن را مطرح خواهم کرد.

خلاصه

حالا که شما درک عمیقی از انواع معماری‌های داده دارید، فصل بعدی درباره چگونگی تعیین بهترین معماری داده برای استفاده خواهد بود: یک فرآیند به نام جلسه طراحی معماری.

فصل سوم: جلسات طراحی و معماری

من صدها جلسه طراحی معماری را انجام داده‌ام و آن‌ها را برای ساخت یک راه حل موفق در زمینه داده ضروری می‌دانم. این فصل شما را در طول چگونگی برگزاری یک جلسه طراحی معماری موفق هدایت می‌کند که راه حل داده‌ی شما را در مسیر صحیح قرار می‌دهد.

بیان دقیق از جلسات معماری داده

یک جلسه طراحی معماری (ADS) یک بحث ساختاری با ذی‌نفعان تجاری و فنی است که توسط کارشناسان فنی هدایت می‌شود و بر روی تعریف و برنامه‌ریزی طراحی سطح بالای یک راه حل برای جمع‌آوری داده‌ها برای فرصت‌های تجاری خاص متمرکز است. اولین ADS شروع فرآیند معماری است و منجر به بسیاری از بحث‌های بیشتر (شاید حتی برخی از جلسات ADS دیگر) می‌شود تا پروژه راه حل داده را پشتیبانی کند. ADS باید دو محصول تولید کند:

- یک معماری (یا "طرح نقشه") که می‌تواند نقطه شروع برای راه حل داده باشد

- یک برنامه عملیاتی سطح بالا، که ممکن است شامل نمایش‌های پیوسته، اثبات‌های مفهومی، و بحث‌های محصول باشد

یک ADS نیست یک کارگاه فنی، یک آموزش فنی، یک نمایش فنی، یا یک جلسه نیازمندی‌های سطح پایین.

چرایی برگزاری جلسات معماری

بسیاری از دلایل وجود دارد که باید یک جلسه طراحی معماری (ADS) برگزار کنید. اولین دلیل، این است که اغلب چندین ذی‌نفع از همان سازمان را با هم جمع می‌کند، از جمله کسانی که هرگز به صورت مستقیم با هم ملاقات نکرده‌اند. حتی فقط این اقدام برای برگزاری ملاقات در شخص یا به صورت مجازی می‌تواند به ایجاد خلاقیت و حل مشکل کمک کند. علاوه بر این، ممکن است شامل ذی‌نفعانی باشد که کسب و کار خود را بسیار خوب درک می‌کنند و می‌توانند به طور دقیق موارد استفاده خاصی که قصد حل آن‌ها را دارند را توصیف کنند.

ADS همچنین پروژه‌هایی را با یک مورد تجاری قوی از "پروژه‌های علمی" که برای آزمایش برخی از فناوری یا محصول قرار دارد، تمایز می‌دهد. این باعث می‌شود دلایل پروژه را بیان کند و اگر واقعاً یک پروژه علمی است، منابع اختصاص یافته به آن را محدود می‌کند.

علاوه بر این، یک ADS یک چارچوب ساختاری برای "فکر کردن بزرگ، شروع کردن کوچک" فراهم می‌کند. به عنوان توضیح داده شده است، این شامل تولید یک معماری سطح بالا است، اما باید همچنین در مورد چگونگی شروع کوچک برای کسب پیروزی‌های سریع بحث کند. به عنوان مثال، معماری در نهایت ممکن است شامل دریافت داده از ده‌ها منبع باشد، اما ADS ممکن است یک رویکرد کم ریسک، با پاداش بالا را انتخاب کند، مانند شروع با دو یا سه منبع داده ارزشمند که نیاز به ساعت‌های کمتری برای تکمیل دارند.

شما همچنین می‌توانید مشتریان سازمان خود و/یا سازمان‌های همکار را در یک ADS شامل کنید. وقتی من در حال برگزاری اولین ADS خود بودم، یک معمار دیگر از Microsoft فرآیند را به عنوان "درمان تجاری" توصیف کرد. به سرعت متوجه شدم که چه معنی دارد. در حدود دو ده شرکت‌کننده از یک شرکت مشتری داشتیم، بیشتر آن‌ها هرگز با هم ملاقات نکرده بودند. ما شروع به بحث در مورد محیط فعلی آن‌ها کردیم و چندین نقطه درد را برای ما برداشتند که پیشرفت را مسدود می‌کند. به شگفت من، یک شرکت‌کننده دیگر از آن سازمان با مشاوره در مورد چگونگی حل مسدود کننده به ما کمک کرد. سپس آن‌ها دو مسدود دیگر را حل کردند در حالی که من به عقب گذاشتم و گوش دادم. (به طور کلی، همسرانم و من همچنین به حذف مسدود کمک می‌کنیم، اما برای من آشکار بود کمتر صحبت کردن و اجازه دادن به دیگران برای حل کردن مسائل بین خودشان بود.)

شما همچنین از مشتری خود به سایر روش‌ها یاد خواهید گرفت، به خصوص اگر تیم حساب شما به اندازه کافی جامع نبوده باشد. شما می‌توانید در هر سوالی که در مورد محیط فعلی آن‌ها، نقاط درد آن‌ها، اهداف آن‌ها، و سایر اطلاعاتی که نیاز دارید دارید بپردازید.

یک ADS مشتری همچنین به تیم حساب شما بسیاری از راه‌های پیگیری فراهم می‌کند، مانند نمایش‌ها، اثبات‌های مفهومی، و پاسخ‌های به سوالات فنی. این یک روش عالی برای شروع یک حساب است که تا حالا به اندازه کافی پیشرفت نکرده است و سریع‌تر سیکل فروش را تسریع کند.

یک ADS مشتری همچنین به تیم حساب شما بسیاری از راه‌های پیگیری فراهم می‌کند، مانند نمایش‌ها، اثبات‌های مفهومی، و پاسخ‌های به سوالات فنی. این یک روش عالی برای شروع یک حساب است که تا حالا به اندازه کافی پیشرفت نکرده است و سریع‌تر سیکل فروش را تسریع کند.

پیش از جلسه

در این بخش، من فرض می‌کنم که شما معمار هستید که قصد دارید جلسه طراحی معماری (ADS) را هدایت کنید. حتی اگر شما نیستید، توصیه می‌کنم که قبل از شرکت در یک ADS، این بخش را بخوانید.

آماده سازی

برای آماده‌سازی یک کارگاه آموزشی ADS، حداقل یک روز را صرف آماده‌سازی قرار دهید. شما نیاز به آماده‌سازی برای منابع کلی جلسه دارید. آیا این یک جلسه فیزیکی است یا مجازی؟ اگر فیزیکی، آیا یک اتاق بزرگ دارید؟ اگر مجازی، کدام پلتفرم را استفاده خواهید کرد؟ آیا این پلتفرم تجهیزات لازم مانند یک تخته سفید، صدا/ویدیو، و سایر تجهیزات را دارد؟ کسی که برای ناهار مسئولیت خواهد داشت؟

اگر ADS در شخص باشد و به خصوص اگر یک گروه بزرگ برای رسیدن به آن سفر کند، معمولاً یک روز کامل هفت تا هشت ساعتی را صرف می‌کنم، زیرا منابع ممکن است باعث شود که همه را دوباره در همان اتاق جمع کنید. اگر بسیاری از فناوری برای مشتری جدید است، معمولاً شش یا هفت ساعت کافی است. اگر متوجه شوید که شرکت‌کنندگان احساس مغز آسیب دیده‌اند، از ابتدای ADS خارج شوید، زیرا در آن نقطه هر چیز دیگری که شما پوشش می‌دهید فراموش خواهد شد.

برای ADSهای دور از راه، دو روز چهار ساعته معمولاً بهتر از یک روز کامل است. من سعی می‌کنم دو جلسه چهار ساعته را در روزهای متوالی یا حداقل در یک هفته از هم برگزار کنم.

مطمئن شوید که بودجه و زمان پروژه را بدانید و تصمیم‌گیرنده را شناسایی کنید. این مهم است چه جلسه داخلی باشد چه خارجی، اما اگر لیست طولانی از مشتریانی دارید که می‌خواهند یک ADS برگزار کنند، این اطلاعات می‌تواند به شما کمک کند تا بر بهترین امکانات خود تمرکز کنید. اگر ADS برای یک مشتری است، گزارش سازمانی آخرین آن‌ها را برای دید بیشتر در مورد علاقه‌مندی‌ها و نگرانی‌های آن‌ها بخوانید.

من همچنین توصیه می‌کنم که یک تماس قبل از اتصال برگزار کنید تا با تیم حساب شرکت خود که مشتری را پشتیبانی می‌کند صحبت کنید. شما احتمالاً می‌خواهید در مورد موارد زیر صحبت کنید:

- جزئیات در مورد مشتری و پس‌زمینه و زمینه پروژه

- مشکلات مشتری با معماری فعلی خود

- چگونه می‌توانید به مشتری کمک کنید

- اهداف تیم حساب و نتیجه‌ای که از جلسه می‌خواهند (به عنوان مثال، آیا می‌خواهند مشتری یک اثبات مفهومی درخواست کند؟)

- چقدر مشتری با محصولات شما آشنا است

- چگونه شما این ADS را اجرا خواهید کرد و چه انتظاراتی از تیم حساب دارید

محصولات این جلسه یک ایمیل به تیم حساب که ملاقات را خلاصه می‌کند، یک پیش‌نویس برنامه که آن‌ها می‌توانند به مشتری بفرستند، یک نقشه نشستن (اگر ADS در شخص باشد)، و یک یادآوری برای تیم حساب برای تنظیم یک تماس قبل از اتصال با مشتری است.

و اگر با یک همکار کار می‌کنید، مانند یک شرکت مشاوره، در نظر بگیرید که آن‌ها را در همراه مشتری در ADS شامل کنید تا هر دو را آموزش دهید که چگونه از فناوری استفاده کنند. مطمئن شوید که این را ابتدا با مشتری مشورت کنید، به خصوص اگر قصد دارید اطلاعات حساسی را بحث کنید.

مطمئن شوید که به مشتری بگویید اصلی‌ترین دلیل برای تماس قبل از اتصال با آن‌ها این است که به شما کمک کند تا به مقدار زیادی برای ADS آماده شوید. اطلاعاتی که به زودی به دست می‌آید، به شما اجازه می‌دهد تا بر اساس

نیاز بر روی موضوعات مختلف تمرین کنید یا متخصصان موضوع (SMEs) را برای کمک به زمینه‌هایی که با آن‌ها آشنا نیستید دعوت کنید. شما همچنین برنامه را به اشتراک خواهید گذاشت و با هم می‌توانید مطمئن شوید که آن شامل هیچ مورد غیرمرتبطی نیست. به عنوان مثال، اگر تمرکز ADS بر روی معماری‌های داده است، بحث‌های امنیتی یا DevOps بهتر است که خارج از ADS مدیریت شوند.

تماس قبل از اتصال با مشتری باید شامل چند چیز باشد:

این چیزی که تا حالا فهمیدم...

ابتدا، خلاصه‌ای از درک شما از مشکل یا فرصت بگذارید تا مشتری فرصت داشته باشد تا هرگونه اشتباه درک را اصلاح کند.

درباره شرکت‌کنندگان...

بگویید که کی شرکت خواهند کرد و نقش‌هایی که در سازمان بازی می‌کنند. این به شما کمک می‌کند تا تصمیم بگیرید که چقدر عمیق یا سطح بالا باید بحث فنی را برگزار کنید.

این چیزی که اتفاق خواهد افتاد...

یک برنامه را ارائه دهید و با مشتری روی آن حرکت کنید، اما اطمینان حاصل کنید که بحث می‌تواند در صورت نیاز تغییر کند. رویکردی که قصد دارید در اجرای ADS اتخاذ کنید را توضیح دهید و انتظارات آن‌ها را بر اساس آن تنظیم کنید. به عنوان مثال، خوب است به آن‌ها بگویید که باید ۱۰۰٪ متمرکز باشند - هیچ تلفن همراه یا ایمیلی برای هر کسی که در آن شرکت می‌کند، از شما نیز شامل.

چه چیزی برای ما برای نگاه کردن دارید؟

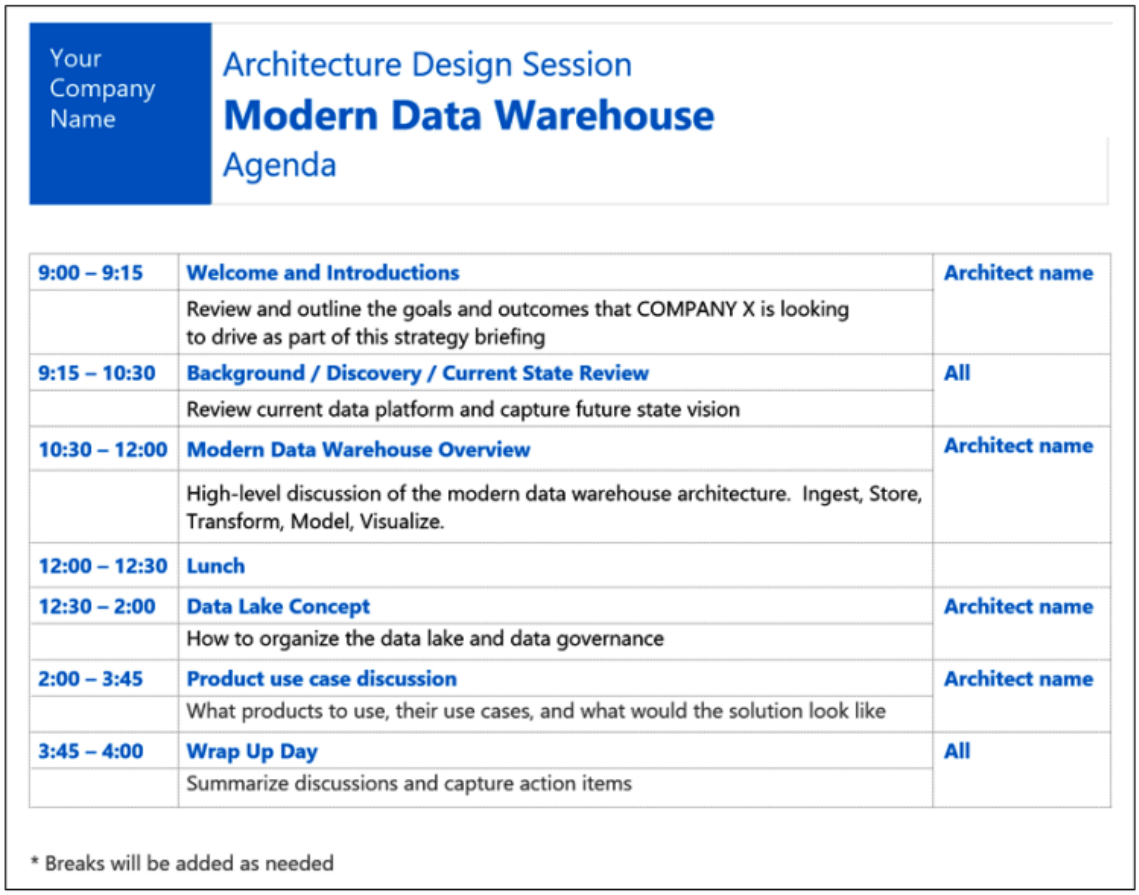
این مرحله کشف است. تشویق به آن‌ها که هر گونه نقشه معماری برای راه حل فعلی خود، هر نقشه معماری‌ای که ممکن است برای راه حل آینده آن‌ها طراحی کرده باشند، و هر سندی که راه حل فعلی و برنامه‌های آینده آن‌ها را توصیف می‌کند را به شما بفرستند.

چه چیزی برای شما یک ADS موفق به نظر می‌رسد؟

"آیا چیزی دیگری که باید آگاه باشم؟" این سوال را بپرسید. این سوال می‌تواند سوال‌ها یا اهدافی را که در غیر این صورت ممکن است گم شوند را به سطح بیاورد.

شکل ۳-۱ یک نمونه برنامه ADS را نشان می‌دهد.

مطمئن شوید که مکالمات برنامه‌ریزی شده را در برنامه شامل کنید - و آن‌ها را نه انجام دهید. ممکن است تمایل داشته باشید که یک مکالمه را در صورتی که در حال حرکت با بحث هستید از بین ببرید، اما این معمولاً یک اشتباه است. از کسی- در تیم حساب خود بخواهید که به شما یادآوری کند تا یک مکالمه بگیرید در صورتی که فراموش کنید.



Your Company Name

Architecture Design Session

Modern Data Warehouse

Agenda

9:00 – 9:15	**Welcome and Introductions**	**Architect name**
	Review and outline the goals and outcomes that COMPANY X is looking to drive as part of this strategy briefing	
9:15 – 10:30	**Background / Discovery / Current State Review**	**All**
	Review current data platform and capture future state vision	
10:30 – 12:00	**Modern Data Warehouse Overview**	**Architect name**
	High-level discussion of the modern data warehouse architecture. Ingest, Store, Transform, Model, Visualize.	
12:00 – 12:30	**Lunch**	
12:30 – 2:00	**Data Lake Concept**	**Architect name**
	How to organize the data lake and data governance	
2:00 – 3:45	**Product use case discussion**	**Architect name**
	What products to use, their use cases, and what would the solution look like	
3:45 – 4:00	**Wrap Up Day**	**All**
	Summarize discussions and capture action items	

* Breaks will be added as needed

Figure 3-1. Sample agenda for an ADS about a modern data warehouse project

برای آماده‌سازی برای استفاده از هر ابزاری که برای تخته سفید (مانند Surface Hub) استفاده خواهید کرد، بهتر است زمانی قبل از اتفاق اختصاص دهید تا با جزئیات آن آشنا شوید، به طوری که در طول ADS چیزی را برای یادگیری بیابید. تصور کنید که در یک ADS هستید و از تخته سفید برای نوشتن اهداف، نقاط درد، و پیگیری‌ها و برای رسم معماری بر اساس تخمینات شما در مورد چگونگی پاسخ مشتری به سوالات شما استفاده می‌کنید. حتی می‌توانید از یک همکار خود بخواهید که به عنوان مشتری نقش‌بازی کند تا به شما کمک کند.

دعوت حاضرین

شرکت‌کنندگانی که باید در ADS شرکت کنند بسته به اینکه مشتری داخلی (یک گروه درون شرکت شما) یا خارجی (یک شرکت خارجی که با آن تجارت می‌کنید) است، کمی متفاوت خواهد بود.

از سوی مشتری (یا اگر ADS داخلی است، از گروهی که برای آن این ADS انجام می‌دهید)، شرکت‌کنندگان باید شامل:

- حامیان (حداقل یکی از بخش کسب و کار و یکی از بخش IT)

- نمایندگان کسب و کار

- نمایندگان فنی

- مدیر پروژه

- مشاوران، معماران، توسعه‌دهندگان، و افراد مربوط به زیرساخت یا عملیات، در صورت لزوم

از تیم شما، شامل:

- یک معمار برای هماهنگی جلسه و اطمینان از اینکه ADS هدف‌های خود را برآورده می‌کند

- از تیم حساب، یک اجرایی حساب، یک متخصص حساب، و/یا یک معمار راه حل ابری

- متخصصان موضوع (SMEs) برای ارائه دانش عمیق در موضوعات خاص

- حداقل یک نفر برای گرفتن یادداشت (احتمالاً یک عضو از تیم حساب)

اگر مشتری می‌خواهد در مورد یک زمینه موضوعی صحبت کند که شما با آن آشنا نیستید، در بیشتر موارد بهتر است یک SME را به جلسه ببخشید تا خودتان موضوع را یاد بگیرید، به خصوص اگر زمان شما محدود است. مطمئن شوید که به SME بزنید و بعد از جلسه به مدیر آن یک ایمیل خوب بفرستید. وقتی یک SME در ADS شرکت می‌کند، همه می‌گیرند: مشتری خوشحال است که مشاوره اخلاق دریافت کند، تیم حساب خوشحال است زیرا مشتری راضی است، و شما می‌توانید از SME یاد بگیرید.

اگر شما جدید در هماهنگی هستید، ممکن است همچنین می‌خواهید از یک مربی بخواهید که شرکت کند، تا شما را در طول ADS پشتیبانی کند (با پاسخگویی به سوالاتی که نمی‌توانید پاسخ دهید)، و بعد از آن به شما بازخورد دهد در مورد اینکه چه چیزی خوب کردید و چه چیزی می‌توانستید بهتر انجام دهید.

گاهی اوقات من برنامه را به گونه‌ای ترتیب می‌دهم که افراد فقط بخشی- از روز را شرکت کنند. به عنوان مثال، ممکن است برنامه صبح را بیشتر به موضوعات کسب و کار و کشف بگذارم، و بخش فنی را برای عصر- بعد نگه دارم که به معماری پیشنهادی و فناوری‌های مورد نظر نزدیک خواهیم شد. در این مثال، اکثر اجرایان سطح بالا احتمالاً در جلسه صبح حضور خواهند داشت، و افراد فنی در عصر- بعد حضور خواهند داشت. با این حال، هر کسی- می‌توانست برای کل روز حضور داشته باشد - ممکن است برخی از اجرایان سطح بالا به مکالمه فنی علاقه‌مند باشند، در حالی که برخی از افراد فنی به کشف علاقه‌مند هستند.

کنداکت جلسات

به یاد داشته باشید که شما در حال رهبری هستید، بنابراین به شما بستگی دارد تا نگرش را تنظیم کنید و جلسه را در مسیر مناسب نگه دارید. اگر کسی- نقطه‌ای غیرمرتبط را برای بحث بیان کند، بگویید که می‌توانید آن را در خارج از جلسه بحث کنید؛ سپس آن را بر روی تخته سفید برای پیگیری بنویسید. به طور مشابه، اگر شما یا کسی- دیگر توافق می‌کنید که بر روی یک مورد خاص پیگیری کنید، آن را بر روی تخته سفید اضافه کنید. به تیم حساب اجازه دهید تا یادداشت بگیرند - شما باید تمرکز خود را بر روی صحبت با مشتری و تخته سفید قرار دهید.

در نقطه نیمه جلسه، برنامه و اهداف را بررسی کنید. اگر شما در بالا هستید، به مشتری بگویید که جلسه باید برای رسیدن به تمام اهداف در مسیر مناسب باقی بماند. اگر به نظر می‌رسد که شما زمان کافی برای پوشش تمام اهداف ندارید، از مشتری بخواهید که اهداف را باز مرتب کند تا هر اهدافی که بحث نشده است، اولویت پایین‌تری داشته باشد.

تعریف کلی

در ابتدای ADS، بخواهید همه فردان خود را معرفی کنند، بیان کنند که چه نامی دارند، نقش آن‌ها چیست و آیا چیزی در مورد فناوری که قرار است در مورد آن بحث کنید می‌دانند یا خیر. سپس، احتمالاً مهم‌ترین قسمت، بپرسید که آن‌ها از ADS چه چیزی می‌خواهند ببینند (اهداف یادگیری آن‌ها). اهداف را بر روی تخته سفید بنویسید و آن‌ها را به عنوان یک گروه اولویت‌بندی کنید. اگر برخی از حاضرین در تماس قبل از جلسه نبوده‌اند، مشخص کنید که ADS چه چیزی را پوشش می‌دهد.

از این زمان برای توضیح چگونگی و دلیل آرانی ADS استفاده کنید و قوانین پایه برای بقیه روز را تنظیم کنید. به عنوان مثال، ممکن است بگویید که به آن‌ها یک کپی از تخته سفید نهایی را ارسال خواهید کرد تا نیازی به گرفتن عکس از آن نباشد، که این یک جلسه تعاملی خواهد بود و شما تشویق به سوالات زیاد می‌کنید، برنامه فقط یک پیشنهاد است و گروه می‌تواند موضوعات دیگری را که ارزشمند می‌داند پوشش دهد، و شما می‌توانید جلسات پیگیری برای پوشش موضوعات اضافی داشته باشید اگر زمان شما به اتمام برسد (نیازی به استراحت از مکالمات مهم نیست).

دستاوردها

کشف موقعیتی است که در آن شما در ابتدای ADS یک ساعت یا دو ساعت صرف می‌کنید و سوالاتی در مورد چیزهایی مانند:

- نقاط درد فعلی مشتری

- فناوری و معماری فعلی آن‌ها

- معماری آینده آن‌ها (اگر هرگونه در مورد آن بحث شده باشد)

- هرگونه تصمیماتی که در مورد فناوری‌ها، محصولات یا ابزارهایی که استفاده می‌کنند یا قصد استفاده از آن‌ها را دارند قبلاً گرفته‌اند

- موارد استفاده فعلی و آینده

- جزئیات در مورد کسب و کار آن‌ها

مشتری باید بیشترین بخش از صحبت کنند، به خصوص در ابتدای ADS. شما چیزی در مورد آن‌ها یاد نمی‌گیرید اگر همه صحبت کنید. من همیشه با پرسیدن سوالات زیاد و دریافت پاسخ‌ها شروع می‌کنم قبل از ورود به حالت "تحصیل"، جایی که من برای مدت طولانی صحبت می‌کنم. حتی در این حالت همیشه مطمئن می‌شوم که برای سوالات مکث می‌کنم. شما ممکن است در ابتدا به آن‌ها بگویید که شما در طول جلسه به طور منظم برای سوالات مکث خواهید می‌کنید.

یک معمار خوب سوالات زیادی می‌پرسد. همانطور که می‌گویند، "شما نمی‌دانید که چه چیزی نمی‌دانید." معماران تجربی آگاه از تمام معماری‌ها، تکنیک‌ها و ابزارهای موجود هستند و با منظره متغیر فناوری‌ها و محصولات با آن بروز می‌مانند. آن‌ها توانایی دارند سوالاتی را مطرح کنند که به آن‌ها کمک می‌کند تا فناوری‌ها و محصولات مناسب را انتخاب کنند. مرحله کشف بهترین راه برای کاهش گزینه‌های محصول به تعداد کاربردی که می‌توانید در نظر بگیرید است. سپس می‌توانید بهترین معماری و محصولات را برای مورد استفاده خاص بیابید.

به عنوان مثال، فرض کنید که شما در یک وان با هزار ابزار پزشکی فروش می‌دهید. (این معمولاً این طریقه نیست، اما با من همراه باشید.) شما نمی‌توانستید فقط به یک پزشک نزدیک شوید، به وان اشاره کنید و از آن بخواهید که تمام ابزارها را ببیند تا آن‌هایی را که دوست دارد پیدا کند. به جای آن، شما باید به پزشک سوالات زیادی بپرسید - در مورد منطقه کار او، ابزارها و بودجه - و بر اساس پاسخ‌های او، فقط تعداد کمی از ابزارها را برای نگاه به پزشک بیرون کشید. معماران همان کار را انجام می‌دهند، و مرحله کشف از ADS فرصت عالی است تا در هنگامی که افراد مناسب در اتاق هستند، سوالات خود را بپرسید.

یک توصیه خوب دیگری که در ابتدای منابع من دریافت کردم: هنگامی که شروع به پرسیدن سوالات می‌کنید، مطمئن شوید که از راه حل دور می‌مانید. به عبارت دیگر، تا زمانی که همه سوالات کشف را پرسیده‌اید، در مورد معماری‌ها یا محصولات صحبت نکنید. وقتی من برای اولین بار ADS ها را اجرا می‌کردم، هرگاه مشتری اشاره کرد که می‌خواهد یک انبار داده ساخت، من فوراً به نمایش یک محصول Microsoft بپردازم. باید منتظر بودم و همه سوالات کشف را پرسیده باشم تا مطمئن شوم که مشتری در حال تلاش برای دستیابی به چه چیزی است. در عوض، گاهی اوقات کشف می‌کردم که واقعاً مشتری نیازی به یک انبار داده نداشت، که به این معنی است که تلاش من برای آموزش آن‌ها در مورد محصول انبار داده به معنای هیچ چیز نیست.

وایت برد

از یک تخته سفید استفاده کنید، نه اسلایدهای ارائه. یک ADS در مورد کشف است؛ تعداد زیادی صحبت و سپس شما به تخته سفید می‌روید. اگر تعداد زیادی اسلاید وجود داشته باشد، ADS فقط یک ارائه دیگر می‌شود. تخته سفید شما باید شامل یک نقشه ساده از معماری باشد، همچنین مکان‌هایی برای اهداف، نقاط درد و مواردی برای پیگیری.

شکل ۳-۲ نشان می‌دهد که چه چیزی ممکن است یک تخته سفید نهایی برای یک ADS متمرکز بر یک انبار داده مدرن باشد.

[متاسفانه، من نمی‌توانم تصاویر یا نمودارهای خاصی را نمایش دهم. اما شما می‌توانید با استفاده از تخته سفید خود، یک نقشه ساده از معماری مورد بحث را بکشید و مکان‌هایی برای اهداف، نقاط درد و مواردی برای پیگیری اضافه کنید. این روش به شما کمک می‌کند تا به طور موثر و تعاملی با مشتری و تیم خود برخورد کنید و از طریق کشف و بحث، به جای ارائه‌های ساختگی، به یادگیری و تصمیم‌گیری موثرتری برسید.]

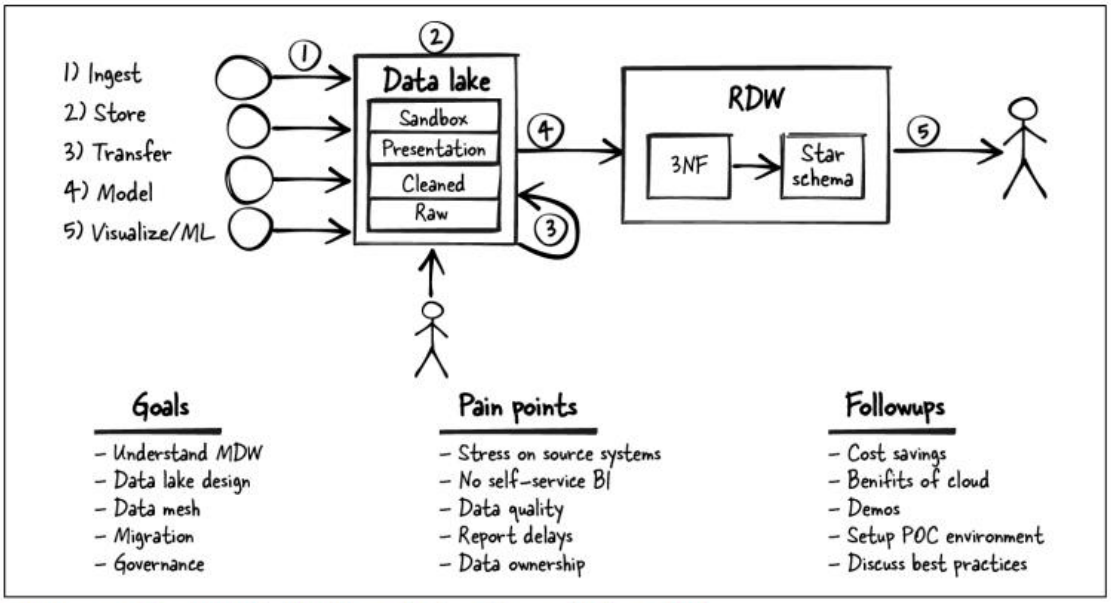


Figure 3-2. A typical whiteboard after the end of an ADS

تخت نقاشی نه تنها معماری را شامل می‌شود بلکه اهداف اولویت‌بندی شده، نقاط درد، موارد "پارکینگ لات" و سایر موارد برای پیگیری را نیز دارد.

لیست کردن تمام نقاط درد روی تخت نقاشی و اطمینان از بررسی و حل آن‌ها به ترتیب است. در پایان استادگی معماری دیجیتال (ADS)، تمام نقاط درد باید در معماری ایجاد شده شما مورد بررسی قرار گیرند. اگر این موارد مورد بررسی نشده‌اند، یک جلسه پیگیری با مشتری برنامه‌ریزی کنید تا نقاط درد نادیده گرفته شده را مورد بررسی قرار دهید. همین مورد برای اهداف نیز صادق است: اگر نتوانستید تمام آن‌ها را مورد بررسی قرار دهید، یک جلسه پیگیری برنامه‌ریزی کنید.

۳۰ دقیقه آخر استادگی معماری دیجیتال را برای بحث در مورد موارد پیگیری اختصاص دهید.

پس از برگزاری جلسات

بعد از استادگی معماری دیجیتال (ADS)، با تیم حساب مشتری مشورت کنید و از آن‌ها بپرسید که چه چیزی خوب انجام شده و چه چیزی می‌توانید بهتر انجام دهید. این بخشی از اعمال نگرش رشد به عمل می‌کند و یک روش عالی برای دریافت بازخورد و بهبود جلسات ADS شما است. شما باید نتایج استادگی معماری دیجیتال را به تمام ذینفعان ارتباط دهید. این ممکن است شامل خلاصه‌ای برای مدیریت بالا و گزارشی دقیق‌تر برای کسانی که مستقیماً در پروژه دخیل هستند باشد. در آخر، تمام موادی که برای ADS استفاده کرده‌اید را در یک پوشه دیجیتال جمع‌آوری کنید که می‌توانید آن را به عنوان مرجع برای استادگی‌های آتی استفاده کنید. به عنوان مثال، من تمام تخته‌های نقاشی قبلی خود را نگهداری می‌کنم؛ متوجه شدم که زمانی که قصد دارید یک استادگی معماری دیجیتال مشابه یکی که قبلاً انجام داده‌اید را برنامه‌ریزی می‌کنید، برای بازبینی بسیار مفید هستند.

کوتاه پس از ADS، به مشتری ایمیل بزنید با اطلاعات زیر:

- سند خلاصه

- نقاط اصلی بحث شده در طول ADS را به طور مختصر خلاصه کنید.

معماری فیزیکی

اگر از یک تخت نقاشی دیجیتال استفاده کرده‌اید، نتیجه نهایی را به یک فایل اکسپورت کنید که می‌توانید آن را به ذینفعان، از جمله مشتری، ارسال کنید.

موارد اقدام

شامل هر گام بعدی که شما موافقت کرده‌اید، مانند "در سه‌شنبه بعدی برای بحث در مورد یک اثبات مفهومی ملاقات کنید" یا "مشتری باید یک نمودار از معماری فعلی خود ایمیل کند" باشید.

موارد پارکینگ لات و پیگیری‌ها

این موارد را روی تخت نقاشی ردیابی کرده‌اید. حالا در ایمیل می‌توانید به جزئیات بیشتری بپردازید و فردی که مسئول پیگیری هر مورد است را لیست کنید. این فرصتی به مشتری می‌دهد تا هر چیزی که به درستی فهمیده نشده است را توضیح دهد.

نظرسنجی

در یک استادگی معماری دیجیتال حضوری، بهتر است در پایان به هر شرکت‌کننده یک نظرسنجی داده شود. به این ترتیب، بیشتر یا تمام حاضران آن را پر خواهند کرد—به خصوص اگر با آن‌ها جوک کنید که نمی‌توانند تا زمانی که آن را پر کنند خارج شوند! در نظر داشته باشید که این درخواست را در آخرین مکرر استادگی روز به جای در پایان استادگی انجام دهید. اگر استادگی دیجیتال است، می‌توانید لینک نظرسنجی را در چت ملاقات کوتاه قبل از پایان جلسه ارسال کنید و به آن‌ها تشویق کنید که "لینک را قبل از خروج از ملاقات کپی کنند تا بتوانند بازخورد خود را ارائه دهند." بگویید که فقط دو دقیقه زمان می‌برد تکمیل آن (و مطمئن شوید که این حقیقت است). شما همچنین می‌توانید لینک نظرسنجی را در ایمیل پیگیری خود اضافه کنید.

نکات مدیریت جلسات

در حالی که نادر است، من تعدادی از افراد دشوار را در طول یک استادگی معماری دیجیتال (ADS) با چالش کردن مواجه شده‌ام. گاهی اوقات یک فرد در اتاق از تغییر می‌ترسد، می‌ترسد که یک معماری جدید آن‌ها را از شغل خود منع می‌کند (به خصوص زمانی که ADS درباره مهاجرت به ابر است)، یا به سادگی فکر می‌کند که پاسخ‌های خود را دارد. اگر با کسی- مشابه این مواجه می‌شوید، نباید در یک جنگ وارد شوید - حتی اگر شما درست هستید. به جای آن، با چیزی پاسخ دهید که جنگ را متوقف کند و ADS را دوباره به مسیر خود برگرداند، به عنوان مثال، "شما نقطه عالی دارید و می‌خواهم بیشتر در مورد آن بحث کنم، اما باید آن را در زمان دیگری آفلاین انجام دهیم تا بتوانیم بر موضوع مانده بمانیم."

همچنین از همراهی استفاده کنید. شما نیازی به گوش دادن به جوک‌ها ندارید، اما اگر راحت هستید، سعی کنید در طول روز نظرات خنده‌دار بگویید تا حال را روشن کنید و توجه همه را در جلسه حفظ کنید. یک استادگی می‌تواند برای یک روز طولانی باشد، و خنده بهترین راه برای افزایش سطح انرژی افراد است.

با احترام باشید - نباشید کسی- که همه چیز را می‌داند. افراد از این نوع نمی‌پسندند. آن‌ها بسیار ترجیح می‌دهند که افرادی را ببینند که اعتراف می‌کنند که همه چیز را نمی‌دانند. اگر به سوالی پاسخ نمی‌دهید، فقط بگویید که نمی‌دانید اما پیدا خواهم شد و به شما سریعاً بازگشت خواهم داد. شما می‌توانید در طول استراحت ناهار پاسخ را جستجو کنید و در عصر- بعدی پاسخ دهید. یا یک همکار را پیدا کنید که پاسخ را می‌داند و به او دعوت کنید تا به طور مستقیم به

سوال پاسخ دهد و سوالات پیگیری را بپذیرد. هرگز سعی نکنید پاسخی را ترکیب یا تغییر دهید. شما ممکن است گیر شوید، و حتی اگر نگیرید، این هم روش درستی برای رفتار با مشتری یا همکار نیست.

البته، مشتریان تنها افرادی نیستند که می‌توانند چیزی را به هم بیفزایند. نادر است که یک عضو از تیم حساب به طول زمان در امید به نمایش خود در مقابل مشتری با ارائه هیچ ارزشی به ADS اضافه کند. اگر این اتفاق بیفتد، در طول یک مکرر برای یک کلمه دوستانه با آن‌ها صحبت کنید و به آن‌ها با احترام می‌خواهید که از صحبت زیاد خودداری کنند. به آن‌ها بگویید که مطمئن می‌شوید که به آن‌ها سوالات مرتبطی را مطرح می‌کنید که می‌توانند در طول ADS پاسخ دهند تا بتوانند تخصص خود را با مشتری برقرار کنند.

همچنین اتاق را بخوانید. شما می‌دانید که کی تصمیم‌گیران هستند، پس آن‌ها را متمرکز و مشارکت‌کننده نگه دارید. توجه داشته باشید به زبان بدنی آن‌ها: اگر از شما نگاه کردن به شما به خواندن ایمیل‌های خود تغییر کند، آن‌ها خسته هستند. اگر این اتفاق بیفتد، به موضوع دیگری تغییر دهید یا یک مکرر بگیرید.

برای ورود به تماس با استفاده از دو دستگاه مختلف: یکی برای اشتراک‌گذاری صفحه نمایش و تخت نقاشی، و دیگری برای ارتباط (بیشتر نرم‌افزارهای ارتباطی این قابلیت را پشتیبانی می‌کنند). به طور ایده‌آل، دستگاه ارتباطی شما یک کامپیوتر با دو یا سه مانیتور بزرگ خواهد بود. به این ترتیب، نیازی به کاهش مکانیزم‌های پنجره یا جابجایی آن‌ها نیست، و شما همیشه می‌توانید مشتری را هنگامی که در حال تخت نقاشی و صحبت کردن هستید ببینید.

من توصیه می‌کنم که یک کانال پشتیبان با تیم حساب (و فقط تیم حساب) در ابزار چت نرم‌افزار ارتباطی که استفاده می‌کنید داشته باشید. این روش عالی برای ارتباط با تیم حساب شما بدون اینکه مشتری بشنود است - به عنوان مثال، برای درخواست تحقیق یک سوال که شما نمی‌توانید پاسخ دهید، برای ورود و صحبت کردن در زمان نیاز، یا برای ارائه جزئیات در مورد شخصی که در حال صحبت کردن است.

• من همچنین این ابزارها را توصیه می‌کنم:

— اگر از Teams Microsoft استفاده می‌کنید، ویژگی Coach Speaker را فعال کنید، که به صورت واقعی و در زمان حاضر به صدای شما در حین ارائه گوش می‌دهد و بازخورد و پیشنهادات برای بهبود را ارائه می‌دهد، و همچنین یک خلاصه پس از آن. فقط شما می‌توانید به دید آن دیدگاه‌های زنده آن بپردازید، و آن‌ها در خلاصه‌های ملاقات‌های ضبط شده ذخیره نمی‌شوند.

— من همچنین ZoomIt را دوست دارم، یک ابزار بزرگنمایی صفحه، انوتاسیون، و ضبط برای ارائه‌های فنی و نمایش برنامه. به طور نامتوجه در صف کار اجرا می‌شود و با کلیدهای سفارشی قابل فعال سازی برای اجازه دادن به شما برای بزرگنمایی یک ناحیه از صفحه نمایش، حرکت در حال بزرگنمایی، و رسم روی تصویر بزرگنمایی شده. این روش عالی برای جلب توجه به بخش‌های خاصی از ارائه شما است - بسیار بهتر از استفاده از موس شما، که ممکن است سخت برای دیدن باشد.

خلاصه

یک استادگی معماری دیجیتال (ADS) بخش حیاتی در ساخت یک معماری داده است. این کمک می‌کند تا تصمیمات طراحی را با اهداف کسب و کار هماهنگ کنید، به ریسک‌ها و چالش‌های بالقوه پاسخ دهید، هزینه‌ها و منابع را بهینه‌سازی کنید و بین ذینفعان همکاری را ترویج کنید.

نتیجه نهایی از یک ADS ساختن یک معماری داده خوب یا حتی بزرگ است. یک معماری داده خوب قوی و قابل مقیاس است و می‌تواند به طور موثر به پروژه‌های مبتنی بر داده پشتیبانی کند. انتقال یک معماری داده از خوب به بزرگ به معنای برآورده کردن این نیازها از ابتدا تا انتها، با بازخورد از همه کاربران در زنجیره ارزش داده، به طوری که

استراتژی داده کلی می‌تواند با اهداف سازمان هماهنگ شود. ساخت یک راه حل داده یک سفر متمرکز بر کاربر در طراحی و بازخورد است و نیاز به استراتژی و برنامه‌ریزی دارد که فقط یک ADS می‌تواند ارائه دهد.

بخش دوم: مفاهیم معمول در معماری داده

قبل از پرش در معماری‌های داده، مهم است که اطمینان حاصل کنید تمام مفاهیم معماری داده که ممکن است در یک معماری استفاده شود، را درک کنید. من معمولاً توجه زیادی به اینکه چندین مفهوم از این مفاهیم با ابهام مواجه هستند، که امیدوارم آن‌ها را روشن کنم. بنابراین، شما در فصل‌های آتی بخش بیش از ۲۰ مفهوم مشابه پیدا خواهید کرد. حداقل، این فصل‌ها یک تازه‌سازی برای کسانی خواهند بود که ممکن است چند سال این مفاهیم را استفاده نکرده باشند. من ادعا نمی‌کنم که تعریف‌های من از این مفاهیم توسط همه مورد توافق قرار گرفته باشد، اما حداقل این فصل‌ها به همه کمک می‌کنند تا به یک صفحه برسند تا بتوانند راحت‌تر در مورد معماری‌ها صحبت کنند.

من معماری انبار داده رابطه‌ای و دیتا لیک را در مفاهیم به جای معماری‌ها قرار دادم. یک زمان در گذشته، وقتی آن‌ها تنها محصولاتی بودند که در یک راه حل استفاده می‌شدند، می‌توانستند به عنوان معماری‌های داده در نظر گرفته شوند. اما حالا، آن‌ها تقریباً همیشه با محصولات دیگر ترکیب می‌شوند تا راه حل را تشکیل دهند. به عنوان مثال، چند سال پیش محصولات معماری انبار داده رابطه‌ای وجود داشت که شامل ذخیره‌سازی رابطه‌ای، محاسبات، نرم‌افزار ETL (استخراج، تبدیل، و بارگذاری)، و نرم‌افزار گزارش‌گیری بودند - در واقع همه چیزی که شما نیاز داشتید در یک بسته از یک فروشنده بسته بندی شده بود.

امروزه، شما محصولات متعددی را از احتمالاً فروشندگان متعدد، با هر محصول یک تمرکز خاص (مثلاً ETL)، برای تکمیل معماری داده خود به هم می‌پیوندید.

فصل چهارم: انبار داده رابطه ای

در میانه دهه ۲۰۰۰، من سال‌ها با پایگاه‌های داده رابطه‌ای کار کرده بودم، اما هرگز با انبارهای داده رابطه‌ای مواجه نشده بودم. من به عنوان یک مدیر پایگاه داده (DBA) برای یک شرکت کار می‌کردم که از یک بسته نرم‌افزار حسابداری برای مدیریت معاملات مالی خود استفاده می‌کرد. گزارشات از بسته محدود و کند بودند، بنابراین شرکت در حال بهبود عملکرد بود و در نظر گرفتن ایجاد داشبوردها برای تقسیم و تحلیل داده‌ها و ترکیب داده‌های مالی خود با داده‌های برنامه‌ای خانگی برای درک بهتر از کسب و کار بود.

کارفرمای من یک شرکت مشاوره‌ای را برای ساخت چیزی که به آن "انبار داده رابطه‌ای" گفته می‌شد، استخدام کرد و در یک تصمیمی که مسیر کاری من را تغییر داد، من را دعوت کرد تا به آن کمک کنم. ما داشبوردهایی را ایجاد کردیم که به کاربران نهایی زمان زیادی را صرفه‌جویی کردند و بازخوردهای تجاری را که قبلاً نداشته بودند، اضافه کردند. وقتی که تحسین آن‌ها را دیدم، متوجه شدم که عشق جدیدی پیدا کرده‌ام. من تغییر دادم تا کاری را که بر روی انبارداری داده تمرکز داشته باشم و هرگز به عقب نگاه نکنم.

تعریف اولیه

یک انبار داده رابطه‌ای (RDW) مکانی است که شما داده‌های ساختاری بزرگ را از منابع داده متعدد کپی می‌کنید و آن‌ها را برای گزارشات تحلیل تاریخی و روند ذخیره می‌کنید تا شرکت شما تصمیمات تجاری بهتری بگیرد. این به دلیل استفاده از مدل رابطه‌ای نامیده می‌شود، که یک رویکرد گسترده برای نمایش و سازماندهی داده برای پایگاه‌های داده است. در مدل رابطه‌ای، داده‌ها در جداول (که همچنین به عنوان روابط شناخته می‌شوند، بنابراین نام) سازماندهی می‌شوند. این جداول شامل سطرها و ستون‌ها هستند، که هر سطر یک موجودیت (مانند مشتری یا محصول) را نمایش می‌دهد و هر ستون یک ویژگی از آن موجودیت را نمایش می‌دهد (مانند نام، قیمت یا تعداد). این به دلیل جمع‌آوری، ذخیره‌سازی، و مدیریت حجم بزرگی از داده‌های ساختاری از منابع مختلف، مانند پایگاه‌های داده معاملاتی، سیستم‌های برنامه، و خوراک‌های داده خارجی، به عنوان یک انبار داده نامیده می‌شود. تمام انبارهای داده بر پایه مدل رابطه‌ای نیستند. انبارهای داده غیررابطه‌ای شامل انواعی مانند ستونی، NoSQL، و انبارهای داده گرافی هستند. با این حال، انبارهای داده رابطه‌ای بسیار محبوب‌تر و گسترده‌تر پذیرفته شده‌اند، عمدتاً به دلیل اینکه پایگاه‌های داده رابطه‌ای برای دهه‌ها پارادیگم مدیریت داده اصلی بوده‌اند. مدل رابطه‌ای برای داده‌های ساختاری که در برنامه‌های تجاری رایج است، بسیار مناسب است. همچنین به دلیل استفاده گسترده از SQL، که برای سال‌ها زبان استاندارد برای انبارهای داده رابطه‌ای بوده است، محبوب است.

یک RDW به عنوان یک مخزن مرکزی برای بسیاری از حوزه‌های موضوعی عمل می‌کند و شامل نسخه یکپارچگی از حقیقت (SVOT) است. یک مفهوم حیاتی در انبارداری داده، SVOT به شیوه‌ای اشاره دارد که در آن یک نمای یکپارچه و یکسان از داده‌های سازمان ایجاد می‌شود. این بدین معنی است که تمام داده‌ها در انبار داده در یک فرمت ساختاری استاندارد ذخیره شده‌اند و نمایانگر یک نسخه دقیق و یکپارچه از اطلاعات هستند. این اطمینان را می‌دهد که تمام کاربران به همان اطلاعات دسترسی دارند، اختلافات یا ناهماهنگی‌ها را حذف می‌کند و سیلوهای داده را حذف می‌کند. این بهبود تصمیم‌گیری، همکاری، و کارایی در سراسر سازمان را فراهم می‌کند. همچنین خطر اشتباهات و اشتباهات که ممکن است از کار با منابع داده ناهماهنگ و متفاوت ناشی شود، را کاهش می‌دهد.

تصور کنید که شما یک انبار داده ندارید و گزارشات را مستقیماً از چندین سیستم منبع و حتی برخی از فایل‌های Excel تولید می‌کنید. اگر یک مشاهده‌کننده گزارش دقت داده‌ها را مطرح کند، چه چیزی می‌توانید به او بگویید؟ "حقیقت" می‌تواند در سیستم‌های منبع بسیاری توزیع شود که پیدا کردن منبع داده سخت است. علاوه بر این، برخی از گزارش‌ها نتایج متفاوتی برای همان داده ارائه می‌دهند - به عنوان مثال، اگر دو گزارش از منابع متعدد داده را با استفاده از منطق پیچیده بکشند و منطق به درستی به‌روز نشده یا همه چیز نشده باشد. داشتن تمام داده‌ها در یک مکان مرکزی بدین معنی است که انبار داده منبع یکپارچگی از حقیقت است؛ هر سوالی در مورد داده‌های گزارش می‌تواند توسط انبار داده پاسخ داده شود. حفظ یک SVOT برای سازمان‌هایی که قصد دارند به تمام پتانسیل داده‌های خود را بهره‌برداری کنند، ضروری است.

خلاصه - رمز گشایی از معماری داده

اگـر یـک انبـار داده (DW) توسـط کـل شرکـت اسـتفاده می‌شـود، اغلـب بـه آن انبـار داده سـازمانی (EDW) گفتـه می‌شـود. ایـن یـک نسـخه جـامع‌تر و قـوی‌تر از یـک انبـار داده اسـت کـه طـراحی شـده اسـت تـا نیازهـای کـل سـازمان را پشـتیبانی کنـد. در حـالی کـه یـک DW اسـتاندارد ممکـن اسـت نیازهـای چنـدین واحـد تجـاری را پشـتیبانی کنـد، بـا چنـدین DW در سراسر سـازمان، EDW از منـابع داده و انـواع داده گسـترده‌تری بـرای پشـتیبانی از تمـام واحـدهای تجـاری اسـتفاده می‌کنـد. EDW یک نمای یکپارچه و یکسان از تمام داده‌های سازمان را فراهم می‌کند.

شـکل ٤-١ دلیـل اصـلی بـرای داشـتن یـک انبـار داده را نشـان می‌دهـد. توضـیحات سـمت چـپ نشـان می‌دهـد کـه چگونـه سـخت اسـت یـک گـزارش را بـا اسـتفاده از داده‌هـای چنـدین برنامـه اجـرا کردنـد وقـتی شـما یـک انبـار داده ندارید. هـر بخـش یـک گـزارش را اجـرا می‌کنـد کـه داده‌هـا را از تمـام پایگاه‌هـای داده مـرتبط بـا هـر برنامـه جمـع‌آوری می‌کنـد. بـه ایـن دلیـل بـه طـور مکـرر پـرس و جوهـا اجـرا می‌شـوند کـه شـما مطمـئن هسـتید کـه بـا مشـکلات عملکـرد و داده‌هـای نادرسـت مواجـه خواهیـد شـد. یـک آشـفتگی اسـت. توضـیحات سـمت راسـت نشـان می‌دهـد کـه بـا کـپی تمـام داده‌هـای برنامـه بـه EDW، برای هر بخش بسیار آسان می‌شود یک گزارش را بدون ارائه کارایی اجرا کردن.

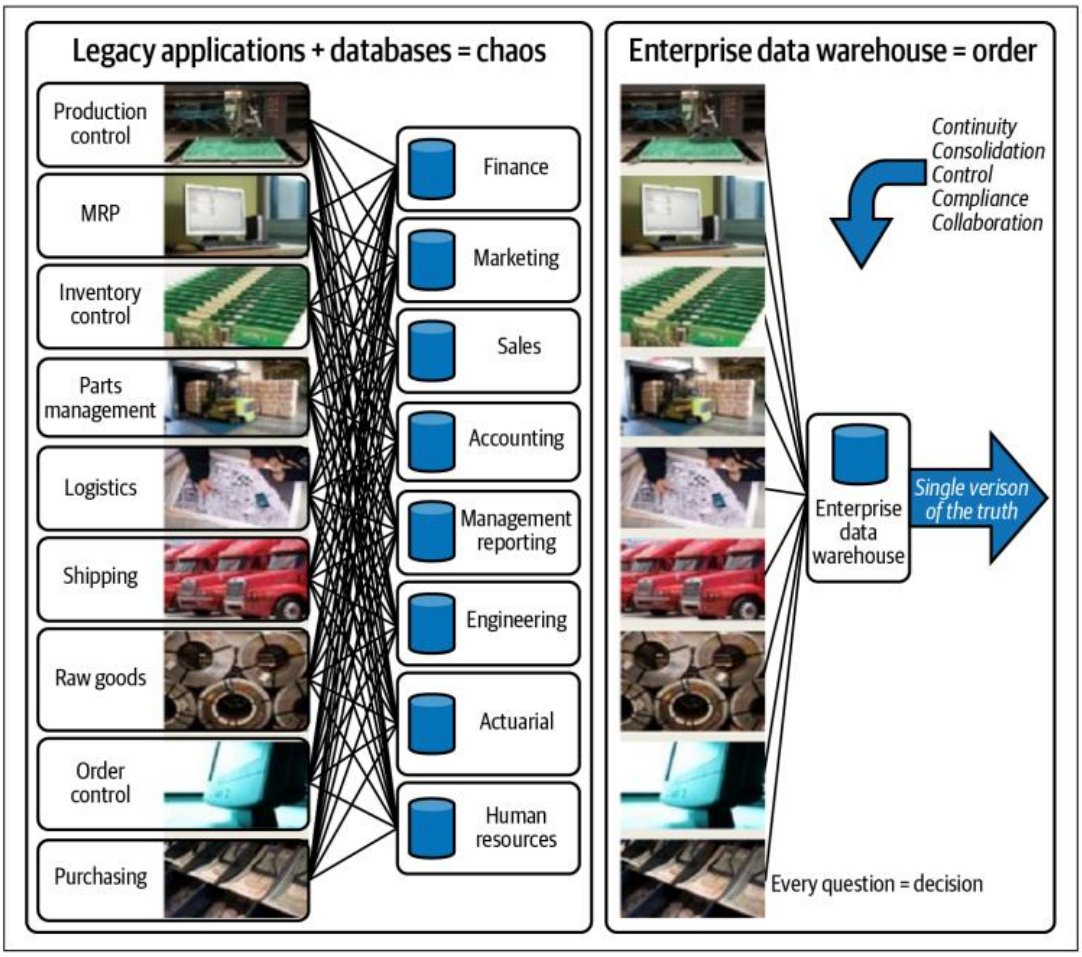


Figure 4-1. Before and after an enterprise data warehouse

بـه طـور معمـول، بـرای سـاخت یـک انبـار داده، شـما پیپلین‌هـای داده‌ای را ایجـاد می‌کنیـد کـه سـه مرحلـه را انجـام می‌دهنـد، که به آن‌ها ETL گفته می‌شود:

١. پیپلین داده‌ها را از سیستم‌های منبع، مانند پایگاه‌های داده و فایل‌های سطحی استخراج می‌کند.

۲. داده‌های استخراج شده سپس تبدیل یا تغییر می‌یابند تا نیازهای سیستم‌های هدف (در این مورد، برای یک انبار داده) را برآورده کنند. این ممکن است شامل تمیز کردن، فیلتر کردن، تجمع یا ترکیب داده‌ها از چندین منبع باشد.

۳. داده‌های تبدیل شده به انبار داده بارگذاری می‌شوند. یک DBA می‌تواند نام پایگاه داده و نام فیلدها را معنادارتر کند، که این باعث می‌شود برای کاربران نهایی ساخت گزارشات راحت‌تر و سریع‌تر شود.

انبار داده چه چیزی نیست

حالا که می‌دانید چه یک انبار داده است، بیایید هدف آن را با نگاهی به راه‌حل‌هایی که نباید به عنوان یک انبار داده در نظر گرفته شوند (اگرچه من دیده‌ام که افراد بسیاری این کار را به طور مکرر انجام می‌دهند) توضیح دهیم:

پیشوند DW

یک انبار داده فقط کپی ای از یک پایگاه داده منبع از یک سیستم عملیاتی با افزودن DW به نام فایل نیست. به عنوان مثال، فرض کنید که شما یک پایگاه داده به نام Finance را که ۵۰ جدول عملیاتی دارد کپی کنید و آن را DW_Finance بنامید، سپس از این ۵۰ جدول برای ساخت گزارشات خود استفاده کنید. این منجر به یک انبار داده طراحی شده برای داده‌های عملیاتی می‌شود، در حالی که به جای آن نیاز به یک انبار داده طراحی شده برای داده‌های تحلیلی دارید. با داده‌های تحلیلی، شما عملکرد خواندن بهتری دارید و می‌توانید مدل‌های داده را ایجاد کنید تا برای کاربران نهایی ساخت گزارشات راحت‌تر شود.

نماها با اتحاد

یک انبار داده نیست کپی چندین جدول از سیستم‌های منبع مختلف که با استفاده از دستور UNION SQL در یک نمای نمایش می‌شوند. برای مثال، اگر شما داده را از سه سیستم منبع که هر کدام مشتریان را دارند کپی کردید، شما در انبار داده سه جدول به نام CustomerSource ۱Source، ۲Customer، و ۳CustomerSource خواهید داشت. بنابراین شما باید یک نمای نمایش به نام CustomerView ایجاد کنید که یک دستور SELECT است که جدول‌های CustomerSource ۱Source، ۲Customer، و ۳CustomerSource را با هم ادغام می‌کند. شما این فرآیند را برای جداول دیگری مانند محصولات و سفارشات تکرار خواهید کرد.

به جای آن، داده‌ها از سه جدول باید به یک جدول در انبار داده کپی شوند، که نیاز به کار اضافی برای ایجاد یک مدل داده است که برای همه سه جدول مناسب است. شما احتمالاً می‌خواهید در این نقطه از مدیریت داده‌های اصلی (MDM) استفاده کنید، که در فصل ٦ توضیح داده شده است، تا تکرارها را جلوگیری کنید و دسترسی و عملکرد را بهبود بخشید.

محل دفن

یک انبار داده نیست محلی برای جمع‌آوری جداول. بسیاری از این عملیات زمانی رخ می‌دهد که یک شرکت یک انبار داده ندارد و یک کاربر نهایی می‌خواهد یک گزارش را از یک زیرمجموعه از داده‌ها از چندین سیستم منبع ایجاد کند. برای کمک به آن‌ها به سرعت، یک نفر از IT یک انبار داده بدون فکر زیاد ایجاد می‌کند، داده‌ها را از این دو سیستم منبع به انبار داده کپی می‌کند. سپس سایر کاربران نهایی مفاد اولین کاربر نهایی را می‌بینند و آن‌ها می‌خواهند داده‌های اضافی از همان سیستم‌های منبع و چندین سیستم دیگر برای ساخت گزارشات خود دریافت کنند. بنابراین دوباره این نفر از IT به سرعت داده‌های درخواستی را به انبار داده کپی می‌کند. این فرآیند به طور مکرر تکرار می‌شود تا انبار داده به یک آشفتگی از پایگاه‌های داده و جداول تبدیل شود.

بسیاری از انبارهای داده به عنوان راه‌حل‌های یکباره برای چند کاربر شروع می‌شوند، سپس به انبارهای داده کامل ولی به طراحی ضعیف برای کل شرکت تبدیل می‌شوند. راه بهتری وجود دارد.

به جای آن، وقتی درخواست اولیه کاربر نهایی وارد می‌شود، نیازهای گزارش‌گیری شرکت خود را ارزیابی کنید. بیایید ببینیم آیا این درخواست واقعاً یک یکباره است یا آن باید شروعی برای ساخت یک انبار داده سازمانی (EDW) باشد. اگر باید، این فرصتی است که به رهبران سنیور نشان دهید چرا شرکت شما نیاز به یک انبار داده دارد. اگر این مورد صادق است، مطمئن شوید که به اندازه کافی زمان اولیه نیاز دارید تا یک انبار داده طراحی کنید که می‌تواند چندین منبع داده و کاربران نهایی را پشتیبانی کند. (از "چرا از یک انبار داده رابطه‌ای استفاده کنید؟" در صفحه ٤٩ برای حمایت از مورد شما استفاده کنید.)

رویکرد بالا به پایین

در یک انبار داده‌های متناظر (RDW)، شما قبل از اینکه بتوانید از داده‌ها برای ایجاد گزارشات استفاده کنید، بسیاری از کارها را انجام می‌دهید. انجام تمام این کارها قبل از اینکه به توسعه انجام شود، یک رویکرد طراحی و پیاده‌سازی است که به عنوان رویکرد از بالا به پایین شناخته می‌شود. این رویکرد برای گزارش‌گیری از نوع تاریخی کار می‌کند، در که شما سعی دارید بفهمید چه اتفاق رخ داده است (تحلیل توصیفی) و چرا اتفاق افتاده است (تحلیل تشخیصی-). در رویکرد از بالا به پایین، ابتدا برنامه‌ریزی کلی، طراحی و معماری انبار داده‌ها را برقرار می‌کنید و سپس اجزای خاص را توسعه می‌دهید. این روش اهمیت تعریف یک دیدگاه کل سازمانی و درک اهداف استراتژیک و نیازهای اطلاعاتی سازمان را قبل از پرش به توسعه انبار داده‌ها تاکید می‌کند.

تحلیل توصیفی و تحلیل تشخیصی- دو نوع مهم تحلیل داده‌ها هستند که در کسب و کار به طور متداول استفاده می‌شوند. تحلیل توصیفی شامل تحلیل داده‌ها برای توصیف رویدادهای گذشته یا فعلی است، اغلب از طریق استفاده از آمار خلاصه یا نمایش داده‌ها. این نوع تحلیل برای درک اینکه چه چیزی در گذشته اتفاق افتاده است و برای شناسایی الگوها یا روندهایی در داده‌ها که می‌تواند در تصمیم‌گیری کمک کند، استفاده می‌شود.

تحلیل تشخیصی- برای بررسی علل رویدادهای گذشته استفاده می‌شود، معمولاً با بررسی روابط بین متغیرها یا عوامل مختلف. این نوع تحلیل می‌تواند علل ریشه‌ای مشکلات را شناسایی کند یا مشکلاتی را که ممکن است بر عملکرد کسب و کار تأثیر بگذارد، تشخیص دهد.

فرض کنید یک شرکت می‌خواهد داده‌های فروش سال گذشته را تحلیل کند. تحلیل توصیفی شامل محاسبه آمار خلاصه مانند درآمد کل فروش، فروش میانگین روزانه و فروش بر اساس دسته‌بندی محصول برای درک اینکه چه چیزی اتفاق افتاده است، خواهد بود. در مقابل، تحلیل تشخیصی- روابط بین عوامل (مانند هزینه فروش و بازاریابی، یا فصلیت و نشانه‌های تومسن) را بررسی می‌کند تا درک کند که چرا فروش در طول سال تغییر کرده است. با ترکیب هر دو رویکرد، شرکت‌ها می‌توانند درک عمیق‌تری از داده‌های خود بدست آورند و تصمیمات بهتری بگیرند.

شکل ٤-٢ معماری یک انبار داده‌های متناظر معمولی را نشان می‌دهد. ETL برای وارد کردن داده‌ها از منابع متعدد به RDW استفاده می‌شود، که در آن گزارش‌گیری و سایر تحلیل‌ها می‌توانند انجام شوند.

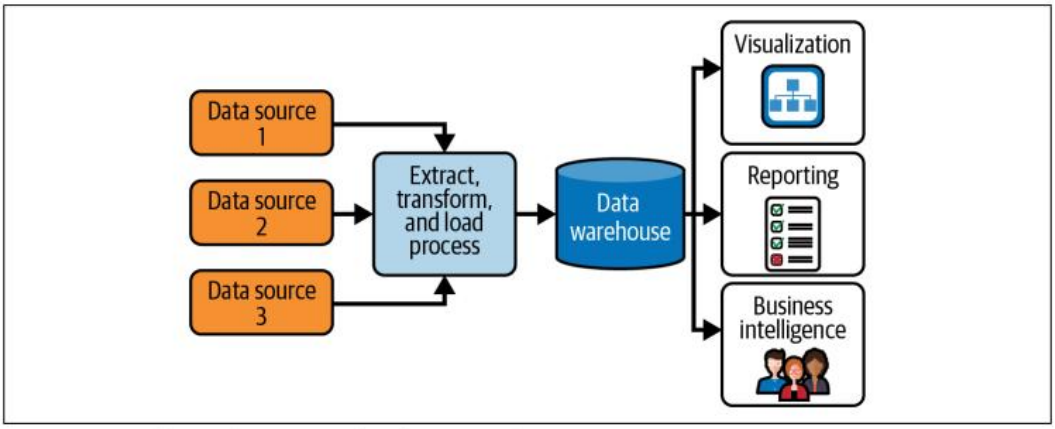


Figure 4-2. The architecture of a data warehouse

رویکرد از بالا به پایین معمولاً شامل مراحل زیر است:

۱. فرمولاسیون برخی از فرضیات در ابتدا.

شروع با درک واضح از استراتژی سازمانی. سپس مطمئن شوید که بدانید که سوالاتی را از داده‌ها می‌خواهید بپرسید.

۲. تعریف نیازهای کسب و کار.

شناسایی اهداف، اهداف و شاخص‌های کلیدی عملکرد (KPI) سازمان. جمع‌آوری و تحلیل نیازهای اطلاعاتی بخش‌ها و کاربران مختلف. می‌توانید این مرحله را نیز به عنوان تعریف نیازهای گزارش‌گیری خود در نظر بگیرید.

۳. طراحی معماری انبار داده‌ها.

بر اساس نیازهای کسب و کار، یک معماری سطح بالا برای انبار داده‌ها ایجاد کنید، از جمله ساختار آن، مدل‌های داده و فرآیندهای یکپارچگی داده. این موارد نیازهای فنی شما خواهند بود.

٤. توسعه مدل داده.

یک مدل داده جزئی را برای انبار داده‌ها طراحی کنید، با توجه به روابط بین موجودیت‌های داده مختلف و دقت داده.

٥. ساخت معماری.

پایگاه‌های داده مناسب، ساختارها، جداول و فیلدهای مورد نیاز برای انبار داده‌ها را توسعه دهید. این رویکرد قبلاً توصیف شده به نام schema-on-write است.

٦. توسعه ETL.

فرآیندهای ETL را برای استخراج داده از سیستم‌های منبع مختلف، تبدیل آنها به فرمت مورد نظر و بارگذاری آنها در انبار داده‌ها توسعه دهید.

۷. توسعه و اجرای ابزارها و برنامه‌های BI.

ابزارها و برنامه‌های BI را پیاده‌سازی کنید که به کاربران اجازه دسترسی، تحلیل و گزارش‌گیری در مورد داده‌های ذخیره شده در انبار داده‌ها را می‌دهد.

۸. آزمایش و بهینه‌سازی انبار داده‌ها.

آزمایش انجام دهید تا کیفیت داده، عملکرد و قابلیت اطمینان را تضمین کنید. هر تنظیمات لازم را برای بهینه‌سازی سیستم انجام دهید.

۹. نگهداری و گسترش انبار داده‌ها.

همانطور که نیازهای سازمان تکامل می‌یابد، انبار داده‌ها را به طور متناسب به‌روزرسانی و گسترش دهید.

رویکرد از بالا به پایین چندین مزیت دارد، مانند دید جامع از نیازهای داده سازمان، بهبود یکپارچگی داده و بهبود حاکمیت. با این حال، ممکن است زمان‌بر باشد و منابع محرمانه باشد، و زمان بیشتری برای ارائه ارزش نسبت به رویکرد از پایین به بالا که توسط دریاچه داده در فصل ٥ توصیف شده است، نیاز دارد.

معماری انبار داده‌های مدرن، که در فصل ۱۰ توصیف شده است، هر دو رویکرد از بالا به پایین و از پایین به بالا را ترکیب می‌کند.

چرا از انبار داده رابطه ای استفاده کنیم

داشتن یک انبار داده‌های متناظر (RDW) باعث می‌شود ساخت هر نوع راه حل BI بسیار آسان‌تر شود، زیرا راه حل‌های BI می‌توانند فقط از RDW داده را بکشند بدون اینکه نیاز به ایجاد منطق پیچیده برای بکشیدن داده از سیستم‌های منبع متعدد داشته باشند. همچنین، نیازی به تمیز کردن یا ادغام داده‌ها نیست زیرا RDW قبلاً آن را انجام داده است. راه حل BI که از RDW ساخته می‌شود می‌تواند یک داروخانه داده (که شامل یک زیرمجموعه از

داده‌های RDW برای گروه خاصی از افراد است، همانطور که در فصل ٦ توضیح داده شده است)، داده‌ها را برای سرعت بخشیدن به پرس و جوها و گزارشات تجمیعی کند و حتی قابل استفاده در داخل Excel Microsoft باشد. خط اصلی این است که با یک RDW، شما قاعده مستقر دارید که بر روی آن ساخته شود.

بیایید به طور دقیق به برخی از مزایای اصلی که می‌توانید از استفاده از RDW به دست آورید نگاه کنیم:

- کاهش فشار بر سیستم تولید

شما ممکن است قبلاً با این مشکل مواجه شده باشید: یک تماس ناراحت از یک کاربر نهایی که شکایت می‌کند که وارد کردن سفارشات از طریق برنامه ورود سفارش برای همیشه است. شما به آن نگاه می‌کنید و متوجه می‌شوید که یک کاربر نهایی دیگری یک گزارش را از طریق برنامه ورود سفارش اجرا می‌کند که تمام منابع را در سرور که برنامه در آن قرار دارد اشغال می‌کند. این وضعیت به خصوص متداول است زمانی که کاربران نهایی اجازه ایجاد پرس و جوهای تصادفی را دارند و آنها با نوشتن SQL بد مواجه می‌شوند. با کپی پایگاه داده برنامه ورود سفارش به یک DW و بهینه‌سازی آن، می‌توانید تمام گزارشات و پرس و جوهای تصادفی را بر روی DW اجرا کنید و از این مشکل کاملاً خودداری کنید، به خصوص اگر کاربر نهایی نیاز به اجرای یک گزارش دارد که بر روی چندین پایگاه داده برنامه متداول است.

- بهینه‌سازی برای دسترسی خواندن

پایگاه‌های داده برنامه برای پشتیبانی از تمام عملیات CRUD به طور مساوی بهینه‌سازی می‌شوند، بنابراین خواندن داده به سرعت نیست که می‌تواند. انبار داده، از سوی دیگر، یک سیستم نوشتن یکبار، خواندن بسیاری است، به این معنی که بیشتر برای خواندن داده استفاده می‌شود. بنابراین، می‌تواند برای دسترسی خواندن بهینه‌سازی شود، به خصوص برای اسکن‌های دیسک توالی زمان‌بر که به طور مکرر زمانی که گزارشات یا پرس و جوها اجرا می‌شوند رخ می‌دهد. روش‌های زیادی پایگاه داده وجود دارد که می‌توانند دسترسی خواندن در یک DW را تسریع کنند، برخی از آنها به معنی کاهش دسترسی نوشتن هستند، که ما در آن نگران نیستیم.

یکپارچگی چندین منبع داده

توانایی یکپارچگی چندین منبع داده برای ایجاد گزارشات مفیدتر یکی از دلایل مهم برای ساخت یک انبار داده (DW) است. قرار دادن تمام داده‌ها در یک مکان به جای پخش آن‌ها در سیستم‌های پایگاه داده مختلف نه تنها ساخت گزارش را آسان‌تر می‌کند بلکه عملکرد گزارش‌گیری را به شدت بهبود می‌بخشد.

اجرای گزارشات تاریخی دقیق

بدون یک DW، کاربران نهایی برنامه‌ها معمولاً تمام گزارشات خود را در یک روز خاص هر ماه (معمولاً آخرین) اجرا می‌کنند. سپس آن‌ها را به دیسک ذخیره می‌کنند تا نسخه‌هایی که می‌توانند در آینده به آن‌ها ارجاع دهند. به عنوان مثال، کاربر می‌خواهد گزارشی از چند ماه پیش را ببیند که فروش مشتریان بر اساس استان را لیست می‌کند. با این حال، یک مشتری اخیراً به یک استان دیگر مهاجرت کرده است. اگر کاربر گزارش فعلی را اجرا کند، به طور نادرست فروش این مشتری در استان جدید خود را نشان می‌دهد به جای استان قدیمی خود (زیرا رکورد آن در پایگاه داده به استان جدید آن به‌روز شده است).بنابراین، کاربر باید به گزارش قدیمی‌تری نگاه کند به جای اجرای گزارش فعلی. یک DW می‌تواند این مشکل را با پیگیری زمانی که یک مشتری مهاجرت می‌کند (با پیگیری تاریخچه مکان مشتری با تاریخ شروع و پایان) و همچنین هر فیلد دیگری که نیاز به پیگیری دارد (برای مثال، کارفرما یا درآمد) مراقبت کند، حل کند. حالا کاربر می‌تواند گزارشی امروز را اجرا کند اما بخواهد که داده‌ها را در تاریخ خاصی در گذشته بکشد و گزارش دقیق خواهد بود. علاوه بر این، ذخیره فایل‌های گزارش ماهانه دیگر لازم نیست.

بازساختن و تغییر نام جدول‌ها

بسیاری از پایگاه‌های داده برنامه نام جدول و فیلد دارند که بسیار دشوار برای فهم هستند، به خصوص محصولات ERP و CRM قدیمی‌تر (فکر کنید به نام‌های جدولی مانند T۱۱۶ و نام‌های فیلد مانند RAP۱۶). در انبار داده، می‌توانید داده را از این جداول منبع را به چیزی که بسیار راحت‌تر است برای فهم (برای مثال، مشتری به جای T۱۱۶) کپی کنید. شما همچنین احتمالاً می‌توانید مدل داده بهتری برای تمام جداول بدست آورید. کاربران نهایی می‌توانند گزارشات را بسیار آسان‌تر ایجاد کنند زمانی که نیازی به ترجمه ناشناخته نام جدول و فیلد ندارند.

خلاصه - رمز گشایی از معماری داده

محافظت در برابر به‌روزرسانی برنامه

تصور کنید که شما یک DW ندارید و کاربران به جای ایجاد گزارشات در پایگاه داده برنامه انجام می‌دهند. همه چیز به خوبی در حال اجرا است و سپس به طور ناگهانی، تعداد زیادی از گزارشات خطا می‌دهند. با این حال، برنامه به روزرسانی شده و نسخه جدیدی را نصب کرده است که تعدادی از جداول و فیلدها را تغییر نام داده است. بنابراین، حالا باید از طریق هر یک از صدها گزارش، تغییر نام جداول و فیلدهای تغییر یافته را انجام دهید. این کار می‌تواند ماه‌ها طول بکشد، که نتیجه کاربران نهایی ناراحت شده باشد. حتی پس از آن، هر گزارشی که گذشته شده است هنوز ممکن است خطا دهد. یک DW می‌تواند شما را در برابر این موضوع محافظت کند. پس از به‌روزرسانی برنامه، فقط ETL که داده‌ها را از پایگاه‌های داده برنامه به DW کپی می‌کند، نیاز به به‌روزرسانی دارد - یک کار سریع. گزارشات نیازی به تغییر ندارند. کاربران نهایی تا زمانی که ETL تصحیح شود، هیچ داده جدیدی را نمی‌بینند، اما گزارشات آن‌ها خطا ندارند.

کاهش نگرانی‌های امنیتی

بدون یک DW، تیم شما باید به هر کاربر نهایی دسترسی امنیتی به هر پایگاه داده برنامه‌ای که برای اهداف گزارش‌گیری نیاز داشته باشد، بدهد. ممکن است دزدیز ها باشد؛ فرآیند ارائه دسترسی می‌تواند چند هفته طول بکشد و گاهی اوقات هنوز به همه چیزی که نیاز دارند دسترسی نداشته باشند. با یک DW، هر کاربر نهایی فقط به جداول مناسب نیاز دارد و ارائه این موارد بسیار سریع‌تر و آسان‌تر است.

حفظ داده‌های تاریخی

بسیاری از سیستم‌های تولید محدودیت در مقدار داده‌های تاریخی که نگهداری می‌کنند (به عنوان مثال، داده‌های سه سال گذشته) را قرار می‌دهند. آن‌ها این کار را برای صرفه‌جویی در فضا و بهبود عملکرد و در برخی موارد برای رعایت قوانین انجام می‌دهند. داده‌های قدیمی معمولاً سالانه یا ماهانه حذف می‌شوند. از سوی دیگر، یک DW می‌تواند تمام تاریخچه را نگه دارد، بنابراین شما هرگز نیازی به اجرای گزارش برای سال‌های قدیمی و نداشتن هیچ داده‌ای ندارید.

مدیریت داده‌های اصلی (MDM)

هنگامی که داده‌ها را از چندین سیستم منبع جمع‌آوری می‌کنید، بسیاری از اوقات شما نیاز به استفاده از MDM برای حذف رکوردهای تکراری برای چیزهایی مانند مشتریان، محصولات و دارایی‌ها خواهید داشت. (برای توضیح بیشتر در مورد MDM، به فصل ٦ مراجعه کنید.) DW مکان کامل برای انجام MDM است. علاوه بر این، بسیاری از ابزارهای MDM به شما اجازه می‌دهند تا سلسله‌های سلسله را ایجاد کنید (به عنوان مثال، شرکت → بخش → کارمند)، ارزش بیشتری به مستری کردن داده‌ها اضافه می‌کند.

بهبود کیفیت داده با پر کردن شکاف‌ها در سیستم‌های منبع

شما ممکن است ببینید که بسیاری از داده‌هایی که از سیستم‌های منبع مختلف دریافت می‌کنید، نیاز به تمیز کردن دارند، با وجود آنکه مالکان برنامه‌ها می‌گویند "داده‌های ما تمیز هستند" (من آن‌ها را بسیاری بار شنیدم، تا اثبات اشتباه شوم). به عنوان مثال، یک برنامه ورود سفارش ممکن است نیاز به تاریخ تولد مشتری داشته باشد، و اگر شخصی ـ که داده‌ها را وارد می‌کند تاریخ تولد مشتری را نمی‌داند، ممکن است یک تاریخ در آینده یا یک تاریخ بیش از ۱۰۰ سال قبل را وارد کند تا بتواند سفارش را تکمیل کند. یا شاید برنامه دقت دو رقم وارد شده برای کد استان را بررسی نکند. همیشه دزدیزهایی در سیستم منبع وجود دارد. شما نه تنها می‌توانید داده‌ها را در DW تمیز کنید، بلکه می‌توانید نیز افرادی که برنامه‌ها را نگهداری می‌کنند را از شکاف‌های سیستم خود آگاه کنید تا آن‌ها را تصحیح کنند. به این ترتیب، شما به جلوگیری از ورود داده‌های بد در آینده کمک می‌کنید.

حذف مشارکت IT در ایجاد گزارش

این موضوع به خودخدمت BI که در فصل ۳ ذکر شده است باز می‌گردد: ساخت یک DW مناسب نیاز به مشارکت IT در ساخت گزارشات را حذف می‌کند و این وظیفه را در دست کاربر نهایی قرار می‌دهد. بدون گلوگاه

محـدودیت منـابع IT محـدود، گزارشـات و داشـبوردها می‌تواننـد زودتـر سـاخته شـوند. و IT بـا تشـکر خواهـد بـود کـه می‌توانند بر روی پروژه‌های جذاب‌تر تر کار کنند تا ایجاد گزارشات!

معایب استفاده از انبار داده های رابطه ای

همیشـه تعادل‌هـایی وجـود دارد و در زیـر نقـاط مـنفی را کـه در هنگـام سـاخت یـک داده‌خـواری رابطـه‌ای (RDW) بایـد در نظر گرفت می‌شود، می‌بینید:

پیچیدگی

- **پیچیـدگی**: داده‌خواری‌هـا می‌تواننـد پیچیـده و زمـان‌بر بـرای طـراحی، سـاخت و نگهـداری باشـند. مهارت‌هـا و منـابع متخصص مورد نیاز می‌توانند هزینه‌ها را افزایش دهند.

هزینه‌های بالا

- **هزینـه‌های بـالا**: پیاده‌سـازی یـک داده‌خـواری می‌توانـد گـران باشـد، نیـاز بـه سرمایه‌گـذاری‌های قابـل تـوجهی در سخت‌افزار، نرم‌افزار و افراد دارد. نگهداری مداوم و به‌روزرسانی‌ها نیز می‌توانند هزینه‌ها را افزایش دهند.

چالش‌های یکپارچگی داده

- **چالش‌هـای یکپـارچگی داده**: یکپـارچگی داده‌هـا از منـابع مختلـف می‌توانـد چـالش‌برانگیز باشـد، زیـرا ممکـن اسـت شـامل مقابلـه بـا فرمت‌هـای داده مختلـف، سـاختارها و مسـائل کیفیـت باشـد. ایـن می‌توانـد منجـر بـه صرف زمـان و تـلاش در تمـیز کـردن و پیش‌پـردازش داده‌هـا شـود. عـلاوه بـر ایـن، بـرخی از داده‌هـا، ماننـد داده‌هـای جریـانی از دسـتگاه‌های IoT، بـرای مصرـف در یـک داده‌خـواری رابطـه‌ای بسـیار چـالش‌برانگیز هسـتند و بنـابراین بینش‌هـای احتمـالی از ایـن اطلاعـات از دست می‌رود.

تبدیل داده زمان‌بر

- **تبـدیل داده زمـان‌بر**: بـرای بارگـذاری داده در یـک داده‌خـواری، ممکـن اسـت نیـاز بـه تبـدیل بـه مـدل داده انبـار بـرای تطبیـق داشـته باشـد. ایـن فرآینـد می‌توانـد زمـان‌بر باشـد و خطاهـایی در تبـدیل داده می‌توانـد منجـر بـه تحلیل‌هـای نادرسـت شود.

تاخیر در داده

- **تـاخیر در داده**: بـه دلیـل طـراحی داده‌خواری‌هـا بـرای کـار بـا حجم‌هـای بـزرگی از داده، آن‌هـا می‌تواننـد نسـبت بـه سـایر انـواع پایگاه‌داده‌هـا کنـدتر باشـند. ایـن می‌توانـد منجـر بـه تـاخیر در داده شـود، جـایی کـه داده‌هـا در انبـار بـا تغییـرات اخیر در پایگاه‌داده‌های منبع به‌روز نیستند.

پنجره نگهداری

- **پنجـره نگهـداری**: بـا یـک داده‌خـواری رابطـه‌ای، معمـولاً نیـاز بـه یـک پنجـره نگهـداری داریـد. بارگـذاری و تمـیز کـردن داده بسـیار منـابع مصرـفی اسـت و اگـر کـاربران در همـان زمـان سـعی می‌کننـد گزارشـات را اجـرا کننـد، بـا عملکـرد بسـیار کنـد مواجـه خواهنـد شـد. بنـابراین کـاربران بایـد در حـین نگهـداری از انبـار خـارج شـوند، کـه دسـترسی ۷/۲٤ را می‌گـیرد. اگـر هـر گونـه مشـکلی در طـول پنجـره نگهـداری اتفـاق بیفتـد، ماننـد یـک کـار ETL شکسـته، ممکـن اسـت نیـاز باشـد تـا پنجـره

نگهـداری را گسـترش دهیـد. اگـر کـاربران سـعی می‌کننـد گزارشـات را اجـرا کننـد و هنـوز خـارج شـده‌اند، کـاربران ناراحـت خواهید داشت که نمی‌توانند کارهای خود را انجام دهند.

انعطاف‌پذیری محدود

- **انعطاف‌پـذیری محـدود**: داده‌خواری‌هـا بـرای پشـتیبانی از انـواع خـاصی از تحلیـل طـراحی شـده‌اند، کـه می‌توانـد انعطاف‌پـذیری آن‌هـا را بـرای سـایر انـواع پـردازش یـا تحلیـل داده محـدود کنـد. ابزارهـا یـا سیسـتم‌های اضـافی ممکـن اسـت نیاز باشد تا با انبار یکپارچگی شوند تا نیازهای خاص را برآورده کنند.

نگرانی‌های امنیتی و حریم خصوصی

- **نگرانی‌هـای امنیـتی و حـریم خصـوصی**: ذخـیره مقـادیر زیـادی از داده‌هـای حسـاس در یـک مکـان متمرکـز می‌توانـد خطر نقض داده‌ه

دستکاری و اصلاح انبار داده

چـون جـداول منـبعی کـه بـه یـک داده‌خـواری مصرف می‌شـوند در طـول زمـان تغیـیر می‌کننـد، نیـاز اسـت کـه داده‌خـواری تغییرات آن‌ها را بازتاب کند. ایـن بـه نظـر سـاده می‌رسـد، امـا تعـداد زیـادی تصـمیم بایـد گرفتـه شـود: چقـدر اغلـب بایـد داده را اسـتخراج (یـا کشـیدن) کنـیم، کـدام روش اسـتخراج را بایـد اسـتفاده کنـیم، چگونـه بایـد داده را بـه طـور فـیزیکی اسـتخراج کنـیم، و چگونـه بایـد تعیـین کنـیم کـه کـدام داده از آخـرین اسـتخراج تغییر کـرده اسـت. مـن بـه طـور مختصر در مـورد هـر یک از این موارد صحبت خواهم کرد.

چند وقت یکبار اطلاعات استخراج میشوند

چقـدر اغلـب نیـاز بـه به‌روزرسـانی داده‌خـواری وابسـته بـه چقـدر اغلـب سیسـتم‌های منبـع بـه‌روز می‌شـوند و چقـدر زمـانی کـاربر نهـایی نیـاز بـه گزارش‌هـا دارد. اغلـب کـاربران نهـایی می‌خواهنـد داده‌هـای روز فعـلی را ببیننـد، تـرجیح می‌دهنـد تمـام داده‌هـا را تـا پایـان روز قبـل دریافـت کننـد. در ایـن مـورد، می‌توانیـد کارهـای خـود را بـرای اسـتخراج داده از سیسـتم‌های منبـع بـا اسـتفاده از ابزارهـای ETL هـر شـب پـس از اتمـام به‌روزرسـانی پایگاه‌داده‌هـای سیسـتم منبـع اجـرا کنیـد، یـک پنجـره نگهـداری شـبانه را بـرای انجـام تمـام انتقـال داده ایجـاد کنیـد. اگـر کـاربران نهـایی نیـاز بـه به‌روزرسـانی‌ها در طـول روز دارنـد، آنگاه یک استخراج بیشتر، به عنوان مثال هر ساعت، لازم خواهد بود.

یـک نکتـه کـه بایـد در نظـر گرفتـه شـود انـدازه داده‌هـای اسـتخراج شـده بـرای هـر اسـتخراج اسـت. اگـر بسـیار بـزرگ باشـد، به‌روزرسـانی داده‌خـواری ممکـن اسـت زمـان زیـادی بگـیرد، بنـابراین ممکـن اسـت بخواهیـد به‌روزرسـانی را بـه قطعـات کـوچکتر تقسـیم کنیـد و اسـتخراج‌ها و به‌روزرسـانی‌های بیشـتری انجـام دهیـد (بـه عنـوان مثـال، هـر سـاعت بـه جـای روزانـه). عـلاوه بـر ایـن، ممکـن اسـت زمـان زیـادی بـرای انتقـال مقـادیر زیـادی از داده‌هـا از سیسـتم‌های منبـع بـه داده‌خـواری لازم باشـد، بـه خصـوص اگـر داده‌هـای منبـع در محـل باشـد و شـما نیـازی بـه یـک لولـه بـزرگ از سیسـتم منبـع بـه اینترنـت نداریـد. ایـن دلیـل دیگـری اسـت کـه ممکـن اسـت بخواهیـد از یـک انتقـال شـبانه بـزرگ بـه انتقال‌هـای سـاعتی کوچکتر طی روز بروید.

روشهای استخراج

دو روش برای استخراج داده از سیستم‌های منبع وجود دارد. بیایید به هر یک از آن‌ها نگاه کنیم:

استخراج کامل

در اسـتخراج کامـل، تمـام داده‌هـا بـه طـور کامـل از یـک یـا چنـد جـدول در سیسـتم منبـع اسـتخراج می‌شـوند. ایـن روش بـرای جـداول کـوچکتر بهـتر کـار می‌کنـد. چـون ایـن اسـتخراج تمـام داده‌هـای موجـود در سیسـتم منبـع را بازتـاب می‌دهـد، نیـازی بـه ردیـابی تغیـرات نیسـت، بنـابراین ایـن روش بسـیار سـاده اسـت تـا سـاخته شـود. داده‌هـای منبـع بـه صـورت اسـت فـراهم شده و شما نیازی به اطلاعات اضافی (به عنوان مثال، زمان‌بندی‌ها) ندارید.

استخراج تدریجی

در استخراج تدریجی، فقط داده‌هایی که از زمان مشخصی- (مانند آخرین استخراج یا پایان یک دوره مالی) تغییر کرده‌اند را استخراج می‌کنید، به جای کل جدول. این روش برای جداول بزرگتر بهتر کار می‌کند و فقط زمانی کار می‌کند که ممکن است تمام اطلاعات تغییر یافته را شناسایی کنید (که در زیر بحث می‌شود).

با اکثر سیستم‌های منبع، شما از ترکیب این دو روش استفاده خواهید کرد. چه استخراج کامل ای یا استخراج تدریجی ای که انجام می‌دهید، دو روش برای استخراج داده وجود دارد: آنلاین و آفلاین.

در استخراج آنلاین، فرآیند استخراج می‌تواند مستقیماً به سیستم منبع متصل شود تا به جداول منبع دسترسی پیدا کند، یا ممکن است به یک سیستم میانی متصل شود که تغییرات در داده‌ها را به صورت پیش‌فرض شده (به عنوان مثال، در لاگ‌های تراکنش یا جداول تغییر) ذخیره می‌کند.

با این حال، دسترسی مستقیم به سیستم منبع همیشه در دسترس نیست. در چنین مواردی، داده‌ها به صورت مرحله‌ای خارج از سیستم منبع اصلی قرار می‌گیرند و توسط یک روتین استخراج ایجاد می‌شوند که از سیستم منبع منشأ می‌شود (به عنوان مثال، یک مینی‌کامپیوتر یک روتین استخراج روی یک جدول انجام می‌دهد و داده‌ها را در یک پوشه در یک سیستم فایل قرار می‌دهد). داده‌های استخراج شده معمولاً در یک فایل ساده قرار می‌گیرند که در یک فرمت مشخص و عمومی است (به عنوان مثال، CSV یا JSON).

نحوه مشخص کردن تازه ترین تغییرات پس از آخرین استخراج

متاسفانه، برای بسیاری از سیستم‌های منبع ممکن است دشوار باشد تا داده‌های اخیراً تغییر یافته را شناسایی کنند و یک استخراج تدریجی انجام دهند. در زیر چندین تکنیک برای شناسایی داده‌های اخیراً تغییر یافته و پیاده‌سازی استخراج تدریجی از سیستم‌های منبع آورده شده است. این تکنیک‌ها می‌توانند با روش‌های استخراج داده که بحث شده است، همکاری کنند. برخی از این تکنیک‌ها بر اساس ویژگی‌های سیستم‌های منبع است؛ دیگران ممکن است نیاز به تغییرات در سیستم‌های منبع داشته باشند. مالکان سیستم منبع باید با دقت هر تکنیکی را قبل از پیاده‌سازی ارزیابی کنند:

۱. **استفاده از لاگ‌های تراکنش**: بسیاری از سیستم‌های منبع لاگ‌های تراکنش را نگهداری می‌کنند که تغییرات اخیر در داده‌ها را ثبت می‌کنند. این لاگ‌ها می‌توانند برای شناسایی داده‌های تغییر یافته و انجام استخراج تدریجی استفاده شوند.

۲. **استفاده از جداول تغییر**: برخی سیستم‌های منبع جداول تغییر را پشتیبانی می‌کنند که تغییرات اخیر در داده‌ها را ثبت می‌کنند. این جداول می‌توانند برای شناسایی داده‌های تغییر یافته و انجام استخراج تدریجی استفاده شوند.

۳. **استفاده از تایم‌استمپ‌ها**: برخی سیستم‌های منبع تایم‌استمپ‌ها را پشتیبانی می‌کنند که زمان آخرین تغییر در داده‌ها را ثبت می‌کنند. این تایم‌استمپ‌ها می‌توانند برای شناسایی داده‌های تغییر یافته و انجام استخراج تدریجی استفاده شوند.

٤. **استفاده از تریگرها و مقادیر مشترک**: برخی سیستم‌های منبع امکان استفاده از تریگرها و مقادیر مشترک را برای شناسایی تغییرات در داده‌ها فراهم می‌کنند. این روش می‌تواند برای شناسایی داده‌های تغییر یافته و انجام استخراج تدریجی استفاده شود.

٥. **تغییرات در سیستم منبع**: در برخی موارد، ممکن است نیاز به تغییرات در سیستم منبع برای پشتیبانی از استخراج تدریجی باشد. این ممکن است شامل افزودن جداول تغییر یا لاگ‌های تراکنش باشد.

هر یک از این تکنیک‌ها ممکن است بسته به ویژگی‌های خاص سیستم منبع و نیازهای خاص پروژه مناسب باشد. مالکان سیستم منبع باید با دقت هر تکنیکی را قبل از پیاده‌سازی ارزیابی کنند تا از اینکه آن برای نیازهای خود مناسب است یا خیر اطمینان حاصل کنند.

زمان‌بندی‌ها (Timestamps)

زمان‌بندی‌ها گزینه مورد ترجیح و ساده‌ترین برای پیاده‌سازی هستند. جداول برخی سیستم‌های عملیاتی ستون‌های زمان‌بندی دارند که زمان و تاریخی که یک ردیف خاص آخرین بار تغییر کرده است را نشان می‌دهند، که باعث می‌شود تا آخرین داده‌ها را به راحتی شناسایی کنیم. در پایگاه‌داده‌های رابطه‌ای، ستون زمان‌بندی معمولاً با نوع داده time stamp یا datetime همراه با یک نام ستون مانند Timestamp یا Modified Last ارائه می‌شود. برنامه منبع سپس این ستون را پر می‌کند. اگر این کار انجام نشود، می‌توانید پایگاه داده رابطه‌ای را به گونه‌ای تنظیم کنید که به طور پیش‌فرض به تاریخ فعلی زمانی که رکورد ذخیره می‌شود، تنظیم شود، یا می‌توانید تریگرهای پایگاه داده را اضافه کنید تا ستون را پر کنند.

گرفتن تغییرات داده (Change Data Capture)

بسیاری از پایگاه‌داده‌های رابطه‌ای پشتیبانی از گرفتن تغییرات داده (CDC) را دارند، که تغییرات UPDATE، INSERT و DELETE اعمال شده بر روی جداول پایگاه داده را ثبت می‌کند و یک رکورد جدول را در دسترس قرار می‌دهد که نشان می‌دهد چه چیزی تغییر کرده، کجا و کی بر اساس لاگ تراکنش پایگاه داده رابطه‌ای. اگر نیاز به داده‌خواری نزدیک به زمان واقعی دارید، که در آن تغییرات در سیستم منبع در داده‌خواری در طی چند ثانیه نمایش داده می‌شوند، CDC می‌تواند تکنولوژی فعال‌ساز کلیدی باشد.

تقسیم‌بندی (Partitioning)

برخی سیستم‌های منبع از تقسیم‌بندی با محدوده استفاده می‌کنند، که در آن جداول منبع بر اساس یک کلید تاریخ تقسیم‌بندی می‌شوند، که باعث می‌شود تا داده‌های جدید را به راحتی شناسایی کنیم. به عنوان مثال، اگر از یک جدول سفارشات که بر اساس روز تقسیم‌بندی شده است استخراج می‌کنید، برای شناسایی داده‌های روز فعلی یا قبلی آسان است.

تریگرهای پایگاه داده (Database Triggers)

می‌توانید یک تریگر برای UPDATE، INSERT و DELETE روی یک جدول تکی اضافه کنید و این تریگرها اطلاعات در مورد تغییر رکورد را به یک "جدول تغییر" بنویسند. این مشابه گرفتن تغییرات داده است، بنابراین اگر محصول پایگاه داده شما CDC را پشتیبانی می‌کند؛ در غیر این صورت، از تریگرها استفاده کنید.

دستور MERGE

گزینه کمتر مورد ترجیح این است که از سیستم منبع به یک محل میانی در داده‌خواری استخراج کامل کنید، سپس این جدول را با یک استخراج کامل قبلی از سیستم منبع با استفاده از یک دستور MERGE مقایسه کنید تا داده‌های تغییر یافته را شناسایی کنید. شما نیاز دارید تمام فیلدهای منبع را با تمام فیلدهای مقصد مقایسه کنید (یا از یک تابع هش استفاده کنید). این رویکرد احتمالاً تأثیر قابل توجهی بر سیستم منبع ندارد، اما می‌تواند بر روی داده‌خواری بزرگ تحمل قابل توجهی ایجاد کند، به خصوص اگر حجم داده‌ها بزرگ باشد. این گزینه معمولاً آخرین گزینه است اگر هیچ گزینه دیگری ممکن نباشد.

مرگ انبار داده های رابطه ای بسیار اغراق شده است

در دهه ۲۰۱۰ اوایل، مردم در حوزه IT شروع به مطرح کردن سوال کردند که آیا پایگاه داده رابطه‌ای (RDW) هنوز نیاز دارد یا خیر، با پرسیدن، "آیا پایگاه داده رابطه‌ای مرده است؟" بسیاری از افراد این را به معنای پرسیدن می‌فهمیدند که آیا کسب و کارها هنوز نیاز به DWs دارند. آنها هستند، همانطور که این فصل نشان می‌دهد. اما سوال واقعاً در مورد معماری پایگاه داده است - آیا می‌توانید فقط از یک دیتا لاک استفاده کنید یا باید هر دو دیتا لاک و RDW را استفاده کنید؟

وقتی دیتا لاک‌ها برای اولین بار ظاهر شدند، آنها بر روی تکنولوژی Hadoop Apache ساخته شدند و بیشتر تامین‌کنندگان Hadoop پایگاه داده رابطه‌ای را مرده می‌گفتند. "فقط تمام داده‌های خود را در دیتا لاک قرار دهید و RDW خود را حذف کنید"، آنها توصیه می‌کردند. همانطور که در فصل ۲ ذکر شد، پروژه‌هایی که سعی کردند این کار را انجام دهند، همه شکست خوردند.

برای سال‌ها، من احساس می‌کردم که RDWs همیشه نیاز خواهند داشت زیرا دیتا لاک‌ها همه بر پایه Hadoop بودند و محدودیت‌ها خیلی زیاد بودند. اما هنگامی که راه‌حل‌هایی مانند Delta lake (ببینید فصل ۱۲) موجود شدند و دیتا لاک‌ها شروع به استفاده از محصولات بهتر و آسان‌تر از Hadoop کردند (ببینید فصل ۱۶)، شروع به دیدن برخی موارد استفاده کردم که یک راه حل می‌تواند بدون RDW کار کند. این نوع راه حل معماری دیتا لاک‌هاس است که در فصل ۱۲ پوشش داده خواهد شد.

با این حال، هنوز بسیاری از موارد استفاده که یک RDW لازم است. و در حالی که تکنولوژی‌های دیتا لاک ادامه دارند تا بهبود یابند، بنابراین نگرانی‌های درباره دور زدن RDW را کاهش یا حذف می‌کنند (ببینید فصل ۱۲)، ما هرگز کاملاً از RDWs دور نخواهیم افتاد. من فکر می‌کنم سه دلیل برای این است. اول، هنوز سخت‌تر است از یک دیتا لاک گزارش بدهید تا از یک DW. دوم، RDWs همچنان نیازهای اطلاعاتی کاربران را برآورده می‌کنند و ارزش ارائه می‌دهند. سوم، بسیاری از افراد از دیتا لاک‌ها استفاده می‌کنند، به آنها وابسته هستند و آنها را می‌اندیشند و نمی‌خواهند آنها را با دیتا لاک‌ها جایگزین کنند.

دیتا لاک‌ها منبع غنی از داده برای دانشمندان داده و مصرف‌کنندگان داده Self Service (کاربران قدرتمند) ارائه می‌دهند و نیازهای تحلیل و بزرگ داده را خوب خدمت می‌کنند. اما همه کارکنان داده و اطلاعات نمی‌خواهند تبدیل شوند به کاربران قدرتمند. اکثر هنوز نیاز به داده‌های رابطه‌ای بهتر ادغام شده، به طور سیستماتیک پاک شده و آسان دسترسی دارند که شامل یک لاگ تاریخی است که چگونگی تکامل یا پیشرفت چیزها را طی یک دوره زمانی ضبط می‌کند. این افراد بهترین خدمات را با یک پایگاه داده می‌گیرند.

خلاصه

این فصل تکنولوژی اولیه و گسترده‌ای را که برای مرکزیت کردن داده‌ها از منابع متعدد و گزارش دادن بر روی آن‌ها استفاده شده است، پوشش داده است: پایگاه داده رابطه‌ای (RDW). انقلاب RDW روش مدیریت و مدیریت داده‌های کسب و کارها و سازمان‌ها را تغییر داد، با ارائه یک مخزن مرکزی برای ذخیره‌سازی و بازیابی داده، امکان مدیریت و تحلیل داده به صورت کارآمدتر را فراهم می‌کند. با توانایی ذخیره و سازماندهی داده به شکل ساختاری، RDWs به کاربران اجازه می‌دهد تا پرس و جوهای پیچیده و گزارشات را به سرعت و آسان تولید کنند، با ارائه بینش‌های ارزشمند و حمایت از تصمیم‌گیری حیاتی.

امروزه، پایگاه داده رابطه‌ای هنوز یک مؤلفه اساسی از بسیاری از معماری‌های داده است و می‌توان آن را در بازه گسترده‌ای از صنایع، از مالی و بهداشت تا بازاریابی و تولید، مشاهده کرد. فصل بعدی تکنولوژی بعدی را که برای مرکزیت کردن و گزارش دادن بر روی داده به عنوان یک عامل اصلی در میان می‌آید، معرفی می‌کند: دیتا لاک.

فصل پنجم: دریاچه داده

داده‌های بزرگ در دهه ۲۰۱۰ اوایل به حجم‌های بی‌سابقه ظاهر شدند به دلیل افزایش منابعی که داده‌های نیمه‌ساختاری و غیرساختاری مانند سنسورها، ویدیوها و رسانه‌های اجتماعی را خروجی می‌دادند. داده‌های نیمه‌ساختاری و غیرساختاری مقدار بزرگی ارزش را نگه می‌دارند - فکر کنید به بینش‌های موجود در سال‌های طولانی از ایمیل‌های مشتری! با این حال، در آن زمان، پایگاه‌های داده رابطه‌ای فقط می‌توانستند داده‌های ساختاری را مدیریت کنند. آنها همچنین با مدیریت مقادیر زیادی از داده یا داده‌هایی که باید به طور مکرر مصرف شوند، مشکل داشتند، بنابراین برای ذخیره این انواع داده‌ها گزینه نبودند. این امر منجر شد که صنعت باید به دنبال یک راه حل باشد: دیتا لاک‌ها. دیتا لاک‌ها می‌توانند به راحتی داده‌های نیمه‌ساختاری و غیرساختاری را مدیریت کنند و داده‌هایی که به طور مکرر مصرف می‌شوند را مدیریت کنند.

سال‌ها پیش، با تحلیلگرانی از یک زنجیره بزرگ بازاریابی صحبت کردم که می‌خواستند داده‌ها را از تو پیتر مصرف کنند تا ببینند چه چیزی مشتریان در مورد فروشگاه‌های آن‌ها فکر می‌کنند. آنها می‌دانستند مشتریان ممکن است از اینکه شکایات را به کارمندان فروشگاه بیان کنند مردد باشند، اما سریع‌تر برای آن‌ها را روی تو پیتر قرار می‌دهند. من به آن‌ها کمک کردم تا داده‌های تو پیتر را در یک دیتا لاک مصرف کنند و بینش نظرات مشتریان را ارزیابی کنند، آن‌ها را به عنوان مثبت، خنثی یا منفی دسته‌بندی می‌کردم. وقتی آن‌ها نظرات منفی را خواندند، تعداد بیش از حد شکایات در مورد اتاق‌های تغییر پوشیدن را یافتند - آن‌ها خیلی کوچک بودند، خیلی افراد زیادی در آن‌ها بودند و کافی از خصوصیت نداشتند. به عنوان یک آزمایش، شرکت تصمیم گرفت تا اتاق‌های تغییر پوشیدن در یک فروشگاه را تغییر دهد. یک ماه پس از تغییر، تحلیلگران تعداد بیش از حد نظرات مثبت در مورد اتاق‌های تغییر پوشیدن در آن فروشگاه را یافتند، همراه با یک افزایش ۷٪ در فروش. این امر باعث شد که شرکت تصمیم گرفت تا اتاق‌های تغییر

پوشیدن در تمام فروشگاه‌های خود را تغییر دهد، که منجر به یک افزایش ٦٪ در فروش در سراسر کشور و میلیون‌ها در سود شد. همه به دلیل یک دیتا لاک!

تعریف اولیه

اصطلاح "دیتا لاک" یک متافور است که برای توصیف مفهوم ذخیره کردن مقادیر بزرگی از داده‌های خام در فرمت طبیعی آن‌ها استفاده می‌شود. همانطور که یک دریاچه آب را بدون تغییر طبیعت آب نگه می‌دارد، یک دیتا لاک داده را بدون نیاز به ساختاردهی یا پردازش اولیه نگه می‌دارد. همچنین، همانطور که دریاچه‌ها گونه‌های مختلف زندگی ویلندری و گیاهی را نگه می‌دارند، یک دیتا لاک انواع مختلفی از داده‌ها (ساختاری، نیمه‌ساختاری و غیرساختاری) را نگه می‌دارد. این را با پایگاه داده مقایسه کنید، که در آن داده بیشتر ساختاری و پردازش شده است، مانند کالاهای بسته شده یا بطری در یک انبار.

هنگامی که داده در یک دیتا لاک قرار می‌گیرد، باید پاک شود، با هم متصل شود و احتمالاً تجمع یابد تا مفید شود. این جایی است که باید نوعی از محاسبات (یعنی قدرت پردازشی که برای مدیریت، تغییر زدن و تحلیل داده لازم است) به دیتا لاک متصل شود، داده را بگیرد و تبدیل کند، سپس آن را دوباره در دیتا لاک قرار دهد.

چرایی دریاچه داده

استفاده از یک دیتا لاک برای مدیریت داده‌ها به دلایل متعددی مزیت دارد، به خصوص هنگامی که با یک پایگاه داده رابطه‌ای (RDW) همراه است. همانطور که اشاره شده است، شما می‌توانید به سرعت داده را در یک دیتا لاک ذخیره کنید بدون کار اولیه انجام شده - رویکردی که به عنوان "schema-on-read" شناخته می‌شود (ببینید فصل ۲). این به شما اجازه می‌دهد تا به داده بسیار سریع‌تر دسترسی پیدا کنید، که می‌تواند به کاربران قدرتمند اجازه دهد تا گزارشات را به سرعت اجرا کنند (برای بازده سریع‌تر سرمایه‌گذاری یا ROI) و به دانشمندان داده را برای آموزش مدل یادگیری ماشین به سرعت دسترسی دهند. همچنین برای بررسی داده‌ها مفید است. اگر یک کاربر نهایی از شما درخواست کند که داده منبع را در یک DW کپی کنید، می‌توانید به سرعت داده منبع را به دیتا لاک کپی کنید و آن را برای اطمینان از ارزش آن قبل از اینکه تلاش خود را برای ایجاد schema در پایگاه داده و نوشتن ETL انجام دهید، بررسی کنید.

من در فصل ٤ اشاره کردم که معمولاً با یک DW، شما دارای یک پنجره نگهداری در شب هستید که در آن کاربران نهایی را برای بارگذاری داده منبع به جداول مرحله‌ای DW از سیستم حذف می‌کنید. سپس داده را تمیز می‌کنید و آن را به جداول تولید DW کپی می‌کنید. مشکل این است که اگر با مشکلات مواجه شدید و نگهداری بیش از انتظار طول بکشد، وقتی کاربران نهایی در صبح سعی می‌کنند به داده دسترسی پیدا کنند، آن‌ها قفل شده‌اند. گزینه بهتری این است که داده را در دیتا لاک تبدیل کنید به جای تبدیل آن در DW. مزایای این شامل موارد زیر است:

- صرفه‌جویی در هزینه، زیرا محاسبات DW معمولاً بسیار گران‌تر از سایر انواع محاسباتی هستند که می‌توانید روی داده‌ها در دیتا لاک استفاده کنید.

- عملکرد بی‌نظیر برای تبدیلات (اگر لازم باشد)، با داشتن گزینه‌های محاسباتی متعدد که هر یک می‌تواند به پوشه‌های مختلف دارای داده دسترسی داشته باشد و به صورت موازی (یا به صورت موازی روی همان داده) اجرا شود.

- توانایی برای تنظیم داده به طور بسیار بیشتری نسبت به DW به دلیل امکان استفاده از گزینه‌های متعددی از محاسباتی روی داده‌ها در دیتا لاک. به عنوان مثال، می‌توانید از کتابخانه‌های کد با روتین‌های از پیش ساخته شده برای انجام تبدیلات پیچیده استفاده کنید که بسیار سخت یا غیرممکن است با استفاده از SQL در DW انجام شود.

- نیازی به پنجره نگهداری DW نیست، که این امکان را به شما می‌دهد تا DW را برای پرس و جوها ۷/۲٤ استفاده کنید. همچنین کاربران را از رقابت برای منابع محاسباتی یکسان در هنگام اجرای گزارشات در حالی که تبدیلات داده در حال اجرا هستند، جلوگیری می‌کنید. از آنجا که تبدیلات داده مصرف‌کنندگی منابع قابل توجهی دارند، می‌توانند هم یکدیگر را به طور قابل توجهی کندرت کنند.

یـک دیتـا لاک راهی ارزان بـرای ذخـیره کـردن مقـدار نامحـدودی از داده اسـت. بـرخلاف ذخیره‌سـازی در محـل، ارائه‌دهنـدگان ابـری ذخیره‌سـازی بی‌نهایـت دارنـد، بنـابراین شـما هرگـز نیـازی بـه نگـرانی از اینکـه فضـای خـالی در دیتـا لاک شـما تمـام شـود، ندارید. عـلاوه بـر ایـن، ذخیره‌سـازی ابـری نسـبتاً ارزان اسـت و بیشـتر ارائه‌دهنـدگان ابـری دسـته‌های مختلفی از ذخیره‌سازی را فراهم می‌کنند که به مشتریان آن‌ها اجازه می‌دهد هزینه‌های بیشتری را صرف کنند.

دیتـا لاک بـه شـما اجـازه می‌دهـد تـا هـر نـوع داده‌ای "فقـط بـرای مـواردی کـه ممکـن اسـت در آینـده نیـاز داشـته باشـید" را جمع‌آوری کنیـد. ایـن جمع‌آوری داده‌هـا بـه نـدرت بـا یـک DW انجـام می‌شـود زیـرا ذخیره‌سـازی در یـک DW معمـولاً بسـیار گران‌تـر از ذخیره‌سـازی در یـک دیتـا لاک اسـت و نگهـداری بیشـتر داده در DW می‌توانـد بـر عملکـرد تـأثیر بگـذارد. و بـه دلیل اینکه ذخیره‌سازی در یک دیتا لاک بسیار ارزان است، داده‌ها به ندرت حذف می‌شوند مگر برای دلایل قانونی.

بـه عنـوان مکان مرکـزی بـرای تمـام موضـوعات داده، یـک دیتـا لاک می‌توانـد "یـک نسـخه واحـد از حقیقـت" باشـد. اگـر هـر نـوع داده‌ای قبـل از اینکـه بـه جـای دیگـری کـپی شـود، در دیتـا لاک قـرار بگـیرد، دیتـا لاک می‌توانـد مکانی باشـد کـه بـه آن برگردیـد اگـر در مـورد دقـت داده‌هـا در هـر یـک از مکان‌هـای دیگـری کـه در آن‌هـا اسـتفاده می‌شـود، ماننـد پـرس و جوهـا، گزارشـات و داشـبوردها، سـوالات وجـود داشـته باشـد. یـک دیتـا لاک همچنـین بـه شـما اجـازه می‌دهـد تـا هـر کاربر نهـایی از هـر مکـانی دسـترسی بـه هـر نـوع داده داشـته باشـد و آن را بـرای نیازهـای تحلـیلی و مـوارد اسـتفاده مجـدداً اسـتفاده کنـد. وقـتی شـما یـک دیتـا لاک را بـا یـک DW همـراه اسـتفاده می‌کنیـد، دیتـا لاک بـه جـای DW بـه عنـوان "یـک نسـخه واحـد از حقیقـت" تبـدیل می‌شـود. عـلاوه بـر ایـن، یـک دیتـا لاک مکـانی اسـت کـه می‌تـوان داده‌هـای جریـانی، ماننـد از دسـتگاه‌های IoT، را در آن قرار داد که تقریباً امکان‌پذیر نیست در DW قرار گیرد.

دیتـا لاک همچنـین بـه عنـوان یـک آرشـیو آنلایـن عمـل می‌کنـد. بـه عنـوان مثـال، می‌توانیـد داده‌هـای سـفارش سـال‌های آخـر را در DW نگهـداری کنیـد و داده‌هـای سـفارش قـدیمی‌تر از سـه سـال در دیتـا لاک نگهـداری کنیـد. بیشـتر فناوری‌هـای مـورد اسـتفاده بـرای سـاخت DW بـه شـما اجـازه می‌دهـد کـه بـا اسـتفاده از SQL داده‌هـا را در دیتـا لاک پـرس و جـو کنیـد، بنـابراین کـاربران هنـوز می‌تواننـد داده‌هـا را در دیتـا لاک در گزارشـات اسـتفاده کننـد (بـا تعـویض عملکـرد کنـدتر). ایـن می‌تواند هزینه‌ها را صرفه‌جویی کند و از فضای ناکافی در DW جلوگیری کند.

عـلاوه بـر ایـن، شـما می‌توانیـد داده‌هـای بـا سـاختارهای مختلـف را در یـک دیتـا لاک یکپارچـه کنیـد: از فایل‌هـای CSV و JSON تـا اسـناد Word و فایل‌هـای رسـانه‌ای. می‌تـوان داده را از ایـن فایل‌هـا اسـتخراج کـرد تـا بـه شـما ارزش بیشـتری بدهد.

بـا یـک پایگاه داده رابطـه‌ای (DW)، داده‌هـای خـامی کـه از یـک سیسـتم منبـع بـه جـداول مرحلـه‌ای در DW کـپی می‌شـوند، سـپس تبـدیل می‌شـوند و بـه جـداول تولیـد بارگـذاری می‌شـوند، معمـولاً پـس از یـک روز یـا دو حـذف می‌شـوند تـا فضـای محـدود ذخیره‌سـازی رابطـه‌ای را صرفـه‌جویی کننـد. مشـکلی ایجـاد می‌شـود اگـر یـک خطـای ETL از یـک اجـرای ETL چنـدین روز پـیش کشـف شـود؛ اگـر ETL نیـاز بـه اجـرای مجـدد داشـته باشـد امـا داده‌هـای خـام حـذف شـده‌اند، شـما بایـد بـه سیسـتم منبـع برگردیـد و احتمـالاً عملکـرد آن را تحـت تـأثیر قـرار دهیـد. بـا انجـام تبـدیل در دیتـا لاک، جـایی کـه ذخیره‌سـازی ارزان‌تـر اسـت، می‌توانیـد تاریخچـه طـولانی از داده‌هـای خـام را حفـظ کنیـد و هرگـز نیـازی بـه بازگشـت بـه سیستم منبع ندارید.

رویکرد پایین به بالا

بـه دلیـل اینکـه یـک دیتـا لاک schema-on-read اسـت، کمـترین کـار اولیـه لازم اسـت تـا شروع بـه اسـتفاده از داده کنیـد. ایـن کـار بسـیار خـوب کـار می‌کنـد اگـر هنـوز نمی‌دانیـد کـه چـه سـوالاتی را از داده مطـرح کنیـد - شـما می‌توانیـد بـه سـرعت داده را بـررسی کنیـد تـا سـوالات مـرتبط را بـه بیـان برسـانید. ایـن منجـر بـه آن می‌شـود کـه بـه آن بـه عنـوان رویکـرد پـایین بـه بـالا اشـاره می‌شـود، همـانطور کـه در شـکل ۵-۲ مشـاهده می‌شـود، جـایی کـه شـما داده را قبـل از تولیـد هرگونـه نظریـه یـا فرضـیه جمع‌آوری می‌کنیـد. ایـن بسـیار متفـاوت از رویکـرد بـالا بـه پـایین یـک پایگاه داده رابطـه‌ای اسـت، همـانطور کـه در شـکل ۵-۱ مشاهده می‌شود.

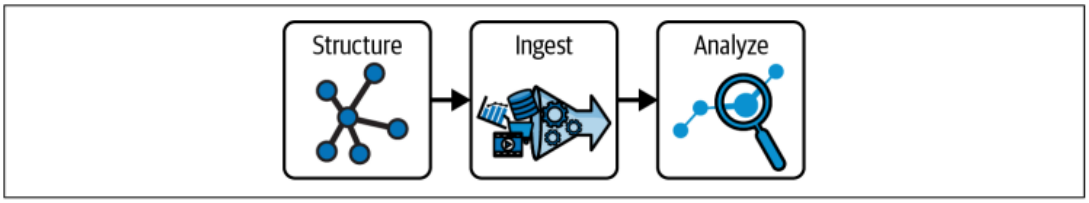


Figure 5-1. Top-down approach

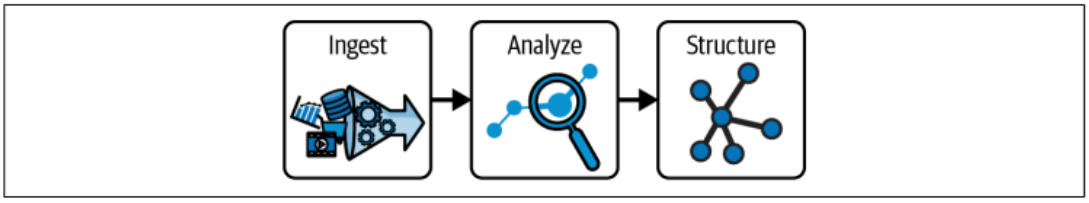


Figure 5-2. Bottom-up approach

این امکان را به دانشمندان داده، که معمولاً از نرم‌افزارهایی استفاده می‌کنند که ترجیح می‌دهند داده را در فایل‌ها نگهداری کنند، می‌دهد تا با استفاده از یادگیری ماشین بر روی داده‌ها در دیتا لاک تصمیم بگیرند که چه اتفاق رخ خواهد داد (تحلیل پیش‌بینی) و چگونه می‌توان این اتفاق را ایجاد کرد (تحلیل توصیه‌دهنده). تحلیل پیش‌بینی از داده‌ها، الگوریتم‌های آماری و تکنیک‌های یادگیری ماشین برای پیش‌بینی نتایج آینده بر اساس داده‌های تاریخی استفاده می‌کند. این شامل تکنیک‌های آماری متنوعی مانند استخراج داده‌ها، مدل‌سازی پیش‌بینی و یادگیری ماشین است تا واقعیت‌های فعلی و تاریخی را برای پیش‌بینی درباره رویدادهای آینده یا دیگر ناشناخته تحلیل کند. این به شما اجازه می‌دهد تا پیش‌بینی کنید، نه فقط پاسخگویی. به عنوان مثال، تحلیل پیش‌بینی می‌تواند در بهداشت برای پیش‌بینی نرخ بازگشت بیماران، در بازاریابی برای پیش‌بینی فروش آینده یا در بانکداری برای پیش‌بینی عفو وام استفاده شود.

تحلیل توصیه‌دهنده یک مرحله بعدی از تحلیل پیش‌بینی است. این از الگوریتم‌های بهینه‌سازی و شبیه‌سازی برای مشاوره در مورد نتایج ممکن استفاده می‌کند. تحلیل توصیه‌دهنده نه تنها پیش‌بینی می‌کند که چه اتفاق در آینده رخ خواهد داد، بلکه نیز عملیات را پیشنهاد می‌کند تا این نتایج را تأثیر بگذارد. هدف این است که بر اساس سناریوهای آینده پیش‌بینی شده مشاوره بدهد تا تصمیم‌گیری را بهینه‌سازی کند. تحلیل توصیه‌دهنده می‌تواند گزینه‌های تصمیم‌گیری را در مورد چگونگی استفاده از فرصت آینده یا کاهش خطر آینده پیشنهاد دهد و می‌تواند پیامدهای هر گزینه تصمیم را نشان دهد. به عنوان مثال، در لجستیک، تحلیل توصیه‌دهنده می‌تواند برای پیدا کردن بهترین مسیرهای تحویل استفاده شود و حتی مسیرهای جایگزین را در صورت بستن ناگهانی جاده‌ها پیشنهاد دهد.

دیتا لاک‌ها ابتدا برای تحلیل‌های پیش‌بینی و توصیه‌دهنده استفاده شدند تا از مشکلات انجام تحلیل‌های پیشرفته با پایگاه داده رابطه‌ای سنتی جلوگیری کنند. اما حالا دیتا لاک‌ها برای بسیاری از کاربردهای دیگر استفاده می‌شوند، همانطور که در "چرا از یک دیتا لاک استفاده می‌کنیم؟" در صفحه ۶۰ نشان داده شده است.

اگر شما در هنگام بررسی دیتا لاک داده‌های ارزشمندی پیدا کرده و می‌خواهید آن‌ها را به راحتی برای کاربران نهایی قابل دسترسی کنید، می‌توانید همیشه آن‌ها را با کپی کردن آن‌ها به یک RDW بعداً مدل‌سازی کنید. مدل‌سازی داده مانند ایجاد طرح برای سازماندهی و درک داده است. این کمک می‌کند تا تعیین کنید که کدام داده‌ها مهم هستند، چگونه با هم مرتبط هستند و چگونه باید در RDW ذخیره و استفاده شوند. دیتا لاک‌ها ابزارهای محدودی برای مدل‌سازی داده دارند، در حالی که DW ها از سال‌ها پیش ابزارهای مدل‌سازی داشته‌اند.

بهترین روش ها برای طراحی دریاچه داده

طراحی یک دیتا لاک باید یک فرآیند مصرف زمان باشد. من اغلب متوجه می‌شوم که شرکت‌ها کافی زمان را برای فکر کردن در مورد تمام موارد استفاده خود هنگام طراحی دیتا لاک‌های خود ندارند و در نهایت باید آن‌ها را بازطراحی و بازساخت کنند. بنابراین، مطمئن شوید که در تمام منابع داده‌ای که حالا استفاده می‌کنید و که در آینده استفاده خواهید کرد، فکر کرده‌اید و توانایی درک اندازه، نوع و سرعت داده را دارید. سپس همه اطلاعاتی که می‌توانید در مورد طراحی دیتا لاک پیدا کنید را می‌گیرید و طراحی مناسب برای شرایط خود را انتخاب می‌کنید.

یک دیتا لاک به طور کلی ساختار خاصی را بر روی داده‌هایی که می‌گیرد اعمال نمی‌کند؛ در واقع، این یکی از ویژگی‌های کلیدی آن است. این متفاوت از یک پایگاه داده سنتی یا دیتا وارهاس است که نیاز دارد داده‌ها قبل از اینکه مدل‌سازی یا ساختاردهی شوند. با این حال، برای ایجاد استفاده از داده و جلوگیری از تبدیل شدن دیتا لاک به "مرطوب داده" (یک مجموعه ناسازمانی و غیرقابل مدیریت از داده‌ها)، مهم است که برخی از روش‌های سازماندهی و حاکمیت را اعمال کنید. این بخش برخی از بهترین روش‌ها را معرفی می‌کند تا شما را شروع کند.

بهترین روش اول این است که به طور منطقی دیتا لاک را به چندین لایه (همچنین نامیده می‌شوند مناطق) تقسیم کنید که با سطوح افزایشی ـ کیفیت داده متناسب هستند، همانطور که در شکل ۵-۳ نشان داده شده است. در غیر این صورت، شما تمام فایل‌های خود را در یک پوشه داشته خواهید بود، که باعث می‌شود عملکرد، قابلیت مدیریت و امنیت به شدت تحت تأثیر قرار بگیرد.

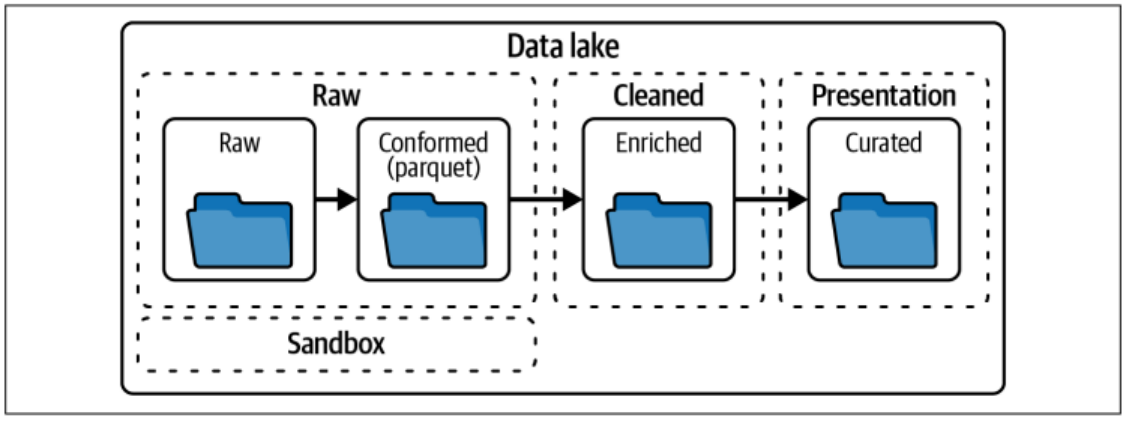


Figure 5-3. Data lake layers

برای درک بهتر ساختار و کارکرد یک دیتا لاک، بیایید به طور دقیق به هر یک از لایه‌های مختلف آن نگاه کنیم، هر کدام نمایانگر یک مرحله در بهبود کیفیت و قابلیت استفاده از داده است. این لایه‌ها در یک سلسله مراتبی ترتیب داده شده‌اند، از داده‌های خام و بدون پردازش به اطلاعات بسیار تمیز و آماده برای کسب و کار حرکت می‌کنند. در اینجا نگاهی به هر لایه می‌اندازیم:

لایه خام (Raw layer)

رویدادهای خام، که برای مرجع تاریخی نگهداری می‌شوند، معمولاً برای همیشه نگهداری می‌شوند (غیرقابل تغییر). لایه خام را به عنوان یک حوضچه در نظر بگیرید که داده‌ها را در حالت طبیعی و اصلی خود (بدون اعمال هیچ تبدیلی) نگهداری می‌کند. این لایه فیلتر نشده و تمیز نیست. این لایه همچنین به عنوان لایه نقره‌ای، لایه مرحله‌ای، یا منطقه نقل و انتقال شناخته می‌شود.

لایه مطابقت (Conformed layer)

بسیاری از اوقات، داده‌ها در لایه خام به انواع مختلفی مانند JSON، CSV، و Parquet (یک فرمت فایل ذخیره‌سازی متداول برای پردازش بزرگ داده بهینه‌سازی شده) ذخیره می‌شوند. لایه مطابقت جایی است که تمام انواع فایل به یک فرمت، معمولاً Parquet، تبدیل می‌شوند. فکر کنید که دیتا لاک دارای دو حوضچه است، یکی از آب مرطوب و دیگری از آب شیرین، و استفاده از فرآیند تصفیه برای تبدیل آب مرطوب به آب شیرین. این لایه همچنین به عنوان لایه پایه یا لایه استاندارد شناخته می‌شود.

لایه پاک‌شده (Cleansed layer)

در اینجا، رویدادهای خام تبدیل می‌شوند (داده پاک می‌شود، یکپارچه می‌شود و ترکیب می‌شود) به مجموعه‌های داده‌ای که مستقیماً قابل مصرف هستند. لایه پاک‌شده را به عنوان یک لایه فیلتراسیون در نظر بگیرید. این لایه آلودگی‌ها را حذف می‌کند و ممکن است داده را غنی‌سازی کند. هدف این است که فایل‌های ذخیره شده در مورد رمزگذاری، ساختار، فرمت، انواع داده، و محتوا (مانند رشته‌ها و اعداد صحیح) یکنواخت باشند. این لایه همچنین به عنوان لایه نقره، تبدیل شده، تمیز شده، یادگیری ماشین، پردازش شده، یا غنی‌سازی شده شناخته می‌شود.

لایه ارائه (Presentation layer)

منطق تجاری بر روی داده‌های پاک‌شده اعمال می‌شود تا داده‌هایی تولید شوند که آماده مصرف توسط کاربران نهایی یا برنامه‌ها هستند، معمولاً در یک فرمتی که قابل فهم و استفاده است. تبدیل ممکن است شامل تجمع یا خلاصه‌سازی باشد، یا ممکن است به معنای قرار دادن فایل‌ها در یک طرح خاص برای استفاده در ابزارهای گزارش‌گیری (مانند ساختار ستاره، که در فصل ۸ مورد بحث قرار گرفته است) باشد، اغلب شامل اطلاعات در مورد داده (متادیتا) درون هر فایل. این لایه همچنین به عنوان لایه برنامه، فضای کاری، معتبر، طلایی، آماده برای تولید، مدیریت شده، مراقبت شده، خدمت دهنده، تحلیل، یا مصرف شناخته می‌شود.

لایه ساندباکس (Sandbox layer)

لایه ساندباکس (Sandbox layer) یک لایه اختیاری است که برای "بازی کردن" استفاده می‌شود. به طور کلی، ساندباکس توسط دانشمندان داده استفاده می‌شود. این معمولاً یک کپی از لایه خام است که در آن داده نه تنها می‌توان آن را خواند، بلکه می‌توان آن را تغییر داد. شما ممکن است چندین لایه ساندباکس ایجاد کنید. این لایه همچنین به عنوان لایه بازنمایی، لایه توسعه، یا فضای کاری دانش محاسباتی شناخته می‌شود.

این فقط یک روش برای تنظیم لایه‌ها در دیتا لیک شما است. من طراحی‌های بسیار خوب دیگری را دیده‌ام که لایه‌های اضافی را شامل می‌شوند یا برخی از لایه‌ها را ترکیب می‌کنند، مانند لایه‌های مطابقت داده شده و پاکسازی شده.

برای یک مثال با استفاده از لایه‌ها در شکل ۵-۳، بیایید یک شرکت فروشگاهی را در نظر بگیریم که منابع داده‌ای متفاوتی مانند سیستم‌های نقد و برداشت (POS)، فروش آنلاین، بازخورد مشتریان و سیستم‌های موجودی دارد. چهار لایه به شرح زیر داده‌های مختلف کیفیت را شامل می‌شوند:

- لایه خام

- لاگ‌های خام از سیستم POS، از جمله هر جزئیات معامله (مانند زمان برچسب، موارد خریداری شده، تعداد، قیمت‌ها، مبلغ کل، شناسه صندوق‌دار، و مکان فروشگاه)

- داده‌های فروش آنلاین از وب‌سایت یا برنامه، مانند شناسه کاربر، موارد خریداری شده، تعداد، قیمت‌ها، مبلغ کل، و زمان‌بندی‌ها

- داده‌های موجودی از انبارها و فروشگاه‌های مختلف، از جمله موارد، تعداد، و تاریخ‌های بازسازی

- بازخورد مشتریان خام که از طریق نظرسنجی‌ها، بررسی‌ها، و امتیازدهی‌ها جمع‌آوری شده است

- لایه مطابقت داده شده

- تمام فایل‌های POS، فروش آنلاین، موجودی، و بازخورد مشتریان به فرمت Parquet تبدیل می‌شوند اگر در این فرمت نباشند.

- لایه پاکسازی شده

- داده‌های POS و فروش آنلاین با حذف یا اصلاح هر گونه خطا (مانند ناهماهنگی در نامگذاری موارد، خطاها در تعداد، و زمان‌بندی‌های گم شده). داده‌ها همچنین به یک ساختار مشترک تبدیل شده‌اند (برای مثال، قوانین نامگذاری مشترک و فرمت‌ها برای موارد، فرمت زمان مشترک، یا سیستم شناسه فروشگاه مشترک).

- داده‌های موجودی با استانداردسازی در نامگذاری موارد و با اصلاح خطاها یا ناهماهنگی‌ها. داده‌ها همچنین برای هماهنگی با نام‌های موارد مشترک و شناسه‌های فروشگاه استفاده شده در داده‌های فروش تبدیل شده‌اند.

- داده‌های بازخورد مشتریان که پاکسازی شده‌اند تا پاسخ‌های غیرمرتبط یا اشتباه حذف شوند، با فرمت‌های استاندارد، و غیره. داده‌ها همچنین به یک فرمت مشترک تبدیل شده‌اند، هماهنگ با سیستم شناسه فروشگاه مشترک و شاید با عناصر مشترک استخراج شده از بازخورد برای تحلیل.

- لایه ارائه

- گزارش فروش متمرکز نشان دادن فروش کل بر اساس فروشگاه، منطقه، یا روز/ماه/سال، و شاید فروش تقسیم بر اساس مورد یا دسته

- گزارش موجودی نشان دادن سطوح موجودی فعلی برای هر مورد در هر فروشگاه یا انبار، به علاوه برنامه‌های بازسازی

- گزارش بازخورد مشتریان خلاصه‌ای از بازخورد، شاید با امتیاز تحلیل احساسات برای هر فروشگاه یا محصول

یکی دیگر از بهترین روش‌ها ایجاد یک ساختار پوشه است، که معمولاً برای هر لایه متفاوت است و می‌تواند به دلایل مختلف به صورت متفاوتی تقسیم شود. در زیر چند مثال از این موارد آورده شده است:

- جداسازی داده‌ها

- سازماندهی داده‌ها بر اساس منبع، واحد کسب و کار، یا نوع داده باعث می‌شود تا دانشمندان داده و تحلیلگران بتوانند به راحتی داده‌های مرتبط را پیدا و استفاده کنند.

- کنترل دسترسی

- تیم‌ها یا افراد مختلف در یک سازمان ممکن است دسترسی‌های مختلفی به داده‌ها داشته باشند. با سازماندهی پوشه‌ها بر اساس نقش‌های کاربری یا دپارتمان‌ها، سازمان‌ها می‌توانند سیاست‌های کنترل دسترسی با دقت بالا را اجرا کنند.

- بهینه‌سازی عملکرد

- سازماندهی داده‌ها در یک روش خاص می‌تواند منجر به بهبود عملکرد شود. برای مثال، برخی عملیات پردازش یا پرس و جو داده ممکن است سریع‌تر باشند اگر داده‌ها بر اساس ویژگی‌های خاص گروه‌بندی شوند.

- مدیریت چرخه عمر داده

- داده‌ها اغلب یک چرخه عمر دارند، از جمله ورود به سیستم تا آرشیو یا حذف. پوشه‌ها مختلف می‌توانند برای جداسازی داده‌ها بر اساس مرحله آن‌ها در چرخه عمر استفاده شوند.

- مدیریت متادیتا

- پوشه‌ها می‌توانند برای مدیریت و جداسازی متادیتا از داده‌های خام استفاده شوند. این جداسازی می‌تواند مدیریت متادیتا را ساده‌تر کند و کشف داده‌ها را تسریع کند.

مشخصات مطابقت

در بسیاری از صنایع، مشخصات مطابقت مشخص می‌کند که برخی از انواع داده‌ها باید در روش‌های خاصی ذخیره و مدیریت شوند. ساختارهای پوشه مختلف می‌توانند به سازمان‌ها کمک کنند تا این مشخصات را برآورده کنند.

پشتیبان‌گیری و بازیابی بعد از فاجعه

خلاصه - دکتر کتابی از معماری داده

داشتن ساختارهای پوشه مختلف می‌تواند در ایجاد یک برنامه پشتیبان‌گیری استراتژیک و بازیابی بعد از فاجعه کمک کند. برخی پوشه‌ها ممکن است با فرکانس بیشتری پشتیبان گرفته شوند یا برای مدت طولانی‌تری نگهداری شوند بر اساس اهمیت آن‌ها.

نسخه‌بندی داده‌ها

پوشه‌های مختلف می‌توانند برای مدیریت نسخه‌های مختلف همان مجموعه داده استفاده شوند.

جداسازی داده‌ها

داده‌ها می‌توانند بر اساس ویژگی‌های کلیدی جداسازی شوند تا عملکرد پرس و جو سریع‌تر شود.

نیازهای ورود و پردازش

بر اساس منبع، داده‌ها ممکن است نیاز به خطوط پردازش مختلف داشته باشند. پوشه‌های مختلف می‌توانند در مدیریت و بهینه‌سازی این فرآیندها کمک کنند.

برای برآورده کردن یک یا چند دلیل فوق، شما می‌توانید پوشه‌ها را به یک یا چند از این روش‌ها تقسیم کنید:

- جداسازی زمانی (سال/ماه/روز/ساعت/دقیقه)، مانند زمانی که داده‌ها به دیتا لیک کپی شدند

- حوزه موضوعی

- منبع داده

- شیء، مانند یک جدول منبع

- مرزهای امنیتی، مانند دپارتمان یا حوزه کسب و کار

- برنامه یا هدف دانشگاهی

- نوع داده، مانند جزئیات یا خلاصه

- سیاست نگهداری داده، مانند داده‌های موقت و دائم، دوره اعتبار یا عمر پروژه

- تأثیر تجاری یا حیاتی، مانند بالا، متوسط، یا پایین

- مالک/مربی/کارشناس موضوع

- احتمال دسترسی به داده‌ها، مانند داده‌های اخیر یا فعلی در مقابل داده‌های تاریخی

- طبقه‌بندی محرمانه، مانند اطلاعات عمومی، فقط برای استفاده داخلی، محرمانه تامین‌کننده/همکار، اطلاعات قابل شناسایی شخصی، یا اطلاعات حساس

شکل ٥-٤ یک مثال از ساختار پوشه برای مناطق خام و پاکسازی شده نشان می‌دهد.

جزوه - رمز گشایی از معماری داده

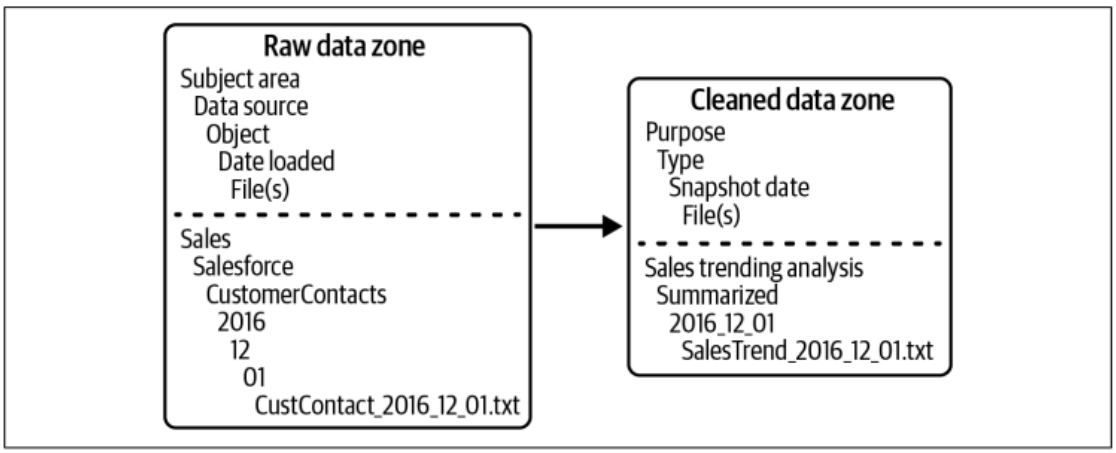


Figure 5-4. Folder structure in two data lake zones

بیشتر اوقات، همه این لایه‌ها تحت یک اشتراک ابری قرار دارند. برخی استثناها وجود دارد، مانند اگر شما نیاز خاصی به مدیریت حسابداری داشته باشید، اگر به حد اشتراک خود برخورد کنید، یا اگر می‌خواهید اشتراک‌های جداگانه برای توسعه، تست، و تولید داشته باشید. (برای استثناهای بیشتر به بخش بعدی مراجعه کنید.)

بیشتر مشتریان یک حساب ذخیره‌سازی برای هر لایه ایجاد می‌کنند (بنابراین سه لایه به معنی سه حساب ذخیره‌سازی است)، همه در یک گروه منابع مرکزی (یک کانتینر که منابع مرتبط برای یک راه حل را نگهداری می‌کند). این کار باعث می‌شود لایه‌ها به طور جداگانه قرار گیرند تا عملکرد قابل پیش‌بینی‌تر شود و امکان استفاده از ویژگی‌ها و کارکردهای مختلف در سطح حساب ذخیره‌سازی را فراهم کند.

ویژگی‌های معمولی که در هر سطح حساب ذخیره‌سازی تنظیم می‌شوند و که توسط بیشتر ارائه‌دهندگان ابری (در این بخش من به مورد Azure مشخص هستم) در دسترس هستند، شامل مدیریت چرخه عمر است که به شما اجازه می‌دهد تا هزینه‌ها را با اتوماسیون وظایف مدیریت داده‌ها (مانند انتقال داده‌ها در طول لایه‌های ذخیره‌سازی مختلف یا حذف داده‌ها وقتی که دیگر نیازی به آن‌ها ندارید) کاهش دهید. شما همچنین می‌توانید قوانین فایروال را تنظیم کنید تا فقط به گروه‌های مورد اعتماد یا افراد خاص دسترسی را اجازه دهید، یا برای جلوگیری از رسیدن به حد محدودیت ذخیره‌سازی یا ترافیک حساب خود.

بیشتر دیتا لیک‌ها از سطوح دسترسی ابر استفاده می‌کنند، با لایه‌های خام و مطابقت داده شده استفاده از سطح آرشیو، لایه پاکسازی شده استفاده از سطح سرد، و لایه‌های ارائه و ساندباکس استفاده از سطح گرم. هر سطح یک تعادل مختلف بین هزینه ذخیره‌سازی و هزینه دسترسی را ارائه می‌دهد، با سطح گرم برای ذخیره‌سازی بسیار گران قیمت و سطح آرشیو با هزینه بازیابی بالاترین قیمت.

من توصیه می‌کنم که برای اطمینان از دقت داده‌ها هنگام حرکت از طریق لایه‌ها، بررسی یا بررسی یکپارچگی را در نظر بگیرید. برای مثال، اگر با داده‌های مالی کار می‌کنید، ممکن است یک پرس و جو ایجاد کنید که مجموع سفارشات روز (تعداد سطر و مجموع فروش) را محاسبه کند و مقادیر را با داده منبع مقایسه کند تا مطمئن شوید که این مقادیر در تمام لایه‌های داده برابر هستند. اگر برابر نباشند، این نشان می‌دهد که مشکلی با خطوط ایجاد شده توسط شما برای حرکت و تبدیل داده‌ها وجود دارد. شما باید مشکل را حل کنید و خطوط را دوباره اجرا کنید.

دریاچه داده چندگانه

به طور ایده‌آل، شما فقط یک دیتا لیک بزرگ ایجاد می‌کنید و برای تمام داده‌های خود از آن استفاده می‌کنید. این رویکرد باعث می‌شود پیدا کردن و ترکیب داده‌ها برای پرس و جوها یا گزارشات ساده‌تر شود. اما بسیاری از دلایلی وجود دارد که شما ممکن است نیاز به ایجاد چندین دیتا لیک جداگانه در مقیاس فیزیکی داشته باشید (همانطور که در شکل ۵-۵ نشان داده شده است) به جای تقسیم یک دیتا لیک به بخش‌های مختلف. بیشتر موارد استفاده که در مورد آن‌ها صحبت می‌کنم نمی‌تواند انجام شود (یا حداقل مناسب نیست) مگر اینکه دیتا لیک‌ها جداگانه فیزیکی باشند.

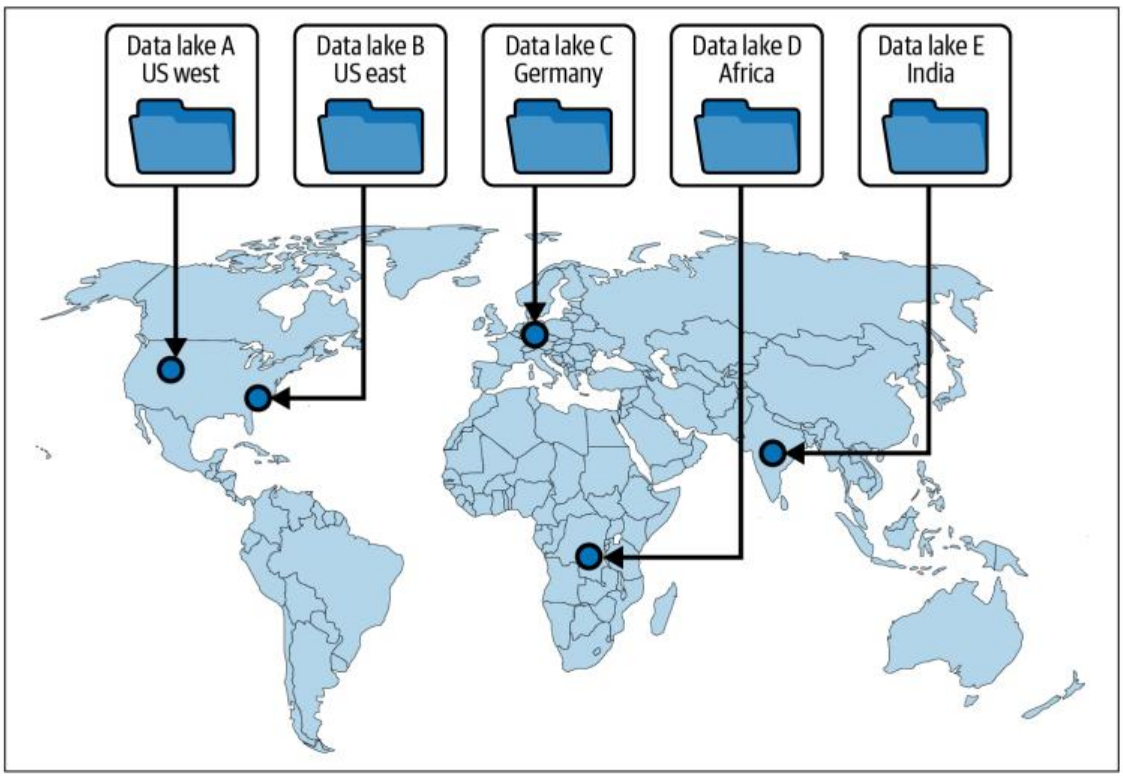


Figure 5-5. Separate data lakes across the world

مزایا

من دلایل ایجاد دیتا لیک‌های جداگانه در مقیاس فیزیکی را به پنج بخش گروه‌بندی کردم، سپس در مورد معایب داشتن چندین دیتا لیک صحبت خواهم کرد.

ساختار سازمانی و مالکیت

حفظ چندین دیتا لیک می‌تواند به دلایل متعددی مفید باشد، که بیشتر آن‌ها به ساختار سازمانی و مالکیت مرتبط هستند. برای مثال، ساختار سازمانی شرکت ممکن است تشویق یا نیاز به حفظ مالکیت داده‌های خود توسط هر واحد سازمانی را داشته باشد، یک ویژگی که معمولاً با یک شبکه داده (مشاهده کنید به فصل ۱۳) مرتبط است.

دلیل دیگری این است که تیم‌ها یا دپارتمان‌های مختلف ممکن است نیاز به دیتا لیک‌های خود متمایز برای موارد استفاده خاص یا پروژه‌های خاص داشته باشند.

علاوه بر این، یک شرکت ممکن است از داشتن یک دیتا لیک مطابقت داده و یک دیتا لیک مطابقت مصرف‌کننده بهره ببرد. داده‌ها از یک منبع خاص می‌توانند به یک دیتا لیک با تبدیلات کم جمع‌آوری شوند. این کار برای کاربرانی که داده‌های منبع را درک می‌کنند، مانند کسانی که در تولید هستند و داده‌ها از یک فرآیند تولید منبع می‌شوند، خوب کار می‌کند؛ با این حال، کسانی که خارج از تولید هستند ممکن است داده‌ها را دشوار بیابند. در چنین مواردی، شما ممکن است داده‌ها را به یک دیتا لیک مطابقت مصرف‌کننده کپی کنید و آن‌ها را تبدیل کنید تا آن‌ها را قابل فهم‌تر کنید. بنابراین، داشتن دیتا لیک‌های جداگانه برای داده‌های مطابقت منبع و مصرف‌کننده به جای یک دیتا لیک می‌تواند فرآیند مدیریت داده‌ها را ساده‌تر کند.

مشخصات مطابقت، حاکمیت، و امنیت

تصمیم برای داشتن چندین دیتا لیک به جای یکی به طور قابل توجهی تحت تأثیر عوامل متنوعی است که تحت دسته‌های گسترده‌ای از مطابقت، حاکمیت، و امنیت قرار دارند. برای مثال، نصب‌های چند منطقه‌ای ممکن است نیاز به چندین دیتا لیک داشته باشند وقتی نیازهای مقیمت یا استقلال داده‌ها در مناطق مختلف متفاوت است. برای مثال، داده‌های منشأ از چین نمی‌توانند خارج از کشور صادر شوند، بنابراین نیاز به یک دیتا لیک منحصر به فرد در منطقه وجود دارد.

یک در نظر گرفتن دیگر حیاتی این است که چندین دیتا لیک به شما اجازه می‌دهد تا داده‌های حساس یا محرمانه را از داده‌های کمتر حساس جدا کنید. کنترل‌های امنیتی محدودتر و سخت‌گیرانه‌تر می‌توانند به طور خاص به دیتا لیک‌های حساس اعمال شوند، که این کار به بهبود کلی امنیت داده‌ها کمک می‌کند.

اگر برخی افراد دسترسی‌های بالاتر دارند، دیتا لیک‌های جداگانه به شما اجازه می‌دهد تا محدوده این دسترسی‌ها را محدود کنید، آن‌ها را به دیتا لیک خاصی که در آن فرد کار می‌کند محدود کنید. این کار باعث می‌شود محیط داده‌ای امن‌تری ایجاد شود.

در نهایت، متنوع بودن نیازهای حاکمیت و مطابقت می‌تواند یک انگیزه برای داشتن چندین دیتا لیک باشد. مقررات مانند قانون حفظ داده‌های عمومی (GDPR) و قانون قابلیت حمل و نقل بیمه سلامت (HIPAA) نیازمند استانداردها و روش‌های مدیریت داده‌های منحصر به فرد هستند. با داشتن چندین دیتا لیک، سازمان‌ها می‌توانند داده‌های خود را بر اساس نیازهای حاکمیت و مطابقت خاص که برای هر مجموعه داده اعمال می‌شود، مدیریت کنند، یک روش که به خصوص در صنایع بسیار مرتبط با قوانین است.

اشتراک ابری، محدودیت‌های سرویس و سیاست‌ها

داشتن چندین دیتا لیک همچنین به دلایل اصلی مرتبط با اشتراک ابری، محدودیت‌های سرویس و سیاست‌ها می‌تواند مفید باشد. برای مثال، داشتن چندین دیتا لیک می‌تواند به شما کمک کند تا محدودیت‌ها، کوتاه‌ها، و محدودیت‌های ارائه شده توسط ارائه‌دهندگان ابری، مانند یک سر بر تعداد حداکثری حساب‌های ذخیره‌سازی در هر اشتراک، را دور بزنید. در چنین سناریوهایی، اگر شما تنها با یک دیتا لیک عمل کنید، نیازهای شما ممکن است از این محدودیت‌ها بیشتر باشد.

علاوه بر این، دیتا لیک‌های چندگانه به شما اجازه می‌دهد تا سیاست‌های ابری متمایز را اجرا کنید. ارائه‌دهندگان ابری معمولاً صدها گزینه سیاست متفاوت را ارائه می‌دهند، که می‌توانند به طور فردی برای هر دیتا لیک سفارشی شوند. این انعطاف‌پذیری به شما کمک می‌کند تا مطابقت با سیاست‌های شرکت را تضمین کنید. یک مثال ممکن است یک مشخصه باشد که برخی از حساب‌های ذخیره‌سازی رمزگذاری زیرساخت دارند، یک قاعده که از یک دیتا لیک به دیگری در عملیات شما متفاوت است.

در نهایت، می‌تواند ساده‌تر باشد که هزینه‌ها را برای منظورات محاسبه‌ی هزینه‌ها ردیابی کنید وقتی شما دیتا لیک‌های جداگانه را حفظ می‌کنید، هر کدام با اشتراک ابری خود. این سطح از جزئیات در ردیابی هزینه‌ها نسبت به روش‌های جایگزین ارائه شده توسط خدمات ابری، مانند استفاده از برچسب‌ها برای هر منبع، مزیت واضحی دارد.

عملکرد، در دسترس بودن، و بازیابی پس از حوادث

عملکرد، در دسترس بودن، و بازیابی پس از حوادث نیز دلایل فراوانی را برای در نظر گرفتن استفاده از چندین دیتا لیک فراهم می‌کنند. یکی از دلایل این است که تاخیر بهبود یافته است. اگر مجموعه‌ای از کاربران جهانی به یک دیتا لیک دسترسی داشته باشند، کسانی که دور از آن هستند ممکن است خدمات را بسیار آهسته یابند. با قرار دادن یک دیتا لیک در همان منطقه که کاربر نهایی یا برنامه‌ای که داده‌ها را پرس و جو می‌کند قرار دارد، می‌توانید زمان دسترسی به داده‌ها را به طور قابل توجهی کاهش دهید.

حفظ چندین دیتا لیک با کپی‌های داده در مناطق مختلف نیز قابلیت‌های بازیابی پس از حوادث را به طور قابل توجهی افزایش می‌دهد. اگر یک دیتا لیک در یک منطقه قابل دسترسی نباشد، کاربران نهایی می‌توانند به یک منطقه جایگزین که داده‌های مشابهی را نگهداری می‌کند، هدایت شوند. داشتن چندین دیتا لیک همچنین انعطاف‌پذیری را فراهم می‌کند تا استراتژی‌های بازیابی داده و بازیابی پس از حوادث متفاوت را برای انواع مختلف داده اجرا کنید. داده‌های حیاتی که نیاز به در دسترس بودن مداوم دارند می‌توانند در یک دیتا لیک نگهداری شوند که داده‌های آن در چندین دیتا لیک در مناطق مختلف تکرار شده‌اند، در حالی که داده‌های کمتر حیاتی که می‌توانند دوره‌های دسترسی ناپذیری را تحمل کنند، می‌توانند در یک دیتا لیک تنها برای صرفه‌جویی هزینه‌ای قابل توجه نگهداری شوند.

در نهایت، با چندین دیتا لیک، شما می‌توانید سطوح خدمات متفاوتی را برای انواع مختلف داده اجرا کنید. برای مثال، می‌توانید یک دیتا لیک را به نگهداری و پردازش داده‌های اولویت بالا اختصاص دهید، که نیاز به دسترسی با تاخیر پایین و در دسترس بودن بالا دارد، و دیگری را به داده‌های اولویت پایین‌تر اختصاص دهید که در آن‌ها تاخیرهای بالاتر و در دسترس بودن پایین‌تر قابل تحمل هستند. این استراتژی هزینه و عملکرد زیرساخت مدیریت

داده‌های شما را بهینه‌سازی می‌کند با اجازه دادن به شما استفاده از منابع ذخیره‌سازی و پردازش کم هزینه‌تر برای داده‌های اولویت پایین‌تر.

حفظ داده‌ها و مدیریت محیط

داشتن چندین دیتا لیک می‌تواند کارایی بیشتری در حفظ داده‌ها و مدیریت محیط فراهم کند. برای مثال، با جداسازی دیتا لیک‌های اختصاص یافته به محیط‌های توسعه، تست، و تولید، سازمان‌ها می‌توانند اطمینان حاصل کنند که هر محیط فضای جداگانه‌ای برای ذخیره‌سازی و پردازش داده‌ها دارد. این کار خطر تداخل یا ناهماهنگی بین مراحل مختلف چرخه عمر داده‌ها را کاهش می‌دهد.

یکی دیگر از مزایای داشتن چندین دیتا لیک این است که می‌توانید سیاست‌های حفظ داده‌های متمایز را اجرا کنید. نیازهای قانونی یا مقررات معمولاً نیاز به حفظ داده‌ها برای دوره‌های خاص را مشخص می‌کنند. اگر شما دیتا لیک‌های جداگانه برای انواع مختلف داده دارید، می‌توانید به راحتی سیاست‌های حفظ متفاوت را که به این دسته‌بندی‌ها سفارشی شده‌اند، اجرا کنید. این رویکرد به مدیریت کارآمد حفظ داده‌ها اجازه می‌دهد، حفظ مطابقت با مقررات مختلف در حالی که منابع ذخیره‌سازی را بهینه‌سازی می‌کند.

معایب

استفاده از چندین دیتا لیک می‌تواند پیچیدگی و هزینه زیادی به زیرساخت مدیریت داده‌های شما اضافه کند و نیاز به منابع بیشتر و دانش بیشتر برای نگهداری دارد، بنابراین اگر انتخاب دارید، مهم است که منافع را در برابر هزینه‌ها وزن کنید. با این حال، در برخی موارد، انتخابی ندارید؛ برای مثال، اگر شما با نیاز به استقلال داده طراحی می‌کنید، باید چندین دیتا لیک داشته باشید.

انتقال داده‌ها بین چندین دیتا لیک با حفظ یکپارچگی آن‌ها ممکن است نیاز به ابزارهای ادغام و مدیریت اضافی (مانند Informatica، Azure Data Factory، یا Apache NiFi) داشته باشد. در نهایت، داشتن چندین دیتا لیک چالش عملکردی را اضافه می‌کند که در زمانی که یک پرس و جو یا گزارش نیاز به داده‌های چندین دیتا لیک دارد، داده‌ها را ترکیب می‌کند. اگر دیتا لیک‌ها در مکان‌های بسیار دور یا حتی در قسمت‌های مختلف جهان قرار دارند، ممکن است زمان‌بر باشد تا تمام داده‌ها را در یک مکان کپی کنید.

خلاصه

این فصل یک بررسی جامع از دیتا لیک، یک راه حل ذخیره‌سازی و مدیریت داده‌های مقیاس بزرگ قابل نگهداری داده‌ها در فرمت اصلی خود ارائه داده است. من نقش و اصول طراحی دیتا لیک را بحث کردم، همچنین چرا ممکن است در برخی زمینه‌ها تصمیم بگیرید که بیش از یک دیتا لیک را حفظ کنید. به طور متفاوت از انبارهای داده رابطه‌ای، دیتا لیک‌ها امکان ذخیره‌سازی سریع داده بدون هیچ آماده‌سازی یا تبدیل اولیه‌ای را فراهم می‌کنند. این اطمینان از یک فرآیند دریافت داده سریع و بدون تلاش می‌دهد، به خصوص در دهه امروزی بزرگ داده.

من مزایای استفاده از یک دیتا لیک را برجسته کردم، تاکید بر انعطاف‌پذیری و سرعت آن برای ذخیره‌سازی داده است. این جنبه، همراه با توانایی آن برای پذیرش محدوده گسترده‌ای از انواع داده (ساختاری، نیمه ساختاری، و غیر ساختاری) یک مزیت کلیدی است، به خصوص در مواردی که جمع‌آوری داده سریع، قابل مقیاس‌بندی، و متنوع حیاتی است.

فصل همچنین رویکرد پایین به بالا را معرفی کرد، یک روش مهم که در آن شما قبل از تولید هرگونه نظریه یا فرضیه داده‌ها را جمع‌آوری می‌کنید. بر خلاف استراتژی سنتی از بالا به پایین، این رویکرد یک فرآیند تصمیم‌گیری مبتنی بر داده و پایدارتر را تشویق می‌کند.

بخش قابل توجهی از فصل بر روی طراحی یک دیتا لیک تمرکز داشته است، مفهوم تقسیم منطقی یک دیتا لیک به چندین لایه را معرفی می‌کند. این ساختار لایه‌بندی به مدیریت داده به شکل سازمان‌یافته کمک می‌کند، راه را برای بازیابی داده و تحلیل‌های کارآمد، و همچنین می‌تواند در امنیت، کنترل دسترسی، و مدیریت چرخه عمر داده کمک کند.

فصل با بررسی دلایلی که یک سازمان ممکن است تصمیم بگیرد که چندین دیتا لیک داشته باشد به پایان می‌رسد. مزایای احتمالی شامل افزایش امنیت، بهبود مطابقت با مقررات، و مدیریت داده بهتر در واحدهای کسب و کار متفاوت یا موارد خاص است.

در فصل ٦، ما به بررسی مخازن داده می‌پردازیم.

فصل ششم: راه کارها و فرایند های ذخیره داده

در دهه دیجیتال، داده به عنوان خون سازمان‌ها شناخته شده است. اما همانطور که هر حرفه‌ای داده می‌داند، فقط داشتن داده کافی نیست. ارزش واقعی در این است که چگونه این داده‌ها به طور موثر مدیریت، ذخیره‌سازی، و پردازش می‌شوند. این دلیل است که این فصل یک راهنمای جامع برای پیمایش در دنیای پیچیده مدیریت داده است. من نگاه عمیقی به نحوه کارکرد راه‌حل‌های ذخیره‌سازی داده و فرآیندها در معماری‌های داده متفاوت خواهم داد.

اجزای مدیریت داده مانند چرخ‌های یک ماشین پیچیده هستند؛ هر کدام نقش منحصر به فرد خود را انجام می‌دهند، اما همه آن‌ها برای دستیابی به یک هدف مشترک به طور همزمان کار می‌کنند. من با برخی از راه‌حل‌های ذخیره‌سازی معمول شروع خواهم کرد: دیتا مارت‌ها، فروشگاه‌های داده عملیاتی، و مراکز داده. نیمه دوم فصل، "فرآیندهای داده"، طیف گسترده‌ای از فرآیندهای مرتبط با مدیریت و استفاده موثر از داده را آشکار می‌کند، مفاهیمی مانند مدیریت داده‌های اصلی، مجازی‌سازی داده، کاتالوگ‌های داده، و بازارهای داده را بررسی می‌کند.

به عنوان مثال، در حالی که دیتا مارت‌ها نمای خاص بخشی- از داده را ارائه می‌دهند، فروشگاه‌های داده عملیاتی نمای گسترده‌تر و واقعی از عملیات کسب و کار را ارائه می‌دهند. از سوی دیگر، مدیریت داده‌های اصلی اطمینان از یکپارچگی و قابلیت اطمینان داده را فراهم می‌کند، در حالی که مجازی‌سازی داده نمای یکپارچگی و انتزاعی از داده از منابع مختلف را فراهم می‌کند. کاتالوگ داده به عنوان راهنمای مرجع عمل می‌کند، که کشف داده را آسان می‌کند، در حالی که بازار داده در مورد به اشتراک‌گذاری داده یا حتی تجارت آن درباره باز می‌شود. در عین حال، مرکز داده به عنوان مرکز داده سیستم عمل می‌کند، داده‌ها را از منابع مختلف برای دسترسی یکپارچه سازمان می‌دهد. هر یک از این اجزا می‌توانند به تنهایی در کارکرد خود قرار بگیرد؛ اما هنگام ادغام، می‌توانند یک استراتژی داده قدرتمند و جامع را تشکیل دهند، که کسب و کار شما را به تصمیم‌گیری آگاهانه، بهبود کارایی عملیاتی، و بهبود مزیت رقابتی هدایت می‌کند.

بدون اینکه شما یک دانشمند داده تجربه‌کار باشید، یک حرفه‌ای داده با اهداف برای شغل در حوزه داده، یا یک تصمیم‌گیرنده استراتژیک، درک این اجزای کلیدی و یادگیری ترکیب آن‌ها به صورت هماهنگ می‌تواند باعث باز کردن پتانسیل واقعی سازمان شما در زمینه داده شود.

راه کارهای ذخیره سازی

آیا می‌خواهیم به داخل بیاییم؟ این فقط در مورد ذخیره‌سازی و پردازش داده نیست—این در مورد آماده‌سازی برای تبدیل مبتنی بر داده است.

دیتا مارت ها"بخش بندی انبار داده"

یک دیتا مارت، همانطور که در شکل ٦-١ نشان داده شده است، یک زیرمجموعه از انبار داده است. این معمولاً به عنوان یک مخزن متمرکز بر داده طراحی می‌شود که برای برآورده کردن نیازهای خاص یک بخش یا خط کسب و کار در یک سازمان (مانند مالی، منابع انسانی یا بازاریابی) بهینه‌سازی شده است. به دلیل دامنه کمتر خود، یک دیتا مارت می‌تواند به کاربران نمایی ساده‌تر و قابل دسترسی‌تر از اطلاعات را ارائه دهد تا انبار داده که مقادیر بزرگی از داده را نگهداری می‌کند. برای کاربران پیمایش در انبارهای داده بزرگ و پیچیده که مقادیر بزرگی از داده را نگهداری می‌کنند، چالش است. دیتا مارت‌ها این چالش را با استخراج و ترکیب داده‌های مرتبط از انبار داده و ارائه آن‌ها به شکلی کاربرپسندتر مقابله می‌کنند. به این ترتیب، کاربران در یک بخش می‌توانند به راحتی داده‌های مورد نیاز خود را پیدا و بازیابی کنند.

need.

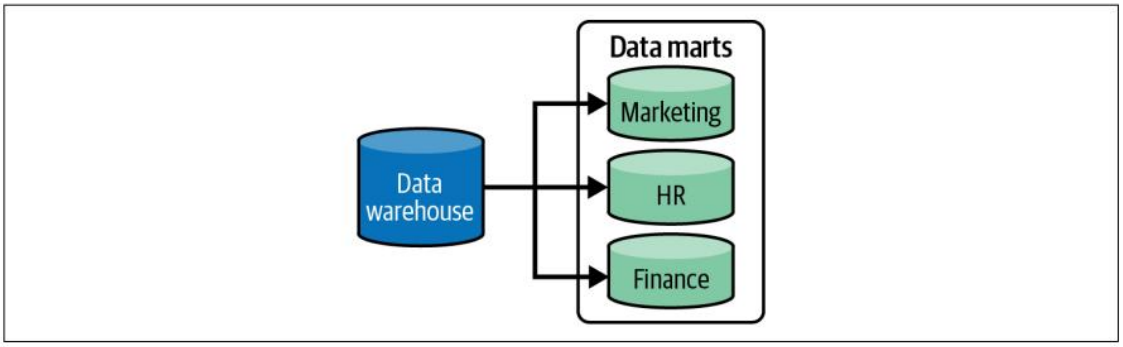


Figure 6-1. Data marts

یکی از مزایای کلیدی دیتا مارت‌ها این است که می‌توانند توسط بخش‌های مختلف سازمان ایجاد و نگهداری شوند، به جای اینکه فقط بر اساس بخش فناوری اطلاعات باید اعتماد کنید. علاوه بر این، هر دیتا مارت می‌تواند به شکل‌های مختلفی ساخته شود تا نیازهای بخشی- که خدمات آن را ارائه می‌دهد را تحمیل کند. این امکان به کاربران کسب و کار کنترل بیشتری بر روی داده می‌دهد؛ آن‌ها می‌توانند آن را به نیازهای منحصر- به فرد خود سفارشی کنند و از ساختارهای داده استفاده کنند که برای حوزه خود مرتبط بیشتر هستند.

دیتا مارت‌های خاص بخش به کاربران دسترسی سریع‌تر و آسان‌تر به داده‌هایی که برای عملیات روزمره و فرآیندهای تصمیم‌گیری آن‌ها نیاز دارند، ارائه می‌دهد. آن‌ها می‌توانند پرس و جوهای تحلیلی انجام دهند، گزارش‌ها را تولید کنند و به درک عملکرد بخش خود برسند بدون اینکه نیاز باشد کل انبار داده را پیمایش کنند.

علاوه بر این، دیتا مارت‌ها حفظ حکمرانی داده و امنیت را ترویج می‌کنند. هر بخش می‌تواند سیاست‌های دسترسی را برای دیتا مارت خود تعریف و اجرا کند، اطمینان حاصل کرده که فقط افراد مجاز دسترسی به مجموعه‌های داده خاص دارند. این کمک می‌کند تا اطلاعات حساس را محافظت کند و اطمینان حاصل کند که با قوانین حفظ حریم خصوصی داده مطابقت دارد.

با توجه به دیتا مارت‌ها با بینش‌های متمرکز و خاص بخش، حالا توجه خود را به فروشگاه‌های داده عملیاتی منتقل می‌کنیم.

منابع ذخیره سازی اطلاعات عملیاتی

فروشگاه داده عملیاتی (ODS) دارای دامنه گسترده‌تری نسبت به دیتا مارت است و نمای واقعی و زمان واقعی از داده را در سطح کل سازمان ارائه می‌دهد. در حالی که دیتا مارت‌ها بینش‌های ارزشمند و عمیق در زمینه‌های خاص ارائه می‌دهند، ODS نمای جامعی از داده‌های عملیاتی سازمان را ارائه می‌دهد. هر ابزار دیدگاه متفاوتی ارائه می‌دهد؛ اگر یک دیتا مارت به شما اجازه می‌دهد تا داده‌های خود را از طریق یک میکروسکوپ ببینید، ODS به شما اجازه می‌دهد تا آن‌ها را از طریق یک عدسه زمان‌بندی گسترده ببینید.

هدف از یک ODS ادغام داده‌های شرکتی از منابع مختلف برای امکان گزارش‌گیری عملیاتی در (یا نزدیک به) زمان واقعی است. این یک انبار داده نیست. همانطور که نام آن نشان می‌دهد، داده‌های عملیاتی را نگهداری می‌کند، نه داده‌های تحلیلی. این داده‌ها را سریع‌تر از یک انبار داده ارائه می‌دهد: هر چند دقیقه، به جای روزانه.

یک ODS ممکن است برای سازمان شما مناسب باشد اگر داده‌هایی که در حال حاضر برای گزارش‌گیری در سیستم‌های منبع استفاده می‌کنید بسیار محدود باشد یا اگر می‌خواهید ابزاری برای گزارش‌گیری بهتر و قدرتمندتر از آنچه سیستم‌های منبع ارائه می‌دهند داشته باشید. اگر فقط چند نفر دارای مجوز امنیتی لازم برای دسترسی به سیستم‌های منبع هستند اما می‌خواهید به دیگران اجازه دهید گزارش‌ها را تولید کنند، یک ODS ممکن است یک راه حل خوب باشد.

معمولاً داده‌ها در یک ODS به شکل مشابهی به ساختار آن‌ها در سیستم‌های منبع ساخته شده‌اند. با این حال، در طول ادغام، راه حل ODS می‌تواند داده‌ها را تمیز کند، نرمال‌سازی کند و قوانین کسب و کار را به داده‌ها اعمال کند تا یکپارچگی داده را تضمین کند. این ادغام به طور منظم در طول روز در سطح کمترین توزیع رخ می‌دهد. معمولاً نیازی به بهینه‌سازی یک ODS برای تحلیل تاریخی و روند نخواهید داشت، زیرا این کار در انبار داده انجام می‌شود.

داده‌ها در ODS نزدیک‌تر به زمان واقعی نسبت به داده‌های مورد نیاز انبار داده هستند. شما می‌توانید یک ODS را به عنوان یکی از مناطق مرحله‌بندی برای DW استفاده کنید، حداقل برای داده‌هایی که آن را نگهداری می‌کند (مشاهده کنید شکل ٦-٢). من توصیه می‌کنم که ابتدا این داده‌ها را تسویه و تمیز کنید، سپس ODS را در زمان نزدیک به زمان واقعی پر کنید تا ارزشی را برای تصمیم‌گیران عملیاتی و تاکتیکی که آن را نیاز دارند ارائه دهید. سپس همچنین می‌توانید آن را برای پر کردن انبار داده استفاده کنید، که معمولاً دارای چرخه‌های بار کمتر تقاضا است. شما هنوز نیاز به منطقه مرحله‌بندی برای داده‌های مورد نیاز DW که در ODS میزبانی نمی‌شوند دارید، اما با دنبال کردن این توصیه، تعداد جریان‌های ETL و اندازه مناطق مرحله‌بندی را کاهش می‌دهید. این کمک می‌کند تا عملکرد را بدون ارزش یا عملکرد را از دست دادن بهبود بخشد.

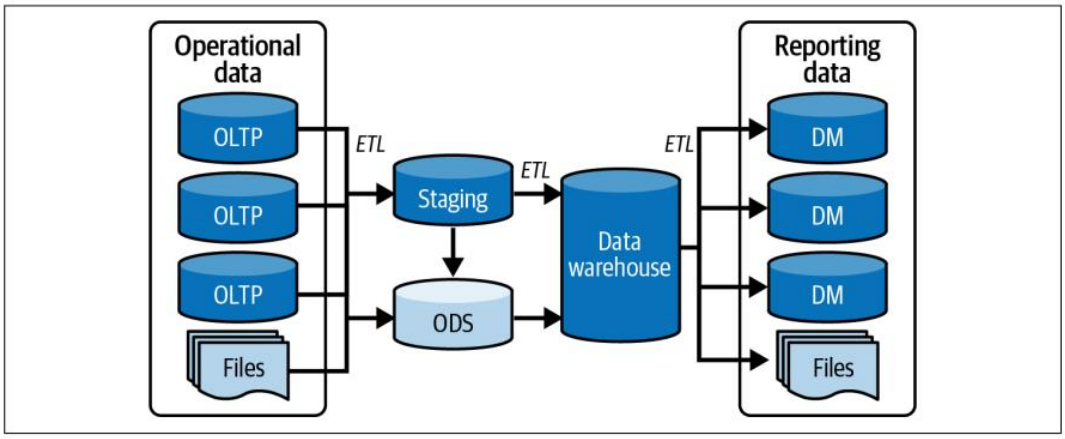


Figure 6-2. An operational data store

به طور خلاصه تفاوت بین اطلاعات/داده ها در دو محیط عملیاتی و انبار داده بشرح زیر هستند.

Table 6-1. Differences between an ODS and a data warehouse

	Operational data store	Data warehouse
Best suited for	Granular, low-level queries against detailed operational data	Complex queries against summary or aggregated analytical data
Purpose	Operational reporting; current or near real-time reporting	Historical and trend analysis reporting on a large volume of data
Data duration	Contains a short window of data	Contains the entire history of the organization's data
Decision making	Supports operational and tactical decisions on current or near real-time data	Provides feedback on strategic decisions, leading to overall system improvements
Data load frequency	Might load data every few minutes or hourly	Might load data daily, weekly, monthly, or quarterly

مورد کاربرد

فرض کنید شما گزارشات موجودی برای یک زنجیره فروشگاهی خیالی به نام ShoesFor-Less اجرا می‌کنید. اگر فقط یک فروشگاه فروش داشته باشید و یک پایگاه داده برای ردیابی سفارشات داشته باشید، می‌توانید مستقیماً گزارش موجودی را در پایگاه داده اجرا کنید. اما از آنجا که شما چندین فروشگاه فروش دارید و هر یک سفارشات خود را در پایگاه داده خود ردیابی می‌کند، تنها راه برای دریافت گزارشات موجودی دقیق و به روز است که داده‌های سفارش از این پایگاه‌های داده را ترکیب کنید. این موضوع برای اطلاع رسانی به مشتریان در مورد کالاهایی که موجود

نیستند و همچنین برای بازسازی سریع کالاها زمانی که تامین کم است، مهم است. یک انبار داده برای این موقعیت به زمان واقعی کافی نیست.

حالا فرض کنید یک نماینده خدمات مشتری تماس بگیرد و از یک مشتری سوال کند که آیا یک سبک و اندازه خاص از کفش در دسترس است یا خیر. نماینده با استفاده از ODS سوال می‌کند تا داده‌های زمان واقعی در مورد در دسترس بودن آن نوع کفش (به عنوان مثال، "کفش دویدن مردانه، اندازه ۱۰") را بازیابی کند و سطح موجودی فعلی را برگرداند. از سوی دیگر، اگر مدیر بازاریابی فروشگاه می‌خواهد تا روند فروش برای این سبک کفش در طول سال گذشته را درک کند، می‌تواند از انبار داده استفاده کند. او یک پرس و جو تجمعی اجرا می‌کند که فروش "کفش دویدن مردانه" را برای هر ماه در طول سال گذشته جمع می‌کند. این یک دید سطح بالا از روندها را فراهم می‌کند که می‌تواند تصمیمات بازاریابی و موجودی استراتژیک را مشروح کند.

مراکز داده "دیتا هاب"

یک هوب داده یک سیستم ذخیره‌سازی و مدیریت داده‌های متمرکز است که به سازمان‌ها کمک می‌کند تا داده‌ها را از منابع مختلف جمع‌آوری، ادغام، ذخیره، سازماندهی و به اشتراک بگذارند و دسترسی آسان به ابزارهای تحلیل، گزارش‌گیری و تصمیم‌گیری را فراهم می‌کند. به طور قابل توجه، هوب‌های داده نیاز به استفاده از صدها یا هزاران رابط نقطه به نقطه برای تبادل داده بین سیستم‌ها را از بین می‌برند. در حالی که انبارهای داده و دریاچه‌های داده هدف اصلی (و اغلب محصول منحصر به فرد) برای ارائه داده برای تحلیل دارند، هوب‌های داده همچنین به عنوان یک راه ارزشمند برای ارتباط و تبادل داده بین سیستم‌های عملیاتی مورد استفاده قرار می‌گیرند.

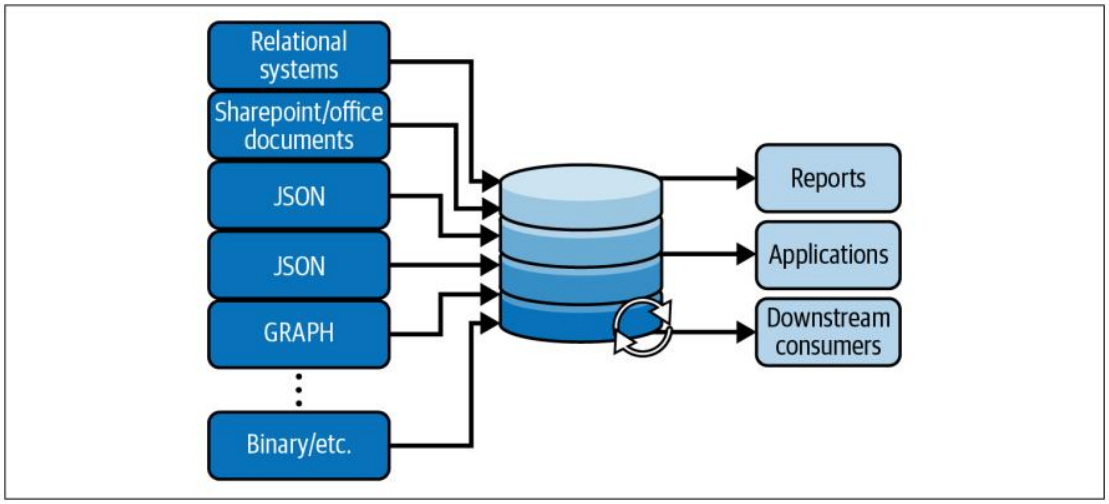


Figure 6-3. Data hub

هوب‌های داده اغلب به همراه دریاچه‌های داده و انبارهای داده برای ایجاد یک زیرساخت مدیریت داده‌های جامع استفاده می‌شوند. همانطور که در فصل‌های ٤ و ٥ یاد گرفتید، دریاچه‌های داده حجم بزرگی از داده‌های خام و پردازش نشده را ذخیره می‌کنند، در حالی که انبارهای داده داده‌های ساختاریافته و پردازش شده را ذخیره می‌کنند که برای پردازش تحلیلی بهینه‌سازی شده‌اند. یک هوب داده این سیستم‌ها را به عنوان یک واسط برای تبادل داده، همکاری و حکمرانی تکمیل می‌کند. برای توضیح بیشتر، بیایید ببینیم چگونه هوب داده با سایر رویکردهای این فصل تعامل دارد و با آن‌ها مقایسه می‌شود. (جدول ٦-٢ این مقایسه را خلاصه می‌کند.)

Table 6-2. Comparison of data solution features

Feature	Data hub	Data lake	Relational data warehouse	Data catalog	Data marketplace
Primary focus	Data storage, integration, distribution	Data storage, processing, analytics	Data storage, processing, analytics	Metadata management, data discovery	Data buying, selling, sharing
Data storage	Raw or lightly processed data	Raw and processed data	Highly structured, processed data	Metadata only	Various formats, provider-dependent
Data structure	Structured, semi-structured, unstructured	Structured, semi-structured, unstructured	Structured	Metadata only	Structured, semi-structured, unstructured
Data processing	Limited processing, mainly integration and distribution	On-demand processing, analytics	Extensive ETL, complex queries	None	Provider dependent
Data sharing	Built-in	Third party	Third party	None	Built-in
Data ownership	Within organization or related groups	Within organization or related groups	Within organization or related groups	Within organization or related groups	Multiple organizations
Use cases	Data ingestion, integration, distribution	Big data analytics, machine learning, artificial intelligence	Complex analytics, reporting, decision making	Data discovery, data lineage, enforcing data governance policies	Monetizing data, acquiring new datasets, data discovery

یک هوب داده کپی‌هایی از داده‌های منبع را ذخیره می‌کند و بر روی مدیریت و توزیع داده‌ها تمرکز دارد، در حالی که یک کاتالوگ داده داده‌های واقعی را ذخیره نمی‌کند و بیشتر به مدیریت متادیتا، پیگیری داده، اجرای سیاست‌های حکمرانی داده و تسهیل کشف داده می‌پردازد. یک هوب داده می‌تواند به عنوان یک منبع برای یک بازار داده عمل کند، که داده را ذخیره نمی‌کند. هوب بر روی مدیریت و توزیع داده در یک سازمان تمرکز دارد، نه معاملات تجاری یک بازار داده. این یک پلتفرم مرکزی است که از سیستم‌های ذخیره‌سازی بیشتری استفاده می‌کند تا یک دریاچه داده یا یک RDW. دریاچه‌های داده معمولاً از ذخیره‌سازی اشیاء برای ذخیره داده استفاده می‌کنند، در حالی که RDWs از ذخیره‌سازی رابطه‌ای استفاده می‌کنند؛ هوب‌های داده می‌توانند سیستم‌های ذخیره‌سازی مانند پایگاه‌های داده رابطه‌ای، پایگاه‌های داده NoSQL و دریاچه‌های داده را ترکیب کنند.

مانند یک دریاچه داده، یک هوب داده پشتیبانی از داده‌های ساختاریافته، نیمه ساختاریافته و غیر ساختاریافته را پشتیبانی می‌کند، همانطور که در شکل ۶-۳ نشان داده شده است، در حالی که یک RDW فقط پشتیبانی از داده‌های ساختاریافته را دارد. هوب‌های داده احتمالاً مکانیزم‌های داخلی برای کاتالوگ‌بندی داده، پیگیری و کنترل‌های دسترسی را فراهم می‌کنند، در حالی که یک دریاچه داده اغلب ابزارهای سومی یا راه‌حل‌های سفارشی را در ادغام می‌کند.

در مورد پردازش، یک دریاچه داده از منابع محاسباتی برای پاکسازی داده و بازگرداندن آن به دریاچه داده استفاده می‌کند. RDWs پردازش گسترده‌ای از داده‌ها انجام می‌دهند، از جمله عملیات ETL، برای پاکسازی، نرمال‌سازی و ساختاردهی داده برای اهداف تحلیلی. با یک هوب داده، با این حال، داده‌ها معمولاً به صورت خارجی تبدیل و تحلیل می‌شوند با استفاده از ابزارها یا برنامه‌های خارجی.

در خلاصه، یک هوب داده یک راه حل انعطاف‌پذیر و چند منظوره برای دریافت، مدیریت، ادغام و توزیع داده از انواع و منابع متنوع است. در مقابل، یک دریاچه داده بر روی ذخیره و پردازش حجم بزرگی از داده‌های خام برای

استفاده در تحلیل و یادگیری ماشین تمرکز دارد، و یک انبار داده برای ذخیره داده‌های ساختاریافته برای استفاده در تحلیل‌های پیچیده، گزارش‌گیری و تصمیم‌گیری بهینه‌سازی شده است.

فرایندهای داده

این فصل ما به بررسی محیط راه‌حل‌های ذخیره‌سازی داده و چگونگی ارائه راه‌های متنوع و کارآمد برای سازماندهی و دسترسی به داده پرداخته است. با این حال، ذخیره داده تنها یک قطعه از معما است. برای واقعی استفاده از پتانسیل داده، ما باید همچنین درک کنیم که چگونه می‌توانیم آن را به طور موثر مدیریت کنیم. بنابراین، ما اکنون از راه‌حل‌های ذخیره‌سازی به پیچیدگی‌های فرآیندهای داده می‌رویم، جایی که ما استراتژی‌ها و تکنیک‌هایی را کشف خواهیم کرد که به ما کمک می‌کنند تا داده‌های ما را تغییر دهیم، حکمرانی کنیم و از دارایی‌های داده خود سود ببریم.

مدیریت اصالت داده ها

مدیریت داده‌های اصلی (MDM) مجموعه‌ای از فناوری، ابزارها و فرآیندها است که به کاربران اجازه می‌دهد تا لیست‌های یکپارچه و دقیق از داده‌های اصلی (مانند لیست مشتریان، محصولات و تامین‌کنندگان) را ایجاد و نگهداری کنند. این شامل ایجاد یک رکورد اصلی تکی برای هر فرد، مکان یا چیزی در یک کسب و کار از منابع داده داخلی و خارجی و برنامه‌های مختلف است. سپس می‌توانید از این رکوردهای اصلی برای ساخت گزارشات، داشبوردها، پرس و جوها و مدل‌های یادگیری ماشین دقیق‌تر استفاده کنید.

بیشتر محصولات MDM ابزارهای بسیار پیشرفته‌ای را برای پاکسازی، تبدیل، ادغام و اعتبارسنجی داده‌ها برای اجرای استانداردهای داده دارند. آن‌ها همچنین به شما اجازه می‌دهند تا سلسله‌های تودرتو (مانند شرکت → بخش → کارمند) را بسازید، که می‌توانید از آن‌ها برای بهبود گزارشات و داشبوردها استفاده کنید. مرحله اول این است که داده‌هایی که می‌خواهید را به MDM می‌کپی کنید. سپس محصول داده‌ها را پاکسازی و استانداردسازی می‌کند و یک رکورد اصلی برای هر موجودیت (مثلاً یک مشتری) که سعی دارید را مستر کنید ایجاد می‌کند. این بیشتر تکرارها را به طور خودکار حذف می‌کند، اما برخی از کارها باید به صورت دستی انجام شود، با اینکه کسی ـ رکوردهایی که ممکن است تکرار باشند را بررسی می‌کند. سپس رکوردهای اصلی به دریاچه داده یا DW کپی می‌شوند. این رکورد اصلی همچنین به عنوان "منبع طلایی" یا "بهترین نسخه از حقیقت" شناخته می‌شود. اگر شما یک ساختار ستاره (مشاهده فصل ۸) ایجاد می‌کنید، رکوردهای مستر شده بخش‌های بُعدی شما خواهند بود که سپس به جدول‌های واقعیت (که مستر نشده‌اند) متصل می‌شوند.

مورد استفاده

برگشتیم به ShoesForLess، زنجیره تجارتی که در آن هر فروشگاه دارای یک پایگاه داده مشتری خود است. شرکت داده‌های مشتری را از تمام این پایگاه‌های داده در یک انبار داده جمع‌آوری می‌کند. به دلیل اینکه برخی مشتریان محصولاتی را از بیش از یک مکان تجاری خریداری کرده‌اند، انبار داده شامل تکرارهای برخی از رکوردهای مشتری خواهد بود.

اگر نام مشتری در هر دو رکورد به طور دقیق نوشته شده باشد، می‌توانید به راحتی تکرارها را فیلتر کنید. اما اگر نام در یک رکورد اشتباه نوشته شده باشد یا به طور متفاوتی وارد شده باشد (برای مثال، اگر شامل یک حرف میانی، خط تیره یا پسوندی مانند Junior یا Senior در یک رکورد باشد اما در دیگری نباشد)، فیلتر نمی‌تواند نام‌ها را به عنوان تکرارها شناسایی کند. این آنها را تکرارهای نزدیک می‌نامند.

اگر MDM را پیاده‌سازی نکنید، زمانی که کاربر نهایی برای اولین بار گزارشی از انبار داده دریافت می‌کند، مشتریان مشابه چندین بار نمایش داده می‌شوند و احتمالاً در مورد دقت گزارش سوال می‌کنند. شما قبلاً اعتماد آن‌ها را از دست داده‌اید - و وقتی این کار را انجام می‌دهید، بسیار دشوار است که آن را دوباره به دست آورید. MDM راه حل است!

مجازی سازی داده ها و فدراسیون داده

مجازی سازی داده‌ها با چندین نام شناخته می‌شود؛ ممکن است به آن به عنوان یک انبار داده منطقی، انبار داده مجازی یا انبار داده غیرمتمرکز اشاره کنید. این یک فناوری است که به شما اجازه می‌دهد تا داده‌ها را از منابع و فرمت‌های مختلف بدون تکرار یا انتقال داده‌های واقعی به صورت منطقی دسترسی پیدا کنید. این یک نقطه دسترسی تکی برای تمام داده‌ها فراهم می‌کند، که اجازه می‌دهد تا داده‌ها را در زمان واقعی ادغام کنید و زمان و هزینه‌های مرتبط با روش‌های ادغام داده‌های سنتی مانند ETL یا ELT را کاهش دهد (مشاهده فصل ۹).

مجازی سازی داده‌ها و مجازی سازی داده‌ها اصطلاحات بسیار مشابهی هستند که اغلب به صورت متقابل استفاده می‌شوند و احتمالاً با اینکه آن‌ها را به هم می‌خورید، به مشکل نمی‌خورید. با این حال، تفاوت‌های نازکی بین دو وجود دارد. مجازی سازی داده‌ها شامل یک سازمان آماری است که شامل سازمان‌های کوچکتر، کاملاً یا به طور جزئی مستقل است که کنترل کامل یا بخشی- از عملیات خود را دارند. (سیستم آمریکایی از دولت‌ها و دولت‌های ایالت یک مثال خوب از یک مجازی سازی است.) مانند مجازی سازی داده‌ها، مجازی سازی داده‌ها نمای متحد از داده‌ها را از منابع داده متعدد ایجاد می‌کند. مجازی سازی داده‌ها، با این حال، مفهوم گسترده‌تری است که شامل مجازی سازی داده‌ها و همچنین تبدیل داده‌ها، کشینگ، امنیت و حکمرانی است. ابزارهای بهتر مجازی سازی داده‌ها ویژگی‌هایی را فراهم می‌کنند که در مورد عملکرد کمک می‌کنند، مانند بهینه‌سازی پرس و جو، پوش کردن پرس و جو و کشینگ. ممکن است ابزارهایی با این ویژگی‌ها را به عنوان مجازی سازی داده‌ها ببینید و ابزارهایی بدون آن‌ها را به عنوان مجازی سازی داده‌ها ببینید. مجازی سازی داده‌ها می‌تواند یک انبار داده را جایگزین کند (مشاهده فصل ٤) یا حرکت داده‌ها/ETL. بیایید ببینیم چگونه.

نسخه آزمایشی - رمز گشایی از معماری داده

مجازی سازی به عنوان جایگزین برای انبار داده

در برخی موارد، به ویژه زمانی که منابع داده زیادی وجود دارد یا به طور مداوم تغییر می‌کنند، ویرتوالیزه داده‌ها می‌توانند جایگزین یک انبار داده باشد. وقتی از این رویکرد استفاده می‌کنید، هیچ کپی و ذخیره‌سازی داده در مکان مرکزی وجود ندارد. شکل ٦-٤ نشان می‌دهد که چگونه بیشتر محصولات ویرتوالیزه داده‌ها کار می‌کنند. شبکه (گام ١) نتایج پرس و جوی یک کاربر نهایی را نشان می‌دهد: یک لیست از آدرس‌ها. وقتی کاربر نهایی از طریق شبکه برای مشاهده داده‌های بیشتر پیمایش می‌کند، شبکه مواردی را که در حال حاضر بارگذاری نشده‌اند از موتور ویرتوالیزه داده‌ها درخواست می‌کند (گام ٢). اگر موتور یک کش از داده‌ها داشته باشد، آن را فوراً برمی‌گرداند و شبکه فقط برای نمایش داده‌ها به‌روز می‌شود. اگر داده‌ها را خارج از کش درخواست کنید، موتور ویرتوالیزه داده‌ها درخواست را به منبع داده از راه دور (گام ٣) منتقل می‌کند، که در آنجا پردازش می‌شود. داده‌های جدید بارگذاری شده به موتور برگردانده می‌شوند (گام ٤) و سپس به شبکه منتقل می‌شوند. در هر دو مورد، شبکه همیشه کاملاً پاسخگو باقی می‌ماند.

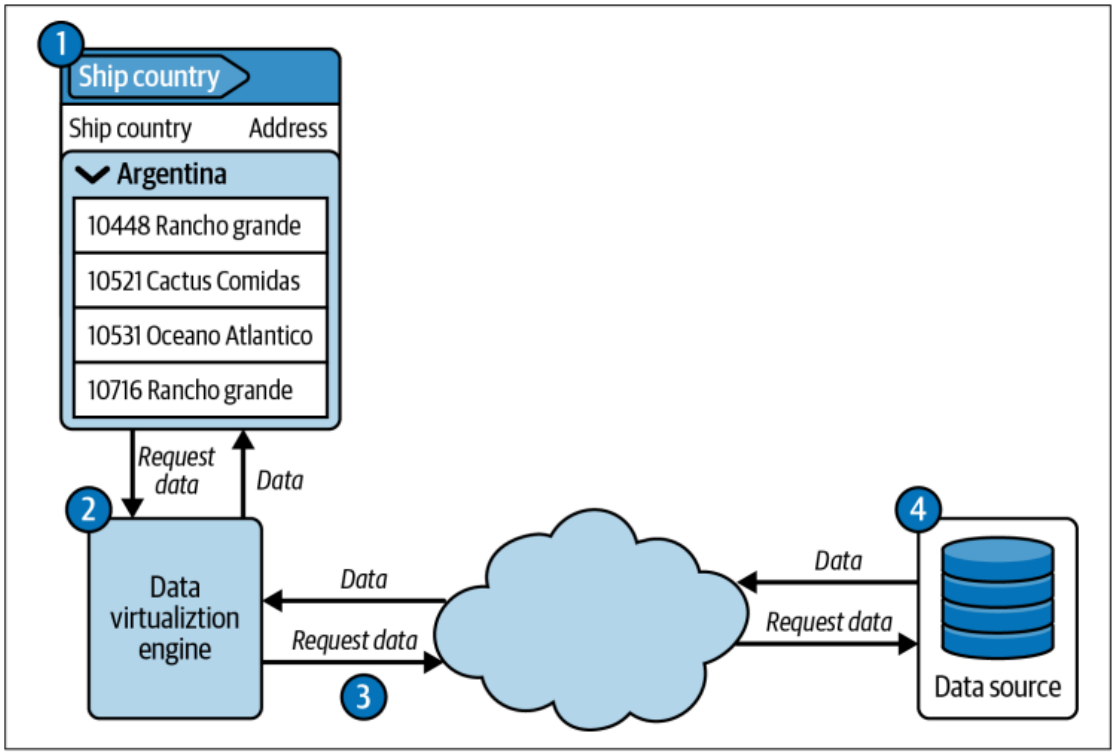


Figure 6-4. Data virtualization

مزیت اصلی یک راه حل ویرتوالیزه داده‌ها این است که برای سرعت به بازار بهینه می‌شود. می‌توانند در زمان کمتری ساخته شود که برای ساخت یک انبار داده لازم است، زیرا نیازی به طراحی و ساخت یک DW و ETL برای کپی داده‌ها در آن و نیازی به گذراندن زمان بیشتری برای آزمایش آن وجود ندارد. کپی داده‌ها (مانند یک DW) به معنای هزینه‌های ذخیره‌سازی بیشتر و حکمرانی بالاتر، جریان‌های ETL بیشتر برای ساخت و نگهداری و ناهماهنگی‌های بیشتر در داده‌ها است، بنابراین استفاده از ویرتوالیزه می‌تواند به شما پول زیادی را صرفه‌جویی کند.

١ تفاوت بین ویرتوالیزه داده‌ها و پرس و جوهای متحده این است که ویرتوالیزه داده‌ها بیشتر در مورد معماری سیستم است، در حالی که پرس و جوهای متحده مکانیسمی است که برای استخراج اطلاعات از این سیستم استفاده می‌شود. در عمل، اگر شما یک سیستم ویرتوالیزه داده‌ها در نظر گرفته‌اید، شما از پرس و جوهای متحده برای بازیابی داده‌های خود استفاده می‌کنید.

٢ در ویرتوالیزه داده‌ها، اصطلاح query push-down به توانایی انتقال بخش‌های خاصی از یک پرس و جو، مانند فیلتر کردن یا تجمع، به منابع داده خود اشاره دارد. این امکان را به لایه ویرتوالیزه داده‌ها می‌دهد تا وظایف پردازشی را

به سرور منبع داده منتقل کند و فقط نتایج مرتبط را بازیابی کند. مزایا شامل بهبود عملکرد و کاهش حجم انتقال داده است؛ معادله این است که منابع محاسباتی را در سرور که منبع داده قرار دارد استفاده می‌کند.

۳ شاخص‌های پایگاه داده نوعی از ساختار داده هستند که سرعت بازیابی عملیات در یک جدول پایگاه داده را افزایش می‌دهند. آن‌ها به شکل زیادی مانند شاخص‌های کتاب کار می‌کنند، کاربران را یک راه سریع برای دسترسی به داده‌هایی که دنبال می‌کنند فراهم می‌کنند.

٤ آمار پایگاه داده مجموعه‌ای از اطلاعات است که یک سیستم مدیریت پایگاه داده (DBMS) در مورد داده‌های ذخیره شده

ویرتوالیزه داده‌ها پشتیبانی از پرس و جوهای متحده را پشتیبانی می‌کند، که در آن می‌توانید یک پرس و جو تکی را صادر کنید که داده‌ها را از منابع مختلف و فرمت‌ها بازیابی کرده و نتایج را در یک مجموعه نتیجه تکی ترکیب کند.

با این حال، برخی از معایب اصلی ویرتوالیزه داده‌ها و پرس و جوهای متحده وجود دارد. اگر شما در نظر گرفتن یک راه حل ویرتوالیزه داده‌ها برای جایگزین کردن انبار داده خود هستید، پیشنهاد می‌کنم سوالات زیر را بپرسید:

- چه سطح عملکردی نیاز دارم؟ این یک راه حلی است که می‌توانم برای یک داشبورد که نیاز به زمان پاسخ زیر ثانیه دارد، استفاده کنم یا بیشتر برای گزارشات عملیاتی است؟

- چقدر را راه حل ویرتوالیزه داده‌ها بر عملکرد سیستم منبع تأثیر می‌گذارد؟ آیا پرس و جوی من به آرامی اجرا می‌شود اگر یک کاربر نهایی دیگری پرس و جویی را در برابر همان منبع داده که تمام منابع سرور را مصرف می‌کند اجرا کند؟ آیا پرس و جو را به پایین می‌فرستد؟

- آیا نیاز دارم که چیزی را در هر سروری که یک منبع داده را دارد نصب کنم؟

- آیا راه حل از شاخص هر فناوری را در منبع داده استفاده می‌کند یا شاخصهای خود را ایجاد می‌کند؟

- آیا راه حل از آمار پایگاه داده هر منبع داده برای پرس و جوها استفاده می‌کند؟

- آیا می‌توانم کاربران مختلف را برای دسترسی به هر منبع داده به منظور امنیت محدود کنم؟

- راه حل چگونه نزدیک تکرارها را مدیریت می‌کند؟

- کجا و چگونه داده‌ها تمیز خواهند شد؟

- آیا گزارشات به دلیل تغییر سیستم منبع شکست خواهند خورد؟

- هزینه نرم‌افزار ویرتوالیزه چقدر است و چقدر پیچیده است برای استفاده؟

این سوالات می‌توانند به شما کمک کنند تا تصمیم بهتری در مورد انتخاب یک راه حل ویرتوالیزه داده‌ها بگیرید و از معایب احتمالی آگاه شوید.

علاوه بر دلایلی که با پرسش‌های مطرح شده کشف شده‌اند، دلایل دیگری نیز وجود دارد که می‌توان برای استفاده از یک انبار داده فیزیکی توجیه کرد، همانطور که در فصل ٤ بحث شده است. این دلایل شامل نیاز به اجرای گزارشات تاریخی دقیق، تغییر نام جداول، و نگهداری داده‌های تاریخی است.

انبار داده فیزیکی می‌تواند برای مواردی که نیاز به دسترسی به داده‌های تاریخی دقیق و بازگشتی دارند، مناسب باشد. این نوع انبار داده می‌تواند به طور مداوم داده‌ها را ذخیره کند و به کاربران اجازه دهد تا به داده‌های تاریخی دسترسی

پیدا کنند و آن‌ها را برای تحلیل و گزارش‌سازی استفاده کنند. همچنین، امکان تغییر نام جداول و مدیریت بهتر ساختار داده‌ها در انبار داده فیزیکی امکاناتی را فراهم می‌کند که در محیط‌های ویرتوالیزه داده‌ها ممکن نیست.

با این حال، انتخاب بین استفاده از یک انبار داده فیزیکی و یک راه حل ویرتوالیزه داده‌ها بستگی به نیازهای خاص سازمان شما دارد. هر دو روش مزایا و معایب خاص خود را دارند و بسته به نیازهای شما به داده‌ها، می‌توانید تصمیم بگیرید که کدام روش برای شما مناسب‌تر است.

مجازی سازی جای گزینی برای انتقال داده و ETL

اگر در حال ساخت یک انبار داده هستید، آیا باید تمام داده‌های منبع را به انبار داده منتقل کنید یا بهتر است داده‌های منبع را در جای خود نگه دارید و یک لایه ویرتوالیزه روی بخشی از آن‌ها ایجاد کنید؟

دلیل متداول تر برای انتقال داده‌ها این است که اگر قصد دارید داده‌ها را یک بار تجمع یا تبدیل کنید و سپس نتایج را بسیاری بار پرس و جو کنید. دلیل دیگری این است که اگر قصد دارید به طور مکرر مجموعه‌های داده را از منابع مختلف به یکدیگر بپیوندید و نیاز به عملکرد بسیار سریع دارید. در این سناریوها، معمولاً توصیه می‌کنم که از راه حل‌های انبار داده استفاده کنید.

با این حال، گاهی اوقات ممکن است داشته باشید پرس و جوهای تصادفی که نیازی به عملکرد بسیار سریع ندارند. و می‌توانید قطعاً یک انبار داده داشته باشید که برای برخی از جداول از انتقال داده استفاده می‌کند و برای دیگران از ویرتوالیزه داده استفاده می‌کند. جدول ٦-٣ انتقال داده و ویرتوالیزه را مقایسه می‌کند.

Table 6-3. Comparison of data movement and data virtualization

	Development and operations costs	Time to solution	Security	Data freshness and quality	Compliance
Data movement	Costly to build and maintain ELT jobs; duplicate data storage costs	Takes time to build and run jobs	Creating copies of the data makes it more vulnerable to hackers	ETL pipelines make the data "stale"; ETL can introduce data errors	Moving data in and out of compliance boundaries can cause data governance issues
Data virtualization	Reduces ongoing maintenance and change management; minimizes storage costs	Allows for rapid iterations and prototyping	Data is kept in one secure place, minimizing the attack surface	Data being queried is always "fresh" and accurate from the source	Having few copies of data and less movement helps meet compliance requirements

ویرتوالیزه داده‌ها چندین مزیت ارائه می‌دهد. اولاً، یک خط داده جامع را فراهم می‌کند، که اطمینان می‌دهد شما می‌توانید مسیر داده‌ها از منبع به لایه ارائه را به طور واضح درک کنید. دوماً، افزودن منابع داده اضافی به صورت بی‌درز است، که نیازی به تغییر بسته‌های تبدیل یا جداول مرحله‌ای نیست. سوماً، نرم‌افزار ویرتوالیزه داده‌ها تمام داده‌ها را از طریق یک رابط SQL متحد ارائه می‌دهد. این به کاربران اجازه دسترسی آسان را بدون توجه به منبع، که می‌تواند فایل‌های ساده، اکسل‌ها، مینی‌کامپیوترها، پایگاه‌های داده رابطه‌ای، یا سایر انواع مخازن داده باشد، می‌دهد.

در حالی که جدول ٦-٣ مزایای ویرتوالیزه داده را نسبت به انتقال داده نشان می‌دهد، این مزایا ممکن است کافی نباشد تا بتواند ارزش از دست دادن عملکرد و سایر معایب استفاده از یک انبار داده رابطه‌ای را جبران کند، که در فصل ٤ لیست شده‌اند. همچنین، به یاد داشته باشید که ابزار ویرتوالیزه‌ای که انتخاب می‌کنید ممکن است تمام منابع داده شما را پشتیبانی نکند.

در اینجا برخی از دلایلی که ممکن است به ویرتوالیزه داده انتخاب کنید را می‌بینید:

- **محدودیت‌های قانونی در انتقال داده**: به عنوان مثال، قوانین حکمرانی سوورین داده‌ها می‌گوید که داده‌های قرار داده شده در چین باید در چین بمانند و فقط می‌توان آن‌ها را توسط یک شخص که در حال حاضر در آنجا قرار دارد پرس و جو کرد.

- **دسترسی کاربر نهایی خارج از شرکت به داده‌های مشتری**: به جای ارسال کپی داده به کاربر، آن‌ها می‌توانند از نرم‌افزار ویرتوالیزه برای پرس و جوی آن استفاده کنند. این به شما اجازه می‌دهد کنترل کنید که کدام کاربران به چه داده‌ای دسترسی دارند و می‌دانید چقدر دسترسی داشته‌اند.

- **داده‌های مرجع در یک پایگاه داده خارج از DW که به طور مکرر تغییر می‌کند**: به جای کپی داده در DW مجدداً، می‌توانید از ویرتوالیزه برای پرس و جوی آن هنگامی که داده‌های پایگاه داده خارجی را با داده‌های DW ادغام می‌کنید، استفاده کنید.

- **می‌خواهید داده‌ها را از مخازن داده مختلف پرس و جو کنید و آن‌ها را با هم ادغام کنید**: نرم‌افزار ویرتوالیزه معمولاً از بسیاری از منابع داده پشتیبانی می‌کند (مثلاً یک دریاچه داده، پایگاه داده رابطه‌ای، و پایگاه داده NoSQL) و می‌تواند این کار را انجام دهد.

- **می‌خواهید کاربران خود تحلیل‌های Self Service را از طریق یک ساندباکس مجازی انجام دهند**: یک ساندباکس مجازی یک محیط امن و جداگانه است که کاربران می‌توانند در آن داده‌ها را بررسی، تحلیل، و تغییر دهند بدون اینکه داده‌های زیربنایی یا سیستم‌ها را تحت تأثیر قرار دهند.

- **می‌خواهید یک راه حل را برای سریع ساختن برای انتقال داده منبع بسازید**: به جای استفاده از یک ابزار ETL سنتی برای کپی داده منبع به مکان مرکزی و سپس ایجاد گزارشات از آن، می‌توانید فقط با استفاده از نرم‌افزار ویرتوالیزه گزارشات را ایجاد کنید. این به معنای این است که شما می‌توانید یک راه حل را بسیار سریع‌تر بسازید، زیرا نیازی به استفاده از یک ابزار ETL

دیتا کاتالوگ

یک کاتالوگ داده مخزن مرکزی است، معمولاً در ابر قرار دارد، که متادیتا درباره منابع داده سازمان، جداول، ساختارها، ستون ها و دیگر دارایی های داده مانند گزارشات و داشبوردها، فرآیندهای ETL، و اسکریپت های SQL را ذخیره و سازماندهی می کند. این به عنوان یک منبع یکپارچه برای کاربران برای کشف، درک و مدیریت داده هایی که در پایگاه های داده برنامه، دریاچه های داده، انبارهای داده رابطه ای، فروشگاه های داده عملیاتی، مارت های داده و هر شکل دیگری از ذخیره سازی داده قرار دارد، عمل می کند.

یک کاتالوگ داده معمولاً اطلاعات زیر را درباره هر دارایی داده ای شامل می شود:

- اطلاعات درباره منبع، مانند مکان، نوع و جزئیات اتصال

- اگر در یک RDW باشد، اطلاعات جدول و ساختار (مانند ساختار، روابط و سازمان) و اطلاعات ستون (مانند انواع داده، فرمت ها، توضیحات و روابط بین ستون ها)

- اگر در یک ذخیره سازی اشیاء مانند یک دریاچه داده باشد، ویژگی های فایل (ذخیره سازی، نام پوشه، نام فایل)

- گذشته داده (چگونه داده از منبع خود رسیده است)، از جمله اطلاعات درباره هر تبدیل، تجمع، و یکپارچگی که داده را تحت تأثیر قرار داده است

- جزئیات حاکمیت داده و مطابقت، مانند کیفیت داده، مالکیت و سیاست ها

- ابزارهای جستجو و کشف برای کاربران برای جستجو، فیلتر کردن و پیدا کردن داده های مرتبط

یک کاتالوگ داده به شما کمک می کند تا داده ها را به طور موثرتری مدیریت کنید با فراهم کردن دید به منظره داده. این در نوبت خود همکاری بهتری بین تیم ها و تصمیم گیری با اطلاعات بیشتر را تسهیل می کند. همچنین به اطمینان کیفیت داده، امنیت و مطابقت کمک می کند با ساده سازی ردیابی گذشته داده، اجرای سیاست های داده و حفظ درک واضح از دارایی های داده.

بسیاری از محصولات در بازار برای ایجاد کاتالوگ های داده وجود دارد، برخی از آنها بیش از نیم قرن در حال استفاده هستند. محصولات محبوب ترین عبارتند از کاتالوگ داده سازمانی Informatica، کاتالوگ داده Collibra و Microsoft Purview. این محصولات قابلیت های اضافی مانند حاکمیت داده و مطابقت، فرهنگ لغت های تجاری و معیارهای کیفیت داده را دارند.

فروشگاه های داده

بازارهای داده برای تجارت داده‌ها طراحی شده‌اند تا به مصرف‌کنندگان داده (مانند کسب و کارها، پژوهشگران و توسعه‌دهندگان) کمک کنند تا داده‌ها را برای اهدافی مانند تحلیل، یادگیری ماشین، هوش مصنوعی و تصمیم‌گیری کشف، ارزیابی و خرید کنند. ارائه‌دهندگان معمولاً شامل سازمان‌ها، دولت‌ها و افرادی هستند که مجموعه‌ای از داده‌های ارزشمند را جمع‌آوری یا تولید کرده‌اند.

یک بازار داده معمولاً شامل یک کاتالوگ داده است. بسیاری از بازارها کیفیت داده‌ها را ارزیابی و بهبود می‌بخشند تا اطمینان حاصل شود که آن‌ها دقیق، کامل، یکنواخت و آسان برای استفاده هستند. برخی ابزارها و خدماتی را برای تمیز کردن، تبدیل و غنی‌سازی داده ارائه می‌دهند که می‌توانند به کاربران کمک کند تا زمان و تلاش خود را در آماده‌سازی آن برای تحلیل صرف کنند. (این به خصوص ارزشمند است وقتی که داده‌ها را از چندین منبع یا با داده‌های ناقص یا نایق می‌توانید یکپارچه کنید.) پس از خرید یک مصرف‌کننده، بازار ممکن است ابزارهایی را برای کمک به آن‌ها در دسترسی به داده و یکپارچگی آن در جریان کاری، برنامه‌ها یا سیستم‌های خود ارائه دهد. برخی حتی اجازه می‌دهند که کاربران مجموعه‌های داده را با ترکیب داده‌ها از چندین منبع یا فیلتر کردن آن‌ها بر اساس معیارهای خاص تنظیم کنند تا دقیقاً داده‌هایی که نیاز دارند را دسترسی داشته باشند.

بازارهای داده باید از معیارهای امنیتی قوی برخوردار باشند تا داده‌های کاربر را محافظت کنند، اطمینان حاصل کنند که با قوانین حفظ حریم خصوصی مطابقت دارند و حفظ محرمانگی اطلاعات حساس را حفظ کنند. برخی راه‌هایی را برای ساده‌سازی آسان داده‌های خصوصی یا حساس ارائه می‌دهند. ساختارهای قیمت‌گذاری و قراردادهای مجوز بازارهای داده شرایط استفاده را به صورت واضح تعریف می‌کنند و اطمینان حقوقی برای هر دو ارائه‌دهنده و مصرف‌کننده داده را تضمین می‌کنند.

بسیاری از بازارهای داده نیز ابزارهای تحلیل داخلی و نمایش داده را ارائه می‌دهند که به شما اجازه می‌دهند تا داده‌ها را مستقیماً در پلتفرم تحلیل و نمایش یا فقط پیش‌نمایش کنید. سایر ویژگی‌ها و ابزارهای متداول عبارتند از:

- امتیازدهی و نقد داده‌ها

- توصیه‌ها برای داده‌های دیگری که ممکن است دوست داشته باشید

- پروفایل‌های خریدار برای سفارش مجدد آسان، لیست مورد علاقه، و غیره

- همکاری و جامعه

- آموزش و پشتیبانی

از دیدگاه ارائه‌دهندگان، بازارهای داده راهی برای تجارت دارایی‌های داده ارائه می‌دهند، جریان درآمد جدیدی ایجاد کردن. این به خصوص مفید برای سازمان‌هایی است که مقادیر زیادی از داده را جمع‌آوری کرده‌اند اما منابع یا دانش لازم برای تحلیل و تجارت آن‌ها را ندارند.

بازارهای داده به دلیل افزایش تقاضا برای بینش‌های مبتنی بر داده و در دسترس بودن مجموعه‌های داده متنوع‌تر در حال رشد هستند. تولید نمودارهای داده برای ایجاد بازارهای داده نزدیک به تعداد محصولات موجود برای ایجاد کاتالوگ‌های داده نیست، زیرا بازارها فقط در ابتدای دهه ۲۰۲۰ ظهور کرده‌اند. محبوب‌ترین‌ها شامل بازار داده‌های Datarade، Snowflake و بازار داده‌های ابری Informatica است.

خلاصه ای از کتاب معماری داده

هـر دو ارائه‌دهنـده و مصرف‌کننده می‌تواننـد از تبـادل داده بـه صـورت کارآمـد و شـفاف‌تر بهره‌منـد شـوند، کـه می‌توانـد بـه ابداع محصولات و خدمات جدید مبتنی بر داده کمک کند.

خلاصه

ایـن فصـل روی مـوارد مختلـف مربـوط بـه ذخیره‌سـازی داده پرداخـت، کـه همـه آن‌هـا نقش‌هـای مهـمی در محـیط کسـب و کـار امـروزی مبتـنی بـر داده ایفـا می‌کننـد. مـن دربـاره ذخیره‌سـازی داده صـحبت کـردم، از جملـه مارت‌هـای داده، فروشـگاه‌های داده عملیـاتی و مراکـز داده، و سـپس تمرکـز را بـه فرآینـدها منتقـل کـردم. مـدیریت داده‌هـای اصـلی (MDM) یـک رویکـرد حیـاتی اسـت کـه بـر روی مـدیریت موجودیت‌هـای کسـب و کار اصـلی تمرکـز دارد تـا اطمینـان حاصـل شـود کـه داده‌هـا دقیـق، یکنواخـت و متناسـب در سیسـتم‌ها و فرآینـدهای متعـدد باشـند. سـپس بـه موضـوع ویرتـوالیزه داده، کاتالوگ‌هـای داده و بازارهـای داده پرداختم.بـا درک ایـن اسـتراتژی‌ها و کارکردهـای مختلـف، شـما می‌توانیـد بهـتر معمـاری داده خود را با نیازهای سازمان خود هماهنگ کنید و ارزش کسب و کار بیشتری را از داده‌های خود به دست آورید.

فصل هفتم: رویکردهای طراحی

ایـن فصـل روی رویکردهـای طـراحی بـرای کنـترل و سـازماندهی داده و چگـونگی کمـک ایـن روش‌هـا بـه سـاخت سیسـتم‌های قدرتمنـد، قابـل تطبیـق و قابـل اعتمـاد بـرای مـدیریت داده می‌پـردازد. بـه طـور سـاده، مـا در اینجـا دربـاره اسـتراتژی‌هایی کـه بـه مـا کمـک می‌کنـد تصـمیم بگـیریم کـه چگونـه داده بایـد ذخـیره شـود، پـردازش شـود و دسـترسی پیـدا شـود تـا سرعـت و قابلیت اعتماد سیستم‌های داده ما را بهینه کنیم.

بـرای توضـیح، بـین طـراحی داده و مدل‌سـازی داده تمـایز دهـیم، کـه موضـوع فصـل ۸ اسـت. طـراحی داده را ماننـد سـاخت یـک شـهر در نظـر بگیریـد. ایـن دربـاره تصـمیم‌گیری دربـاره جایگـاه سـاختمان‌ها، اتصـال بـین بخش‌هـای مختلـف شـهر بـا مسـیرها و جریـان ترافیـک اسـت. در عـین حـال، مدل‌سـازی داده بیشـتر شـبیه بـه طـراحی سـاختمان‌های فـردی اسـت - ایـن درباره ترتیب دادن اتاق‌ها، تصمیم‌گیری درباره چگونگی اتصال آن‌ها و تعیین هدف هر اتاق است.

در ایـن فصـل، مـا بـه نوعهـای مختلـف طـراحی داده نگـاه خـواهیم کـرد. مـا روش‌هـایی ماننـد OLTP و OLAP را مقایسـه و مقابلـه خـواهیم کـرد، کـه در واقـع روش‌هـای متفـاوتی بـرای پـردازش و تحلیـل داده هسـتند. همچنـین مفـاهیمی ماننـد SMP و MPP را کـه اسـتراتژی‌هایی بـرای پـردازش داده بـه صـورت کارآمـدتر هسـتند، بـررسی خـواهیم کـرد. سـپس دربـاره معماری‌هـای Lambda و Kappa کـه نقشـه‌هایی بـرای کنـترل مقـادیر زیـادی از داده در زمـان واقـعی هسـتند، یـاد خـواهیم گرفت. در آخـر، دربـاره یـک رویکـرد بـه نـام persistence polyglot صـحبت خـواهیم کـرد کـه بـه مـا اجـازه می‌دهـد از نوع‌های مختلف فناوری ذخیره‌سازی داده در همان برنامه استفاده کنیم.

هـدف مـن در اینجـا نیسـت کـه یـک رویکـرد را بـه عنـوان بهـترین راه حـل بـرای هـر شرایـط تـرویج کـنم بلکـه بـه شـما کمـک می‌کـنم تـا نقـاط قـوت و ضـعف هـر کـدام را درک کنیـد. بـه ایـن ترتیـب، شـما می‌توانیـد بـر اسـاس نیازهـای خـاص خـود روش‌هـای مناسـب را انتخـاب یـا ترکیـب کنیـد. در پایـان ایـن فصـل، شـما بـه طـور قـوی‌تری درک خواهیـد کـرد کـه چگونـه می‌توانیـد سیسـتم‌های داده کارآمـد را ایجـاد کنیـد کـه می‌تواننـد بـه نیازهـای تغییـر کننـده و پیشـرفت‌های تکنولـوژیکی پاسـخ دهند.

پردازش تراکنش‌های آنلاین در مقابل پردازش تحلیلی آنلاین

پـردازش تراکنش‌هـای آنلایـن (OLTP) یـک نـوع سیسـتم یـا برنامـه اطلاعـاتی اسـت کـه تراکنش‌هـای ایجـاد، خوانـدن، به‌روزرسـانی و حـذف (CRUD) را در محـیط واقـعی پـردازش می‌کنـد (مشـاهده کنیـد فصـل ۲). سیسـتم‌های OLTP بـرای پشـتیبانی از سـطوح بـالای همزمـانی طـراحی شـده‌اند، کـه بـه ایـن معـنی اسـت کـه می‌تواننـد تعـداد زیـادی تـراکنش را در همـان زمـان پـردازش کننـد. آن‌هـا معمـولاً از یـک مـدل رابطـه‌ای اسـتفاده می‌کننـد (مشـاهده کنیـد فصـل ۸) و بـرای کـم تـاخیر بهینه‌سـازی شـده‌اند، کـه بـه ایـن معـنی اسـت کـه می‌تواننـد تراکنش‌هـا را بسـیار سریـع پـردازش کننـد. مثال‌هـا شـامل برنامه‌های نقد و بررسی، وب‌سایت‌های تجارت الکترونیکی و راه‌حل‌های بانکی آنلاین است.

در یـک سیسـتم OLTP، تراکنش‌هـا معمـولاً بـه سرعـت پـردازش می‌شـوند و بـه یـک روش بسـیار خـاص. بـه عنـوان مثـال، خریـد مشـتری در یـک فروشـگاه بـه عنـوان یـک تـراکنش در نظـر گرفتـه می‌شـود. ایـن تـراکنش شـامل کاهش حسـاب مشـتری بـرای مبلـغ خریـد، کـاهش موجـودی فروشـگاه و به‌روزرسـانی سـوابق مـالی فروشـگاه بـرای نشـان دادن فـروش اسـت. سیسـتم‌های OLTP از مجموعـه‌ای از DBMS بـرای ذخـیره و مـدیریت داده اسـتفاده می‌کننـد، ماننـد Microsoft SQL Server و Oracle.

سیستم‌های OLTP اغلب با سیستم‌های پردازش تحلیلی آنلاین (OLAP) مقایسه می‌شوند، که برای تحلیل داده و گزارش‌دهی برای پشتیبانی از هوش تجاری و تصمیم‌گیری استفاده می‌شوند. سیستم‌های OLAP برای عملکرد پرس و جو سریع بهینه‌سازی شده‌اند، که به کاربران نهایی اجازه می‌دهد تا به راحتی و سریع داده‌ها را از چندین دیدگاه تحلیل کنند با برش و تقسیم داده‌ها در گزارشات و داشبوردها بسیار سریع‌تر از سیستم OLTP.

فکر کنید که یک سیستم OLAP مانند "نوشته شده یک بار، خوانده شده چندین بار" است (در مقابل CRUD). اغلب، چندین پایگاه داده OLTP به عنوان منابعی استفاده می‌شوند که به یک انبار داده می‌رسند، که سپس به یک پایگاه داده OLAP می‌رسد، همانطور که در شکل ۷-۱ نشان داده شده است. (با این حال، گاهی اوقات می‌توانید یک پایگاه داده OLAP را مستقیماً از منابع OLTP بسازید.)

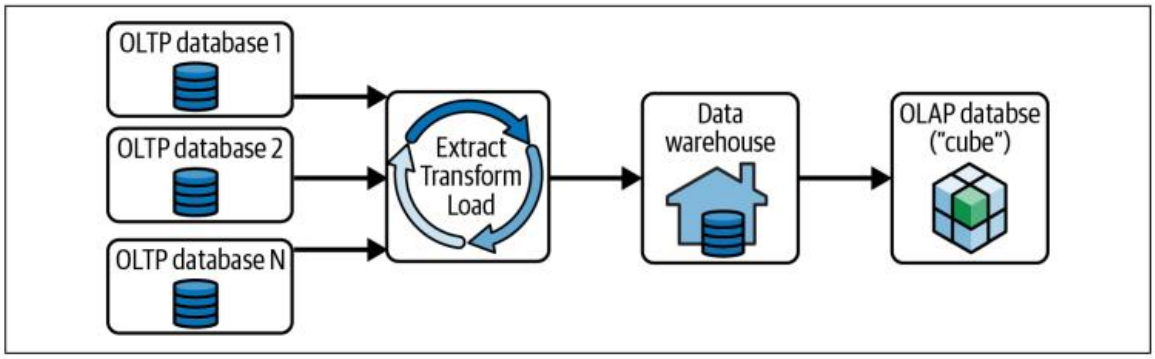


Figure 7-1. OLAP database architecture

یک پایگاه داده OLAP معمولاً از یک یا چند کوب OLAP تشکیل شده است. کوب OLAP محلی است که داده‌ها در آن از قبل تجمع شده‌اند، به این معنی که قبلاً خلاصه و گروه‌بندی شده‌اند بر اساس بعضی ابعاد؛ کاربران نهایی می‌توانند به سرعت از چندین بُعد و سطوح جزئیات به آن دسترسی پیدا کنند بدون اینکه برای انتظار پرس و جوهای طولانی مدت منتظر بمانند. ایجاد یک کوب OLAP معمولاً شامل استفاده از یک مدل چند بُعدی است، که از یک ساختار ستاره یا شکل شنگلاند برای نمایش داده‌ها استفاده می‌کند. (فصل ۸ به طور جزئی این ساختارها را بحث خواهد کرد.)

به عنوان مثال، یک شرکت تجارتی ممکن است از یک پایگاه داده OLAP با کوب‌های OLAP برای تحلیل سریع داده‌های فروش استفاده کند. این داده‌های از قبل تجمع شده بر اساس زمان سازماندهی شده‌اند، که اجازه می‌دهد تحلیل روندهای فروش را بدون توجه به سالانه، روزانه یا هر نوع میانگین دیگری انجام دهد. داده‌ها همچنین بر اساس مکان مرتب شده‌اند، که موجه کردن مقایسات جغرافیایی را تسهیل می‌کند تا به عنوان مثال، فروش محصولات در شهرهای مختلف را ببینید. علاوه بر این، داده‌ها بر اساس محصول دسته‌بندی شده‌اند، که باعث می‌شود ردیابی عملکرد موارد فردی آسان‌تر شود. اگر یک مدیر منطقه نیاز داشته باشد که به سرعت ببیند چگونه یک دسته محصول خاص در منطقه خود در طول فصل تعطیلات گذشته عمل کرده است، کوب OLAP قبلاً داده‌ها را آماده کرده است. به جای انتظار برای یک پرس و جو طولانی برای مرتب سازی هر فروش فردی، مدیر داده‌های مربوطه را پیدا می‌کند - مرتب شده بر اساس محصول، منطقه و زمان - که به راحتی در دسترس است، فرایند تصمیم‌گیری را تسریع می‌کند.

شما می‌توانید از یک مدل داده جدولی جدیدتر و بیشتر استفاده کنید، که عملکرد پرس و جو مشابهی با یک کوب OLAP ارائه می‌دهد. به جای یک مدل چند بُعدی، مدل داده جدولی از یک ساختار جدولی مشابه یک جدول پایگاه داده رابطه‌ای برای نمایش داده‌ها استفاده می‌کند. این آن را بیشتر انعطاف‌پذیر و ساده‌تر از یک مدل چند بُعدی می‌کند. این کتاب به مدل چند بُعدی اشاره می‌کند به عنوان یک "کوب OLAP" و به مدل جدولی به عنوان یک "کوب جدولی"، اما لطفاً توجه داشته باشید که اصطلاح "کوب" در صنعت به طور متقابل برای نشان دادن هر یک از این مدل‌ها استفاده می‌شود.

Table 7-1. Comparison of OLTP and OLAP systems

	OLTP	OLAP/Tabular
Processing type	Transactional	Analytical
Data nature	Operational data	Consolidation data
Orientation	Application oriented	Subject oriented
Purpose	Works on ongoing business tasks	Helps in decision making
Transaction frequency	Frequent transactions	Occasional transactions
Operation types	Short online transactions: Insert, Update, Delete	Lots of read scans
Data design	Normalized database (3NF)	Denormalized database
Common usage	Used in retail sales and other financial transactions systems	Often used in data mining, sales, and marketing
Response time	Response time is instant	Response time varies from seconds to hours
Query complexity	Simple and instant queries	Complex queries
Usage pattern	Repetitive usage	Ad hoc usage
Transaction nature	Short, simple transactions	Complex queries
Database size	Gigabyte database size	Terabyte database size

پایگاه‌های داده OLAP و انبارهای داده (Data Warehouses - DW) (مشاهده فصل ٤) مفاهیم مرتبط اما متمایز هستند و اغلب به همراه استفاده می‌شوند. از طریق یک مدل چند بُعدی یا جدولی، پایگاه‌های داده OLAP راهی برای تحلیل داده‌های ذخیره شده در DW ارائه می‌دهند که از نظر انعطاف‌پذیری و تعاملی بیشتر از اجرای پرس و جوهای SQL سنتی بر روی یک DW که حاوی مقادیر زیادی از داده‌ها است، است. مدل‌های چند بُعدی و مدل‌های جدولی اغلب به عنوان لایه‌ها یا مدل‌های معنایی در نظر گرفته می‌شوند، به این معنی که یک سطح انعطاف‌پذیری بر روی ساختارهای موجود در DW فراهم می‌کنند.

داده‌های عملی و تحلیلی

داده‌های عملیاتی داده‌های واقعی و زمان واقعی هستند که برای مدیریت عملیات روزمره و فرآیندها استفاده می‌شوند. این داده‌ها توسط سیستم‌های OLTP ضبط، ذخیره‌سازی و پردازش می‌شوند. شما می‌توانید از آن‌ها برای گرفتن یک "عکس" از وضعیت فعلی کسب و کار استفاده کنید تا اطمینان حاصل کنید که عملیات به طور صاف و کارآمد اجرا می‌شوند. داده‌های عملیاتی معمولاً حجم بالایی دارند و در تصمیم‌گیری سریع کمک می‌کنند.

داده‌های تحلیلی از جمع‌آوری و تبدیل داده‌های عملیاتی می‌آیند. این یک دیدگاه تاریخی از داده‌ها است که توسط سیستم‌های OLAP/Tabular و DWs نگهداری و استفاده می‌شود. با استفاده از ابزارهای تحلیل داده، می‌توانید تحلیل انجام دهید تا روندها، الگوها و روابط طی زمان را درک کنید. داده‌های تحلیلی برای ایجاد گزارشات و نمایش‌ها بهینه‌سازی شده‌اند و برای آموزش مدل‌های یادگیری ماشین. معمولاً نمایش داده‌ها در یک دوره بلندتری نسبت به داده‌های عملیاتی را فراهم می‌کند، معمولاً حجم کمتری دارد و معمولاً تجمع و تجزیه و تحلیل می‌شود. داده‌ها معمولاً در دسته‌ها مصرف می‌شوند و نیاز به زمان پردازش بیشتری نسبت به داده‌های عملیاتی دارند.

در خلاصه، داده‌های عملیاتی برای نظارت و کنترل فرآیندهای کسب و کار در زمان واقعی استفاده می‌شوند، در حالی که داده‌های تحلیلی برای به دست آوردن بینش‌ها و اطلاع‌رسانی به تصمیم‌گیری طی یک دوره بلندتر استفاده می‌شوند. هر دو نوع داده برای مدیریت کسب و کار موثر ضروری هستند و یکدیگر را تکمیل می‌کنند تا یک دیدگاه کامل از عملیات و عملکرد یک سازمان را فراهم کنند.

OLTP را به عنوان فناوری استفاده شده برای پیاده‌سازی داده‌های عملیاتی در نظر بگیرید و OLAP/Tabular و DWs را به عنوان فناوری استفاده شده برای پیاده‌سازی داده‌های تحلیلی در نظر بگیرید.

پردازش موازی انبوه داده و پردازش متقارن آنها

برخی از اولین پایگاه‌های داده رابطه‌ای از طراحی پردازش موازی متقارن (SMP) استفاده کردند، که در آن پردازش کامپیوتری توسط چندین پردازنده انجام می‌شود که دیسک و حافظه را به اشتراک می‌گذارند، همه در یک سرور - به عنوان مثال Server SQL و Oracle. (این در سمت چپ شکل ۷-۲ نشان داده شده است.) برای به دست آوردن قدرت پردازش بیشتر برای این سیستم‌ها، شما "به صورت افزایش" می‌کنید با افزایش پردازنده‌ها و حافظه در سرور. این کار خوب برای پایگاه‌های داده OLTP عمل می‌کند اما برای محیط نوشتن یک بار، خواندن بسیاری از یک DW خوب نیست.

با رشد محبوبیت انبارهای داده در دهه ۱۹۹۰ و شروع به مصرف مقادیر بزرگی از داده، عملکرد مشکل بزرگی شد. برای کمک به این موضوع، یک نوع جدید از طراحی پایگاه داده ظاهر شد. در طراحی پردازش موازی بسیار گسترده (MPP)، پایگاه داده شامل چندین سرور است، هر کدام با چندین پردازنده، و (متفاوت از SMP) هر پردازنده حافظه و دیسک خود را دارد. این امکان را به شما می‌دهد تا "به صورت گسترش" (به جای افزایش) با افزودن سرورهای بیشتر انجام دهید.

سرورهای MPP بخشی‌ـ از داده‌های پایگاه داده را به دیسک هر سرور (که در حالی که پایگاه داده‌های SMP همه داده‌ها را روی یک دیسک نگهداری می‌کنند) توزیع می‌کنند. سپس پرس و جوها به یک گره کنترل (همچنین گره نام نامیده می‌شود) ارسال می‌شوند که هر پرس و جو را به چندین زیرپرس و جو تقسیم می‌کند که به هر سرور (که گره محاسبه یا گره کارگر نامیده می‌شود) ارسال می‌شوند، همانطور که در سمت راست شکل ۷-۲ نشان داده شده است. در آنجا، زیرپرس و جو اجرا می‌شود و نتایج از هر گره محاسبه به گره کنترل بازگردانده می‌شوند، ترکیب شده، و به کاربر بازگردانده می‌شوند. این چگونگی کارکرد راه حل‌هایی مانند Teradata و Netezza را نشان می‌دهد.

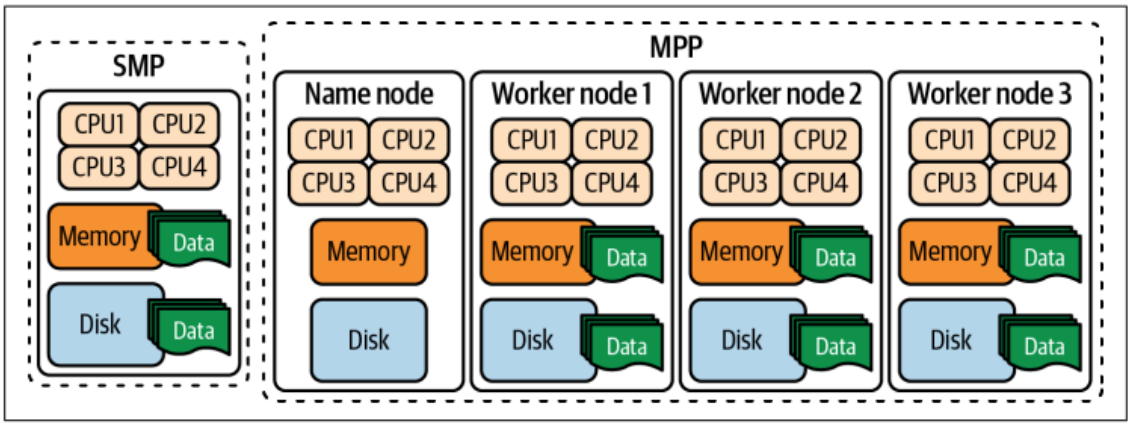


Figure 7-2. SMP and MPP database designs

به عنوان مثال، تصور کنید دوست شما Fiona یک بازی ۵۲ کارت دارد و به دنبال آس کارت قلب است. به طور متوسط برای Fiona حدود ۱۵ ثانیه زمان می‌برد تا کارت را پیدا کند. شما می‌توانید با "افزایش قدرت" با جایگزین کردن Fiona با دوست دیگری که سریع‌تر است، مانند Max، این کار را انجام دهید. استفاده از Max باعث می‌شود زمان متوسط به ۱۰ ثانیه کاهش یابد - یک بهبود محدود. این چگونگی کارکرد پایگاه‌های داده SMP است.

حالا تصور کنید شما به جای "افزایش قدرت" با "گسترش" می‌روید، با جایگزین کردن Fiona با ۲۶ نفر، هر کدام فقط دو کارت دارند. حالا زمان متوسط برای پیدا کردن کارت فقط ۱ ثانیه است.این چگونگی کارکرد پایگاه‌های داده MPP است. پایگاه‌های داده SMP و MPP به عنوان راه حل‌های محلی شروع شدند و این ها هنوز هم رایج هستند، اما اکنون راه حل‌های معادل در ابر وجود دارد.

معماری لامبدا یک معماری پردازش داده است که طراحی شده است تا با استفاده از روش‌های پردازش داده در حالت دسته‌ای و جریان زمان واقعی، مقادیر بزرگی از داده را مدیریت کند. ایده این است که برای دریافت نماهای جامع و دقیق از داده‌های دسته‌ای و تعادل بین تاخیر، عملکرد، مقیاس‌پذیری و تحمل خطا با استفاده از پردازش دسته‌ای، همزمان با استفاده از پردازش جریان زمان واقعی برای ارائه نماهای داده‌های آنلاین (مانند دستگاه‌های IoT، خوراک‌های Twitter یا فایل‌های لاگ کامپیوتر)، نماهای دو خروجی را قبل از لایه ارائه/خدمت به هم متصل کنید.

معماری لامبدا فاصله بین منبع تاریخی "یک منبع واحد حقیقت" و راه حل زمان واقعی بسیار مورد تلاقی با ترکیب سیستم‌های پردازش دسته‌ای سنتی با ابزارهای مصرف جریان برای برآورده کردن هر دو نیاز را پر می‌کند.

معماری لامبدا سه اصل کلیدی دارد:

۱. **مدل داده دوگانه**: معماری لامبدا یک مدل برای پردازش دسته‌ای (لایه دسته‌ای) و یک مدل دیگر برای پردازش زمان واقعی (لایه جریان) استفاده می‌کند. این امکان را می‌دهد تا سیستم هر دو نوع داده را مدیریت کند و هر دو نوع پردازش را به شکل قابل مقیاس‌پذیری و تحمل خطا انجام دهد.

۲. **نمای یکپارچه متحد**: معماری لامبدا یک نمای یکپارچه متحد (که به عنوان لایه ارائه نامیده می‌شود) را برای ارائه نتایج هر دو پردازش دسته‌ای و زمان واقعی به کاربران نهایی استفاده می‌کند. این امکان را می‌دهد تا کاربر یک نمای کامل و به‌روز از داده‌ها را ببیند، حتی اگر توسط دو سیستم مختلف پردازش شده باشد.

۳. **لایه‌های پردازش جدا شده**: معماری لامبدا لایه‌های پردازش دسته‌ای و زمان واقعی را جدا می‌کند تا بتوانند به صورت مستقل مقیاس شوند و به طور جداگانه توسعه و نگهداری شوند، که امکان انعطاف‌پذیری و آسانی توسعه را فراهم می‌کند.

شکل ۷-۳ یک بررسی سطح بالا از معماری لامبدا را نشان می‌دهد. در سمت چپ شکل ۷-۳، لایه مصرف داده قرار دارد. اینجا که داده‌ها را از تمام سیستم‌های منبع وارد می‌کنید. برخی منابع ممکن است داده را جریان دهند، در حالی که دیگران فقط داده را روزانه یا ساعتی ارائه می‌دهند.

در میانه بالا، شما لایه جریان را می‌بینید، همچنین به عنوان لایه سرعت نامیده می‌شود. این لایه برای به‌روزرسانی تدریجی امکان می‌دهد، بنابراین پیچیده‌ترین از دو لایه میانی است. این تعویض دقت با تاخیر کم می‌کند، با نگاه به فقط داده‌های اخیر. داده‌های در اینجا ممکن است فقط چند ثانیه پشت باشند، اما معامله این است که ممکن است تمیز نباشد. داده‌های در این لایه معمولاً در یک دریاچه داده ذخیره می‌شوند.

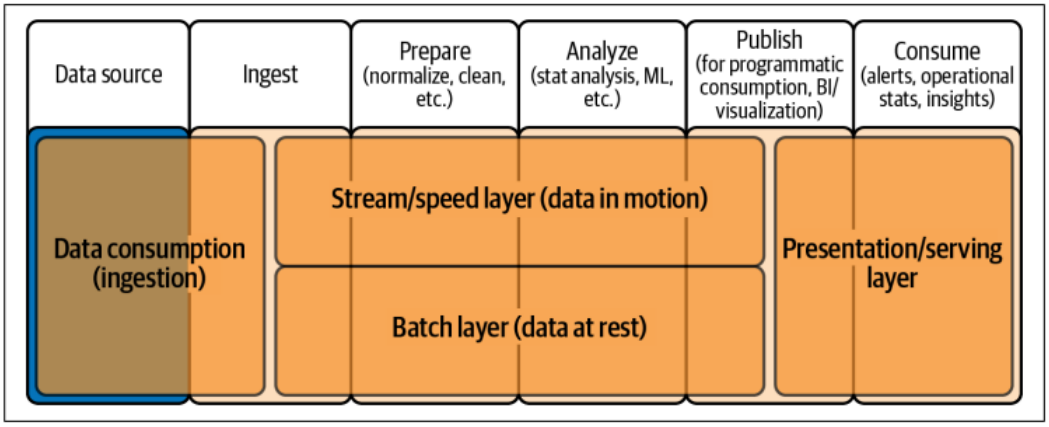


Figure 7-3. Overview of Lambda architecture

زیر از لایه جریان در شکل ۷-۳، لایه دسته‌ای قرار دارد که به طور کلی به تمام داده‌ها نگاه می‌کند و در نهایت داده‌هایی که به لایه جریان وارد می‌شوند را اصلاح می‌کند. این منبع واحد حقیقت است، لایه اعتماد به آن می‌رسد.

در اینجا معمولاً بسیاری از ETL وجود دارد و داده‌ها در یک انبار داده سنتی یا دریاچه داده ذخیره می‌شوند. این لایه با استفاده از یک برنامه زمان‌بندی پیش‌تعیین شده، معمولاً روزانه یا ساعتی، ساخته می‌شود و شامل وارد کردن داده‌های فعلاً در لایه جریان است.

در سمت راست شکل ۷-۳، لایه ارائه نیز وجود دارد، همچنین به عنوان لایه خدمت نامیده می‌شود. این را به عنوان یک متوسط‌کننده تصور کنید؛ وقتی پرس و جوها را پذیرفته می‌شود، تصمیم می‌گیرد که کی لایه دسته‌ای و کی لایه سرعت را استفاده کند. این لایه معمولاً به لایه دسته‌ای می‌پردازد، زیرا داده‌های اعتماد به آن وجود دارد، اما اگر شما از آن برای داده‌های به‌روز به‌روز درخواست کنید (ممکن است با تنظیم هشدارها برای پیام‌های لاگ خاصی که نشان می‌دهند یک سرور در حال خاموشی است)، از لایه جریان استخراج خواهد کرد. این لایه باید بین بازیابی داده‌هایی که می‌توانید به آن اعتماد کنید و بازیابی داده‌هایی که در حال حاضر می‌خواهید بازیابی کند، تعادل برقرار کند.

معماری لامبدا یک انتخاب عالی برای ساخت سیستم‌های توزیع شده است که نیاز دارند تا هر دو نوع داده دسته‌ای و زمان واقعی، مانند موتورهای توصیه و سیستم‌های تشخیص تقلب، را مدیریت کنند. با این حال، این به معنی نیست که برای هر شرایط بهترین انتخاب است. برخی از مشکلات احتمالی معماری لامبدا عبارتند از:

- **پیچیدگی**: معماری لامبدا شامل یک مدل داده دوگانه و یک نمای یکپارچه متحد است. این می‌تواند پیچیده‌تر باشد تا پیاده‌سازی و نگهداری نسبت به معماری‌های دیگر.

- **پردازش زمان واقعی محدود**: معماری لامبدا برای هر دو پردازش دسته‌ای و زمان واقعی طراحی شده است، اما ممکن است به طور کامل به مدیریت حجم بالای داده‌های زمان واقعی با معماری Kappa (که به طور خاص برای پردازش زمان واقعی طراحی شده است) کارآمد نباشد.

پشتیبانی محدود برای پردازش حالت‌محور

معماری لامبدا برای پردازش بدون حالت طراحی شده است و ممکن است برای برنامه‌هایی که نیاز به حفظ حالت در طول رویدادهای متعدد دارند، مناسب نباشد. به عنوان مثال، فرض کنید یک فروشگاه با یک سیستم توصیه‌گر که محصولات را بر اساس الگوهای مرور و خرید مشتریان پیشنهاد می‌دهد. اگر این سیستم از معماری لامبدا استفاده کند، که هر رویداد را به طور جداگانه بدون حفظ حالت پردازش می‌کند، ممکن است سفر خرید مشتری و نیت او را از دست بدهد. اگر مشتری برای کفش‌ها مرور کند، سپس جوراب‌ها، و سپس پولیش کفش، یک سیستم بدون حالت ممکن است محصولات مرتبط مانند رشته‌های کفش یا ذخیره‌سازی کفش را به درستی پیشنهاد نکند، زیرا توانایی درک ترتیب رویدادها را ندارد. همچنین ممکن است محصولاتی را پیشنهاد کند که قبلاً در سبد خرید مشتری قرار دارند.

در کل، شما باید معماری لامبدا را در نظر بگیرید اگر نیاز به ساخت یک سیستم توزیع شده دارید که قادر به مدیریت هر دو نوع داده دسته‌ای و زمان واقعی باشد اما نیاز به ارائه یک نمای یکپارچه از داده‌ها دارد. اگر نیاز به پردازش حالت‌محور دارید یا برای مدیریت حجم بالای داده‌های زمان واقعی نیاز دارید، ممکن است بخواهید معماری Kappa را در نظر بگیرید.

معماری کاپا

در مقابل معماری لامبدا که طراحی شده است تا هر دو نوع داده دسته‌ای و زمان واقعی را مدیریت کند، معماری Kappa فقط برای مدیریت داده‌های زمان واقعی طراحی شده است. مانند معماری لامبدا، معماری Kappa نیز برای مدیریت سطوح بالای همزمانی و حجم بالای داده‌ها طراحی شده است. شکل ۷-۴ یک بررسی سطح بالا از معماری Kappa را ارائه می‌دهد.

سه اصل کلیدی معماری Kappa عبارتند از:

۱. **پردازش زمان واقعی**: معماری Kappa برای پردازش زمان واقعی طراحی شده است، که به این معنی است که رویدادها همان زمانی که دریافت می‌شوند پردازش می‌شوند به جای اینکه بعداً به صورت دسته‌ای پردازش شوند. این کاهش تاخیر و امکان پاسخگویی سریع سیستم به شرایط تغییر کننده را فراهم می‌کند.

۲. **جریان رویداد تکی**: معماری Kappa یک جریان رویداد تکی را برای ذخیره تمام داده‌هایی که از طریق سیستم جریان می‌یابد، استفاده می‌کند. این امکان مقیاس‌پذیری و تحمل خطا را آسان می‌کند، زیرا داده‌ها می‌توانند به راحتی در سراسر چندین گره توزیع شوند.

۳. **پردازش بدون حالت**: در معماری Kappa، همه پردازش‌ها بدون حالت است. این به این معنی است که هر رویداد به طور مستقل پردازش می‌شود، بدون اعتماد به حالت رویدادهای قبلی. این باعث می‌شود تا سیستم راحت‌تر قابل مقیاس‌پذیری باشد، زیرا نیازی به حفظ حالت در سراسر چندین گره وجود ندارد.

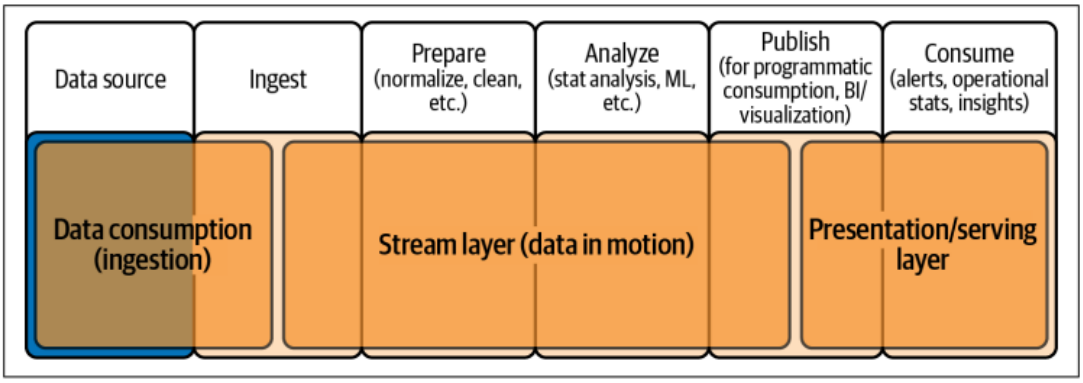


Figure 7-4. Overview of Kappa architecture

لایه‌های معماری Kappa دقیقاً همان لایه‌های معماری Lambda هستند، به جز اینکه معماری Kappa لایه دسته‌ای ندارد. برخی از مشکلات احتمالی معماری Kappa عبارتند از:

- **پیچیدگی**: معماری Kappa شامل یک جریان رویداد تکی و پردازش بدون حالت است، که می‌تواند پیچیده‌تر باشد تا پیاده‌سازی و نگهداری نسبت به معماری‌های دیگر.

- **پردازش دسته‌ای محدود**: معماری Kappa برای پردازش زمان واقعی طراحی شده است و به راحتی پشتیبانی از پردازش دسته‌ای از داده‌های تاریخی را نمی‌دهد. اگر نیاز به انجام پردازش دسته‌ای دارید، ممکن است بخواهید معماری Lambda را به عنوان گزینه دیگر در نظر بگیرید.

- **پشتیبانی محدود برای پرس و جوهای تصادفی**: به دلیل طراحی معماری Kappa برای پردازش زمان واقعی، ممکن است برای پرس و جوهای تصادفی که نیاز به پردازش مقادیر زیادی از داده‌های تاریخی دارند، مناسب نباشد.

در کل، معماری Kappa یک انتخاب عالی برای ساخت سیستم‌های توزیع شده است که نیاز به مدیریت مقادیر زیادی داده در زمان واقعی دارند و نیاز به قابلیت مقیاس‌پذیری، تحمل خطا، و کم تاخیر دارند. مثال‌هایی شامل پلتفرم‌های جریانی و سیستم‌های معاملاتی مالی است. با این حال، اگر نیاز به انجام پردازش دسته‌ای یا پشتیبانی از پرس و جوهای تصادفی دارید، ممکن است معماری Lambda گزینه بهتری باشد.

توجه داشته باشید که معماری‌های Lambda و Kappa برنامه‌های طراحی سطح بالا هستند که می‌توانند در هر یک از معماری‌های داده توصیف شده در قسمت سه کتاب پشتیبانی شوند. اگر از یکی از این معماری‌ها برای ساخت یک راه حل استفاده کنید که هر دو نوع داده دسته‌ای و زمان واقعی را پشتیبانی می‌کند، آن معماری معماری Lambda را پشتیبانی می‌کند؛ اگر از یکی از این معماری‌ها برای ساخت یک راه حل استفاده کنید که فقط داده‌های زمان واقعی را پشتیبانی می‌کند، آن معماری معماری Kappa را پشتیبانی می‌کند.

چندنوعی در ذخیره سازی و منابع داده

پایگاه داده‌های چندزبانه (Polyglot Persistence) اصطلاحی است که به معنی استفاده از تکنولوژی‌های مختلف ذخیره‌سازی داده برای ذخیره انواع مختلف داده در یک برنامه یا سیستم تکی است، بر اساس اینکه چگونه داده استفاده خواهد شد. انواع مختلف داده بهتر است در فرودگاه‌های داده مختلف نگهداری شوند. به طور کلی، پایگاه داده‌های چندزبانه به انتخاب ابزار مناسب برای هر مورد است. این ایده همان ایده است که پشت برنامه‌نویسی‌ـ چندزبانه قرار دارد، در که برنامه‌ها با استفاده از ترکیبی از زبان‌ها نوشته می‌شوند تا از قوت‌های مختلف زبان‌ها در حل مشکلات مختلف بهره ببرند.

در مقابل، یک فرودگاه داده چندزبانه به استفاده از چندین فرودگاه داده در سراسر یک سازمان یا کارآفرینی اشاره دارد. هر فرودگاه داده برای یک نوع خاص از داده یا مورد استفاده بهینه‌سازی شده است. این رویکرد به سازمان‌ها اجازه می‌دهد تا برای پروژه‌ها یا واحدهای تجاری مختلف از فرودگاه‌های داده مختلف استفاده کنند، به جای یک رویکرد یک اندازه مناسب برای کل سازمان.

به عنوان مثال، فرض کنید که در حال ساخت یک پلتفرم تجارت الکترونیکی هستید که با انواع مختلف داده مانند سبد خرید، موجودی، سفارشات تکمیل شده و غیره کار می‌کند. به جای تلاش برای ذخیره همه انواع مختلف داده در یک پایگاه داده، که نیاز به تبدیل زیادی دارد، می‌توانید یک رویکرد پایگاه داده‌های چندزبانه را بگیرید و هر نوع داده را در پایگاه داده مناسب برای آن ذخیره کنید. بنابراین، یک پلتفرم تجارت الکترونیکی ممکن است مانند نمودار در شکل ۷-۵ باشد.

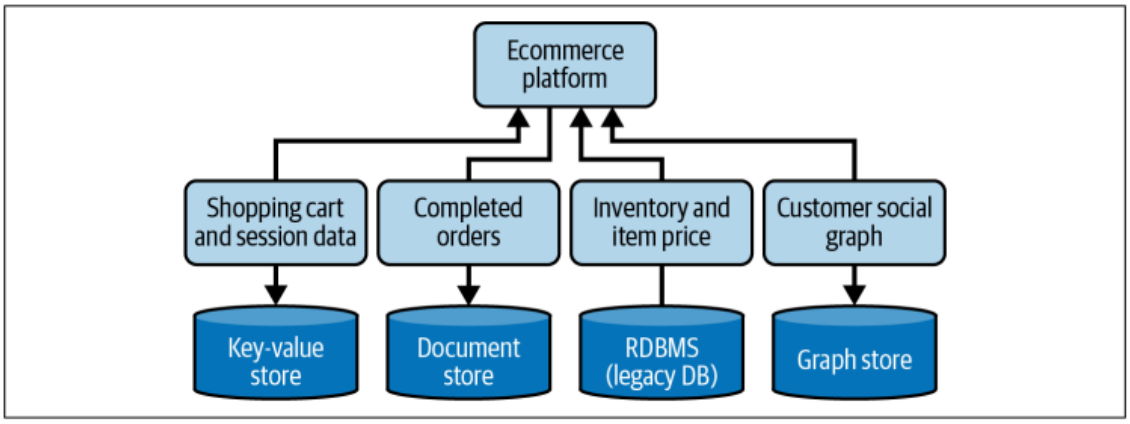


Figure 7-5. An ecommerce platform with a polyglot persistence design

این منجر به استفاده بهترین ابزار برای هر نوع داده می‌شود. در شکل ۷-۵، می‌توانید ببینید که پایگاه داده از یک فرودگاه مقادیر کلیدی-مقداری برای داده‌های سبد خرید و جلسه (که باعث بازیابی بسیار سریع می‌شود)، یک فرودگاه سندگری برای سفارشات تکمیل شده (که ذخیره و بازیابی داده‌های سفارش سریع و آسان می‌کند)، یک RDBMS برای موجودی و قیمت موارد (زیرا این موارد بهتر در یک پایگاه داده رابطه‌ای ذخیره می‌شوند به دلیل طبیعت ساختاری داده‌ها)، و یک فرودگاه گرافی برای گراف‌های اجتماعی مشتریان (زیرا بسیار دشوار است که داده‌های گرافی را در یک فرودگاه غیر گرافی ذخیره کنید) استفاده می‌کند.

این منجر به هزینه‌ای در پیچیدگی می‌شود، زیرا هر راه حل ذخیره‌سازی داده به یادگیری یک تکنولوژی جدید منجر می‌شود. اما منافع آن ارزش خواهد داشت. به عنوان مثال، اگر سعی کنید از پایگاه‌های داده رابطه‌ای برای داده‌های غیر رابطه‌ای استفاده کنید، طراحی می‌تواند به طور قابل توجهی به توسعه برنامه و عملکرد آن کاهش بزند؛ استفاده از نوع ذخیره‌سازی مناسب در سرعت برای شما می‌آید.

خلاصه

این فصل مفاهیم معماری و فلسفه‌های طراحی که پایه سیستم‌های داده موثر را تشکیل می‌دهند را بررسی کرد. ابتدا، شما درباره دو نوع اصلی سیستم‌های پردازش داده: پردازش تراکنش آنلاین (OLTP) و پردازش تحلیلی آنلاین (OLAP) یاد گرفتید. سیستم‌های OLTP برای تراکنش‌های سریع، قابل اعتماد و کوتاه طراحی شده‌اند، که معمولاً در پایگاه‌های داده عملیاتی که عملیات کسب و کار روزانه را تحت تأثیر قرار می‌دهند، استفاده می‌شوند. در مقابل، سیستم‌های OLAP پشتیبانی از پرس و جوهای تحلیلی پیچیده، تجمع‌ها و محاسبات را که برای تصمیم‌گیری استراتژیک استفاده می‌شوند، در یک انبار داده ارائه می‌دهند. سپس شما درباره تفاوت بین داده‌های عملیاتی و تحلیلی یاد گرفتید.

همچنین شما درباره تفاوت بین پردازش چند پردازشگر متقارن (SMP) و پردازش چند پردازشگر بزرگ (MPP) یاد گرفتید. سپس به دو معماری پردازش داده‌های بزرگ مدرن: Lambda و Kappa پرداختیم. در نهایت، مفاهیم پایگاه داده‌های چندزبانه و فرودگاه‌های داده چندزبانه را که تشویق به استفاده از بهترین تکنولوژی پایگاه داده برای نیازها و ویژگی‌های بار کار خاص داده‌های داده شده می‌کنند، بررسی کردیم.

از فصل بعدی به بعد، تمرکز ما از ذخیره‌سازی و پردازش داده به اصول و روش‌های مدل‌سازی داده منتقل می‌شود: پل مهمی که بین داده‌های خام و بینش‌های معنی‌دار ایجاد می‌کند. رویکردهای مدل‌سازی داده، مانند رویکردهای رابطه‌ای و بُعدی و مدل داده مشترک، به عنوان یک ساختار پایه عمل می‌کنند که به شما اجازه می‌دهد تا داده‌ها را به طور کارآمد در برنامه‌های متنوع استفاده و تفسیر کنید.

همانطور که در این موضوعات عمیق‌تر می‌شویم، می‌بینید که چگونه مدل‌سازی داده به شما اجازه می‌دهد تا از راه‌حل‌های ذخیره‌سازی و پردازش که ما مطالعه کردیم به طور کارآمد استفاده کنید، به عنوان یک طرح برای تبدیل داده‌های خام به بینش‌های قابل اعمال کمک می‌کند.

فصل هشتم: رویکردهای مدلسازی داده ها

مدل‌سازی داده یک تکنیک فنی بالایی است که برای طراحی پایگاه داده استفاده می‌شود. چندین نوع مدل‌سازی داده وجود دارد، از جمله مدل‌سازی رابطه‌ای و مدل‌سازی بُعدی. فرآیند شامل شناسایی داده‌هایی است که نیاز به ذخیره شدن دارند، سپس ایجاد یک نمایش ساختاری از آن داده‌ها و روابط بین داده‌ها، سازماندهی شده در جداول و ستون‌ها. جداول و ستون‌ها را به عنوان نمایش منطقی پایگاه داده در نظر بگیرید، جایی که داده‌های فیزیکی ذخیره شده در این جداول و ستون‌ها می‌توانند در یک محصول پایگاه داده رابطه‌ای یا یک دریاچه داده باشند.

شما می‌توانید مدل‌سازی داده را با هر نوع پایگاه داده استفاده کنید: رابطه‌ای، بُعدی، NoSQL و غیره. من توصیه می‌کنم که بسیاری از زمان را صرف مدل‌سازی داده کنید تا اطمینان حاصل کنید که پایگاه داده شما منطقی، کارآمد و آسان برای استفاده است. این باعث می‌شود که عملکرد بهینه را تضمین کند و بازیابی و تحلیل داده‌ها را آسان‌تر کند.

مدلسازی رابطه ای

مدل‌سازی رابطه‌ای، که توسط Edgar F. Codd در سال ۱۹۷۰ توسعه یافته است (همانطور که در فصل ۲ ذکر شده است)، یک تکنیک مدل‌سازی جزئی است که برای طراحی پایگاه داده استفاده می‌شود. این فرآیند شامل سازماندهی داده‌ها در جداول و تعریف روابط بین جداول است. در پایگاه‌های داده رابطه‌ای و انبارهای داده رابطه‌ای، هر جدول از سطرها (همچنین به عنوان رکوردها یا توپل‌ها شناخته می‌شوند) و ستون‌ها (همچنین به عنوان فیلدها یا ویژگی‌ها شناخته می‌شوند) تشکیل شده است. هر سطر نمایانگر یک نمونه منحصر به فرد از داده است، و هر ستون نمایانگر یک قطعه خاص از اطلاعات در مورد داده است.

مدل‌سازی رابطه‌ای به طور کلی به ما اجازه می‌دهد تا داده‌ها را به صورت ساختاری و منطقی در نظر بگیریم، که این ساختار و منطق به ما کمک می‌کند تا روابط بین داده‌ها را به طور واضح و قابل فهم تعریف کنیم. این رویکرد به ما امکان می‌دهد تا داده‌ها را به صورت منطقی و ساختاری در پایگاه داده ذخیره کنیم، که این امر برای ایجاد پایگاه داده‌های قابل اعتماد و کارآمد بسیار مهم است.

کلید ها

در مدل‌سازی رابطه‌ای، شما روابط بین جداول، سطرها و ستون‌ها را با استفاده از کلیدهای اصلی (Primary Keys) و کلیدهای خارجی (Foreign Keys) تعریف می‌کنید. یک کلید اصلی یک شناسه منحصر به فرد برای هر رکورد در یک جدول است که اطمینان می‌دهد که دو سطر داده‌ای هیچ کلید مشابهی نداشته باشند. یک کلید خارجی یک ستون در یک جدول است که به کلید اصلی در یک جدول دیگر اشاره می‌کند. این برای برقراری رابطه بین دو جدول استفاده می‌شود تا اطمینان حاصل کند که یکپارچگی داده‌ها حفظ شود. به عنوان مثال، ممکن است در یک جدول محصولات یک کلید منحصر به فرد با نام ProductKey داشته باشید که به یک کلید خارجی در یک جدول واقعیت‌های فروش با نام ProductKey اشاره می‌کند. یک کلید طبیعی یک فیلد است که در یک جدول وجود دارد و برای هر سطر منحصر به فرد است، مانند شماره بیمه اجتماعی یک فرد یا شماره سریال یک محصول. در مدل‌سازی رابطه‌ای، یک کلید طبیعی معمولاً به عنوان کلید اصلی استفاده می‌شود.

نمودار روابط موجودیت ها

معمولاً شروع به مدل‌سازی رابطه‌ای با یک نمودار رابطه موجودیت (ER) می‌کنید: یک ساختار بصری سطح بالا از پایگاه داده که موجودیت‌ها (داده‌ها) و روابط بین آن‌ها را نمایش می‌دهد. این نمودار از جعبه‌ها برای موجودیت‌ها و خطوط اتصال برای روابط استفاده می‌کند. شکل ۸-۱ یک مثال از یک نمودار ER را نشان می‌دهد. پس از تکمیل آن، می‌توانید یک مدل رابطه‌ای جزئی‌تر را بسازید که جداول و ستون‌های واقعی یک پایگاه داده را نمایش می‌دهد.

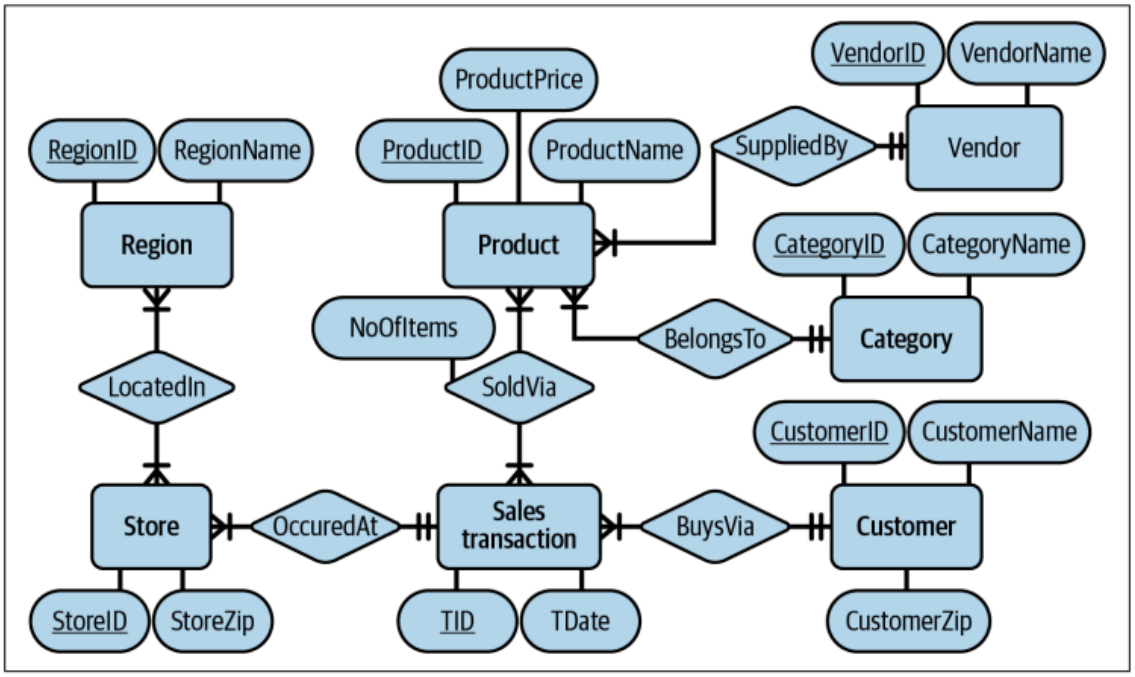


Figure 8-1. An entity–relationship diagram

نرمال/عادی سازی نقش ها و فرم ها

بعد از اینکه شما با استفاده از نمودار ER شروع به مدل‌سازی رابطه‌ای کردید، قوانین نرمال‌سازی را اعمال می‌کنید، که روشی برای تجزیه یک پایگاه داده پیچیده به جداول کوچکتر و ساده‌تر است. این کاهش تکرار و وابستگی را حداقل می‌کند، یکپارچگی داده را بهبود می‌بخشد و پایگاه داده را کارآمدتر و آسان‌تر برای نگهداری و مدیریت می‌کند. این کار را با اعمال سطوح متوالی نرمال‌سازی انجام می‌دهید.

یک جدول در فرم اول نرمال (NF۱) است اگر تمام شرایط زیر را برآورده کند:

- جدول یک کلید اصلی دارد. به عنوان مثال، در یک جدول دانشجویان، StudentID می‌تواند کلید اصلی باشد.

- هر ویژگی در جدول یک مقدار تکی دارد، نه یک لیست از مقادیر. به عنوان مثال، در جدول دانشجویان، هر ستون (ویژگی)، مانند نام، یک مقدار تکی نگهداری می‌کند.

- جدول گروه‌های تکراری ستون‌ها ندارد. به عنوان مثال، در جدول دانشجویان، نباید چندین ستون "درس" مانند Course ،۱Course ،۲Course، و ۳Course داشته باشید. به جای آن، هر جفت دانشجو-درس باید خود را در یک سطر داشته باشد.

یک جدول در فرم دوم نرمال (NF۲) است اگر تمام شرایط NF۱ را برآورده کند، به علاوه:

- هر جزئیات (ویژگی غیر کلیدی) در رکورد پایگاه داده باید به طور کامل بر اساس شناسه منحصر به فرد خود (کلید اصلی) و نه بر اساس هر جزئیات دیگری بستگی داشته باشد. به عنوان مثال، در یک جدول دانشجویان با ستون‌های StudentID، نام، و رشته تخصصی، رشته تخصصی دانشجو (ویژگی غیر کلیدی) باید تنها بر اساس StudentID (کلید اصلی) تعیین شود، نه بر اساس نام یا هر ویژگی دیگری در جدول.

یک جدول در فرم سوم نرمال (NF۳) است اگر و فقط اگر تمام شرایط NF۱ و NF۲ را برآورده کند، به علاوه:

- هر جزئیات غیر کلیدی در جدول باید به طور مستقیم به شناسه اصلی (کلید اصلی) بستگی داشته باشد و نه از طریق یک جزئیات دیگر. به عنوان مثال، در یک جدول دانشجویان با ستون‌های StudentID، رشته تخصصی، و رئیس دانشکده (که رئیس رشته تخصصی است)، رئیس دانشکده نباید بر اساس رشته تخصصی باشد، که در نوبت خود نباید بر اساس StudentID باشد.

بیشتر مدل‌های رابطه‌ای، به خصوص برای پایگاه‌های داده OLTP (مشاهده فصل ۷)، در NF۳ هستند. این باید هدف نهایی شما باشد.

یک مدل رابطه‌ای از یک طرح ساختار پایگاه داده نرمال‌سازی شده استفاده می‌کند، که در آن داده در یک مکان ذخیره شده و به صورت چند جدول با روابط به طور سخت و سریع تعریف شده بین آن‌ها سازماندهی شده است. این کمک می‌کند تا یکپارچگی را تضمین کند، اما همچنین می‌تواند باعث شود پرس و جو زمان‌بر باشد، زیرا پایگاه داده احتمالاً نیاز دارد تا چندین جدول را با هم ادغام کند تا داده‌های مورد نظر را بازیابی کند.

شکل ۸-۲ نشان می‌دهد که چگونه یک مدل رابطه‌ای برای داده‌های فروش ممکن است به نظر برسد.

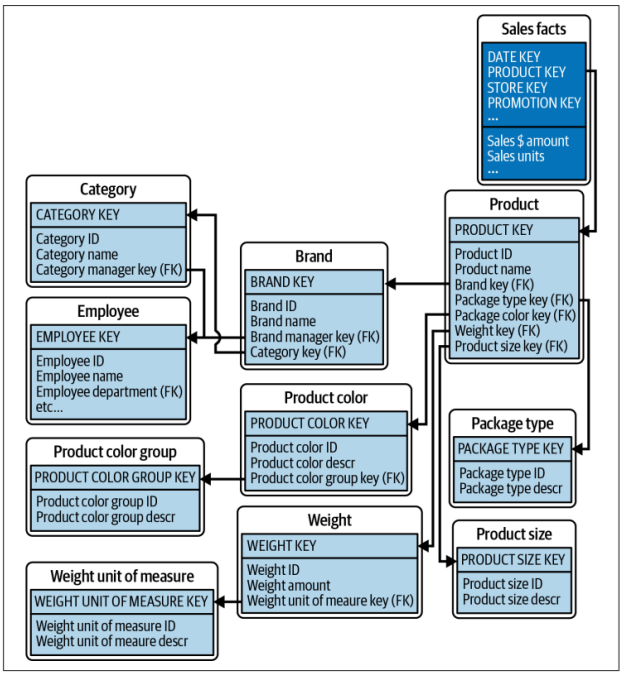


Figure 8-2. Example of a relational model for sales data

رهگیری تغییرات

ردیابی تغییرات در داده‌های مدل‌سازی شده رابطه‌ای در طول زمان مهم است. برای حفظ یک رکورد از داده‌های گذشته و تغییرات، اغلب از جداول تاریخچه استفاده می‌شود. یک جدول تاریخچه معمولاً یک کپی از جدول اصلی است با ستون‌های اضافی که برای ردیابی تغییرات اضافه شده‌اند، مانند یک برچسب تاریخ/زمان، یک شناسه کاربر، و یک مقدار "قبل" و "بعد" برای هر ستون در جدول. جداول تاریخچه می‌توانند برای بازرسی، گزارش‌گیری، و بازیابی داده‌ها مفید باشند، کمک کردن به حفظ یکپارچگی داده‌ها و تسهیل درک چگونگی تغییر داده‌ها در طول زمان.

مدلسازی چند بعدی

مدل‌سازی بُعدی در سال ۱۹۹۶ شروع شد به عنوان یک روش برای پشتیبانی از پرس و جوی کارآمد و تحلیل با سازماندهی داده به واقعیت‌ها و بُعدها. مدل‌های بُعدی زمانی نیاز دارند که پرس و جو و گزارش‌گیری از مدل رابطه‌ای خیلی آهسته یا پیچیده شود. مدل‌های بُعدی معمولاً از یک مدل رابطه‌ای به عنوان منبع داده استفاده می‌کنند.

جداول فکت و ابعاد و کلید ها

مدل‌های بُعدی از واقعیت‌ها و بُعدها استفاده می‌کنند:

واقعیت‌ها (Facts):

واقعیت‌ها، در این زمینه، قطعات داده هستند، معمولاً مقادیر عددی، که برای اندازه‌گیری چیزی مانند فروش، سفارشات، یا درآمد استفاده می‌شوند. واقعیت‌ها می‌توانند برای دلایل عملکردی، مانند محاسبه میانگین‌ها یا شمارش رکوردها، تجمع یا خلاصه شوند، به طوری که این مقادیر قبلاً محاسبه شده‌اند و نیازی به محاسبه آن‌ها به عنوان بخشی از اجرای پرس و جو ندارند.

بُعدها (Dimensions):

بُعدها ویژگی‌های داده را توصیف می‌کنند، مانند زمان، محصول، مشتری، یا مکان. آن‌ها معمولاً به صورت سلسله‌مراتبی نمایش داده می‌شوند، با هر سطح سلسله‌مراتبی یک توصیف دقیق‌تر از داده را فراهم می‌کند.

یک روش دیگر برای درک مدل‌های بُعدی این است که به یاد داشته باشید که جداول واقعیت معیارها را نگهداری می‌کنند، در حالی که جداول بُعد ویژگی‌های معیارها در جداول واقعیت را نگهداری می‌کنند.

به جای کلیدهای طبیعی استفاده شده در مدل‌سازی رابطه‌ای، مدل‌سازی بُعدی از کلیدهای جایگزین (Surrogate Keys) استفاده می‌کند: مقادیر مصنوعی که به طور خاص (و معمولاً به طور خودکار) برای خدمت به عنوان کلید

اصلی برای یک جدول ایجاد شده‌اند. یک کلید جایگزین معمولاً زمانی استفاده می‌شود که یک کلید طبیعی در دسترس نیست یا مناسب برای استفاده به عنوان کلید اصلی نیست، که این موضوع در مدل‌سازی بُعدی به طور مکرر رخ می‌دهد.

کلیدهای طبیعی اغلب معنادارتر و آسان‌تر برای فهم نسبت به کلیدهای جایگزین هستند، بنابراین می‌توانند استفاده از پایگاه داده را آسان‌تر و قابل نگهداری کنند. با این حال، نیز مشکلاتی در استفاده از کلیدهای طبیعی وجود دارد:

- آن‌ها می‌توانند طولانی‌تر و پیچیده‌تر از کلیدهای جایگزین باشند و بنابراین بیشتر به کار با آن‌ها چالش برانگیز باشد.

- آن‌ها اغلب اطلاعات حساس را شامل می‌شوند، که می‌تواند مسائل حریم خصوصی را ایجاد کند.

- آن‌ها می‌توانند به دلیل احتمال تکرار و فرمت‌های غیر استاندارد چالش‌هایی ایجاد کنند. به عنوان مثال، اگر شما از شناسه‌های مشتری به عنوان کلید طبیعی استفاده می‌کنید و شرکت شما یک شرکت دیگر را خریداری می‌کند که شناسه‌های مشتری مطابقت دارد یا یک فرمت متفاوت (مانند کاراکترها به جای اعداد صحیح) استفاده می‌کند، تضاداتی می‌تواند پیش آمده باشد. اصل اینجا این است که تکرارات می‌توانند مشکل باشند هنگام ادغام یا مقایسه داده‌ها در سیستم‌های مختلف، و ناهماهنگی در فرمت‌ها می‌تواند فرآیند یکپارچگی را بیشتر پیچیده کند.

رهگیری تغییرات

برای ردیابی تغییرات ایجاد شده در یک جدول در مدل‌سازی بُعدی، شما از بُعدهای تغییر کننده به آرامی (SCDs) استفاده خواهید کرد، که سه نوع دارند:

نوع ۱:

بُعدهای تغییر کننده نوع ۱ داده‌های موجود را با داده‌های جدید جایگزین می‌کنند و داده‌های قدیمی را حذف می‌کنند. این نوع ساده‌ترین و متداول‌ترین نوع از SCD است و معمولاً زمانی استفاده می‌شود که تغییرات در داده‌ها مهم نیستند یا زمانی که داده‌های قدیمی دیگر لازم نیستند، مانند زمانی که شماره تلفن یک مشتری را اصلاح می‌کنید.

نوع ۲:

بُعدهای تغییر کننده نوع ۲ چندین نسخه از داده‌ها را حفظ می‌کنند: داده‌های جدید و یک رکورد از داده‌های قدیمی. این نوع از SCD را زمانی استفاده کنید که نیاز به ردیابی تغییرات در داده‌ها در طول زمان دارید و یک رکورد از داده‌های قدیمی را حفظ می‌کنید. به عنوان مثال، فرض کنید که شش ماه پیش، یک شرکت داده‌های فروش خود را تجزیه و تحلیل کرد و متوجه شد که نیویورک برترین استان فروش آن است. حالا اگر برخی از مشتریان از نیویورک به نیوجرزی مهاجرت کرده‌اند، اجرای همان گزارش بدون در نظر گرفتن این تغییرات می‌تواند نشان‌دهنده ارزش فروش کمتر برای نیویورک باشد. این می‌تواند منجر به یک درک اشتباه از داده‌های تاریخی که نشان می‌دهد فروش در نیویورک در حال کاهش است، شود که می‌تواند تصمیمات استراتژیک را تحت تأثیر قرار دهد. بنابراین، اگر یک مشتری از یک استان آمریکا به یک استان دیگر مهاجرت کند، SCD نوع ۲ شرکت هم استان قدیمی و هم استان جدید را حفظ می‌کند.

نوع ۳:

بُعدهای تغییر کننده نوع ۳ یک رکورد جدید برای هر تغییر ایجاد می‌کند تا شما بتوانید تاریخچه کامل داده‌ها را حفظ کنید. این نوع از SCD پیچیده‌ترین است اما همچنین بسیار انعطاف‌پذیرتر. به عنوان مثال، وقتی یک مشتری از یک استان آمریکا به یک استان دیگر مهاجرت می‌کند، یک SCD نوع ۳ رکورد کامل قدیمی و یک رکورد کامل جدید را حفظ می‌کند، که می‌تواند شامل ده‌ها فیلد باشد و فقط فیلد استان آمریکا نباشد.

یک تفاوت کلیدی بین جداول تاریخچه در مدل‌سازی رابطه‌ای و SCDs در سطح جزئیات آن‌ها است. جداول تاریخچه تغییرات در داده‌ها را در سطح رکوردهای فردی ردیابی می‌کنند، در حالی که SCDs تغییرات را در سطح بُعد ردیابی می‌کنند. و در حالی که جداول تاریخچه معمولاً برای ردیابی تغییرات در هر نوع داده‌ای استفاده می‌شوند، SCDs به طور خاص برای ردیابی تغییرات در جداول بُعد در یک مدل بُعدی استفاده می‌شوند.

شکل ۸-۳ یک مدل بُعدی برای فروش را نشان می‌دهد، که از مدل رابطه‌ای در شکل ۸-۲ مشتق شده است.

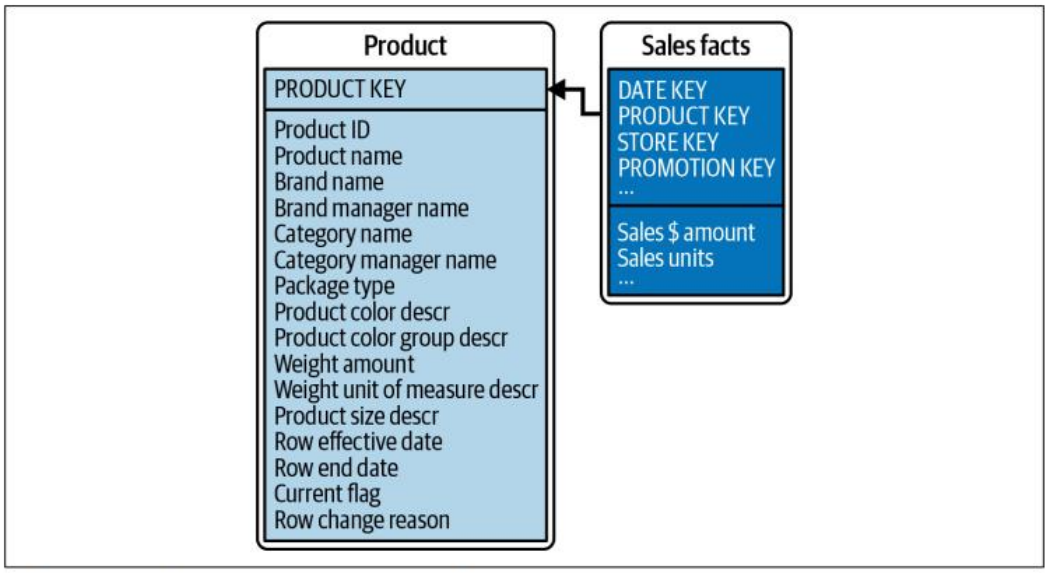


Figure 8-3. A dimensional model for sales data

غیر عادی سازی

مدل‌های بُعدی از یک فرآیند به نام غیر نرمال‌سازی (denormalization) استفاده می‌کنند، که در آن کپی‌های اضافی از داده‌ها در چندین جدول شامل می‌شوند. این کاهش تعداد جداول را ایجاد می‌کند. وقتی شما پایگاه داده را پرس و جو می‌کنید، نیازی به ادغام بسیاری از جداول به هم نیست، بنابراین پرس و جو بسیار سریع‌تر است. کمتر ادغام همچنین پیچیدگی را برای کاربران نهایی که گزارش‌ها را ایجاد می‌کنند، کاهش می‌دهد.

با این حال، این بدان معناست که کپی‌های اضافی از داده‌ها باید برای حفظ یکپارچگی در هماهنگی باشند، و این نیازمند نگهداری دقیق است. داده‌های اضافی همچنین استفاده از بیشتری از فضای ذخیره‌سازی را دارند، که ممکن است هزینه‌ها را کمی افزایش دهد.

در شکل ۸-۳، می‌توانید ببینید که CategoryName در لیست کلیدهای محصول شامل شده است. بنابراین، نام دسته‌بندی در تمام رکوردهای محصول تکرار می‌شود. در مقابل، در مدل رابطه‌ای در شکل ۸-۲، رکوردهای محصول نشان می‌دهند CategoryID. در مدل بُعدی، اگر نام دسته‌بندی تغییر کند، شما باید نام را در چندین رکورد در جدول محصول تغییر دهید؛ در مدل رابطه‌ای، شما فقط باید آن را یک بار تغییر دهید (در جدول دسته‌بندی).

بیشتر جداولی که در یک مدل رابطه‌ای دارید، مدل بُعدی بیشتر مفید خواهد بود. اگر فقط چندین جدول در مدل رابطه‌ای دارید، شاید نیازی به مدل بُعدی نداشته باشید.

یک طرح پایگاه داده یک طرح منطقی است که ساختار، روابط، محدودیت‌ها، و سایر عناصر چگونگی سازماندهی و ارتباط داده‌ها درون یک پایگاه داده را توصیف می‌کند. بسیاری از انواع طرح‌ها در مدل‌سازی بُعدی استفاده می‌شوند،

مانند طرح‌های شکل برف و طرح‌هایی با چندین جدول واقعیت. یک طرح ستاره یک تکنیک مدل‌سازی بُعدی است با یک جدول واقعیت مرکزی که اطراف آن توسط چندین جدول بُعد احاطه شده است.

نمودار طرح شبیه به یک ستاره است (شکل ۸-٤) و به طور خلاصه، یک مدل رابطه‌ای راه حل تجاری را برای چگونگی کارکرد بخشی- از کسب و کار را می‌گیرد، در حالی که مدل بُعدی جزئیاتی را که کسب و کار نیاز به آن‌ها برای پاسخ به سوالات در مورد عملکرد کسب و کار دارد، می‌گیرد. مدل رابطه‌ای ساده‌تر است برای ساخت اگر سیستم منبع قبلاً رابطه‌ای باشد، اما مدل بُعدی ساده‌تر برای کاربران تجاری استفاده است و معمولاً برای پرس و جوهای تحلیلی عملکرد بهتری خواهد داشت.

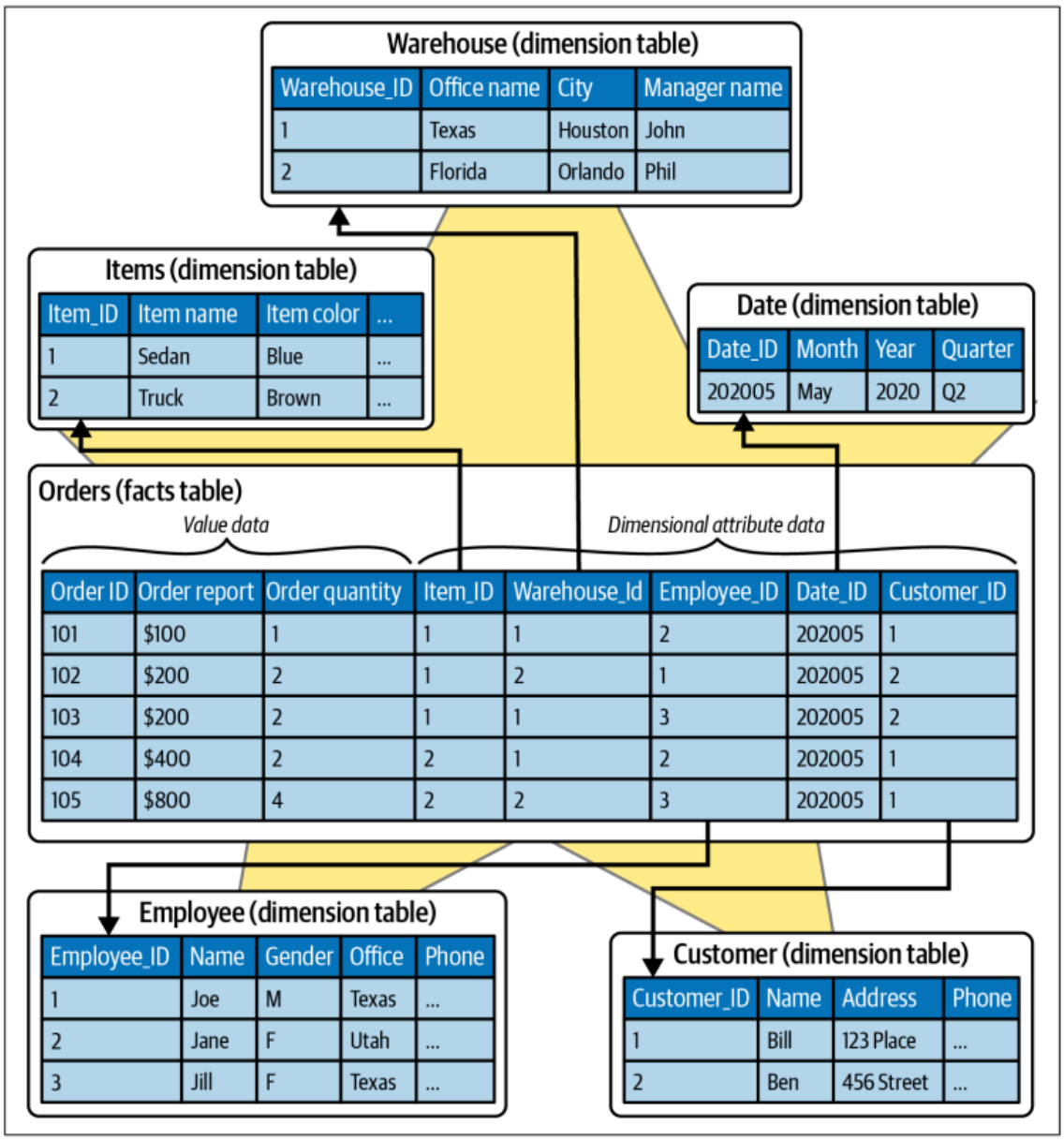


Figure 8-4. Example of a star schema

مدل داده مشترک

یک مدل داده مشترک (CDM) یک ساختار استاندارد برای ذخیره و سازماندهی داده‌ها است که معمولاً زمانی استفاده می‌شود که یک راه حل انبار داده ساخته می‌شود. این ساختار راهی یکپارچگی را برای نمایش داده‌ها درون جداول و روابط بین جداول فراهم می‌کند، که باعث می‌شود هر سیستم یا برنامه‌ای بتواند داده‌ها را بفهمد.

تصور کنید که شما دارید بسیاری از سیستم‌های منبع که می‌خواهید داده‌ها را به انبار داده خود وارد کنید. برخی از آن‌ها برنامه‌های مدیریت رابطه با مشتری (CRM) از شرکت‌های مختلف هستند که فرمت‌های مختلفی دارند (یعنی جداول و فیلدها متفاوت هستند).

شما باید تصمیم بگیرید که چه فرمت جدولی را در انبار داده استفاده کنید. آیا یکی از مدل‌های فروشندگان CRM را انتخاب می‌کنید؟ نه. بهتر است یک مدل جدید را ایجاد کنید که بتواند فرمت‌های همه این فروشندگان را مدیریت کند.

این مدل جدید (شکل ۸-۵) مدل داده مشترک شما است. ساخت آن نیاز به بسیاری از کار دارد، اما اگر به درستی انجام دهید، CDM شما قادر به پشتیبانی از داده‌های فعلی و آینده هر فروشنده CRM خواهد بود. خوشبختانه، شما نیازی به ساخت آن از صفر ندارید. بسیاری از ارائه‌دهندگان ابر و تولیدکنندگان نرم‌افزار مدل‌های CDM قوی و مناسب برای صنایع مانند بانکداری، سلامت، و بازاریابی را دارند. اگر شما قادر به سفارشی‌سازی یک مدل ساخته شده باشید، شما برای ساعات مدل‌سازی خود صرفه‌جویی خواهید کرد، خطر را کاهش خواهید داد، و به تمام برنامه‌های سازمان شما اجازه می‌دهد تا با استفاده از زبان مشترکی دسترسی به داده‌ها داشته باشند.

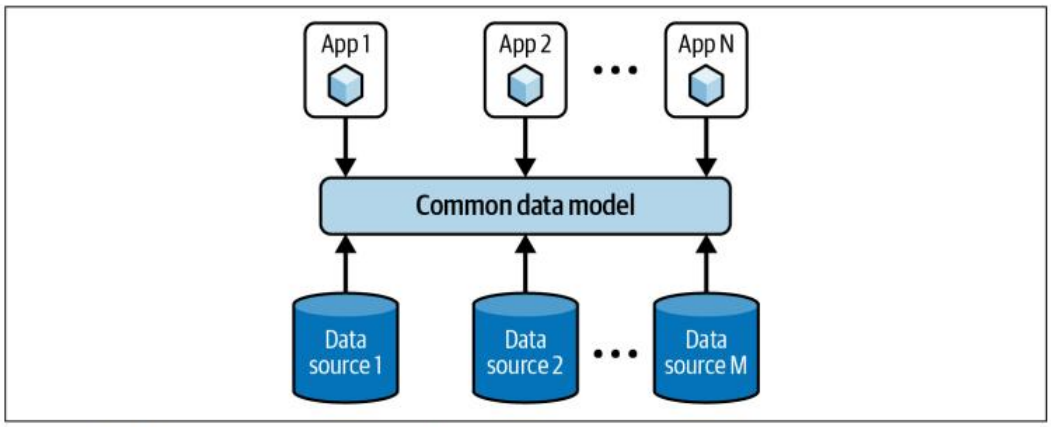


Figure 8-5. The common data model architecture

معماری داده

خزانه داده

توسط دانیل لینستدت در سال ۲۰۰۰ ایجاد شده، مدل‌سازی داده واولت (Data Vault Modeling) یک روش کامل است که به طور خاص برای استفاده در سیستم‌های انبار داده و سیستم‌های هوش مصنوعی طراحی شده است. هدف اصلی آن ارائه یک روش انعطاف‌پذیر، قابل مقیاس‌پذیر و استاندارد برای مدل‌سازی و مدیریت داده‌های تاریخی است.

یک مدل داده واولت (که در شکل ۸-۶ نشان داده شده است) اطراف سه نوع موجودیت ساخته شده است:

هاب‌ها (Hubs):

هاب‌ها موجودیت‌های کسب و کار مرکزی در مدل هستند و مفاهیم کسب و کار کلیدی مانند مشتریان یا محصولات را نمایش می‌دهند. هاب‌ها معمولاً به عنوان یک جدول تکی با یک شناسه منحصر به فرد (مانند کلید اصلی) و مجموعه‌ای از ویژگی‌ها مدل‌سازی می‌شوند.

پیوندها (Links):

پیوندها روابط بین هاب‌ها را مدل می‌کنند، معمولاً به عنوان یک جدول جداگانه. یک جدول پیوند شامل مجموعه‌ای از کلیدهای خارجی است که به کلیدهای اصلی هاب‌های مرتبط اشاره می‌کنند.

ماهواره‌ها (Satellites):

ماهواره‌ها ویژگی‌های توصیفی در مورد یک هاب یا پیوند را ذخیره می‌کنند، مانند تغییرات در داده‌ها در طول زمان. آن‌ها معمولاً به عنوان جداول جداگانه مدل‌سازی می‌شوند و از طریق یک کلید خارجی به یک جدول هاب یا پیوند متصل می‌شوند.

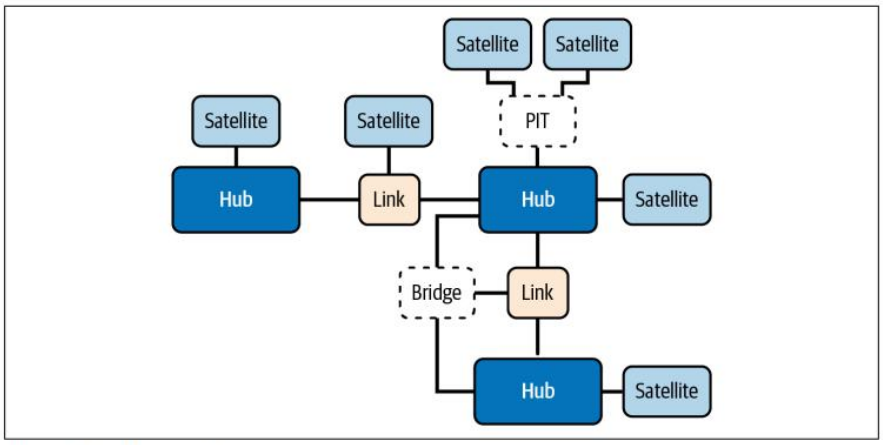


Figure 8-6. Data vault

مدل‌های مبتنی بر داده واولت به شما امکان می‌دهند تا پیگیری ارث‌بری داده، بازرسی داده‌ها، و اجرای قوانین و استانداردها را پیگیری کنید. آن‌ها اغلب با تکنیک‌های دیگر مدل‌سازی داده‌ها مانند مدل‌سازی بُعدی برای ارائه یک معماری داده جامع و انعطاف‌پذیر استفاده می‌شوند که به راحتی می‌توانند با سیستم‌ها و برنامه‌های دیگر یکپارچه شود. بر اساس مدل‌هایی که تا به حال بررسی کردیم، مدل داده واولت بین داده‌های NF۳ و یک مدل ستاره قرار می‌گیرد؛ این برای سازمان‌هایی با مقادیر زیادی از داده‌ها و روابط پیچیده داده مناسب است، در حالی که مدل کیمبال (که در "روش کیمبال از پایین به بالا" در صفحه ۱۱۵ بحث شده است) برای سازمان‌هایی با نیازهای داده ساده‌تر بهتر است.

برخی از عواقب اصلی مدل‌سازی داده واولت عبارتند از:

پیچیدگی:

مدل‌سازی داده واولت می‌تواند برای اجرا و نگهداری پیچیده باشد، به خصوص برای سازمان‌هایی که با تکنیک آشنا نیستند. این نیاز به سرمایه‌گذاری قابل توجهی از منابع و دانش متخصص دارد و درک و پیمایش در مدل داده بدون مستندات مناسب می‌تواند دشوار باشد.

تکرار داده:

مدل‌سازی داده واولت اغلب منجر به تکرار می‌شود، زیرا داده در سطح اتمی ذخیره می‌شود. این می‌تواند هزینه‌های ذخیره‌سازی را افزایش دهد و باعث شود تضمین یکپارچگی داده سخت‌تر شود.

عملکرد:

مدل‌سازی داده واولت می‌تواند منجر به تعداد زیادی از جداول و روابط شود، که می‌تواند عملکرد پرس و جو را کاهش دهد. شما ممکن است نیاز به استفاده از پرس و جوها و استراتژی‌های ایندکس پیچیده‌تر برای رسیدن به عملکرد قابل قبول داشته باشید.

کمبود استانداردسازی:

مدل‌سازی داده واولت یک تکنیک نسبتاً جدید و کمتر استفاده شده است و دارای یک رویکرد استانداردسازی نیست. پیدا کردن مهندسان برای ساخت و نگهداری آن می‌تواند چالش‌برانگیز باشد.

روش های ایجاد انبار داده کیمبال و اینمون

بیل اینمون به عنوان "پدر انبار داده" شناخته می‌شود به دلیل کار پیشروی او در زمینه انبار داده در اواخر دهه ۱۹۸۰. او کتاب اولیه در مورد انبار داده را در سال ۱۹۹۲ نوشت و نقش قابل توجهی در شناخت و پیشرفت مفهوم داشت.

رویکرد اینمون به انبار داده اغلب به عنوان یک رویکرد از بالا به پایین به مارت‌های داده که از مدل‌سازی رابطه‌ای استفاده می‌کند، شناخته می‌شود.

اصطلاحات "از بالا به پایین" و "از پایین به بالا" در اینجا هیچ ارتباطی با رویکردهای از بالا به پایین و از پایین به بالا برای انبارهای داده رابطه‌ای و دریاچه‌های داده که در فصل‌های ٤ و ٥ توضیح داده شده‌اند، ندارند. این رویکردها مربوط به کاری است که در ساخت معماری داده‌ای که شامل یک انبار داده رابطه‌ای (RDW) و یک دریاچه داده است (بین سایر جزئیات) انجام شده است. رویکردهای کیمبال و اینمون بیشتر بر روی چگونگی استفاده از مارت‌های داده در ساخت معماری داده‌ای که فقط شامل یک انبار داده رابطه‌ای است و دریاچه داده نیست (بین سایر جزئیات) تمرکز دارند.

رالف کیمبال، یک پیشرو دیگر در زمینه انبار داده، با نوشتن کتاب اولش، " The Data Warehouse Toolkit: Practical Techniques for Building Dimensional Data Warehouses" (Wiley, ۱۹۹٦)، مدل ستاره را در طراحی انبار داده محبوب کرد. رویکرد کیمبال اغلب به عنوان یک رویکرد از پایین به بالا شناخته می‌شود و همچنین به عنوان مدل‌سازی بُعدی یا متودولوژی کیمبال نیز شناخته می‌شود.

متودولوژی‌های اینمون و کیمبال برای ساخت یک انبار داده سال‌ها منجر به بحث‌ها شده‌اند. این بخش هر دو را با جزئیات بیشتری بررسی می‌کند و برخی از این دروغ‌های رایج را برطرف می‌کند.

اینمون رویکرد و روش بالا به پایین

روش متودولوژی اینمون تاکید بر ادغام داده‌ها از منابع متعدد و ارائه آن‌ها به گونه‌ای که در تصمیم‌گیری کمک کند، دارد. رویکرد از بالا به پایین یک روش سنتی، سختگیرانه و مشخص است و اغلب در سازمان‌های بزرگ و پیچیده که دارای مقدار زیادی از داده‌ها و مدیریت ارزش، کیفیت داده‌ها و تطابق با قوانین و مقررات هستند، ترجیح داده می‌شود. این رویکرد عمدتاً توسط دپارتمان فناوری اداره می‌شود؛ کاربران نهایی به طور منفی شرکت می‌کنند.

این معماری، که در شکل ۸-۷ نشان داده شده است، ابتدا داده‌ها را از هر یک از سیستم‌های منبع OLTP (ببینید فصل ۷) به طور سریع به تابل‌های موقت مرتبط با رابطه‌ای بکشیده می‌کند، بدون تبدیل یا تمیز کردن آن‌ها.

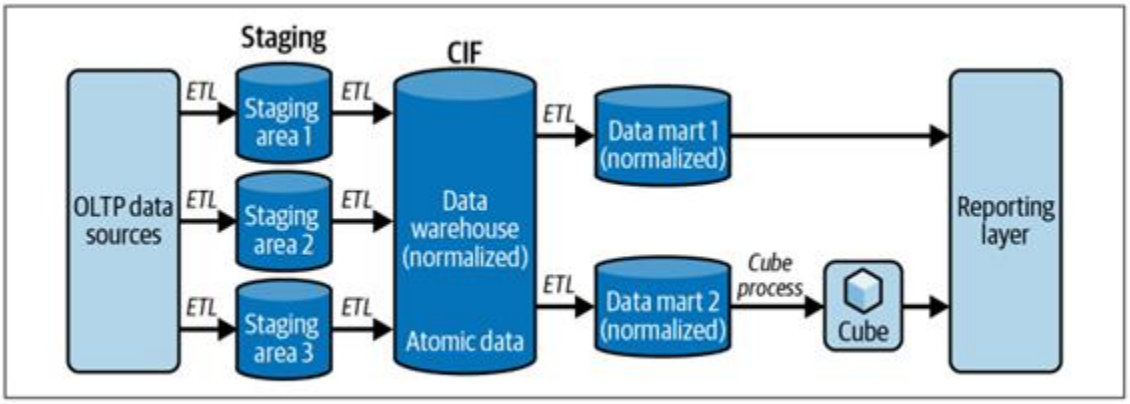


Figure 8-7. A data warehouse designed using the Inmon methodology

تبدیل داده‌ها می‌تواند باعث افزایش قابل توجهی از CPU و پهنای باند در سیستم‌های منبع شود، بنابراین این رویکرد برای جلوگیری از کاهش سرعت سیستم‌های منبع طراحی شده است. تابل‌های مرحله‌ای همچنین برای اهداف بازرسی مفید هستند و اگر نیاز به اجرای مجدد تبدیل‌ها داشته باشید، می‌توانید داده‌ها را از آن‌ها بکشید به جای بازگشت به سیستم منبع.

در مرحله بعدی، داده‌ها کپی می‌شوند به آنچه که اینمون آن را می‌نامد کارخانه اطلاعات شرکتی (CIF)، جایی که تمام داده‌ها متمرکز و در سطح اتمی (به اندازه‌ای که امکان‌پذیر است) در قالب سومین شکل نرمال ذخیره می‌شوند. شما می‌توانید آن را به عنوان یک انبار داده سازمانی (EDW) در نظر بگیرید که یک نسخه تکی از حقیقت است. در این بخش از این معنا استفاده می‌کنم که انبار داده سازمانی به جای فقط انبار داده، زیرا بسیاری از راه‌حل‌های ساخته شده در سال‌های اول انبار داده از تمام زمینه‌های یک سازمان داده‌ها را در بر گرفته است. امروزه برخی از شرکت‌ها ممکن است داشته باشند چندین انبار داده، اما بیشتر راه‌حل‌ها با استفاده از یک "انبار داده سازمانی" تکی ساخته می‌شوند.

CIF بر روی طراحی مدل داده و طراحی تمرکز دارد، با تأکید قوی بر مدیریت و کیفیت داده‌ها. از آن، می‌توانید مارت‌های داده وابسته را ایجاد کنید: زیرمجموعه‌های فیزیکی جداگانه از داده‌ها، معمولاً برای ارائه داده‌های خاصی که هر دپارتمان نیاز دارد.

مارت‌های داده به عنوان "وابسته" نامیده می‌شوند زیرا آن‌ها به داده‌های CIF وابسته هستند و هیچ داده‌ای از جای دیگری دریافت نمی‌کنند. این همچنین به عنوان یک معماری هاب و پیچ نامیده می‌شود (برای شکل آن). کاربران می‌توانند مارت‌های داده را برای گزارشات مراجعه کنند یا زیرمجموعه‌های کوچک با مدل‌های بعدی که کوب‌ها نامیده می‌شوند (ببینید فصل ۷) را برای اهداف خاص مراجعه کنند. به عنوان مثال، تحلیلگران ممکن است فقط نیاز به مشاهده حجم فروش داشته باشند. داده‌ها در یک کوب اغلب تجمع می‌شوند.

CIF برای کاربران نهایی محدود است و آن‌ها از طریق مارت‌های داده یا کوب‌ها به داده‌ها دسترسی دارند. یکی از مشکلات این است که این بدان معناست که داده‌ها تا سه بار به طور دائمی تکرار می‌شوند: برای CIF، مارت داده، و کوب. این منجر به افزایش هزینه‌های ذخیره‌سازی، پیچیدگی بیشتر در نگهداری، و چالش نگهداری سه نسخه متمرکز می‌شود.

روش کیمبال رویکرد پایین به بالا

روش متودولـوژی "از پـایین بـه بـالا" یـا "انعطاف‌پـذیر" کیمبـال بـه شـکل مشـابهی بـا روش اینمـون شروع می‌شـود، بـا کشـیدن داده‌هـای خـام از هـر یـک از سیسـتم‌های منبـع OLTP (ببینیـد فصـل ۷) بـه تابل‌هـای موقـت مـرتبط بـا رابطـه‌ای بـدون تبـدیل یـا تمـیز کـردن. سـپس روش کیمبـال بـه مرحلـه‌ای از ادغـام داده‌هـا و اطمینـان از کیفیـت آن‌هـا می‌رود، در پی ایـن مراحـل، مـراحلی بـرای طـراحی مـدل، طـراحی، و دسـترسی وجـود دارد. معمـاری آن در شـکل ۸-۸ نشـان داده شـده است.

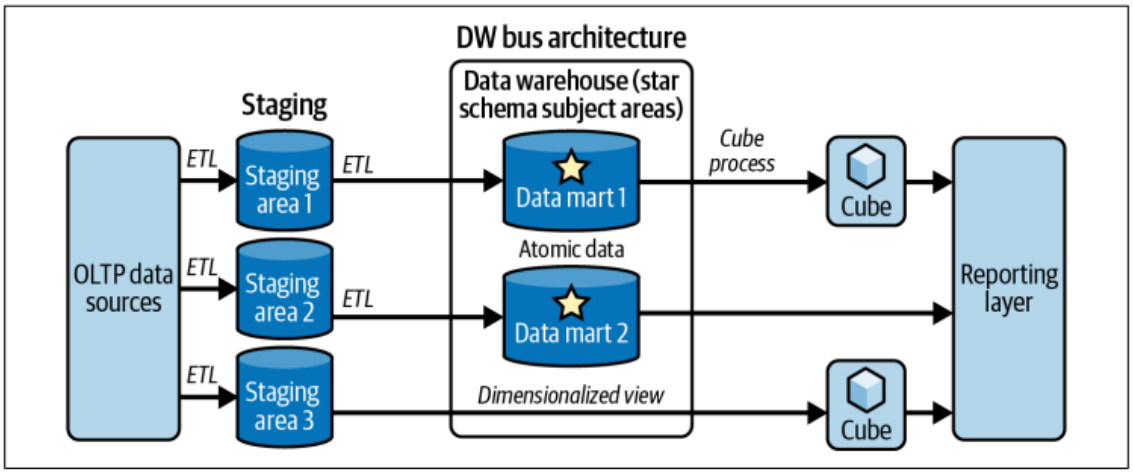


Figure 8-8. A data warehouse designed using the Kimball approach

داده‌هـا از تابل‌هـای مرحلـه‌ای سـپس بـه مارت‌هـای داده مسـتقل بـرای مـوارد حیـاتی کـپی می‌شـوند کـه نیازهـای تحلـیلی بخش‌هـا را خـدمت می‌کننـد. مارت‌هـای داده مسـتقل هسـتند زیـرا بـر روی یـک حـوزه موضـوعی یـا فرآینـد کسـب و کـار خـاص تمرکـز دارنـد، بـه جـای ذخیره و مـدیریت تمـام داده‌هـا در یـک مخـزن مرکـزی تـکی ماننـد CIF اینمـون. ایـن مارت‌هـای داده مستقل از یک مدل بعدی استفاده می‌کنند و در سطح اتمی یا خلاصه ذخیره می‌شوند، بسته به نیازهای کاربران.

بـرای ارائـه یکپـارچگی در منـابع داده، مارت‌هـای داده بـا اسـتفاده از بُعـدهای متناسـب یـا متناسـب شـده ادغـام می‌شـوند، کـه بُعـدهایی هسـتند کـه اسـتاندارد شـده و در تمـام جـدول‌های واقـعی متناسـب شـده‌اند تـا داده‌هـا بتواننـد در تمـام جـدول‌های واقـعی بـدون هـیچ مشـکلی اسـتفاده و مقایسـه شـوند. بـه عنـوان مثـال، اگـر یـک بُعـد مشـتری در چنـدین جـدول واقعی استفاده شود، نام‌ها و شناسه‌های مشتری در تمام جدول‌های واقعی باید یکسان باشند.

از مارت‌هـای داده، زیرمجموعـه‌هایی از داده‌هـا می‌تـوان بـه کوب‌هـا کـپی کـرد. شـما حـتی می‌توانیـد یـک کـوب را مسـتقیماً از تابل‌هـای مرحلـه‌ای بـا اسـتفاده از یـک نمـایش بُعـدی سـاخته کنیـد اگـر آن تمـام انـواع دسـترسی مـورد نیـاز را فـراهم کنـد. (یـک نمـایش بُعـدی یـک نمـایش SQL اسـت کـه داده‌هـا را بـه یـک مـدل بُعـدی سـازماندهی می‌کنـد.) گزارشـات می‌تـوان از کـوب یـا از مـارت داده انجـام شـود. روش متودولـوژی کیمبـال توسـط کسـب و کـار هـدایت می‌شـود و کـاربران نهـایی شرکت‌کننـدگان فعـال هسـتند. داده‌هـا فقـط یـک بـار کـپی می‌شـوند اگـر از یـک نمـایش بُعـدی اسـتفاده می‌کننـد، یـا دو بـار، بـه یک مارت داده و به یک کوب.

انتخاب روش مناسب

در واقـع، متفاوتیـت دو روش بـه دو تفـاوت اصـلی برمی‌گـردد. اول، روش اینمـون یـک انبـار داده نرمال‌شـده (DW) را قبـل از ایجـاد مارت‌هـای داده ایجـاد می‌کنـد، در حـالی کـه روش کیمبـال ایـن مرحلـه را حـذف می‌کنـد. دوم، روش اینمـون از یـک انبـار داده سـازمانی فـیزیکی (EDW) اسـتفاده می‌کنـد، در حـالی کـه روش کیمبـال از یـک انبـار داده سـازمانی متناسـب (conformed EDW) اسـتفاده می‌کنـد امـا انبـار داده سـازمانی فـیزیکی نـدارد. هـر دو روش قبـل از شـکل‌گیری دریاچـه‌های داده بودنـد، امـا هنگـامی کـه دریاچـه‌های داده شروع بـه محبـوب شـدن کردنـد، هـر دو روش آن‌هـا را در معمـاری انبـار داده ادغـام کردنـد. کیمبـال دریاچـه‌های داده را کنـار مارت‌هـای داده در بـاس انبـار داده (DW bus) ادغـام می‌کنـد و هـر دو روش اینمون و کیمبال اجازه می‌دهند که دریاچه‌های داده جایگزین جدول‌های مرحله‌ای رابطه‌ای شوند.

با این حال، برای داشتن یک نسخه تکی از حقیقت، چندین دلیل وجود دارد که یک انبار داده سازمانی فیزیکی داشته باشید، مانند:

- یک انبار داده سازمانی فیزیکی باعث می‌شود که داشتن یک نسخه تکی از حقیقت بسیار آسان‌تر باشد؛ داشتن چندین مارت داده که یک انبار داده متناسب را تشکیل می‌دهند، می‌تواند برای کاربران دشوار و گیج کننده باشد. داشتن داده‌ها فیزیکی در همان پایگاه داده آسان‌تر است برای فهم.

- ساخت از جداول نرمال‌شده کم در یک انبار داده سازمانی فیزیکی آسان‌تر است تا ساخت مستقیم از منبع OLTP.

- یک انبار داده سازمانی فیزیکی نرمال‌شده یک یکپارچگی کل سازمان را فراهم می‌کند. این باعث می‌شود تا ساخت مارت‌های داده آسان‌تر شود، اگرچه با معامله تکرار داده‌ها.

- انبارهای داده سازمانی فیزیکی نیاز به تعمیر و هماهنگ‌سازی ETL کمتری دارند؛ با چندین منبع داده و مارت‌های داده بُعدی در چندین پایگاه داده، شما نیاز به تعداد بسیار بیشتری از تعمیر و هماهنگ‌سازی روزانه یا ساعتی برای نگهداری همه چیز در هماهنگی دارید.

- در یک انبار داده سازمانی فیزیکی، فقط یک مکان برای کنترل داده وجود دارد، بنابراین شما تلاش خود را یا داده‌های خود را تکرار نمی‌کنید.

با این حال، اگر شما فقط چندین منبع داده دارید و نیاز به گزارشات سریع دارید، ممکن است بدون یک انبار داده سازمانی فیزیکی بهتر باشید.

کدام مدل باید استفاده کنید؟ در واقع، دو مدل در طول سال‌ها تغییر کرده‌اند و اکنون بسیار شبیه به هم هستند. در واقع، آن‌ها می‌توانند یکدیگر را تکمیل کنند. شما می‌توانید یک انبار داده سازمانی نرمال‌شده را به یک رویکرد مبتنی بر کیمبال اضافه کنید یا مارت‌های داده بُعدی را به یک رویکرد مبتنی بر اینمون اضافه کنید. خط اصلی این است که شما نیازی به انتخاب فقط یک رویکرد و به آن بستگی داشتن ندارید و باید به آن بسته باشید. شما باید هر دو رویکرد را فهمید و قسمت‌هایی را از هر دو که برای شرایط شما کار می‌کنند، انتخاب کنید. با این حال، هیچ راه حلی موثر نخواهد بود مگر اینکه تیم شما مهارت‌های قوی در رهبری، ارتباطات، برنامه‌ریزی، و روابط بین انسانی داشته باشد (ببینید فصل ۱۵).

مدل/روش ترکیبی

برای بررسی امکانات یک ترکیب از دو روش متدولوژیک، مانند آنچه در شکل ۸-۹ نشان داده شده است، این مدل به شکل دیگر دو مدل دیگر شروع می‌شود: کشیدن داده‌های خام از هر یک از سیستم‌های منبع OLTP مستقیماً به تابل‌های مرتبط موقت با رابطه‌ای بدون تبدیل یا تمیز کردن. هنوز هیچ تبدیل یا تمیز کردن داده وجود ندارد، اما شما یک OLTP مرابط را اضافه می‌کنید (بیشتر در مورد آن در لحظه آینده صحبت می‌کنیم).

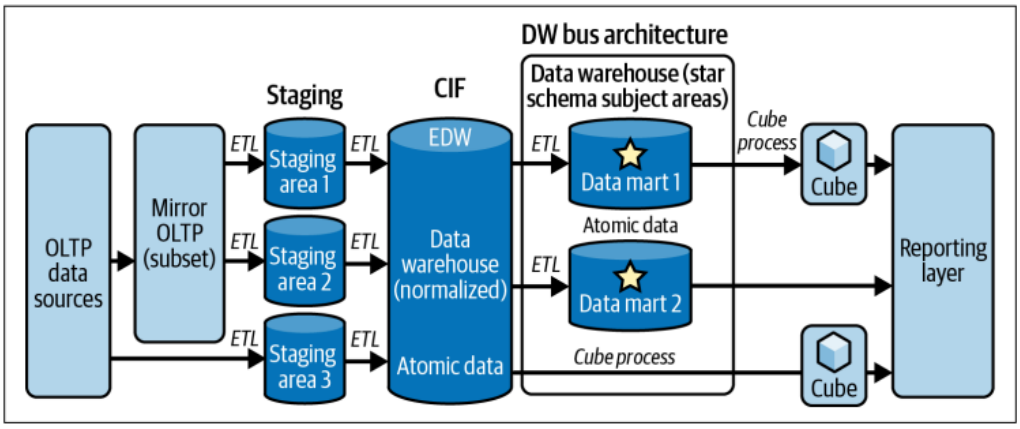


Figure 8-9. A hybrid of the Kimball and Inmon approaches

بعد، شما داده‌ها را به یک CIF کپی می‌کنید تا مرکزی شود و در سطح اتمی در NF۳ ذخیره شود، مانند روش اینمون. سپس داده‌ها از CIF به مارت‌های داده مستقل و بُعدی کپی می‌شوند. برخی از مارت‌های داده در سطح اتمی ذخیره می‌شوند و دیگران در سطح خلاصه، بسته به نیازهای تحلیلی هر بخش.

از مارت‌های داده یا از CIF، شما همچنین می‌توانید زیرمجموعه‌ای از داده‌ها را به کوب‌ها کپی کنید. شما می‌توانید از کوب گزارش کنید یا از مارت داده، اما من توصیه می‌کنم که هر زمان امکان داشته باشید از کوب گزارش کنید. مزایای کوب شامل:

- به عنوان یک لایه معنایی عمل می‌کند.

- می‌تواند با تعداد زیادی از کاربران همزمان کنار بیاید.

- داده‌های آن برای بهبود عملکرد تجمع می‌شوند.

- شما نیازی به مقابله با اتصالات یا روابط ندارید.

- می‌تواند شامل سلسله‌ها و KPIها باشد.

- دارای امنیت سطح ردیف است که امنیت داده‌ها را با محدود کردن دسترسی کاربران به ردیف‌های خاص در یک پایگاه داده افزایش می‌دهد.

- پیشرفته محاسبات زمانی را ارائه می‌دهد که به شما کمک می‌کند تا الگوها و روندهای داده را در طول زمان درک کنید، مانند رشد فروش سال به سال.

برای استفاده از یک مدل Inmon ،Kimball، یا ترکیبی، در نظر بگیرید از نمایش‌های پایگاه داده استفاده کنید. یک نمایش پایگاه داده یک جدول مجازی است که بر اساس نتیجه یک دستور SELECT SQL است. این داده‌ها را ذخیره نمی‌کند، فقط کد SQL؛ هر بار که پرس و جو می‌شود، داده‌ها را از یک یا چند جدول بازیابی می‌کند. استفاده از نمایش‌ها در فرآیند ETL باعث می‌شود کد درون ETL ساده‌تر باشد، و شما نیازی به دیدن کد SQL درون نمایش ندارید تا بفهمید چه چیزی را می‌خواند. یک نمایش باعث می‌شود پرس و جوی یک پایگاه داده برای اهداف اشکال‌زدایی آسان‌تر باشد، و ETL می‌تواند با به‌روزرسانی نمایش خارج از کد ETL بهینه‌سازی شود. هر کسی می‌تواند نمایش‌ها را خواند، تغییر دهد، یا بهینه‌سازی کند، نه فقط در ابزار ETL بلکه همچنین در ابزارهای دیگر، و شما می‌توانید وابستگی‌ها را در یک نمایش با استفاده از ابزارهای سوم شخص تحلیل و ردیابی کنید (مانند جداول و فیلدهایی که نمایش استفاده می‌کند). نمایش‌ها می‌توانند مقادیر پیش‌فرض را ارائه دهند و محاسبات ساده را انجام دهند، و شما می‌توانید فیلدها را تغییر نام دهید تا به فهم جریان آن‌ها کمک کنید. نمایش‌ها می‌توانند یک مدل ستاره را ارائه دهند، حتی اگر ساختار زیرین بسیار پیچیده‌تر باشد.

شما همچنین می‌توانید نمایش‌ها را در یک کوب استفاده کنید. اگر این کار را انجام دهید، می‌توانید ستون‌های پایگاه داد.

روش میتس

چون روش‌های اینمون و کیمبال طی سال‌ها به شدت مورد بحث و تحقیر قرار گرفته‌اند، تعداد زیادی از افسانه‌ها در مورد این روش‌ها به وجود آمده است. این بخش با برخوش کردن برخی از این افسانه‌ها پایان می‌یابد:

افسانه: کیمبال یک رویکرد پایین به بالا است بدون تمرکز بر روی سازمان.

برخوش کردن افسانه: رویکرد کیمبال در واقع شامل هر دو رویکرد پایین به بالا و پایین به بالا است. جنبه پایین به بالا در طراحی استراتژیک و طراحی در سراسر سازمان نشان داده می‌شود. کیمبال تاکید دارد که زمان قابل توجهی را در ابتدا صرف کنید تا راه حل خود را با استفاده از یک ابزار به نام ماتریس باس ادوار داده (EDW bus matrix) طراحی کنید. این ابزار به عنوان نقشه معماری اصلی فرآیندهای کسب و کار اصلی سازمان و بعدهای

مرتبط با آن، از جمله بعدهای مطابقت‌دار، عمل می‌کند. ماتریس باس ادوار داده یک دیدگاه استراتژیک پایین به بالا فراهم می‌کند تا اطمینان حاصل شود که داده‌ها در محیط DW/BI می‌توانند در سراسر کل سازمان یکپارچه شوند.

جنبه پایین به بالا در مرحله اجرا به کار می‌رود. در اینجا، رویکرد کیمبال بر روی یک فرآیند کسب و کار در هر زمان تمرکز دارد، اطمینان از تحویل پویا راه حل‌ها را فراهم می‌کند. این رویکرد تأکید بر پیاده‌سازی عملی دارد و همزیستی استفاده از یک استراتژی طراحی پایین به بالا با یک رویکرد اجرایی پایین به بالا را نشان می‌دهد.

افسانه: اینمون نیاز دارد که قبل از شروع ساخت راه حل، مقدار زیادی از طراحی را انجام دهید، مشابه رویکردهای بزرگ برش یا آبشاری.

برخوش کردن افسانه: اینمون از ابتدا اعلام کرده است که راه ساخت انبارهای داده به صورت تکراری است. در واقع، او به اندازه‌ای می‌رود که معیار بسیار حیاتی در ساخت یک انبار داده استفاده از رویکرد بزرگ برش نیست.

این افسانه مرتبط است با افسانه طولانی مدت موجود که می‌گوید ساخت یک معماری انمون نیاز به زمان طولانی و منابع زیادی دارد. کلاودیا ایموف، یکی از بهترین کارشناسان شناخته شده در رویکرد اینمون، و همسایگان نوشته‌اش به این شکل می‌گویند:

"هیچ جایی توصیه نمی‌کنیم که یک انبار داده کامل را بسازید که تمام داده‌های استراتژیک سازمانی که هرگز نیاز ندارید را شامل می‌شود قبل از ساخت قابلیت تحلیلی اول (مارت داده) بسازید. هر برنامه کسب و کار متوالی که توسط یک پیاده‌سازی دیگر مارت داده حل می‌شود، به مجموعه رو به رشد داده‌هایی که به عنوان پایه در انبار داده شما عمل می‌کنند، اضافه می‌کند. در نهایت، مقدار داده‌ای که باید به انبار داده اضافه شود تا یک مارت داده جدید را پشتیبانی کند، بسیار ناچیز خواهد بود زیرا بیشتر آن‌ها در حال حاضر در انبار داده وجود دارد."

بخشی- از سردرگمی از افرادی است که رویکرد طراحی و برنامه‌ریزی اینمون را با خود راه حل معماری ادغام می‌کنند. هر دو رویکرد زمان را برای طراحی، برنامه‌ریزی و جمع‌آوری نیازمندی‌ها قبل از واقعی ساخت چیزی را رزرو می‌کنند. هر دو را با یک رویکرد تکراری/پویا دنبال می‌کنند که در آن شما



Figure 8-10. Percentage of design work done before building a solution

ترجمه فارسی:

یک تشبیه می‌تواند ساخت یک شهر جدید باشد. شما برای شهر یک طرح معماری طراحی می‌کنید که تمام محلات زندگی درون آن را شامل می‌شود، سپس هر محل زندگی را می‌سازید و بعد از تکمیل آن، مردم می‌روند. در طول این فرآیند، معماری کلی شهر در صورت نیاز می‌تواند تغییر کند. شما قبل از اینکه هر محل زندگی را ساخته شود، هیچ کس وارد نمی‌شود و همچنین قبل از شروع ساخت محلات زندگی بدون داشتن یک ایده خوب از اینکه شهر کلی چگونه باید به نظر برسد، شروع نمی‌کنید.

افسانه: مدل اینمون اجازه مارت‌های ساختار ستاره را نمی‌دهد.

در نسخه‌های بعدی از "ساخت انبار داده" و "داستان دو معماری"، اینمون معتقد است که مارت‌های ساختار ستاره برای ارائه دسترسی مستقیم به داده به کاربران نهایی خوب هستند و ساختارهای ستاره برای مارت‌های داده خوب هستند. او در برابر آن‌ها مخالفت ندارد.

افسانه: بسیار چند شرکت از روش اینمون استفاده می‌کنند.

نظرسنجی‌های اخیر نشان می‌دهند که بیشتر شرکت‌ها از روش اینمون استفاده می‌کنند تا از کیمبال. بسیاری از شرکت‌ها یک ادوار داده بدون هیچ مارت داده‌ای استفاده می‌کنند.

افسانه: معماری‌های کیمبال و اینمون با هم سازگار نیستند.

دو روش می‌توانند با هم کار کنند تا یک راه حل بهتری ارائه دهند، همانطور که در "مدل‌های ترکیبی" در صفحه ۱۱۸ یاد گرفتید.

کیمبال همچنین در کتاب‌هایش چرخه زندگی توسعه را تعریف می‌کند، که موضوعاتی مانند برنامه‌ریزی پروژه، تعریف نیازمندی‌های کسب و کار و نگهداری را پوشش می‌دهد (مشاهده شکل ۸-۱۱). در مقابل، اینمون فقط انبار داده را پوشش می‌دهد، نه چگونگی استفاده از آن.

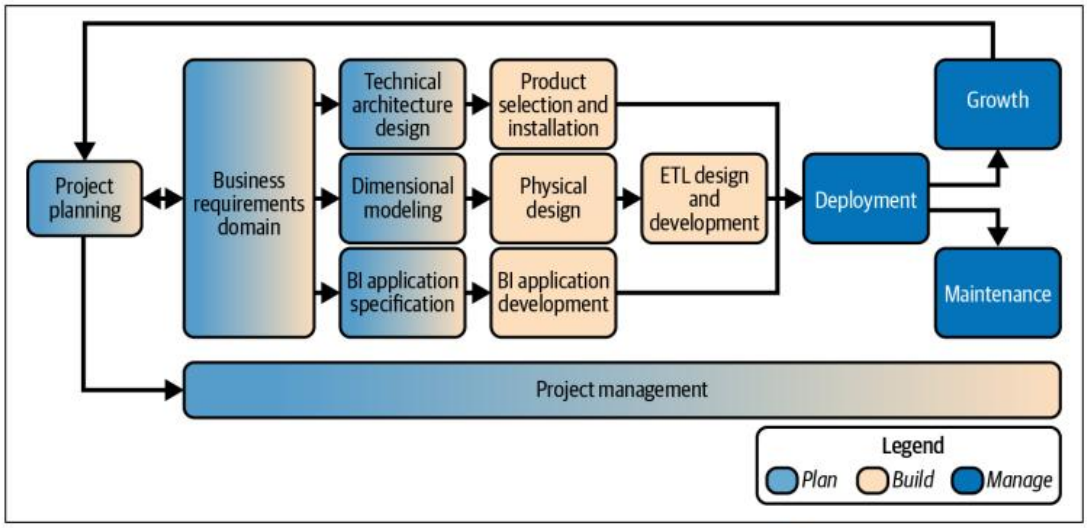


Figure 8-11. Kimball's methodology incorporates a development lifecycle that goes beyond building the data warehouse (source: Kimball, The Microsoft Data Warehouse Toolkit)

وقتی افراد می‌گویند که از روش کیمبال استفاده می‌کنند، اغلب آنچه واقعاً منظور آن‌ها مدل‌سازی بُعدی است - دو اصطلاح به طور معمول به عنوان مترادف در نظر گرفته می‌شوند. کیمبال اصول اساسی واقعیت‌ها و بُعدها را اختراع نکرد؛ با این حال، او یک مجموعه گسترده از تکنیک‌ها و واژگان مدل‌سازی بُعدی را برقرار کرد، از جمله بُعدهای مطابقت‌دار، بُعدهای تغییر کننده به آرامی، بُعدهای زباله، بُعدهای کوچک، جداول پل‌ساز، و جدول‌های واقعیت لحظه‌ای و جمع‌آوری مکرر.

خلاصه

این فصل به شما معماری‌ها و روش‌های اساسی مربوط به سازماندهی و تعریف ساختارهای داده، روابط و محدودیت‌ها برای پردازش، ذخیره‌سازی و بازیابی داده به صورت کارآمد معرفی کرد. ما فرآیند ایجاد یک مدل داده را که به عنوان نمایش انتزاعی از داده‌های سازمانی عمل می‌کند، بررسی کردیم. این گام اولیه حیاتی در مدیریت داده است، زیرا یک طرح برای نحوه ذخیره‌سازی، مصرف و یکپارچگی داده در سیستم‌ها فراهم می‌کند.

سپس، ما به مدل‌سازی رابطه‌ای پرداختیم، یک رویکرد کلاسیک که بر اساس مفهوم نرمال‌سازی داده به جداول (یا روابط) است که در آن هر سطر یک رکورد را نمایش می‌دهد و ستون‌ها ویژگی‌ها را نمایش می‌دهند. این مدل کلیدی برای جلوگیری از تکرار داده و افزایش یکپارچگی داده است و در هسته بیشتر سیستم‌های پایگاه داده رابطه‌ای سنتی قرار دارد. سپس مدل‌سازی بُعدی را بحث کردم، یک رویکرد که در انبار داده به طور معمول استفاده می‌شود و داده‌ها را به واحدهای قابل اندازه‌گیری (مقادیر قابل اندازه‌گیری) و بُعدها (اطلاعات زمینه‌ای) دسته‌بندی می‌کند؛ مدل داده مشترک (CDM)، یک زبان داده مشترک که امکان تعامل داده در بین برنامه‌ها و فرآیندهای کسب و کار مختلف را فراهم می‌کند؛ و مدل انبار داده، یک رویکرد ترکیبی مقاوم که بهترین ویژگی‌های NF۳ و ساختار ستاره را ترکیب می‌کند. بقیه فصل یک مقایسه حیاتی بین دو فلسفه متقابل انبار داده انجام داد. رویکرد پایین به بالا کیمبال

پیشنهاد می‌کند که ابتدا مارت‌های داده فردی را بسازید و سپس آن‌ها را در یک انبار داده کامل ترکیب کنید، در حالی که رویکرد بالا به پایین اینمون توصیه می‌کند که ابتدا یک انبار داده جامع را بسازید، سپس مارت‌های داده را از آن استخراج کنید.

درک این روش‌ها می‌تواند به شما کمک زیادی کند تا معماری‌های داده موثری را طراحی کنید که به نیازهای خاص سازمان شما پاسخ می‌دهد در حالی که کارایی عملیاتی را با تحلیل‌های استراتژیک متعادل می‌کند.

فصل نهم: رویکرد های تزریق داده

در دنیای دیجیتال همیشه رشد یافته ما، کنترل و درک تمام انواع داده برای کسب و کارهای همه انواع بسیار مهم شده است. این فصل درباره اولین قدم در کنترل داده است - دریافت داده به سیستم در اولین مرحله.

در اینجا، ما برخی از روش‌های کلیدی را که داده می‌تواند به سیستم‌های مختلف برود را بررسی می‌کنیم. من با توضیح دو روش رایج به نام ETL و ELT شروع می‌کنم که به راحتی قابل فهم است. همچنین به یک ایده جدید به نام reverse ETL معرفی می‌کنم و توضیح می‌دهم که چگونه این روش روش‌های سنتی را بر روی سر قرار می‌دهد.

از آنجا که نیازهای داده همگی یکسان نیستند، ما روش‌های مختلفی مانند پردازش دسته‌ای و پردازش زمان واقعی را بررسی خواهیم کرد. این کمک می‌کند تا بفهمید که چه چیزی بر اساس مقدار داده‌ای که با آن مواجه هستید و چقدر سریع نیاز دارید، بهترین کار را انجام می‌دهد. در نهایت، ما درباره اهمیت حاکمیت داده صحبت خواهیم کرد - اطمینان از دقت، یکپارچگی، و دسترسی داده‌های شما.

این فصل قصد دارد این ایده‌های پیچیده را ساده کند و نشان دهد که چگونه همه آن‌ها به تصویر بزرگتر کنترل داده متصل هستند. چه شما یک دیدار داده باشید یا یک مبتدی کامل، من خوشحالم که شما را از این دنیای جذاب دریافت داده هدایت کنم. به سمت شما بپیوندید!

ETL در مقایسه با ELT

برای سال‌ها، روش ETL (extract-transform-load) روش رایج‌ترین برای انتقال داده از یک سیستم منبع به یک انبار داده رابطه‌ای بود. اما اخیراً، روش ELT (extract-load-transform) محبوب شده است، به خصوص با دیتا لیک‌ها.

فرآیند ETL شامل استخراج داده از منابع خارجی، تبدیل و تمیز کردن آن در حال پرواز برای تناسب با فرمت و ساختار مقصد، و سپس بارگذاری آن در مقصد است (معمولاً یک دیتا لیک یا RDW).

ETL

- **مراحل**: استخراج، تبدیل، بارگذاری

- **محل تبدیل**: در حال پرواز به مقصد

- **مزایا**: کنترل بیشتر بر فرمت و ساختار داده، امکان اعمال تنظیمات قبل از بارگذاری

- **معایب**: ممکن است برای داده‌های بزرگ کند و ممکن است زمان بیشتری برای تبدیل و تمیز کردن داده نیاز داشته باشد

ELT

- **مراحل**: استخراج، بارگذاری، تبدیل

- **محل تبدیل**: پس از بارگذاری در مقصد

- **مزایا**: سرعت بیشتر در بارگذاری داده، کارایی بالاتر برای داده‌های بزرگ

- **معایب**: کنترل کمتر بر فرمت و ساختار داده در مرحله اول، ممکن است نیاز به تنظیمات پس از بارگذاری

نسخه زیر گیتی از معماری داده

ELT به خصوص برای دیتا لیک‌ها محبوب است زیرا این روش امکان می‌دهد تا داده‌ها به سرعت بارگذاری شوند و سپس تبدیل و تمیز کردن آن‌ها در مقصد انجام شود، که برای دیتا لیک‌ها که معمولاً برای تحلیل‌های بزرگ داده استفاده می‌شوند، مناسب است.

چگونه از ابهام بین ETL و ELT جلوگیری کنیم

اصطلاحات ETL و ELT، به دلیل شباهت آن‌ها، ممکن است به راحتی نادرست تفسیر یا اشتباه زده شوند. برای کمک به حفظ یاد شما که کدام است، اینجا یک حافظه کلیدی مفید ارائه می‌شود:

”تبدیل زودهنگام راهنمایی می‌کند“ = **ETL (Extract-Transform-Load) -**

این برای تأکید بر اینکه تبدیل در ابتدای فرآیند رخ می‌دهد، مستقیماً پس از استخراج و قبل از بارگذاری، استفاده می‌شود. این مفهوم اصلی فرآیند ETL را منتقل می‌کند، که در آن داده‌ها قبل از بارگذاری به سیستم هدف تمیز و تبدیل می‌شوند.

رای ELT، فکر کنید:

”هر بار بارگذاری تبدیل می‌کند“ = **ELT (Extract-Load-Transform) -**

این نشان می‌دهد که پس از هر دسته از داده‌های خام بارگذاری شده به سیستم، تبدیل می‌شود؛ به عبارت دیگر، تبدیل پس از بارگذاری داده به سیستم رخ می‌دهد.

با استفاده از این حافظه کلیدی، می‌توانید به راحتی بین فرآیندهای ETL و ELT تمایز داده و به یاد داشته باشید که ETL شامل تبدیل داده قبل از بارگذاری است، در حالی که ELT شامل بارگذاری داده اول و سپس تبدیل آن است.

چندین معیار اصلی با تکنیک ETL وجود دارد. اول، تبدیل داده‌ها زمان می‌برد و استخراج می‌تواند منبع مصرف زیادی باشد. هرچه استخراج در سیستم منبع طولانی‌تر اتفاق بیفتد، احتمال بیشتر شدن مشکلات عملکرد برای کاربران نهایی در آن سیستم است. دوم، اگر در فرآیند ETL خطایی وجود داشته باشد و نیاز به اجرای مجدد آن باشد، باید به سیستم منبع برگردید تا داده‌ها را دوباره استخراج کنید، که احتمالاً دوباره بر عملکرد تأثیر می‌گذارد. سوم، اگر مقدار زیادی از داده‌ها در حال کپی باشد، ابزارهای ETL سنتی ممکن است قادر به مدیریت آن نباشند به دلیل قدرت پردازش محدود خود. چهارم، ELT تبدیلات را یک رکورد در هر بار انجام می‌دهد، که همچنین می‌تواند باشد کند. در نهایت، برخی از ابزارهای ETL در انواع داده‌هایی که پشتیبانی می‌کنند محدودیت‌هایی دارند.

از سوی مثبت، ETL اغلب برای مجموعه‌های داده‌ای کوچکتر مناسب است، جایی که فرآیند تبدیل بسیار پیچیده نیست. این کنترل بهتری بر کیفیت داده‌ها ارائه می‌دهد، زیرا پاکسازی و تبدیل داده‌ها قبل از بارگذاری کمک می‌کند تا خطاها را کاهش دهد. ETL می‌تواند کلیدی برای امنیت داده‌ها باشد، زیرا فقط داده‌های لازم و پاکسازی شده بارگذاری می‌شوند، کاهش ریسک‌های امنیتی احتمالی. در نهایت، ETL زمانی ترجیح می‌شود که هر دو سیستم منبع و مقصد پایگاه‌های داده رابطه‌ای هستند، زیرا معمولاً کارآمدتر است.

ELT مشابه ETL است، با این تفاوت که ترتیب متفاوت است. ELT همچنین داده‌ها را از منابع خارجی به یک سیستم مقصد کپی می‌کند، اما داده‌ها در حال پرواز تبدیل نمی‌شوند. به جای آن، در سیستم مقصد به صورت خام وارد می‌شوند و برای مطابقت با فرمت و ساختار مقصد تبدیل و پاکسازی می‌شوند. با این حال، می‌توانید در هنگام استخراج داده‌ها از منبع، سطرها و ستون‌های نیازمند را فیلتر کنید تا منابع را برای داده‌های نیازمند نخواهید دور کرد.

ELT اغلب رویکرد ترجیحی برای دریاچه‌های داده است و به خصوص زمانی مفید است که با داده‌های نامتناسب یا نیمه ساختاری با حجم بالا کار می‌کنید. انعطاف‌پذیری آن در قابلیت بارگذاری تمام داده‌های خام اولیه به سیستم مقصد و سپس اعمال تبدیلات در صورت نیاز است. هنگامی که داده‌ها در دریاچه داده هستند، ELT می‌تواند از قدرت پردازش فراوان تکنولوژی امروز بهره ببرد، که آن را برای مدیریت مقادیر بزرگی از داده‌ها کارآمد می‌کند. ELT همچنین می‌تواند در تبدیلات سریع‌تر باشد زیرا آن‌ها را در دسته‌ها پردازش می‌کند، که یک مزیت قابل توجه نسبت به رویکرد تبدیل رکورد به رکورد ETL است.

این همچنین اطمینان می‌دهد که تبدیلات می‌توانند بدون نیاز به استخراج مجدد داده‌ها تغییر یا بهینه‌سازی شوند. یک مزیت دیگر از ELT مجموعه گسترده‌ای از گزینه‌های ابزار موجود است، از جمله آن‌هایی که به طور خاص برای

پلتفرم‌های بزرگ داده طراحی شده‌اند. این بدین معنی است که فرآیندهای ELT می‌توانند انواع بیشتری از داده‌ها را پشتیبانی کنند و به نیازهای تغییر کننده محیط داده تطبیق پیدا کنند.

Reverse ETL

فرآیندی است که داده‌ها را از یک داده‌کاوی مدرن به یک یا چند سیستم شرکت ثالث منتقل می‌کند تا داده‌ها را عملی کند. (شکل ۹-۱ ELT، ETL و Reverse را مقایسه می‌کند.) به طور سنتی، داده‌های ذخیره شده در یک داده‌کاوی (DW) برای بارهای کاری تحلیلی و هوش تجاری استفاده می‌شوند - برای شناسایی روندهای طولانی مدت و تأثیرگذاری بر استراتژی طولانی مدت - و خارج از DW کپی نمی‌شوند. با این حال، برخی از شرکت‌ها اکنون همچنین از این داده‌ها برای تحلیل عملی استفاده می‌کنند.

تحلیل عملی به تصمیم‌گیری‌های روزانه کمک می‌کند و هدف بهبود کارایی و اثربخشی را دارد. به اصطلاح‌های ساده‌تر، این داده‌های شرکت را برای کمک به هر کس در تصمیم‌گیری بهتر و هوشمندتر در مورد کسب و کار به کار می‌برد. به عنوان مثال، فرض کنید که داده‌های مشتری را به DW خود وارد کرده و سپس آن‌ها را پاکسازی و مستر کرده‌اید (ELT). سپس می‌توانید این داده‌های مشتری را به چندین سیستم خدمات نرم‌افزاری به عنوان خدمات (SaaS)، مانند Salesforce، کپی کنید تا نمای یکسانی از هر مشتری در تمام سیستم‌ها را ببینید. شما همچنین می‌توانید داده‌های مشتری را به یک سیستم پشتیبانی مشتری و/یا یک سیستم فروش کپی کنید.

شما حتی می‌توانید مشتریانی را شناسایی کنید که در خطر خروج از محصولات یا خدمات شرکت هستند با نمایش داده‌های استفاده در یک CRM.

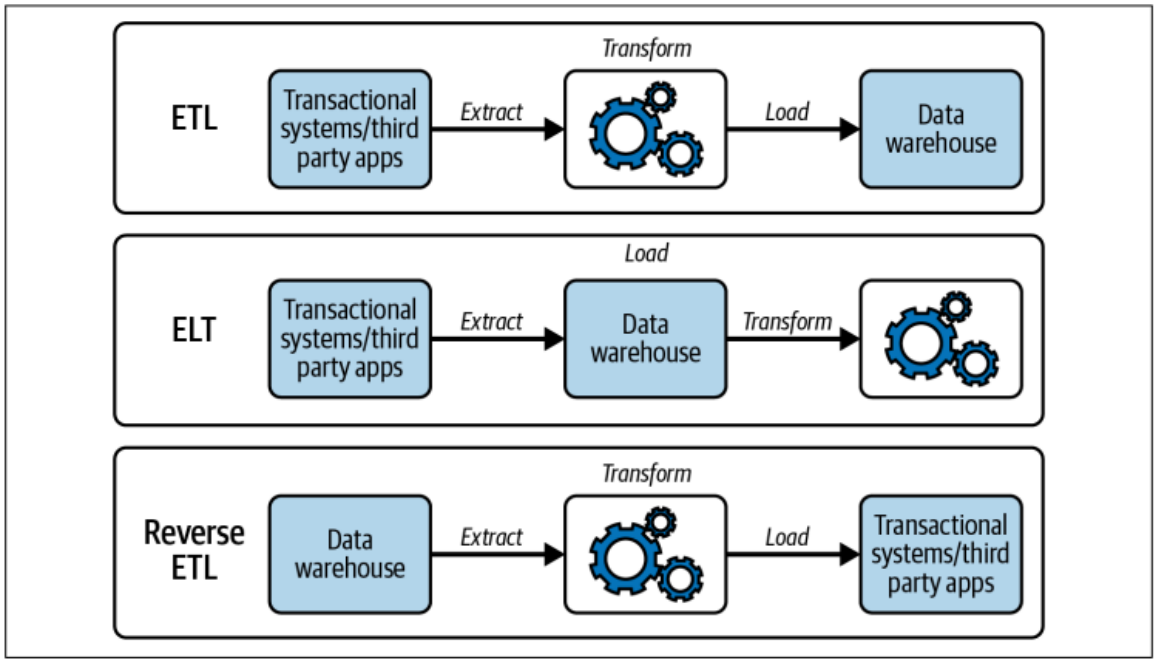


Figure 9-1. Comparison of ETL, ELT, and reverse ETL

شرکت‌ها در ساخت معیارهای ضروری در SQL بر روی داده‌کاوی‌های خود برای درک بهتر و ارتباط بهتر با مشتریان کار می‌کنند. این معیارها ممکن است شامل ارزش در طول عمر (یک پیش‌بینی از سود خالص کلی که یک شرکت انتظار دارد از روابط آینده خود با یک مشتری کسب کند)، مشتری مؤهل به محصول (یک مشتری پتانسیلی که با یک محصول مشارکت کرده و ممکن است به یک مشتری پرداخت‌کننده تبدیل شود)، و امتیاز احتمالی (احتمال اینکه یک کاربر یک محصول خریداری کند)، بین دیگران باشد.

در حالی که می‌توانید به راحتی گزارشات و نمایش‌ها با استفاده از این داده در ابزارهای BI یا SQL ایجاد کنید، با این حال، با اینکه این داده‌ها می‌توانند برای ایجاد گزارشات و نمایش‌ها در ابزارهای BI یا SQL بسیار قدرتمند باشند، اما اگر این داده‌ها توانایی راهنمایی روزمره تیم‌های شما در ابزارهایی که هر روز استفاده می‌کنند را تحریک کنند، قدرت بیشتری خواهند داشت. به عنوان مثال، شما می‌توانید گزارشات BI را از DW بسازید و نمایندگان فروش را آموزش

دهید تا از آن‌ها استفاده کنند—یا تحلیل عملی خود را با داشتن یک تحلیلگر داده که امتیازات مؤهل را از DW به یک فیلد سفارشی در Salesforce وارد می‌کند، اجرا کنید.

به عنوان یک مثال دیگر، فرض کنید تیم علم داده شما یک امتیاز احتمالی محاسبه کرده و آن را در یک DW یا دریاچه داده ذخیره کرده است. با استفاده از ETL Reverse، می‌توانید این امتیاز احتمالی را به یک پایگاه داده تولید عملی حرکت دهید تا تجربیات شخصی‌سازی شده در اپلیکیشن به مشتریان در زمان واقعی ارائه دهید. در عمل، این بدین معنی است که شرکت می‌تواند تجربه مشتری را بهبود بخشد، محتوای مرتبط و سفارشی شده، توصیه‌ها یا پیشنهادات بیشتری را بر اساس احتمال خرید محصول توسط مشتری ارائه دهد. اپلیکیشن به ترجیحات و رفتارهای فردی مشتری پاسخگو و متعهد تر می‌شود.

پردازش دسته ای در مقابل پردازش زمان واقعی

در ETL و ELT، شما دو گزینه برای زمان و فرکانس استخراج داده از سیستم منبع دارید: پردازش دسته‌ای (batch processing) و پردازش زمان واقعی (real-time processing). بیایید به هر کدام نگاه بیشتری بیندازیم:

پردازش دسته‌ای (Batch Processing)

پردازش دسته‌ای روشی کارآمد برای پردازش حجم زیادی از داده‌ها است. یک مجموعه از معاملات مشابه از یک سیستم منبع با هم گروه‌بندی می‌شوند ("دسته‌بندی شده‌اند") برای یک دوره زمانی خاص. در بازه‌های منظم، سیستم به طور خودکار یک کار را اجرا می‌کند و کل دسته از سیستم منبع به مقصد (یک دریاچه داده یا داده‌کاوی) کپی می‌شود، معمولاً در ساعات غیر اوج کسب و کار. تابع اصلی یک سیستم پردازش دسته‌ای اجرای دسته‌ها در زمان‌های برنامه‌ریزی شده به طور منظم (به عنوان مثال، هر شب در طول شب) یا در صورت نیاز است.

بیل برق شما یک مثال از پردازش دسته‌ای است. ماهواره برق شما داده‌های مصرف شما را در طول یک دوره مشخص (معمولاً یک ماه) جمع‌آوری می‌کند و سپس شرکت برق کل دسته را به شکل قبضه ماهانه شما پردازش می‌کند.

پردازش زمان واقعی (Real-time Processing)

پردازش زمان واقعی روشی است که داده‌ها را به طور مداوم هنگام جمع‌آوری آن‌ها، در ماده ثانیه یا میلی‌ثانیه، پردازش می‌کند و بازخوردهای نزدیک به فور ارائه می‌دهد. یک سیستم زمان واقعی به سرعت به اطلاعات جدید در یک معماری مبتنی بر رویداد پاسخ می‌دهد. وقتی یک فایل یا رکورد جدید در سیستم منبع تشخیص داده می‌شود، یک رویداد فعال می‌شود و فایل یا رکورد را به مقصد کپی می‌کند.

دستگاه‌های IoT، پیام‌های رسانه‌های اجتماعی و سیستم‌های معاملات مالی همه از سیستم‌های پردازش زمان واقعی استفاده می‌کنند. به عنوان مثال، اگر یک معامله کارت اعتباری مشکوک وجود داشته باشد، بانک نیاز دارد تا به صاحب کارت بدون تاخیر هشدار دهد، که دلیل استفاده از پردازش زمان واقعی برای تشخیص تقلب است. شما نمی‌خواهید آن‌ها تا روز بعد، پس از اجرای دسته شبانه‌روزی معاملات، منتظر بمانید!

برنامه‌های ترافیک نیز مثال دیگری خوب هستند. برنامه نقشه‌رسانی Waze به طور خودکار سطوح انباشت ترافیک را در زمان واقعی به‌روز می‌کند، با استفاده از اطلاعات کسب شده از دستگاه‌های موبایل مختلف و حسگرهای خیابان در منطقه برای پیشنهاد مسیر بهینه و کوتاه‌ترین برای رسیدن به مقصد شما. به طور واضح، اگر شما باید تا اجرای دسته شبانه‌روزی معاملات منتظر بمانید تا اطلاعات ترافیک را بدست آورید، کاملاً مفید خواهد بود.

پردازش زمان واقعی سیستم مقصد را فوراً به‌روز می‌کند، بنابراین گزارشات یا پرس و جوها هرگز قدیمی نمی‌شوند. این امکان را می‌دهد تا عملیات کسب و کار که نیاز به توجه فوری دارند و به‌روزرسانی‌هایی برای برنامه‌ها یا گزارشات که بر اساس داده‌های زمان واقعی اعتماد می‌کنند، به سرعت شناسایی شوند.

دریاچه‌های داده (RDWs) تقریباً فقط از پردازش دسته‌ای استفاده کرده‌اند. داده‌های زمان واقعی در سال‌های اخیر بسیار محبوب شده است. سیستم‌هایی که از آن استفاده می‌کنند، بیشتر داده‌های خود را در یک دریاچه داده

مزایای پردازش دسته‌ای (Batch Processing)

- **کارآمدی برای حجم بالای داده‌ها**: پردازش دسته‌ای برای حجم زیادی از داده‌ها کارآمد است، زیرا تمام داده‌ها در یک لحظه پردازش می‌شوند به جای پردازش مجزا.

- **برنامه‌ریزی در زمان‌های غیر اوج**: وظایف پردازش دسته‌ای می‌توانند در زمان‌های غیر اوج کسب و کار برنامه‌ریزی شوند، که باعث کاهش تأثیر بر استفاده عادی از سیستم می‌شود.

- **کمتر خطر از خرابی سیستم**: پردازش دسته‌ای خطر کمتری از خرابی سیستم دارد، زیرا وظایف شکسته می‌توانند بدون تأثیرات قابل توجه دوباره امتحان شوند.

معایب پردازش دسته‌ای (Batch Processing)

پردازش دسته‌ای برای حجم بالای داده‌ها کارآمد است، زیرا تمام داده‌ها در یک لحظه پردازش می‌شوند به جای پردازش مجزا. این همچنین امکان برنامه‌ریزی وظایف پردازش دسته‌ای در زمان‌های غیر اوج را فراهم می‌کند، که باعث کاهش تأثیر بر استفاده عادی از سیستم می‌شود. علاوه بر این، پردازش دسته‌ای خطر کمتری از خرابی سیستم دارد، زیرا وظایف شکسته می‌توانند بدون تأثیرات قابل توجه دوباره امتحان شوند.

معایب پردازش دسته‌ای شامل تاخیر احتمالی در در دسترس بودن داده‌ها است، زیرا زمان لازم برای پردازش مقادیر زیادی از داده در یک لحظه وجود دارد. این می‌تواند منجر به استفاده ناکام از منابع شود اگر به درستی مدیریت نشود. همچنین، پردازش دسته‌ای برای برنامه‌هایی که نیاز به بازخورد زمان واقعی دارند مناسب نیست، زیرا به‌روزرسانی‌ها بزرگتر و کمتر فراوان هستند و هر تغییر در داده‌ها یا محاسبات نیاز به اجرای مجدد کامل دسته دارد، که می‌تواند بر عملکرد تأثیر بگذارد.

معایب پردازش لحظه ای

در مقابل پردازش دسته‌ای، پردازش زمان واقعی بازخوردهای تا دقیقه را فراهم می‌کند، که امکان اقدام فوری بر اساس داده‌ها را فراهم می‌کند. این به خصوص مفید برای سیستم‌هایی است که نیاز به به‌روزرسانی‌های مداوم دارند و می‌توانند داده‌های جریانی را به طور موثر مدیریت کنند. همچنین، پردازش زمان واقعی بیشتر انعطاف‌پذیر و پاسخگو به نیازهای تغییر کننده کسب و کار است.

پردازش زمان واقعی همچنین معایب خود را دارد. این نیاز به منابع سیستم بیشتری برای پردازش مداوم دارد و خطر بیشتری از خرابی سیستم را ایجاد می‌کند. مدیریت خطاها و بازیابی در سیستم‌های زمان واقعی می‌تواند پیچیده باشد و نیاز به ابزارهای قوی دارد. اطمینان از یکپارچگی داده‌ها ممکن است به دلیل به‌روزرسانی‌های مداوم چالش‌برانگیزتر باشد. علاوه بر این، هزینه‌های مرتبط با پردازش زمان واقعی ممکن است بیشتر باشند به دلیل نیاز به پردازش مداوم.

تصمیم‌گیری بین پردازش دسته‌ای و پردازش زمان واقعی به وزن‌دادن عواملی مانند نوع داده، نیازهای پردازش شما و تحمل شما از تاخیر یا تاخیر معنی می‌شود. این انتخاب معمولاً شامل تعادل بین تقاضای برای داده‌های فوری و توانایی سیستم برای تخصیص منابع لازم برای پردازش زمان واقعی - که اغلب به عنوان تحمل تاخیر شناخته می‌شود - است. اگر یک فرآیند کسب و کار یا سیستم می‌تواند تاخیر کوچکی در دسترسی به داده‌ها را تحمل کند - به عبارت دیگر، اگر آن تحمل تاخیر بالایی دارد - پردازش دسته‌ای ممکن است رویکرد مناسب باشد. برعکس، اگر نیاز به داده‌های فوری حیاتی است و سیستم مجهز به مدیریت منابع مورد نیاز است - که نشان‌دهنده تحمل تاخیر پایین است - پردازش زمان واقعی ممکن است بهتر باشد.

مدیریت داده‌ها (Data Governance)

مدیریت کلی داده‌ها در یک سازمان است. این شامل تعریف سیاست‌ها و رویه‌ها برای جمع‌آوری، ذخیره‌سازی، امنیت‌سازی، تبدیل، و گزارش داده‌ها است. به طور خاص، این برای اطمینان از اینکه سازمان با نیازمندی‌های قانونی و مقررات مطابقت دارد، استفاده می‌شود. همچنین شامل نظارت بر کیفیت و دقت داده‌ها است - به عنوان مثال، اطمینان از اینکه داده‌ها به درستی تمیز شده و تبدیل شده‌اند.

مدیریت داده‌ها باید شامل یک چارچوب باشد که افراد و نقش‌های مسئول برای مدیریت، نگهداری، و استفاده از داده‌ها در سازمان را تعریف می‌کند. یک روش برای این کار ایجاد یک مرکز توانمندی مدیریت داده‌ها (CoE) است. CoE به عنوان یک مرکز برای توسعه سیاست‌ها، رویه‌ها، و استانداردهای مدیریت داده‌های سازمان عمل می‌کند و نقش‌ها، مسئولیت‌ها، و فرآیندهای تصمیم‌گیری را برای فعالیت‌های مرتبط با داده تعریف می‌کند.

مهم است که زمان اولیه را در تعریف چارچوب مدیریت داده‌ها و ساخت مرکز توانمندی مدیریت داده‌ها (CoE) قبل از ساخت هر راه حل دریاچه داده سرمایه‌گذاری کنید. بسیاری از پروژه‌ها به دلیل عدم توجه کافی به مدیریت داده‌ها شکست می‌خورند.

شکل ۹-۲ یک مدل بلوغ مدیریت داده‌ها را نشان می‌دهد که من برای کمک به شما در شناسایی زمینه‌هایی که سازمان شما باید بر روی آن‌ها کار کند، ایجاد کرده‌ام.

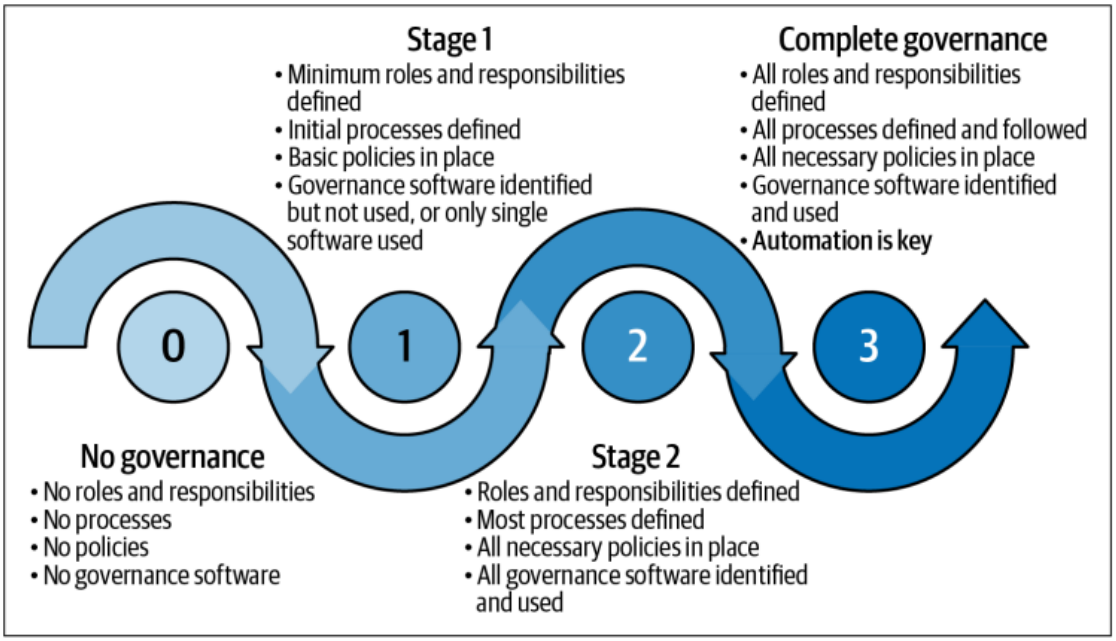


Figure 9-2. Data governance maturity model

خلاصه

مدیریت داده‌ها (Data Governance) مدیریت کلی داده‌ها در یک سازمان است. این شامل تعریف سیاست‌ها و رویه‌ها برای جمع‌آوری، ذخیره‌سازی، امنیت‌سازی، تبدیل، و گزارش داده‌ها است. به طور خاص، این برای اطمینان از اینکه سازمان با نیازمندی‌های قانونی و مقررات مطابقت دارد، استفاده می‌شود. همچنین شامل نظارت بر کیفیت و دقت داده‌ها است - به عنوان مثال، اطمینان از اینکه داده‌ها به درستی تمیز شده و تبدیل شده‌اند.

مدیریت داده‌ها باید شامل یک چارچوب باشد که افراد و نقش‌های مسئول برای مدیریت، نگهداری، و استفاده از داده‌ها در سازمان را تعریف می‌کند. یک روش برای این کار ایجاد یک مرکز توانمندی مدیریت داده‌ها (CoE) است. CoE به عنوان یک مرکز برای توسعه سیاست‌ها، رویه‌ها، و استانداردهای مدیریت داده‌های سازمان عمل می‌کند و نقش‌ها، مسئولیت‌ها، و فرآیندهای تصمیم‌گیری را برای فعالیت‌های مرتبط با داده تعریف می‌کند.

مهم است که زمان اولیه را در تعریف چارچوب مدیریت داده‌ها و ساخت مرکز توانمندی مدیریت داده‌ها (CoE) قبل از ساخت هر راه حل دریاچه داده سرمایه‌گذاری کنید. بسیاری از پروژه‌ها به دلیل عدم توجه کافی به مدیریت داده‌ها شکست می‌خورند.

شکل ۹-۲ یک مدل بلوغ مدیریت داده‌ها را نشان می‌دهد که من برای کمک به شما در شناسایی زمینه‌هایی که سازمان شما باید بر روی آن‌ها کار کند، ایجاد کرده‌ام.

معماری داده

حالا که شما درک خوبی از مفاهیم معماری داده دارید، زمان آن است که به هسته کتاب بپردازیم. پنج فصل قسمت سوم شامل چهار معماری هستند، به ترتیب زمان اولین ظهور آن‌ها: دریاچه داده مدرن، فابریک داده، لاک‌هاوس داده، و شبکه داده.

این تفسیرات من از معماری‌های داده هستند؛ دیگران ممکن است نظرات متفاوتی داشته باشند. در نهایت، راه حل شما ممکن است ترکیبی از این معماری‌ها باشد.

فصل دهم: انبار داده مدرن

در قسمت دوم این کتاب، شما درباره دریاچه‌های داده رابطه‌ای (RDWs) و دریاچه‌های داده یاد گرفتید، دو مؤلفه کلیدی در چشم‌انداز مدیریت داده. حالا بیایید به دنیای کسب و کار مدرن نگاه کنیم. هر روز، سازمان‌ها باید از مقادیر بزرگی از داده‌ها برای به دست آوردن بازخوردها، تصمیم‌گیری و تحریک رشد عبور کنند. تصور کنید یک سوپرمارکت شهری که از پایگاه‌های داده سنتی به یک دریاچه داده مدرن (MDW) می‌پردازد. مدیران حالا می‌توانند به داده‌های موجودی زمان واقعی دسترسی پیدا کنند، روندهای خرید را پیش‌بینی کنند و تجربه خرید را برای مشتریان خود بهینه‌سازی کنند. این قدرت MDW است. این دو جهان بهترین را ترکیب می‌کند: ساختار RDWs و انعطاف‌پذیری دریاچه‌های داده.

چرا باید در مورد MDWs نگران باشید؟ زیرا آن‌ها در قلب اکوسیستم داده‌های در حال تکامل سریع ما قرار دارند، که به سازمان‌ها اجازه می‌دهد تا اطلاعاتی که نیاز دارند را برای نوآوری و رقابت کنند. در این فصل، من توضیح می‌دهم که چه MDWs هستند و چه چیزی با آن‌ها می‌توانید دست آورید، و برخی از موارد مهمی را نشان می‌دهم که باید در نظر گرفت. ما راه رفتار، عملکرد و گام‌های متداول به یک MDW می‌بریم، با پایان یک مطالعه موردی با دیدگاه بازنگری. حالا بیایید به دنیای دریاچه‌های داده مدرن بپردازیم که داده بیش از اعداد است - این سوخت برای موفقیت است.

معماری انبارداده مدرن

شکل ۱۰-۱ طبیعت ترکیبی MDW را نشان می‌دهد، که یک RDW را با یک دریاچه داده ترکیب می‌کند تا محیطی ایجاد کند که اجازه می‌دهد تا داده‌ها را به طور انعطاف‌پذیر تغییر دهید و تحلیل‌های قوی انجام دهید.

همانطور که در فصل ٤ توضیح داده شد، یک RDW بر اساس اصل بالا به پایین عمل می‌کند. ساخت یکی شامل آماده‌سازی قابل توجهی در ساخت دریاچه قبل از هر گونه بارگذاری داده (schema-on-write) است. این رویکرد به گزارشات تاریخی کمک می‌کند با امکان برای تحلیل‌گران برای ورود به تحلیل‌های توصیفی (کشف چه اتفاق افتاده است) و تحلیل‌های تشخیصی (بررسی چرا اتفاق افتاده است).

خلاصه کتاب معماری داده

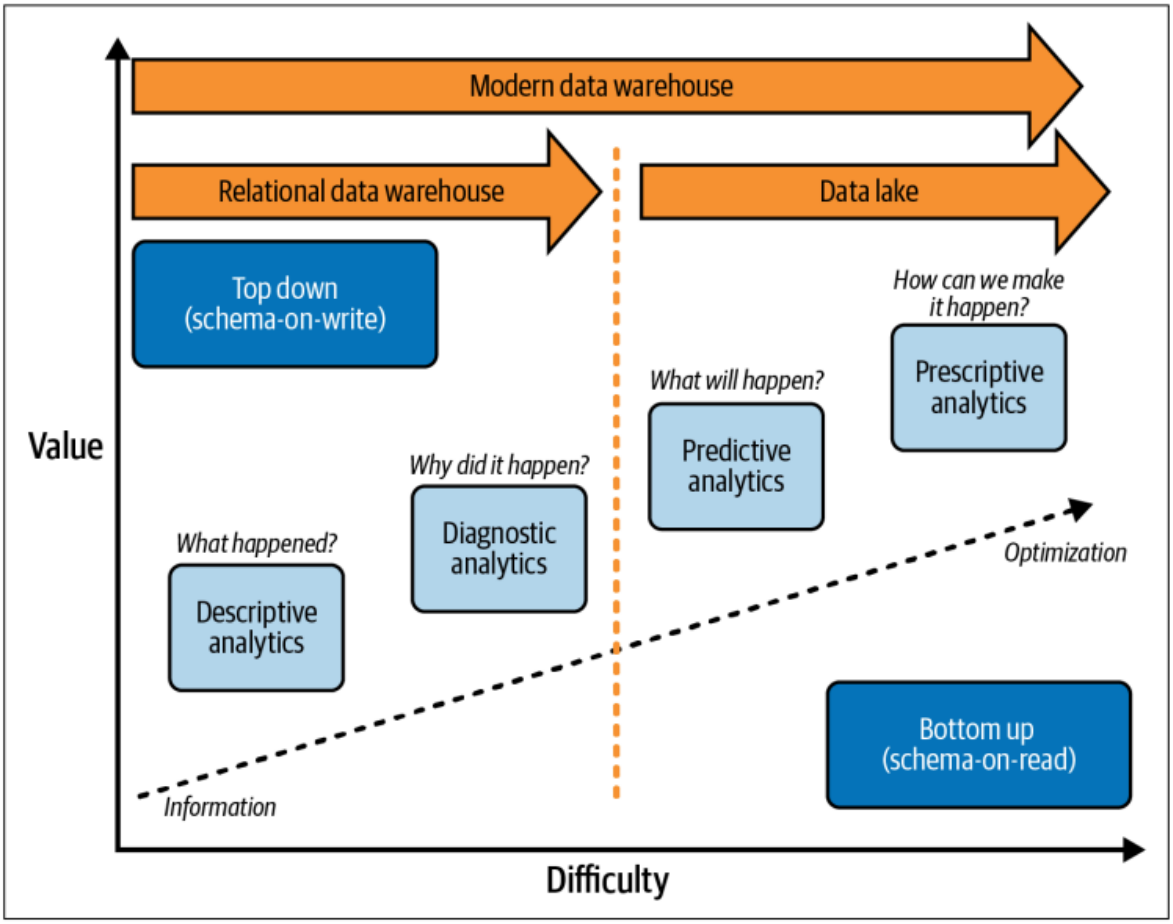


Figure 10-1. The full range of analytics facilitated by a modern data warehouse

در مقابل، یک دریاچه داده توسط فلسفه آن از پایین به بالا تعریف می‌شود، همانطور که در فصل ۵ توضیح داده شد. کار اولیه حداقلی برای شروع استفاده از داده‌ها (schema-on-read) لازم است، که اجازه می‌دهد تا مدل‌های یادگیری ماشین به سرعت راه‌اندازی شوند تا تحلیل‌های پیش‌بینی شده (پیش‌بینی چه اتفاق می‌افتد) و تحلیل‌های تجربی (تعیین راه‌حل‌ها برای ایجاد نتایج مورد نظر) را بررسی کنند.

در معماری MDW، یک دریاچه داده فقط یک مخزن برای ذخیره‌سازی مقادیر زیادی اطلاعات و آموزش مدل‌های یادگیری ماشین نیست؛ این یک مؤلفه پویا است، مسئول تبدیل داده‌ها و هدف‌های دیگری که در فصل ۵ بررسی شده‌اند. این ممکن است، به عنوان یک مرکز دریافت برای داده‌های جریانی، یک دروازه برای کاربران برای انجام سریع تحقیق داده‌ها و گزارش‌گیری، و منبع مرکزی برای حفظ یک نسخه واحد از حقیقت عمل کند.

یک ویژگی تمیز کلیدی معماری MDW این است که حداقل بخشی از داده‌های آن باید به یک RDW تکرار شود. بدون این تکرار، این یک معماری دریاچه-دریاچه خواهد بود - موضوعی که برای فصل ۱۲ مورد بحث قرار می‌گیرد.

در میانه دهه ۲۰۱۰، بسیاری از سازمان‌ها سعی کردند از دریاچه داده برای تمام موارد استفاده از داده استفاده کنند و از استفاده از یک RDW دور زدند. پس از اینکه این رویکرد شکست خورد، MDW به عنوان معماری محبوب ترین شناخته شد، تحت تأثیر افزایش نسبی در حجم، تنوع، و سرعت داده‌ها. سازمان‌های بسیاری در بخش‌های مختلف مانند بهداشت، مالی، بازاریابی، و فناوری، نیاز به رویکردی پویاتر و قابل مقیاس‌پذیری برای ذخیره‌سازی و تحلیل داده‌ها را شناختند. رویکردهای آن‌های RDW دیگر کافی نبودند تا چالش‌های داده‌های بزرگ را مدیریت کنند، اما MDWs یک ادغام پویا از RDWs و دریاچه‌های داده را ارائه دادند.

بازیگران کلیدی به عنوان ارائه‌دهندگان رتبه اول ظهور کردند - Microsoft با Azure Synapse Analytics، Amazon با Redshift، Google با BigQuery، و Snowflake - که روش کسب و کارها را در دسترسی و استفاده از داده‌های خود تغییر دادند.

همانطور که داده‌های جریانی، داده‌های بزرگ، و داده‌های نیمه ساختاری به شدت محبوب شدند، سازمان‌های بزرگ شروع به استفاده از معماری‌های فابریک داده و دریاچه‌هاوس که در فصل‌های ۱۱ و ۱۲ بحث شده‌اند کردند. با این حال، برای مشتریانی که داده‌های زیادی ندارند (مثلاً کمتر از ۱۰ ترابایت)، MDW هنوز محبوب است و احتمالاً به این ترتیب باقی خواهد ماند، حداقل تا زمانی که دریاچه‌ها به همان اندازه کار کنند که RDWs می‌توانند.

در واقع، هنوز شرکت‌هایی وجود دارند که حل‌های ابری را ساخته‌اند که از دریاچه‌های داده استفاده نمی‌کنند در برخی موارد استفاده که شرکت مقادیر کمی از داده دارد و از یک راه حل در محل که دریاچه داده نداشته باشد، مهاجرت می‌کند. من هنوز این را به عنوان یک رویکرد قابل قبول می‌بینم، با شرطی که شرکت باید مطمئن باشد که دریاچه داده به شدت رشد نخواهد کرد (که برای اطمینان کامل بسیار سخت است). همچنین، اگر شما مقدار کمی از داده دارید، ممکن است برای RDW خود با یک راه حل پردازش چندگانه متقابل (SMP) کمتر هزینه‌ای تر از یک راه حل پردازش چندگانه متقابل (MPP) انتخاب کنید (مشاهده فصل ۷).

حالا بیایید به جزئیات MDW بپردازیم. شکل ۱۰-۲ یک جریان معمولی از داده از طریق دریاچه داده و RDW در یک MDW را نشان می‌دهد.

در معماری MDW، یک دریاچه داده فقط یک مخزن برای ذخیره‌سازی مقادیر زیادی اطلاعات و آموزش مدل‌های یادگیری ماشین نیست؛ این یک مؤلفه پویا است، مسئول تبدیل داده‌ها و هدف‌های دیگری که در فصل ۵ بررسی شده‌اند. این ممکن است، به عنوان یک مرکز دریافت برای داده‌های جریانی، یک دروازه برای کاربران برای انجام سریع تحقیق داده‌ها و گزارش‌گیری، و منبع مرکزی برای حفظ یک نسخه واحد از حقیقت عمل کند.

یک ویژگی تمیز کلیدی معماری MDW این است که حداقل بخشی- از داده‌های آن باید به یک RDW تکرار شود. بدون این تکرار، این یک معماری دریاچه-دریاچه خواهد بود - موضوعی که برای فصل ۱۲ مورد بحث قرار می‌گیرد.

در میانه دهه ۲۰۱۰، بسیاری از سازمان‌ها سعی کردند از دریاچه داده برای تمام موارد استفاده از داده استفاده کنند و از استفاده از یک RDW دور زدند. پس از اینکه این رویکرد شکست خورد، MDW به عنوان معماری محبوب ترین شناخته شد، تحت تأثیر افزایش نسبی در حجم، تنوع، و سرعت داده‌ها. سازمان‌های بسیاری در بخش‌های مختلف مانند بهداشت، مالی، بازاریابی، و فناوری، نیاز به رویکردی پویاتر و قابل مقیاس‌پذیری برای ذخیره‌سازی و تحلیل داده‌ها را شناختند. رویکردهای آن‌های RDW دیگر کافی نبودند تا چالش‌های داده‌های بزرگ را مدیریت کنند، اما MDWs یک ادغام پویا از RDWs و دریاچه‌های داده را ارائه دادند.

بازیگران کلیدی به عنوان ارائه‌دهندگان رتبه اول ظهور کردند - Microsoft با Azure Synapse Analytics، Amazon با Google Redshift، با BigQuery، و Snowflake - که روش کسب و کارها را در دسترسی و استفاده از داده‌های خود تغییر دادند.

همانطور که داده‌های جریانی، داده‌های بزرگ، و داده‌های نیمه ساختاری به شدت محبوب شدند، سازمان‌های بزرگ شروع به استفاده از معماری‌های فابریک داده و دریاچه‌هاوس که در فصل‌های ۱۱ و ۱۲ بحث شده‌اند کردند. با این حال، برای مشتریانی که داده‌های زیادی ندارند (مثلاً کمتر از ۱۰ ترابایت)، MDW هنوز محبوب است و احتمالاً به این ترتیب باقی خواهد ماند، حداقل تا زمانی که دریاچه‌ها به همان اندازه کار کنند که RDWs می‌توانند.

در واقع، هنوز شرکت‌هایی وجود دارند که حل‌های ابری را ساخته‌اند که از دریاچه‌های داده استفاده نمی‌کنند در برخی موارد استفاده که شرکت مقادیر کمی از داده دارد و از یک راه حل در محل که دریاچه داده نداشته باشد، مهاجرت می‌کند. من هنوز این را به عنوان یک رویکرد قابل قبول می‌بینم، با شرطی که شرکت باید مطمئن باشد که دریاچه داده به شدت رشد نخواهد کرد (که برای اطمینان کامل بسیار سخت است). همچنین، اگر شما مقدار کمی از داده دارید، ممکن است برای RDW خود با یک راه حل پردازش چندگانه متقابل (SMP) کمتر هزینه‌ای تر از یک راه حل پردازش چندگانه متقابل (MPP) انتخاب کنید (مشاهده فصل ۷).

حالا بیایید به جزئیات MDW بپردازیم. شکل ۱۰-۲ یک جریان معمولی از داده از طریق دریاچه داده و RDW در یک MDW را نشان می‌دهد.

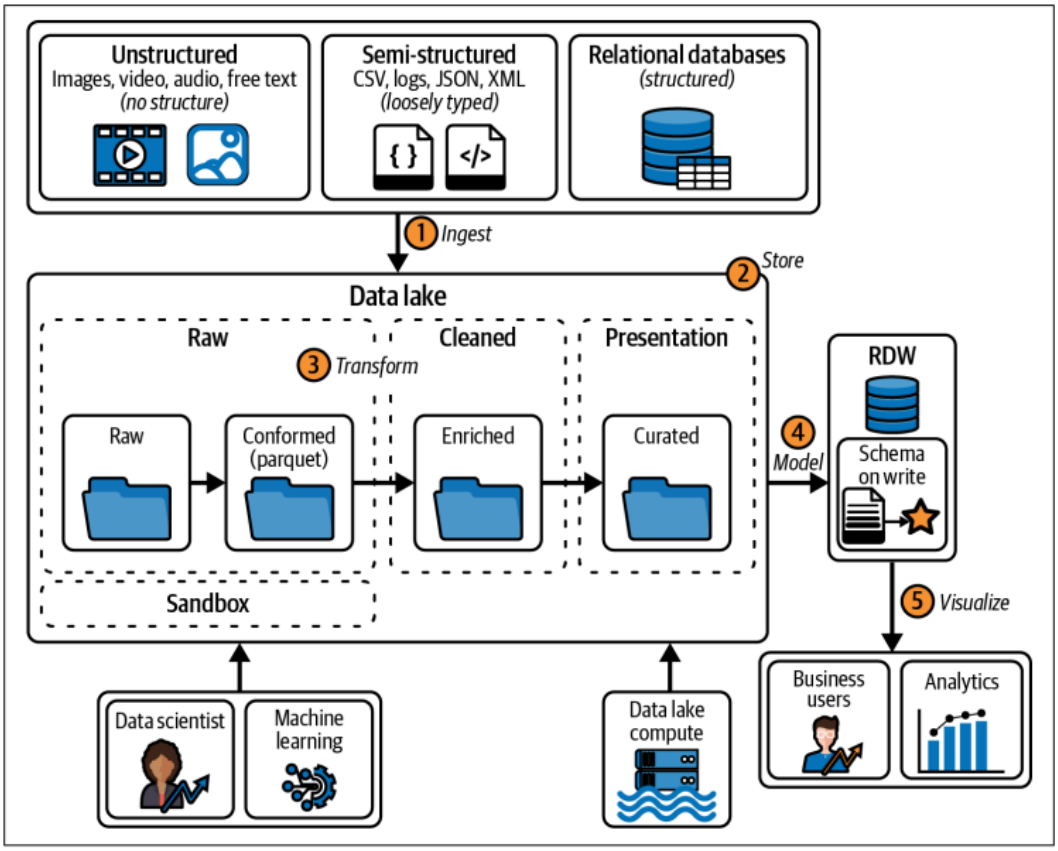


Figure 10-2. Overview of the journey that data takes through a modern data warehouse architecture

هنگامی که داده‌ها از طریق معماری MDW می‌روند، از طریق پنج مرحله می‌رسند که در شکل ۱۰-۲ شماره‌گذاری شده‌اند. بیایید داده‌ها را از طریق هر مرحله بررسی کنیم تا به شما نمایش معماری MDW بدهیم:

مرحله ۱: دریافت داده (Ingestion)

یک MDW می‌تواند نزدیک به هر نوع داده‌ای بپردازد و می‌تواند از منابع متعددی باشد، هم در محیط محلی (on-prem) و هم در ابر (cloud). داده‌ها ممکن است در اندازه، سرعت و نوع متفاوت باشند؛ می‌توانند ساختار نداشته باشند، نیمه ساختار یافته باشند یا رابطه‌ای باشند؛ می‌توانند در دسته‌ها یا از طریق جریان زمان واقعی بارگذاری شوند؛ می‌توانند در فایل‌های کوچک یا بزرگ بارگذاری شوند. این تنوع می‌تواند در طول دریافت داده چالش برانگیز باشد.

مرحله ۲: ذخیره‌سازی (Storage)

هنگامی که داده دریافت می‌شود، در یک دریاچه داده قرار می‌گیرد که تمام لایه‌های مختلف توضیح داده شده در فصل ۵ را شامل می‌شود، اجازه می‌دهد که کاربران نهایی بتوانند بدون محدودیت دسترسی به آن داشته باشند (به شرطی که دسترسی به ابر را داشته باشند). هر ارائه‌دهنده ابر ذخیره‌سازی بی‌نهایت در دریاچه داده را ارائه می‌دهد و هزینه نسبتاً کم است. در دریاچه داده دارای در دسترسی بالا و بازیابی پس از حوادث مقاوم است، همراه با بسیاری از گزینه‌ها برای امنیت و رمزنگاری.

مرحله ۳: تبدیل (Transformation)

اصل یک دریاچه داده این است که یک مرکز ذخیره‌سازی است. با این حال، برای واقعاً درک و کار با داده‌ها، به چند قدرت نیاز دارید - که اینجا قدرت محاسباتی وارد می‌شود. یکی از مزایای قابل توجه MDW تفاوت واضح بین

ذخیره‌سازی داده (در دریاچه داده) و پردازش آن (با استفاده از قدرت محاسباتی) است. نگهداری دو بخش مجزا به شما اجازه می‌دهد تا از مجموعه‌ای از ابزارهای محاسباتی انتخاب کنید، از ارائه‌دهندگان ابر یا منابع دیگر. این ابزارها، طراحی شده برای دریافت و درک داده‌ها از دریاچه، می‌توانند داده‌ها را در فرمت‌های مختلف خوانده و سپس آن‌ها را به یک فرمت استانداردتر مانند Parquet تنظیم کنند. این جداسازی انعطاف‌پذیری را فراهم می‌کند و کارایی را افزایش می‌دهد.

ابزار محاسباتی سپس فایل‌ها را از لایه مطابقت‌داده می‌گیرد، داده‌ها را تبدیل می‌کند (با افزایش و تمیز کردن آن‌ها) و آن‌ها را در لایه تمیز شده دریاچه داده ذخیره می‌کند. در نهایت، ابزار محاسباتی فایل‌ها را از لایه افزوده می‌گیرد و تبدیلات بیشتری برای عملکرد یا آسانی استفاده (مانند ادغام داده‌ها از چندین فایل و تجمیع آن‌ها) را انجام می‌دهد و سپس آن را به لایه ارائه در دریاچه داده می‌نویسد.

مرحله ٤: مدل‌سازی داده (Data modeling)

گزارش مستقیم از داده‌ها در یک دریاچه داده می‌تواند برای کاربران نهایی با سرعت کم، ناامن و گیج کننده باشد. به جای آن، می‌توانید تمام یا بخشی‌ از داده‌ها را از دریاچه داده به یک انبار داده رابطه‌ای کپی کنید. این بدین معناست که شما یک مدل رابطه‌ای برای داده‌ها در RDW ایجاد خواهید کرد، که مدل معمولاً در فرم سوم نرمال است (به فصل ۸ مراجعه کنید). شما همچنین ممکن است می‌خواهید آن را در یک مدل ستاره درون RDW کپی کنید برای دلایل عملکرد و ساده‌سازی.

مرحله ٥: تصویرسازی (Visualization)

هنگامی که داده در RDW به یک فرمت قابل فهم قرار می‌گیرد، کاربران تجاری می‌توانند آن را با استفاده از ابزارهای آشنا، مانند گزارشات و داشبوردها تجزیه و تحلیل کنند.

برای مرحله دریافت، باید تعیین کنید که چقدر اطلاعات را باید استخراج کنید و آیا باید استخراج تدریجی یا کامل استفاده کنید (به فصل ٤ مراجعه کنید). در نظر داشته باشید که اندازه و پهنای باند پایپلین از منابع داده تا ابر که دریاچه داده در آن قرار دارد، چگونه است. اگر نیاز به انتقال فایل‌های بزرگ دارید و پهنای باند شما کوچک است، ممکن است نیاز باشد که داده‌ها را چندین بار در قطعات کوچکتر بارگذاری کنید به جای یک قطعه بزرگ روزانه. ارائه‌دهنده ابر شما ممکن است گزینه‌هایی برای خرید بیشتر پهنای باند به اینترنت داشته باشد.

در مراحل مختلف این سفر، دانشمندان می‌توانند از یادگیری ماشین برای آموزش و ساخت مدل‌ها از داده‌ها در MDW استفاده کنند. بسته به نیازهایشان، می‌توانند داده‌ها را از لایه‌های خام، تمیز، یا ارائه در دریاچه داده، از یک لایه ساندباکس در دریاچه داده (یک فضای اختصاصی برای انجام آزمایش، کاوش و تجزیه و تحلیل اولیه بدون تأثیر بر لایه‌های دیگر)، یا از RDW بگیرند.

برخی از استثناها در سفری که من تا حالا توضیح دادم وجود دارد، در که داده‌ها از طریق معماری MDW جریان می‌یابند، بستگی به ویژگی‌های داده منبع دارد. به عنوان مثال، همه داده‌ها در لایه ارائه دریاچه داده نیازی به کپی شدن به RDW ندارند، به خصوص با ابزارهای جدیدتر که اجازه می‌دهند تا به راحتی از داده‌ها در یک دریاچه داده پرس و جو و گزارش کنید. این بدین معناست که در مرحله تصویرسازی، کاربر تجاری نیازی به رفتن به RDW برای تمام داده‌ها ندارد—آن‌ها می‌توانند به داده‌های دریاچه داده که کپی نشده به RDW دسترسی پیدا کنند.

یک استثنا دیگر این است که همه داده‌های منبع نیازی به کپی شدن به دریاچه داده ندارند. در برخی موارد، به خصوص پروژه‌های جدیدی که در ابر ساخته می‌شوند، سریع‌تر است که داده‌ها را مستقیماً از منبع به RDW کپی کنید، دریاچه داده را دور زده و به خصوص برای داده‌های منبع ساختاری (رابطه‌ای) استفاده کنید. به عنوان مثال، جداول مرجعی که به عنوان جداول بعدی استفاده می‌شوند، نیازی به تمیز کردن ندارند، استخراج کامل هستند و می‌توانند به راحتی از منبع بازیابی شوند اگر باید بسته ETL را دوباره اجرا کنید. در نهایت، ممکن است کار زیادی باشد که داده‌ها را از یک پایگاه داده رابطه‌ای استخراج کنید، آن‌ها را به دریاچه داده کپی کنید (که از داده‌ها اطلاعات ارزشمند مانند انواع داده، محدودیت‌ها و کلیدهای خارجی را از دست می‌دهد)، و سپس آن‌ها را به یک پایگاه داده رابطه‌ای دیگر (انبار داده) وارد کنید.

این به خصوص در مورد یک مهاجرت معمول از یک راه حل محلی که از یک دریاچه داده استفاده نمی‌کند و دارای بسیاری از بسته‌های ETL است که داده‌ها را از یک پایگاه داده رابطه‌ای به RDW کپی می‌کنند، صادق است. برای کاهش تلاش مهاجرت، شما باید فقط بسته‌های ETL موجود را کمی تغییر دهید—تغییر مقصد فقط. با گذشت زمان، می‌توانید تمام بسته‌های ETL را تغییر دهید تا از دریاچه داده استفاده کنند. برای راه‌اندازی سریع، با این حال، شاید فقط چندین بسته ETL کند را تغییر دهید تا از دریاچه داده استفاده کنند و بقیه کارها را بعداً انجام دهید.

داده‌های منبعی که دریاچه داده را دور زده‌اند، از برخی از مزایای آن از دست خواهند داد—به خصوص پشتیبان‌گیری، در صورتی که نیاز به اجرای مجدد بسته ETL داشته باشید. دور زدن همچنین می‌تواند بر روی RDW فشار غیرمناسبی ایجاد کند، که سپس مسئول تمیز کردن داده می‌شود. علاوه بر این، افرادی که از دریاچه داده استفاده می‌کنند، متوجه می‌شوند که داده‌ها گم شده‌اند، که جلوگیری می‌کند دریاچه داده به عنوان منبع یکتای حقیقت عمل کند.

معایب معماری مدرن

این رویکرد منافع بسیاری دارد اما معایبی هم دارند که به طرح ذیل هستند:

یکپارچگی منابع داده متفاوت

MDWs قادر به مدیریت هم داده‌های ساختاری و هم داده‌های غیرساختاری از منابع مختلف هستند، از جمله دریاچه‌های داده و انبارهای داده رابطه‌ای (RDWs)، ارائه یک دید جامع از اطلاعات را فراهم می‌کنند.

Scalability

MDWs طراحی شده‌اند تا با رشد کسب و کار همراه باشند، به راحتی می‌توانند برای مدیریت بارهای داده‌ای افزایش یابند.

Real-time data analysis

MDWs تحلیل‌های زمان واقعی را تسهیل می‌کنند، که به کسب و کارها اجازه می‌دهد تا بر اساس داده‌های فعلی تصمیمات به موقع بگیرند.

Improved performance

با استفاده از فناوری پیشرفته و پردازش داده‌های بهینه‌سازی شده، MDWs می‌توانند عملکرد پرس و جو سریع‌تر و بازیابی بینش‌ها را ارائه دهند.

Flexibility

MDWs انعطاف‌پذیری در مدل‌سازی داده را فراهم می‌کنند، اجازه می‌دهند که هم پرس و جوهای ساختاری سنتی و هم تکنیک‌های پردازش داده‌های بزرگ استفاده شود.

Enhanced security

ارائه‌دهندگان برتر MDW اقدامات امنیتی قوی را برای حفاظت از داده‌های حساس اجرا می‌کنند.

برخی از معایب معماری MDW عبارتند از:

خلاصه - رمزگشایی از معماری داده

Complexity

پیاده‌سازی و مدیریت MDW می‌تواند پیچیده باشد، به خصوص در معماری‌های ترکیبی که انواع مختلفی از ذخیره‌سازی و پردازش داده را ترکیب می‌کنند.

Cost

راه‌اندازی اولیه، نگهداری مداوم و مقیاس‌بندی می‌تواند گران باشد، به خصوص برای کسب و کارهای کوچک تا متوسط. علاوه بر این، هزینه‌های اضافی برای ذخیره‌سازی و برای ایجاد مدارهای داده اضافی برای ایجاد کپی‌های چندگانه از داده‌ها (برای هر دو RDW و دریاچه داده) وجود خواهد داشت.

Skill requirements

برای استفاده به طور کامل از MDW نیاز به دانش و مهارت تخصصی وجود دارد، که ممکن است منجر به چالش‌های استخدام یا هزینه‌های آموزش اضافی شود.

Potential data silos

بدون یکپارچگی و حاکمیت مناسب، MDWs می‌توانند منجر به سیلوهای داده شوند، یا مکان‌هایی که اطلاعات می‌شود و در سراسر سازمان دسترسی سخت می‌شود.

Compliance challenges

رعایت مطابقت با قوانین و مقررات در محیطی که انواع مختلف داده و منابع را مدیریت می‌کند، می‌تواند یکی از چالش‌های مدیریت MDW باشد.

وابستگی به تامین‌کننده

اگر از یک سرویس MDW مبتنی بر ابر استفاده می‌کنید، این ممکن است به معنای وابستگی به یک تامین‌کننده خاص باشد، که می‌تواند منجر به قفل‌شدن به طور موقت و محدود کردن انعطاف‌پذیری در آینده شود.

همانطور که داده‌ها در یک MDW کپی می‌شوند و از طریق دریاچه داده و به RDW حرکت می‌کنند، فرمت آن‌ها تغییر می‌کند و با هر بار ساده‌تر و ساده‌تر می‌شود. این کمک می‌کند تا یک سیستم Self Service BI کاربرپسند را فراهم کند، که در آن کاربران نهایی می‌توانند با کشیدن فیلدها از یک لیست به یک فضای کاری گزارش ساده بسازند بدون اینکه هر جدولی را به آن متصل کنند. بخش IT باید کارهای اضافی اولیه انجام دهد تا داده‌ها واقعاً آسان برای استفاده باشند، اما این معمولاً ارزش دارد، زیرا این بخش را از مشارکت در (و احتمالاً یک گلوگاه برای) ایجاد هر گزارش و داشبورد آزاد می‌کند.

ترکیب انبارداده و دریاچه داده

در یک MDW، دریاچه داده برای مرحله بندی و آماده سازی داده ها استفاده می شود، در حالی که RDW برای خدمات، امنیت و انطباق. بیایید دقیق تر به کجای عملکرد نگاه کنیم زمانی وجود دارد که از دریاچه RDW و داده استفاده می کنید.

دریاچه داده

در یک دریاچه داده، دانشمندان داده و کاربران قدرتمند - به خصوص آن‌هایی که مهارت‌های فنی بالاتری دارند - دسترسی محصول خاص خود را خواهند داشت، به دلیل ساختار پوشه-فایل پیچیده و جداسازی متادیتا.

دریاچه‌های داده ممکن است دشوار برای پیمایش باشند و دسترسی ممکن است نیاز به ابزارهای پیشرفته‌تری داشته باشد. توابع دریاچه داده شامل پردازش دسته‌ای است، که در آن داده‌ها در دسته‌ها تبدیل می‌شوند، و پردازش زمان واقعی، که به عنوان مقصد نقل داده‌های جریانی عمل می‌کند (مشاهده کنید فصل ۹). دریاچه داده همچنین برای تمیز کردن و تصفیه داده‌ها استفاده می‌شود، فراهم کردن یک پلتفرم برای استفاده از قدرت محاسباتی به اندازه کافی، با گزینه‌های محاسباتی متعددی برای انتخاب. همچنین بار کارهای ELT را تحمیل می‌کند.

دریاچه داده، در این چارچوب، به عنوان مکانی برای ذخیره داده‌های قدیمی یا پشتیبان عمل می‌کند، به جای نگهداری آن‌ها در RDW. همچنین مکانی برای پشتیبان‌گیری داده‌ها از خود دریاچه داده است. کاربران می‌توانند به راحتی کپی‌هایی از داده‌ها در دریاچه داده برای اهداف ساندباکس ایجاد کنند، اجازه دادن به دیگران که در داده‌ها استفاده کنند و "بازی کنند". دریاچه داده همچنین فرصتی را برای مشاهده و بررسی داده‌ها ارائه می‌دهد اگر مطمئن نیستید که چه سوالاتی را از آن مطرح کنید؛ شما می‌توانید قبل از کپی آن در دریاچه داده ارزش آن را ارزیابی کنید. این به سرعت گزارش‌گیری و دسترسی به داده‌ها را تسهیل می‌کند، به خصوص زیرا این به صورت schema-on-read است بنابراین داده‌ها می‌توانند به سرعت منتقل شوند.

انبار داده رابطه ای

افراد تجاری اغلب به RDWs می‌روند به عنوان مکانی که برای دسترسی به داده‌ها برای افراد غیرفنی است، به خصوص اگر آن‌ها به دیتابیس‌های رابطه‌ای عادت دارند. این دیتابیس‌ها کمترین تاخیر را ارائه می‌دهند، که امکان پرس و جوی سریع‌تر را فراهم می‌کند، به خصوص اگر از فناوری MPP استفاده می‌کنید (مشاهده فصل ۷). آن‌ها می‌توانند تعداد زیادی از ادغام جدول‌ها و پرس و جوهای پیچیده را مدیریت کنند و برای اجرای پرس و جوهای تعاملی آده‌کن بسیار مناسب هستند به دلیل عملکرد سریع MPP فناوری که ارائه می‌دهد. این به شما اجازه می‌دهد تا پرس و جوها را بدون انتظار مدت زمان طولانی تنظیم کنید. RDWs همچنین می‌توانند بسیاری از کاربران همزمان را که پرس و جوها و گزارشات را اجرا می‌کنند پشتیبانی کنند.

ویژگی‌های امنیتی اضافی RDWs در زمینه MDW شامل گزینه‌هایی مانند امنیت سطح ردیف و سطح ستون است. RDWs همچنین بسیاری از پشتیبانی برای ابزارها ارائه می‌دهند - و از آنجا که مدت طولانی‌تر از دریاچه‌های داده وجود داشته‌اند، بسیاری از ابزارهای بیشتری در دسترس هستند. دریاچه‌های داده نمی‌توانند عملکرد لازم برای داشبوردها را پشتیبانی کنند، اما شما می‌توانید داشبوردها را در برابر RDWs اجرا کنید، به دلیل عملکرد برتر آن‌ها تا میلی‌ثانیه. یک لایه متادیتا اجباری بالای داده‌ها نیاز به کار بیشتری در ابتدا دارد اما باعث می‌شود که خدمات Self Service BI بسیار آسان‌تر شود. به طور کلی، هنگامی که شما در حال ساخت یک RDW هستید، شما قبلاً می‌دانید که چه سوالاتی را از داده‌ها می‌خواهید بپرسید و می‌توانید برای آن‌ها برنامه‌ریزی کنید، که باعث می‌شود RDW به عنوان یک ابزار قدرتمند و چندمنظوره شناخته شود.

سنگ پله به MDW

ساخت یک MDW یک مسئله حیاتی اما طولانی و سخت است که نیاز به سرمایه‌گذاری قابل توجهی در زمینه فناوری، منابع انسانی، و زمان دارد. این نمایانگر تکامل در مدیریت داده است، که در آن یکپارچگی، دسترسی، امنیت، و قابلیت مقیاس‌پذیری اصلی هستند. در حالی که این فرآیند در حال شروع است، بیشتر سازمان‌ها نیاز به راه‌حل‌های موقتی برای برآورده کردن نیازهای فعلی خود در دریاچه داده دارند. این راه‌حل‌ها، که به عنوان پایه‌های موقتی به MDW عملیاتی عمل می‌کنند، اطمینان می‌دهند که سازمان هنوز می‌تواند ارزشی را از داده‌های خود به طور موقت به دست آورد و همچنان انعطاف‌پذیر و پاسخگو به نیازهای تجاری باقی بماند. آن‌ها فقط تعمیرات موقتی نیستند بلکه اجزای حیاتی در یک مهاجرت استراتژیک هستند.

سه نوع معمولی از معماری‌های پایه شامل می‌شوند:

- تقویت EDW

- دریاچه داده موقت به همراه EDW

- همه چیز در یک

هر یک از این رویکردها مزایا و چالش‌های منحصر به فردی ارائه می‌دهد. مناسب بودن آن‌ها به عنوان مسیرهایی به MDW می‌تواند بسته به نیازهای سازمان، زیرساخت موجود، بودجه، و اهداف استراتژیک متفاوت باشد. بیایید به هر یک از آن‌ها بیشتر نگاه کنیم.

افزایش EDW

این معماری معمولاً زمانی مشاهده می‌شود که یک شرکت که در طول زمان بزرگی یک EDW در محل خود داشته است، می‌خواهد ارزشی را از داده‌های خود به دست آورد، اما EDW آن نمی‌تواند "داده‌های بزرگ" را مدیریت کند به دلیل کمبود فضای ذخیره‌سازی، قدرت محاسباتی، زمان موجود در پنجره نگهداری برای بارگذاری داده‌ها، یا پشتیبانی کلی برای داده‌های نیمه-ساختاری.

چگونگی عملکرد

شما یک دریاچه داده در ابر ایجاد می‌کنید و داده‌های بزرگ را به آن کپی می‌کنید. کاربران می‌توانند از دریاچه داده پرس و جو کنند و گزارشات را بسازند، اما داده‌های اصلی همچنان در EDW ذخیره می‌شوند، همانطور که در شکل ۱۰-۳ نشان داده شده است.

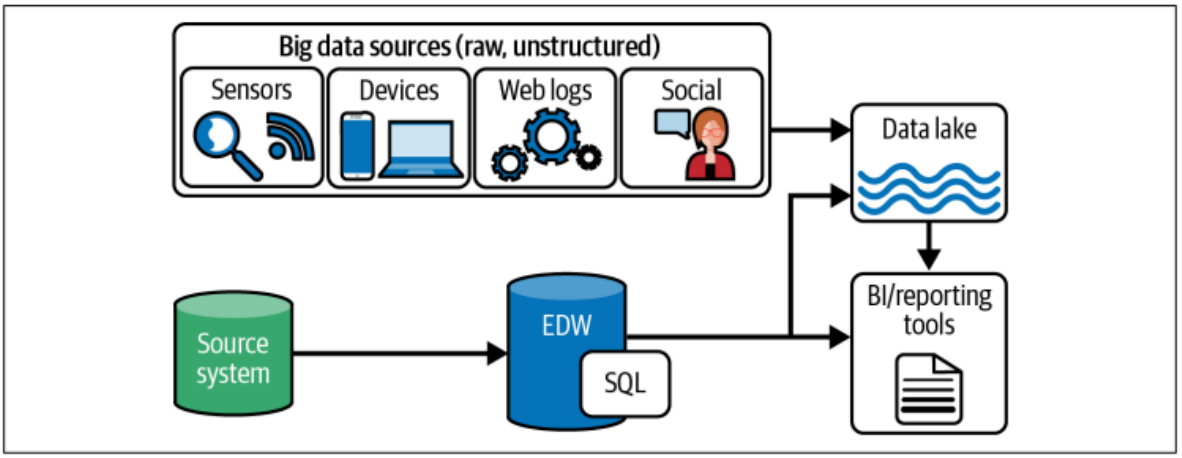


Figure 10-3. EDW augmentation architecture

مزایا

معماری تقویت EDW قابلیت و انعطاف‌پذیری داده را با استفاده از یک دریاچه داده در ابر کنار EDW موجود افزایش می‌دهد. این استراتژی امکان مقیاس‌پذیری و کارایی هزینه را فراهم می‌کند، فرصت‌هایی را برای تحلیل‌های نوآورانه فراهم می‌کند در حالی که ساختارهای فعلی را حفظ می‌کند. این یک رویکرد متعادل ارائه می‌دهد که با رشد و ادامه تجاری هماهنگ است.

چالش ها

این معماری چندین مشکل پتانسیلی را ارائه می‌دهد:

- اگر نیاز به ترکیب داده‌ها از EDW با داده‌ها در دریاچه داده دارید، باید داده‌ها را از EDW به دریاچه داده کپی کنید.

- ابزارهای پرس و جوی موجود شما ممکن است با یک دریاچه داده کار نکنند.

- شما نیاز به یک نوع جدید از محاسبات برای تمیز کردن داده‌ها در دریاچه داده خواهید داشت، که ممکن است گران باشد و نیاز به مجموعه مهارت‌های جدید داشته باشد.

- این معماری شما را از تحمل هیچ یک از بار کار EDW کمک نمی‌کند.

مهاجرت

این معماری می‌تواند شروعی برای رویکرد مراحلی به مهاجرت EDW در محل به ابر باشد. پس از اینکه معماری با داده‌های بزرگ در حال اجرا باشد، می‌توانید شروع به مهاجرت داده‌ها در EDW در محل به دریاچه داده کنید. داده‌ها ابتدا به دریاچه داده منتقل می‌شوند و سپس، در صورت نیاز، به یک RDW جدید در ابر که بخشی از یک MDW واقعی است، منتقل می‌شوند.

دریاچه داده موقت و EDW

شرکت‌ها معمولاً از یک دریاچه داده موقت با EDW استفاده می‌کنند زمانی که یک EDW دارند و نیاز به ادغام داده‌های بزرگ دارند، اما تبدیل آن‌ها زمان زیادی می‌برد. این شرکت‌ها می‌خواهند پنجره نگهداری EDW را با تحمل تبدیل داده‌ها کاهش دهند.

چگونگی عملکرد

این معماری از یک دریاچه داده استفاده می‌کند، اما فقط به عنوان یک منطقه موقتی برای مرحله آماده‌سازی و تمیز کردن. این برای پرس و جو یا گزارش‌گیری استفاده نمی‌شود؛ همه این کارها از EDW انجام می‌شوند، همانطور که شکل ۱۰-٤ نشان می‌دهد. این محدودیت در محدوده باعث می‌شود که دریاچه داده بتواند بسیار سریع‌تر در آن درگیر شود. گاهی اوقات حتی می‌توانید داده‌های EDW را به دریاچه داده کپی کنید، آن‌ها را تمیز کنید و آن‌ها را به EDW برگردانید.

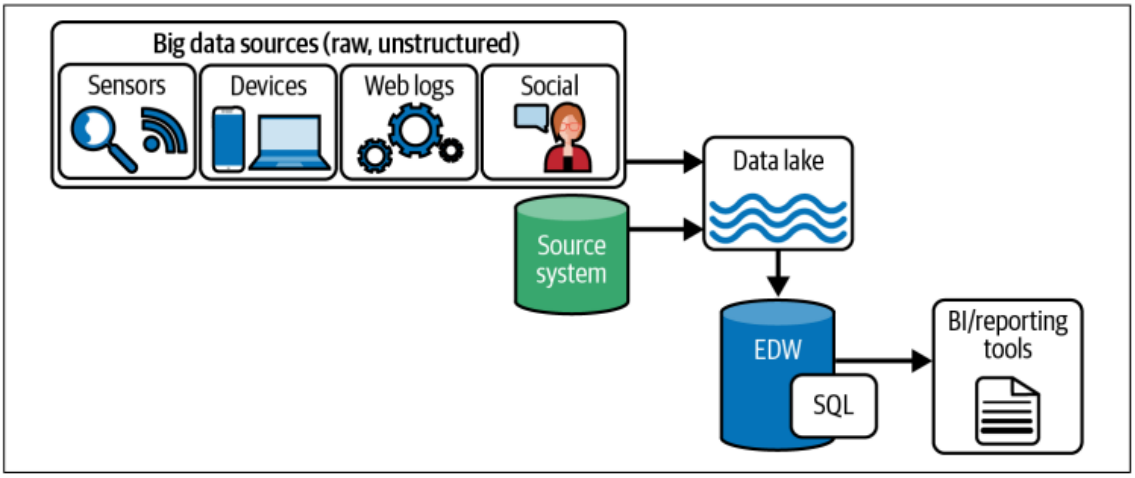


Figure 10-4. Using an EDW with a temporary data lake

مزایا

این معماری امکان تحمل پردازش داده به دریاچه داده را فراهم می‌کند، که باعث کاهش فشار بر روی EDW می‌شود و عملکرد کلی را افزایش می‌دهد. استفاده از نوع‌های دیگری از محاسبات بر روی دریاچه داده می‌تواند سرعت و قابلیت‌های بیشتری را فراهم کند، که امکان مدیریت انعطاف‌پذیر داده‌های بزرگ را فراهم می‌کند. این استراتژی یک راه حل کارایی هزینه برای ادغام مجموعه‌های داده بزرگ بدون ایجاد اختلال در عملیات موجود EDW را ارائه می‌دهد، که آن را به یک رویکرد انعطاف‌پذیر و قابل مقیاس‌پذیری تبدیل می‌کند.

چالش ها

با اینکه شما از یک دریاچه داده استفاده می‌کنید، این معماری شما را از منافع کامل یک دریاچه داده (که در فصل ۵ بحث شده است) محروم نمی‌کند.

مهاجرت

با تغییرات کمی، این معماری می‌تواند به یک MDW کامل تبدیل شود، بنابراین یک پایه مناسب برای مهاجرت است.

همه چیز در یکجا

معماری همه در یک معمولاً توسط سازمان‌هایی که به دنبال یک رویکرد ساده‌شده به مدیریت داده هستند، مانند شرکت‌های نوپا یا کسب و کارهای کوچک، اتخاذ می‌شود. این ممکن است برای انجام پروتایپ‌های سریع یا برای دستیابی به اهداف کوتاه مدت مورد علاقه باشد. همچنین گزینه خوبی است زمانی که کاربران اصلی متخصصان فنی هستند که به یک پلتفرم یکپارچه ترجیح می‌دهند.

چگونگی عملکرد

تمام گزارش‌گیری و پرس و جو با استفاده از دریاچه داده انجام می‌شود، همانطور که شکل ۱۰-۵ نشان می‌دهد. هیچ RDW درگیر نیست. (همه در یک بسیار نزدیک به معماری دریاچه‌خانه که در فصل ۱۲ بحث شده است، متصل است.)

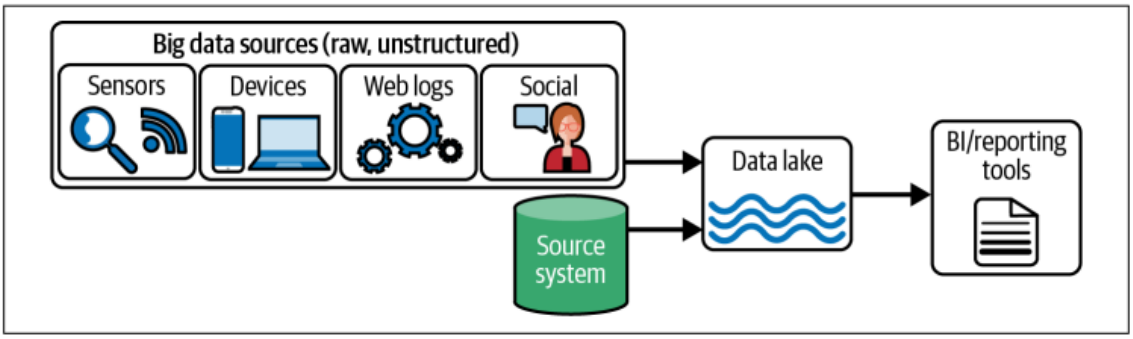


Figure 10-5. All-in-one architecture

مزایا

این رویکرد اجازه می‌دهد تا به سرعت پیاده‌سازی شود و نتایج فوری بدست آید، مزایایی که به خصوص برای تیم‌های فنی که به دنبال پیشرفت سریع هستند، جذاب هستند. با ترکیب گزارش‌گیری و پرس و جو در داخل دریاچه داده، معماری ساده‌تر می‌شود، که به طور مستقیم می‌تواند پیچیدگی‌های نگهداری و یکپارچگی را کاهش دهد. این رویکرد می‌تواند بیشتر انعطاف‌پذیر و قابل تطبیق باشد، که انواع مختلف داده‌ها را پشتیبانی می‌کند و فرآیند توسعه را ساده‌تر می‌کند.

چالش ها

عدم وجود RDW معاملاتی با معایب مربوط به عملکرد، امنیت، یکپارچگی مرجع، یا دوستانه‌بودن کاربر دارد.

مهاجرت

برای برخی شرکت‌ها و موارد استفاده، مانند زمانی که داده‌ها در دریاچه داده فقط توسط دانشمندان داده استفاده می‌شوند، رویکرد فقط دریاچه داده ممکن است کافی باشد. اما برای تبدیل آن به یک پایه برای MDW، باید یک RDW اضافه کنید.

مطالعه موردی: تغییر استراتژیک ویلسون و گانکرک به یک MD

ویلسون و گانکرک، یک شرکت دارویی خیالی متوسط مقیاس قرار دارد در منطقه میان غربی ایالات متحده، بر اساس راه حل های محلی برای مدیریت داده های خود اعتماد داشته است. با توجه به حجم داده ها که به طور مداوم زیر ۱ ترابایت بوده است، شرکت هرگز نیازی به انتقال به راه حل های پیشرفته تر مانند یک دیتا لیک یا یک مدیریت داده های محوری (MDW) نشان داده نمی داد.

چالش

نیاز شرکت به بینش های قابل اعمال در مورد روندهای بازار و رفتارهای مشتری با افزایش شدن محیط کسب و کار رقابتی آن بیشتر شد. سیستم داده موجود شروع به نشان دادن محدودیت های خود کرد و سازمان با یک دیلما مواجه شد: آیا باید به یک راه حل دیتا لیک بیشتر جامع یا یک MDW ارتقا دهد. یک مورد بحرانی دیگر نگرانی در مورد رشد آینده داده ها بود، به خصوص با پروژه های تحقیقاتی و توسعه جدید در پیپلاین.

راهکار

پس از تحلیل دقیق، ویلسون و گانکرک تصمیم گرفتند که یک راه حل MDW را در ابر پیاده سازی کنند. چندین عامل که به این تصمیم کمک کردند عبارتند از:

- حجم داده موجود کاملاً زیر ۱ ترابایت بود و رشد پیش بینی شده در حدود ۱۰ ترابایت بود.

- انتقال به یک MDW مبتنی بر ابر انتقال راحت تری از راه حل محلی موجود را فراهم می کرد بدون نیاز به سازگاری با یک فناوری کاملاً جدید مانند یک دیتا لیک.

- یک MDW از لحاظ هزینه موثر خواهد بود. ویلسون و گانکرک برای یک راه حل SMP به جای گزینه گران تر MPP انتخاب کردند، زیرا نیازهای داده آنها محدود و قابل مدیریت بود.

- انتخاب شده عملکرد قوی و قابلیت مقیاس کافی را برای مدیریت رشد داده های پیش بینی شده در حدوده MDW قابل قبول فراهم می کرد.

دستاورد

انتقال به MDW توسط ویلسون و گانکرک در طی چند ماه با کمترین اختلال به فرآیندهای کسب و کار در حال اجرا کامل شد. این شرکت به سرعت منفعت های فوری را تجربه کرد:

- تحلیل های بهبود یافته

- سیستم جدید تحلیل داده ها را بهبود بخشید و تصمیم گیری بیشتر اطلاعات دار را فراهم کرد. شرکت از تحلیل تاریخی ساده به تحلیل پیش بینی رفت. قبل از اینکه MDW وجود داشت، بینش ها محدود به روندهای گذشته بود. سیستم جدید ادغام منابع داده متنوع و استفاده از الگوریتم های پیشرفته تحقیق داده را تسهیل کرد. این امکان را به شرکت داد تا مدل های پیش بینی ایجاد کند، مانند پیش بینی کارایی داروها بر اساس دموگرافی بیماران و عوامل ژنتیکی. این انتقال منجر به تصمیم گیری بیشتر متمایز شد و رویکرد دقیق تری به توسعه دارو و راه حل های پزشکی شخصی‌سازی شد.

- صرفه جویی در هزینه

- راه حل SMP عملکرد بسیار خوبی را با هزینه کمتر نسبت به راه حل های MPP بالقوه یا یک دیتا لیک کامل ارائه داد.

- آمادگی آینده

- در حالی که در باز داشتن در انتقالات آینده، MDW امکان رشد تدریجی و قابلیت سازگاری بدون سرمایه گذاری بیش از حد در فناوری که شرکت فوراً نیاز نداشت را فراهم کرد.

مورد ویلسون و گانکرک نشان می دهد که رویکرد متعادل به مدیریت راه حل های داده است. برای شرکت هایی که با حجم داده های محدود مواجه هستند و به دنبال یک راه حل قابل مقیاس و هزینه موثر هستند، MDW ها هنوز یک گزینه جذاب را ارائه می دهند. نیازها و مسیر رشد هر سازمان متفاوت است، بنابراین تحلیل دقیق نیازهای حال و آینده برای انتخاب معماری داده مناسب بسیار حیاتی است.

خلاصه

این فصل با توصیف دقیق MDW شروع شد و سپس پنج مرحله کلیدی سفر داده از طریق MDW را توصیف کرد: مصرف (ingestion)، ذخیره سازی (storage)، تبدیل (transformation)، مدل سازی (modeling)، و تصویرسازی (visualization). این مراحل معماری زندگی داده را نشان می دهند که ابتدا به طور اولیه گرفته می شود، به طور امن ذخیره می شود، برای کاربردی بودن تمیز می شود، برای بینش ها مدل سازی می شود، و در نهایت برای تفسیر آسان و تصمیم گیری تصویرسازی می شود.

من مزایای MDW را شامل جنبه هایی مانند ادغام، قابلیت مقیاس پذیری، تحلیل در زمان واقعی، و امنیت توضیح دادم، در حالی که چالش هایی مانند پیچیدگی، هزینه، و وابستگی به تامین کننده را در نظر گرفتم. سپس به ترکیب راه حل های مدیریت داده سنتی (RDWs) با دیتا لیک ها پرداختیم، ویژگی های منحصر به فرد این ترکیب را برجسته کردیم و انعطاف پذیری بهبود یافته ای که در مواجهه با انواع مختلف داده ها و ساختارها ارائه می دهد.

سپس به بررسی دقیق سه معماری انتقالی پرداختم که شرکت ها اغلب به عنوان راه حل های موقتی برای نیازهای خود به انبار داده استفاده می کنند: تقویت انبار داده های کارآمد (enterprise data warehouse augmentation)، دیتا لیک موقت به همراه EDW، و همه چیز در یک.

هر مدل به عنوان یک اقدام موقتی عمل می کند در حالی که سازمان ها به سمت پیاده سازی یک MDW کامل حرکت می کنند.

با مطالعه موردی از ویلسون و گانکرک، یک شرکت دارویی متوسط مقیاس، و تغییر استراتژیک آنها به MDW، به پایان رسیدم.

فصل های آینده به طور عمیق تر به معماری پارچه داده و معماری دیتا لیک هاوس می پردازند، منافع آنها را باز می کنند و موارد استفاده ممکن در منظره در حال تغییر مدیریت داده را باز می گردانند.

فصل یازدهم: دیتا فابریک

معماری پارچه داده (data fabric architecture) یک تکامل از معماری انبار داده مدرن (MDW) است: یک لایه پیشرفته که بر روی MDW ساخته شده است تا دسترسی به داده ها، امنیت، قابلیت کشف و در دسترس بودن را افزایش دهد. تصویر بزنید که پارچه داده به طور کامل در سراسر شرکت شما می رود، تمام داده ها را جمع آوری می کند و آنها را به هر کسی- که نیاز دارد، درون شرکت یا حتی خارج از آن ارائه می دهد. این یک معماری است که می تواند داده ها را بدون توجه به اندازه، سرعت یا نوع مصرف کند. جنبه مهم ترین فلسفه پارچه داده این است که یک راه حل پارچه داده می تواند هر نوع داده ای را در سازمان مصرف کند.

برخی در صنعت آن را به شکل متفاوتی تعریف می کنند. برخی حتی از آن به طور متقابل با انبار داده مدرن استفاده می کنند! به عنوان مثال، شرکت مشاوره ای Gartner با تعریف مشابهی شروع می کند، می نویسد که پارچه داده یک مفهوم طراحی است که به عنوان یک لایه یکپارچه (fabric) از داده ها و فرآیندهای اتصال عمل می کند. یک پارچه داده از تحلیل های مداوم بر روی دارایی های متادیتای موجود، قابل کشف و استنباط شده استفاده می کند تا طراحی، اجرا و استفاده از داده های یکپارچه و قابل استفاده مجدد در تمام محیط ها، از جمله پلتفرم های هیبریدی و چند ابری را پشتیبانی کند.

جایی که دیدگاه Gartner از من متفاوت است در دیدگاهی است که ویرایش داده (که در فصل ٦ آموخته اید) یک قطعه اصلی از تکنولوژی پارچه داده است که نیاز به انتقال یا کپی کردن داده از سیستم های جداگانه را کاهش می دهد. Gartner یک "پارچه داده هوشمند" را تصور می کند که از نمودارهای دانش و AI و ML برای اتوماسیون کشف داده ها و کاتالوگ سازی، یافتن روابط داده، ارکستراسیون و یکپارچه سازی داده ها، و کشف خودکار متادیتا استفاده می کند. با این حال، بسیاری از تکنولوژی مورد نیاز هنوز وجود ندارد و تا به حال، هیچ یک پارچه داده ای وجود ندارد که تعریف Gartner را برآورده کند. من امیدوارم که یک روز تکنولوژی در دسترس باشد، اما در حال حاضر پیشنهاد می کنم که بر روی معماری های پارچه داده که در حال حاضر وجود دارد تمرکز کنید.

معماری پارچه داده (Data Fabric) معماری است که بیشترین استفاده را برای راه حل های جدیدی که در ابر ساخته می شوند، به ویژه برای مقادیر زیادی از داده ها (بیش از ۱۰ ترابایت) دارد. بسیاری از شرکت ها "ارتقا" معماری خود MDW را به یک پارچه داده می کنند با افزودن تکنولوژی اضافی، به خصوص پردازش در زمان واقعی، سیاست های دسترسی به داده ها، و یک کاتالوگ متادیتا. این فصل هشت ویژگی تکنولوژیکی را تعریف می کند که یک MDW را به قرارگاه پارچه داده می برند. اینجا برخی ابهام وجود دارد؛ اگر سه از این ویژگی ها را اضافه کنید، آیا شما یک MDW یا یک پارچه داده دارید؟ هیچ خط روشنی بین دو موجود نیست. شخصاً آن را معماری پارچه داده می نامم اگر حداقل سه از این هشت تکنولوژی را استفاده کند. بسیاری از افراد هیچ یک از آنها را استفاده نمی کنند اما با این حال حل خود را پارچه داده می نامند زیرا با فلسفه پارچه داده مطابقت دارند (من فکر می کنم این کاملاً مناسب است).

این فصل به شما یک بررسی کلی از معماری پارچه داده و هشت مؤلفه کلیدی آن را می دهد: سیاست های دسترسی به داده، یک کاتالوگ متادیتا، مدیریت داده های اصلی (MDM)، ویرایش داده، پردازش در زمان واقعی، API ها، خدمات، و محصولات.

معماری دیتا فابریک

معماری پارچه داده (Data Fabric) شامل تمام مؤلفه های MDW است اما چندین ویژگی اضافی را ارائه می دهد. شکل ۱۱-۱ یک نمودار از این معماری را نشان می دهد، با مراحل سفر داده با شماره هایی که مانند شکل ۱۰-۲ بوده اند. داده ها در یک پارچه داده فرآیند یکسانی را دنبال می کنند که در MDW است؛ همانطور که شما ممکن است از فصل ۱۰ به یاد داشته باشید، این مراحل عبارتند از (۱) مصرف، (۲) ذخیره سازی، (۳) تبدیل، (٤) مدل سازی، و (٥) تصویرسازی.

پس چه چیزی پارچه داده برای فراتر از MDW ارائه می دهد؟ این بخش به بررسی ویژگی های اضافی آن خواهد پرداخت.

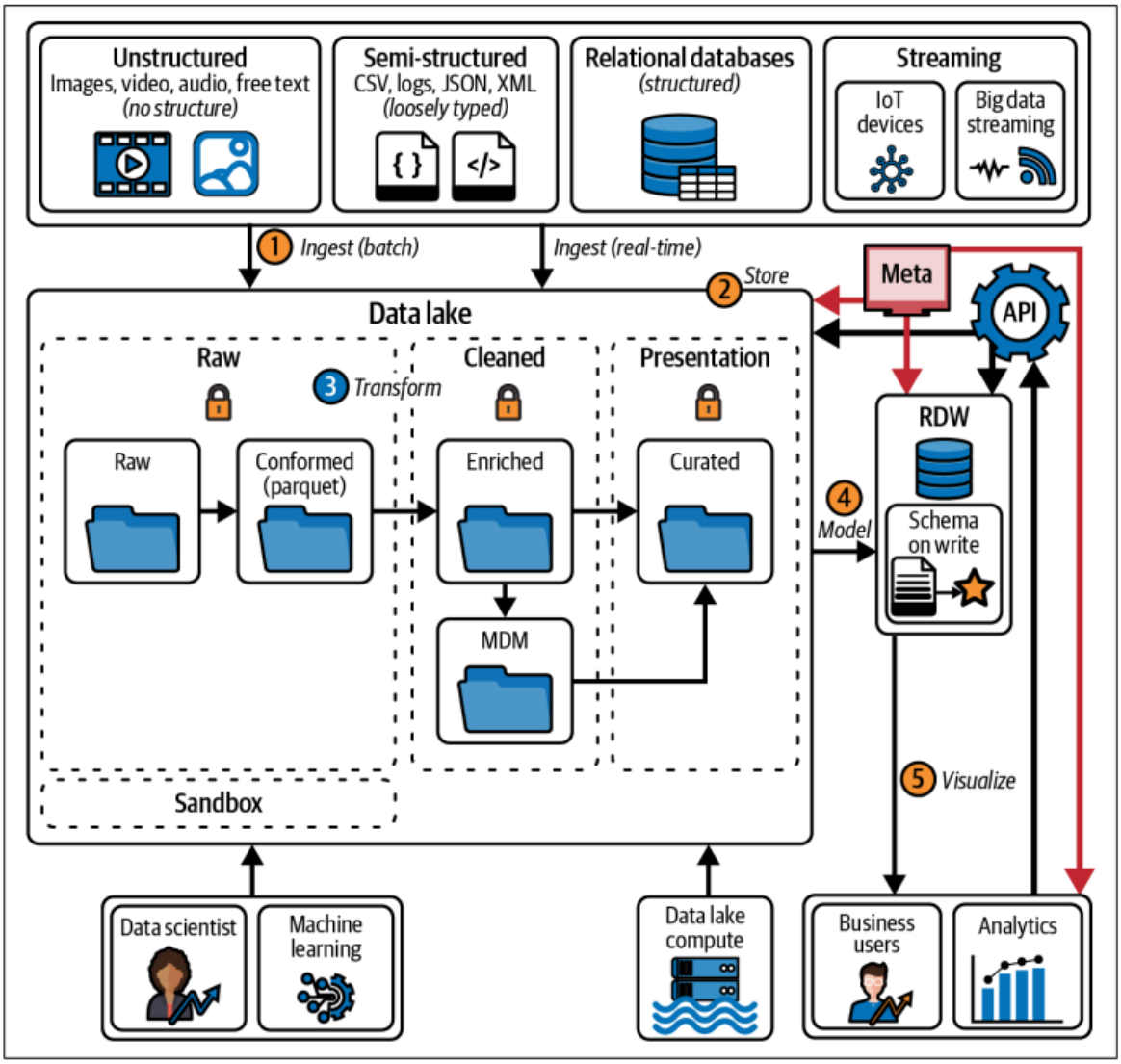


Figure 11-1. An overview of the journey data takes through a data fabric architecture

سیاست های دسترسی به داده ها

سیاست های دسترسی به داده کلیدی برای حاکمیت بر داده ها هستند. آنها شامل مجموعه ای از رهنمودها، قوانین، و رویه ها هستند که کنترل می کنند که چه کسی- به چه اطلاعاتی دسترسی دارد، چگونه این اطلاعات می توانند استفاده شوند، و زمانی که دسترسی می تواند در یک سازمان مجاز یا منها شود. آنها به اطمینان امنیت، حفظ حریم خصوصی، و یکپارچگی داده های حساس کمک می کنند، همچنین با رعایت قوانین و مقررات مانند قانون حفظ داده های عمومی بریتانیا (GDPR) و قانون قابلیت حمل و نقل بیمه و مسئولیت آمریکا (HIPAA) در ایالات متحده موافقت می شوند. این سیاست ها معمولاً موضوعاتی مانند طبقه بندی داده ها، احراز هویت و اجازه کاربران، رمزگذاری داده ها، حفظ داده ها، پشتیبان گیری و بازیابی داده ها، و تخلیه داده ها را پوشش می دهند.

تمام درخواست های داده در یک سازمان باید با سیاست های دسترسی به داده تعریف شده آن سازمان مطابقت داشته باشند و باید از طریق مکانیزم های خاصی مانند API ها، درایورها، یا ویرایش داده ها، پردازش شوند، تأکید بر رعایت قوانین مانند HIPAA. به عنوان مثال، سیاست های دسترسی به داده یک سازمان بهداشتی ممکن است متقاضی باشد که فقط کارکنان پزشکی مجاز به دسترسی به سوابق بیماران داشته باشند. این سیاست ها باید روش های تأیید مدارک کارکنان و استفاده های قابل قبول از اطلاعات را شرح دهند. برای تسهیل دسترسی کنترل شده این، سازمان ممکن است API هایی را پیاده سازی کند که با سیستم های احراز هویت امن رابطه برقرار می کنند. با این

امنیت در جایگاه، وقتی یک ارائه دهنده خدمات بهداشتی سوابق یک بیمار را درخواست می کند، API مدارک ارائه دهنده را با سیاست های دسترسی مقایسه می کند قبل از اعطای دسترسی.

متادیتا کاتالوگ

پارچه داده ها شامل یک کاتالوگ متادیتا هستند: یک مخزن که اطلاعاتی در مورد دارایی های داده، از جمله ساختار، روابط، و ویژگی های آنها را ذخیره می کند. این یک راه مرکزی، سازمان یافته برای مدیریت و کشف داده ها فراهم می کند، که برای کاربران آسان تر می شود تا داده هایی که نیاز دارند را پیدا و درک کنند. به عنوان مثال، اگر یک کاربر در کاتالوگ متادیتا برای "مشتری" جستجو کند، نتایج شامل هر فایل، جدول پایگاه داده، گزارشات، یا داشبوردهایی که داده های مشتری را شامل می شوند، خواهد بود. امکان دیدن چه چیزی از مصرف و گزارش داده شده است به همه کمک می کند تا تلاش های تکراری را جلوگیری کنند.

یک بخش مهم از کاتالوگ گردش داده است، یک سوابق تاریخچه هر قطعه داده ای خاص، از جمله اینکه از کجا آمده، چگونه تبدیل شده است، و کجا ذخیره شده است. گردش داده برای رعایت قوانین مانند قوانین حفظ حریم خصوصی داده استفاده می شود. این جا است که می توانید منشأ داده را ردیابی کنید و تبدیلاتی که آن را تحت عنوان گذشته است، که بخش مهمی از درک چگونگی ایجاد آن و چقدر قابل اعتماد است، را ردیابی کنید. شما نمی خواهید تصمیمات مهم را بر اساس داده ها بگیرید مگر اینکه مطمئن باشید که می توانید آنها را اعتماد کنید! به عنوان مثال، اگر یک کاربر در مورد یک مقدار خاص در گزارش سوال کند، شما می توانید گردش آن مقدار را در کاتالوگ متادیتا برای نشان دادن چگونگی ایجاد آن ببینید.

استادی مدیریت داده

همانطور که در فصل ٦ یاد گرفتید، MDM (مدیریت داده های اصلی) فرآیندی است که داده های متنوع را از منابع مختلف در یک شرکت جمع آوری، ترکیب می کند و آنها را برای ایجاد یک منبع مرجع یکپارچه و دقیق نگهداری می کند. این داده ها معمولاً غیر معاملاتی هستند و موجودیت های کلیدی یک شرکت را توصیف می کنند، مانند مشتریان، محصولات، تامین کنندگان، و کارکنان.

MDM به شرکت ها کمک می کند تا تصمیمات آگاهانه بگیرند و از مشکلات مرتبط با داده ها مانند رکوردهای تکراری، ناهماهنگی ها، و اطلاعات نادرست خودداری کنند.

مصورسازی داده ها

شما همچنین در فصل ٦ با ویرایش داده آشنا شدید. برای بازیابی حافظه، ویرایش داده یک معماری نرم افزاری است که به برنامه ها و کاربران نهایی اجازه می دهد تا به داده ها از منابع متعدد به عنوان اگر آنها در یک مکان تک منظور ذخیره شده باشند دسترسی پیدا کنند. داده های مجازش در یک لایه منطقی ذخیره می شوند که به عنوان یک میانبر بین کاربران نهایی و منابع داده عمل می کند. این لایه مجازش یک نقطه دسترسی تک نقطه است که پیچیدگی منابع داده زیرین را کم کرده و برای کاربران نهایی آسان می کند تا داده ها را از منابع متعدد در زمان واقعی بدون نیاز به کپی یا یکپارچگی فیزیکی آنها دسترسی پیدا کنند.

استفاده از ویرایش داده به معنی نیاز به اتصال تمام منابع داده نیست. در بیشتر موارد، فقط برای تعداد کمی از موارد استفاده می شود، با این حال بیشتر داده ها هنوز متمرکز هستند. برخی از افراد در صنعت ویرایش داده را به عنوان یک ویژگی مشخص پارچه داده می بینند، ادعا می کنند که یک معماری پارچه داده بدون ویرایش داده نیست؛ من با این نظر متفاوت هستم. در تعریف من از پارچه داده، ویرایش داده اختیاری است.

پردازش در لحظه

همانطور که در فصل ۹ مشاهده کردید، پردازش در زمان واقعی به پردازش داده و تولید نتایج فوری هنگامی که داده موجود می شود، بدون هیچ تاخیر قابل تشخیص، اشاره دارد. این امکان را به شما می دهد تا بر اساس اطلاعات به روز (مانند اطلاعات ترافیک هنگام رانندگی) تصمیم بگیرید.

API ها

به جای اعتماد به رشته های اتصال، API ها داده را از منابع متنوعی به صورت استاندارد ارائه می دهند—مانند یک دریاچه داده یا RDW—بدون اشتراک جزئیات مکان داده ها. اگر داده ها منتقل شوند، شما فقط نیاز به به روز رسانی کد داخلی API دارید؛ هیچ یک از برنامه هایی که API را فراخوانی می کنند تحت تأثیر قرار نمی گیرند. این باعث می شود تا استفاده از فناوری های جدید آسان تر شود.

API ها همچنین می توانند شامل چندین نوع اقدامات امنیتی باشند. آنها همچنین امکان انعطاف پذیری با فیلتر امنیتی را فراهم می کنند، کنترل دقیق تری را بر روی کاربرانی که می توانند داده های خاصی را دسترسی داشته باشند، فراهم می کنند.

خدمات

پارچه داده می تواند در "بلاک ها" ساخته شود، که باعث می شود کد را آسان تر بازیافت کنید. به عنوان مثال، ممکن است در شرکت شما دیگران وجود داشته باشند که نیازی به پارچه داده کامل ندارند، اما نیاز به کدی که شما ایجاد کرده اید برای تمیز کردن داده ها یا برای مصرف آنها دارند. شما می توانید این کد را عمومی کنید و آن را در یک سرویس کپسوله بندی کنید که هر کسی می تواند از آن استفاده کند.

محصولات

یک پارچه داده کامل می تواند به عنوان یک محصول بسته بندی شود و فروخته شود. این خصوصاً جذاب خواهد بود اگر به طور خاص برای یک صنعت مانند بهداشت سلامت ساخته شود.

چرایی مهاجرت به دیتا فابریک از انبار داده مدرن

تعدادی دلیل وجود دارد که سازمان ها تصمیم می گیرند از MDW به پارچه داده منتقل شوند. در حالی که MDW ها تاکنون پایه ای برای بسیاری از زیرساخت های داده کسب و کارها بوده اند، آنها می توانند سفت و گاهی اوقات سخت برای مقیاس بندی باشند زیرا انواع داده ها به سرعت در حال توسعه هستند. در مقابل، یک پارچه داده با توجه به قابلیت مقیاس بندی طراحی شده است. انعطاف پذیری اصلی آن امکان سریع تطبیق با انواع مختلف داده ها و منابع را فراهم می کند، که آن را قادر می سازد تا در برابر نیازهای داده ای کنونی مقاوم باشد و در میزان خاصی آینده پذیر باشد.

علاوه بر این، تنوع بی نهایت منابع داده می تواند احساس اضطراب کننده باشد. یک پارچه داده می تواند منابع متفاوت را به طور بی درز با هم پیوسته کند، یک نمای یکپارچه و متحد از داده ها را ارائه می دهد که باعث می شود کنترل و مدیریت جریان های داده چند جانبه بسیار آسان تر و کارآمد تر شود.

در یک محیط کسب و کار در حال تغییر سریع، توانایی پردازش داده در زمان واقعی و استخراج بینش های فوری حیاتی است. یک پارچه داده این نیاز را با پشتیبانی از پردازش داده در زمان واقعی برآورده می کند، کسب و کارها را به دسترسی فوری به آخرین داده ها می دهد.

علاوه بر این، با افزایش نگرانی ها در مورد نقض داده ها و تأکید بیشتر بر رعایت قوانین حفظ حریم خصوصی داده، امنیت و حاکمیت داده ها حتی بیشتر مهم می شوند. یک پارچه داده با سیاست های دسترسی پیشرفته این مشکلات را مواجه می کند، یک محیط داده ای امن تر را فراهم می کند. همچنین حاکمیت داده را با محدود کردن دسترسی به کاربران مجاز به احراز هویت افزایش می دهد، اطلاعات حساس را از تهدیدات احتمالی محافظت می کند. و برای شرکت های چند ملیتی که در جوریجکتان های متفاوت با قوانین حفظ حریم خصوصی داده متفاوت عمل می کنند، قابلیت های حاکمیت پیشرفته پارچه داده کمک می کند تا هر جا که داده ها قرار دارند مطابقت حفظ شود.

در نهایت، برخی از صنایع—مانند معاملات سهام و تجارت الکترونیکی—توسط شرایط بازار سریع تغییر کننده مشخص شده اند. در این دنیاهای ناپایدار، به روز رسانی در زمان واقعی فقط یک مزیت نیست بلکه یک ضرورت است. این فوری جایی است که یک پارچه داده بسیار ارزشمند است؛ پشتیبانی از پردازش داده در زمان واقعی آنها را با حفظ آگاهی، اطلاعات و آمادگی برای تطبیق در لحظه ای کمک می کند.

معایب احتمالی

در حالی که معماری پارچه داده مزایای بسیاری دارد، این معماری از چالش هایی نیز خالی نیست. انتقال از MDW به پارچه داده می تواند منابع محرک باشد، با مشکلات اولیه مربوط به هزینه، آموزش و یکپارچگی. علاوه بر این، همه کسب و کارها نیازی به قابلیت های پیشرفته پارچه داده ندارند—برای کسب و کارهای کوچک با منابع داده محدود و نیازهای پردازش ساده، MDW ممکن است کافی باشد.

علاوه بر این، پیچیدگی اصلی پارچه داده می تواند باعث شود که رفع مشکلات سخت تر باشد. بسیار حیاتی است که اطمینان حاصل کنید که سازمان شما دارای دانش لازم در داخل یا قابل دسترسی از طریق همکاران باشد قبل از اینکه انتقال ایجاد کنید.

در حالی که پارچه داده راه حل برجسته ای برای چالش های مدیریت داده های مدرن ارائه می دهد، برای هر کسب و کار ضروری است که نیازهای منحصر به فرد خود، بازده احتمالی سرمایه گذاری و اهداف استراتژیک بلند مدت را قبل از انتقال ارزیابی کند.

خلاصه

در این فصل، ما به مفهوم معماری پارچه داده پرداختیم، که یک تکامل پیشرفته از MDW است که به منظور افزایش دسترسی به داده، امنیت، قابلیت کشف و در دسترس بودن طراحی شده است. من پارچه داده را تعریف کردم و تعریف خود را با دیدگاه های صنعت مقایسه کردم، تفاوت هایی مانند نقش ویرایش داده را برجسته کردم. فصل هشت تکنولوژی کلیدی را که یک MDW را به قطعه پارچه داده منتقل می کند، بدون ایجاد یک مرز سخت بین دو مورد را شرح داد، با این حال که مرز سخت بین آنها را تعریف نمی کند.

همچنین مؤلفه های اصلی معماری پارچه داده را توضیح دادم، از جمله سیاست های دسترسی به داده، کاتالوگ متادیتا، مدیریت داده های اصلی، پردازش در زمان واقعی، API ها و المان های اختیاری مانند ویرایش داده. فصل تأکید بر قابلیت تطبیق پارچه داده می کند، به خصوص در مدیریت حجم بزرگ داده ها و نیازهای پردازش در زمان واقعی.

ما به دلایل انتقال از MDW به پارچه داده پرداختیم، همچنین مشکلات احتمالی مانند پیچیدگی و انتقال های منابع محرک. به طور کلی، هر سازمان باید با دقت نیازهای منحصر به فرد و توانایی های خود را قبل از پذیرش این راه حل برجسته ارزیابی کند.

فصل دوازده: دریاچه داده و ترکیبی انبار داده

من به طور مختصر در مورد داده لاک هاوس به عنوان یک هماهنگ کننده مفاهیم داده لاک و داده وارهاس صحبت کردم. ایده پشت داده لاک هاوس برای ساده سازی چیزها استفاده از فقط یک داده لاک برای ذخیره تمام داده های شما، به جای داشتن یک داده وارهاس رابطه ای جداگانه است. برای این کار، داده لاک نیاز به قابلیت های بیشتری برای جایگزینی ویژگی های RDW دارد. در اینجا نقش Databricks در Delta Lake ظاهر می شود.

Delta Lake یک لایه نرم افزاری ذخیره سازی تراکنشی است که بر روی یک داده لاک موجود اجرا می شود و ویژگی های شبیه به RDW را اضافه می کند که قابلیت اطمینان، امنیت و عملکرد لاک را بهبود می بخشد. Delta Lake خود یک ذخیره سازی نیست. در بسیاری از موارد، آسان است که یک داده لاک را به Delta Lake تبدیل کنید؛ همه چیزی که نیاز دارید این است که هنگام ذخیره سازی داده ها در داده لاک خود، مشخص کنید که می خواهید آنها را در فرمت Delta Lake ذخیره کنید (در مقابل سایر فرمت ها مانند CSV یا JSON).

پشت صحنه، هنگامی که یک فایل با استفاده از فرمت Delta Lake ذخیره می کنید، به روش خاص خود ذخیره می شود، که شامل فایل های Parquet در پوشه ها و یک لاگ تراکنش برای ردیابی تمام تغییرات ایجاد شده در داده ها است. در حالی که داده های واقعی در داده لاک شما در یک فرمت مشابه آنچه شما به آن عادت دارید قرار دارد، لاگ تراکنش اضافی آن را به یک Delta Lake تبدیل می کند، قابلیت های آن را افزایش می دهد. اما این بدان معنی است که هر چیزی که با Delta Lake تعامل دارد، باید از فرمت Delta Lake پشتیبانی کند؛ بیشتر محصولات این کار را انجام می دهند، زیرا به شدت محبوب شده است.

Delta Lake گزینه یکتا برای ارائه قابلیت های اضافی به یک داده لاک نیست؛ دو گزینه دیگر محبوب هستند که Apache Iceberg و Apache Hudi هستند که ویژگی های بسیار شبیه به هم دارند. با این حال، در این فصل من بر روی Delta Lake تمرکز خواهم کرد.

تـا کنـون، شـما در مـورد داده وارهـاس رابطـه ای (فصـل ٤)، داده لاک (فصـل ٥)، داده وارهـاس مـدرن (فصـل ١٠) و پارچـه داده (فصـل ١١) یـاد گرفتـه ایـد. ایـن فصـل داده لاک هـاوس را اضـافه می کنـد. شـکل ١٢-١ ایـن معمـاری هـا را در یک خط زمانی تاریخی نشان می دهد.

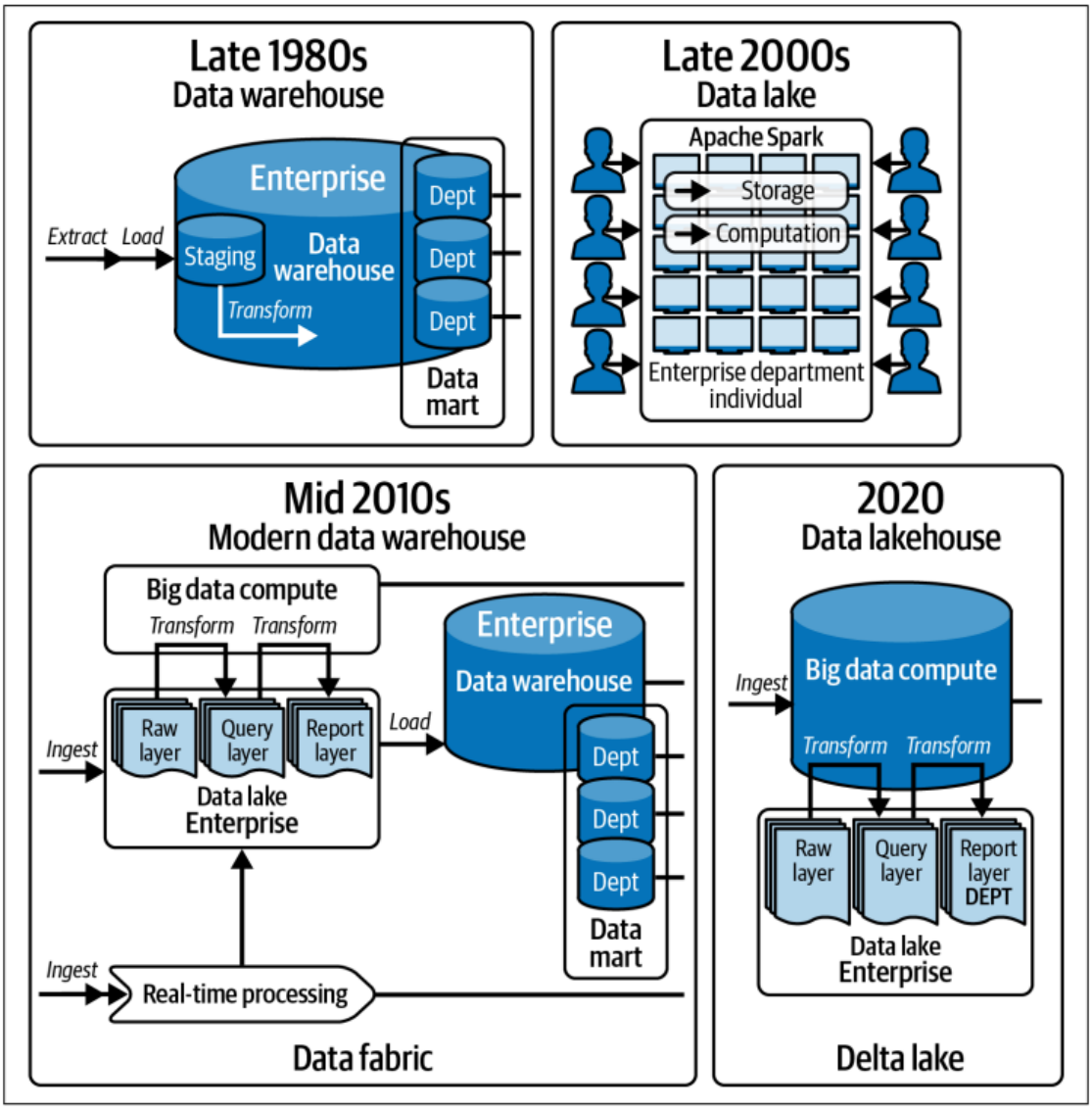


Figure 12-1. Historical timeline of data architectures

ویژگی های دریاچه دلتا

Lake Delta چنـدین ویـژگی شـبیه بـه RDW را بـه یـک داده لاک اضـافه می کنـد. ایـن بخـش شـما را از بـرخی از آنهـا هدایت می کند.

شـاید دلیـل بـزرگتری کـه شرکـت هـا از Lakes Delta اسـتفاده می کننـد، پشـتیبانی از دسـتورات زبـان داده مـدیریت (DML) ماننـد UPDATE، DELETE، INSERT و MERGE اسـت. ایـن دسـتورات وظـایف مـدیریت داده پیچیـده را سـاده می کننـد، کـه در نتیجـه کنـترل داده در Lake Delta را بیشـتر انعطـاف پـذیر و قابـل اعتمـاد می کنـد. داده لاک هـا از طریـق پشـتیبانی مسـتقیم بـرای ایـن عملیـات ارائـه نـمی دهنـد، زیـرا داده لاک هـا بـرای پـردازش دسـته ای و ذخـیره سـازی مقادیر زیادی از داده ها بهینه شده اند، نه برای به روزرسانی های زمان واقعی.

بـه روز رسـانی داده هـا در یـک داده لاک معمـولاً شـامل خوانـدن کـل فایـل، انجـام بـه روزرسـانی هـای لازم و نوشـتن کـل فایـل بـه روز شـده بـه داده لاک اسـت، کـه می توانـد زمـان زیـادی بگـیرد، بـه خصـوص بـرای فایـل هـای بـزرگ. در مقابـل،

وقتی Delta Lake با یک جدول کار می کند - در واقع یک فایل که یک مجموعه سازمان یافته از داده ها در سطرها و ستون ها است - آن را به چندین فایل دیجیتال کوچکتر برای مدیریت آسان تر شکسته می کند. نتیجه یک جدول Delta است. سپس Delta Lake از یک لاگ تراکنش برای ردیابی تغییرات استفاده می کند، که باعث می شود دستورات DML به سرعت بیشتری کار کنند به دلیل استفاده از ذخیره سازی بهینه شده (فرمت ذخیره سازی ستونی)، پردازش در حافظه، و بهینه سازی برای پردازش دسته ای. به عنوان مثال، با یک دستور Delta ،UPDATE Lake تمام فایل هایی را پیدا و انتخاب می کند که داده هایی را که با پیشوند مطابقت دارند و بنابراین باید به روز شوند، حاوی می شوند. سپس هر فایل مطابق را به حافظه می خواند، سطرهای مربوطه را به روز می کند و نتیجه را در فایل های داده جدید خروجی می دهد. این به طور کارآمد داده ها را به روز می کند، بدون اینکه نیاز به بازنویسی کل جدول Delta داشته باشیم. لاگ تراکنش تاریخچه ای از تغییرات در داده ها را حفظ می کند و اطمینان حاصل می کند که داده ها همیشه در یک حالت یکسان هستند، حتی در صورت شکست ها یا سیستم های خراب شده.

یک اختصار رایجی که هنگام کار با پایگاه های داده شنیده می شود وقتی با ACID می باشد، که برای atomicity، isolation ،consistency، و durability است. اینها چهار ویژگی هستند که اطمینان از قابلیت و یکپارچگی یک تراکنش پایگاه داده را تضمین می کنند. آنها تضمین می کنند که یک تراکنش می تواند به عنوان یک عملیات یکپارچه و قابل اعتماد با تغییرات داده خود کاملاً تأیید شود یا کاملاً بازگشت داده شود. در حالی که Delta Lake می تواند گفته شود که پشتیبانی از تراکنش های ACID را ارائه می دهد، این بیان نیاز به توضیح دارد. در مقابل پردازش تراکنش های ACID در یک پایگاه داده رابطه ای مانند SQL Server، پشتیبانی Delta Lake ACID محدود به یک جدول Delta است. اجرای DML بر روی بیش از یک جدول Delta در یک داده لاک Delta تضمین کردن یکپارچگی ACID را نمی دهد. با یک پایگاه داده رابطه ای، یکپارچگی ACID با تراکنش های DML کار می کند که در چندین جدول پوشش دارند.

Delta Lake همچنین امکان "سفر در زمان" را ارائه می دهد، یک ویژگی که به شما اجازه می دهد تا داده های ذخیره شده در جداول Delta را همانطور که در یک نقطه خاص در زمان وجود داشت، مورد سوال قرار دهید. تاریخچه ای از تغییرات در داده ها در لاگ تراکنش Delta حفظ می شود، همراه با متادیتا، مانند زمانی که هر تغییر ایجاد شده است و توسط کدام کاربر. کاربران می توانند نسخه های قبلی از داده ها را مشاهده و دسترسی پیدا کنند، و حتی در صورت نیاز آن را به یک نسخه قبلی بازگردانند. این می تواند برای بازرسی، اشکال زدایی، یا بازیابی داده در صورت تغییرات غیرمورد نظر داده یا سایر مشکلات (مانند بازگشت) مفید باشد.

مشکل "فایل های کوچک"، که در آن تعداد زیادی فایل کوچک می توانند عملیات خواندن و نوشتن را کند کند و هزینه های ذخیره سازی را افزایش دهد، یک مشکل رایج در داده لاک ها است.

Delta Lakes این مشکل را با استفاده از الگوریتم های فشرده سازی بهینه شده حل می کنند تا به طور کارآمد فایل های کوچک را به فایل های بزرگ ترکیب کنند. فرآیند فشرده سازی به صورت خودکار در پس زمینه انجام می شود و می تواند برای اجرا در یک برنامه زمانی یا به صورت دستی فعال شود.

با Delta Lake، کاربران می توانند همزمان پردازش دسته ای و جریان زمان واقعی را بر روی داده های یک جدول Delta انجام دهند، که نیازی به حفظ خطوط داده و سیستم های جداگانه برای پردازش دسته ای و جریان زمان واقعی نیست. این راه حل یکپارچه باعث می شود تا مدیریت و حفظ خطوط داده راحت تر شود و معماری پردازش داده را ساده می کند. این همچنین به این معنی است که Delta Lake پشتیبانی از معماری Lambda را پشتیبانی می کند (مشاهده کنید فصل ۷).

اجرای قوانین برای یک ویژگی جدول Delta است که به شما اجازه می دهد تا ساختار انتظار را برای داده ها در یک جدول Delta مشخص کنید و قوانینی مانند محدودیت های nullable، محدودیت های نوع داده، و محدودیت

های منحصر به فرد را اجرا کنید. این اطمینان می دهد که داده های نوشته شده به جدول Delta با ساختار مشخص شده مطابقت دارد، که به جلوگیری از خرابی داده کمک می کند.

بدون اجرای قوانین ساختار، یک فایل با ساختار نامعتبر می تواند به یک پوشه اضافه شود، که می تواند باعث شود کارهای ELT به خطا برخورد کنند. اگر داده های ورودی با ساختار مطابقت نداشته باشند، Delta Lake عملیات نوشتن را رد می کند و یک خطا می اندازد.

بهبود بازدهی

استفاده از Delta Lake می تواند عملکرد داده لاک را در چندین روش بهبود بخشد، از جمله:

اسکیپ داده

Delta Lake می تواند در خواندن از یک جدول Delta از داده های نامرتبط اسکیپ کند، که می تواند عملکرد پرس و جو را به طور قابل توجهی بهبود بخشد.

کش کردن داده

Delta Lake پشتیبانی از کش کردن داده در Spark را پشتیبانی می کند، که می تواند عملکرد پرس و جوهای تکراری را به طور قابل توجهی بهبود بخشد.

فهرست بندی سریع

Delta Lake از یک ساختار فهرست بندی بهینه شده استفاده می کند تا به سرعت داده ها را پیدا کند، کاهش داده های مورد نیاز برای اجرای پرس و جوها.

بهینه سازی پرس و جو

Delta Lake با SQL Spark ادغام می شود و می تواند از قابلیت های بهینه سازی پرس و جو آن استفاده کند، که منجر به پرس و جوهای سریع تر و کارآمد تر می شود.

انتقال پیشوند پیشوند

Delta Lake از انتقال پیشوند پشتیبانی می کند، که به این معنی است که شرایط فیلتر به لایه ذخیره سازی انتقال داده می شوند، کاهش مقدار داده هایی که نیاز به پردازش دارند.

برش ستون

با برش ستون، فقط ستون های مورد نیاز برای یک پرس و جو خاص خوانده می شوند، کاهش مقدار داده هایی که نیاز به پردازش دارند.

اجرای برداری شده

خلاصه - رمز گشایی از معماری داده

در اجـرای بـرداری شـده، چنـدین نقطـه داده در یـک دسـتور CPU تـکی پـردازش می شـوند، کـه منجـر بـه بهبـود عملکـرد می شود.

پردازش موازی

Delta Lake از پـردازش مـوازی پشـتیبانی می کنـد، کـه بـه ایـن معـنی اسـت کـه چنـدین وظیفـه می تواننـد بـه صـورت مـوازی اجرا شوند، که منجر به بهبود عملکرد می شود.

Z-order

Z-order، همچنـین بـه عنـوان ترتیـب Morton شـناخته شـده، یـک تکنیـک فهرسـت بنـدی داده اسـت کـه در معمـاری های Delta Lake برای سازماندهی داده برای دسترسی و پرس و جوی سریع و کارآمد استفاده می شود.

چگونگی معماری

در یـک داده لاک خانـه، داده هـا از طریـق همـان پـنج مرحلـه کـه در معمـاری هـای MDW و داده فابریـک مشـاهده کردیـد عبور می کنند، همانطور که شکل ۱۲-۲ نشان می دهد. این مراحل عبارتند از:

۱. **انتقـال (Ingestion)**: در ایـن مرحلـه، داده هـا از منـابع مختلـف جمـع آوری می شـوند و بـه داده لاک منتقـل می شوند.

۲. **ذخـیره سـازی (Storage)**: داده هـا در یـک مکـان مناسـب ذخـیره می شـوند، کـه می توانـد یـک داده لاک باشـد یـا سیستم های ذخیره سازی دیگری که برای این منظور طراحی شده اند.

۳. **تبـدیل (Transformation)**: در ایـن مرحلـه، داده هـا تبـدیل می شـوند تـا بـه فـرمتی مناسـب بـرای مـدل سـازی و تحلیل آماده شوند. این می تواند شامل پاک کردن داده ها، تبدیل نوع داده ها، یا اعمال تبدیلات دیگر باشد.

٤. **مـدل سـازی (Modeling)**: در ایـن مرحلـه، داده هـا بـرای تحلیـل و پـیش بیـنی اسـتفاده می شـوند. مـدل هـای مختلفی ماننـد مـدل هـای پـیش بیـنی، مـدل هـای تجزیـه و تحلیـل، و مـدل هـای یـادگیری ماشـین می تواننـد بـر روی داده هـا اعمال شوند.

۵. **تصویرسـازی (Visualization)**: در نهایـت، داده هـا بـرای تصویرسـازی و ارائـه بـه کـاربران نهـایی آمـاده می شـوند. ایـن می توانـد شـامل گـراف هـا، نمودارهـا، داشـبوردها، و رابـط کـاربری هـای تعـاملی باشـد کـه بـه کـاربران اجـازه می دهند تا داده ها را به طور مرتب و معنادار مشاهده و تحلیل کنند.

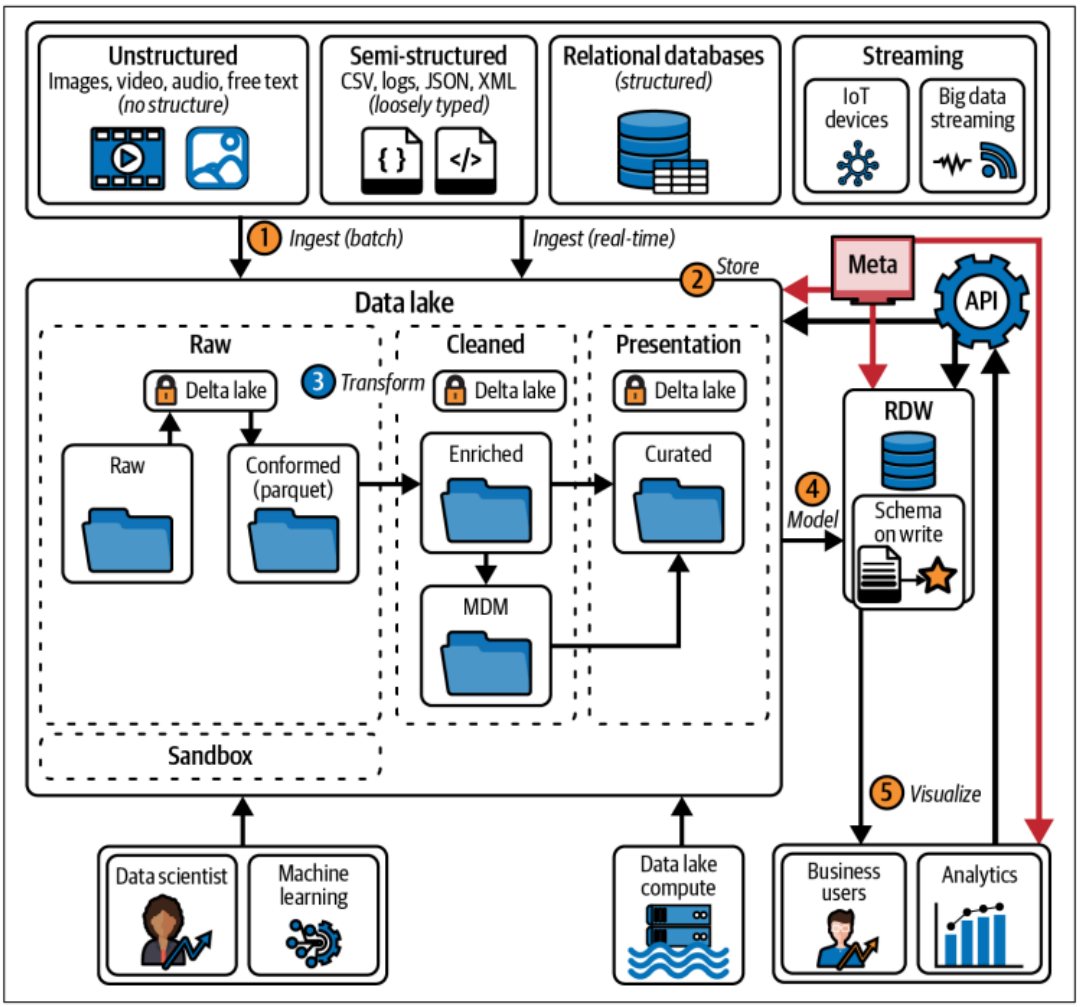


Figure 12-2. Data lakehouse architecture

به عنوان می بینید در شکل ۱۲-۲، با معماری داده لاک خانه، فقط یک مخزن برای داده های شما (داده لاکی که از Delta Lake استفاده می کند) وجود دارد، به جای دو (یک داده لاک و یک RDW). این شش مشکل رایج که در معماری های MDW و داده فابریک مشاهده می شود را حل می کند:

قابلیت اطمینان

حفظ یکپارچگی یک داده لاک و یک RDW می تواند مشکل باشد، به خصوص اگر مقادیر زیادی از داده ها به طور منظم نیاز به کپی از داده لاک به RDW داشته باشند. اگر کارهای کپی داده شکست خورد، داده ها به طور دقیق کپی نشده باشند، یا داده ها را در مکان نادرست در داده لاک قرار دهند، این می تواند مشکلات قابلیت اطمینان ایجاد کند. به عنوان مثال، اجرای گزارشات در برابر RDW ممکن است نتایج متفاوتی نسبت به اجرای گزارشات در برابر همان داده ها در داده لاک باشد. با یک داده لاک خانه، از آنجا که هیچ RDW برای کپی داده وجود ندارد، این مشکل وجود ندارد.

استناد داده ها

داده ها در RDW قدیمی تر از داده های معادل در داده لاک خواهند بود. سن بستگی به چقدر از داده ها از داده لاک به RDW کپی می شود—و برای جلوگیری از تأثیر بر عملکرد پرس و جو و گزارش، نمی خواهید این کارها را به طور منظم اجرا کنید. همانطور که با مشکلات قابلیت اطمینان بالا اشاره شد، این می تواند منجر به گزارشات در

برابـر RDW و داده لاک بـا نتـایج متفـاوت شـود. بـا یـک داده لاک خانـه، از آنجـا کـه هـیچ RDW بـرای کـپی داده وجـود ندارد، این مشکل وجود ندارد.

پشتیبانی محدود برای تحلیل های پیشرفته

تعـداد کـمی از سیسـتم هـا و ابزارهـای AI/ML بـه خـوبی بـر روی RDW هـا کـار می کننـد، زیـرا علمـای داده معمـولاً تـرجیح دارند که با فایل ها در یک داده لاک کار کنند. آنها این را با معماری داده لاک خانه دریافت می کنند.

هزینه کلی مالکیت

حـتی اگـر ذخـیره سـازی نسـبتاً ارزان باشـد، هزینـه هـای اضـافی بـرای محاسـبه مـورد نیـاز بـرای کـپی داده بـه یـک RDW وجـود دارد. همچنـین، نتیجـه دو نسـخه از داده هـا اسـت، کـه هزینـه هـای ذخـیره سـازی اضـافی ایجـاد می کنـد. در مقابـل، در یـک داده لاک خانـه، فقـط یـک نسـخه از داده هـا وجـود دارد. همچنـین، محاسـبات مـورد نیـاز بـرای پـرس و جوهـا و گـزارش در RDW معمـولاً بسـیار بیشـتر از محاسـبات مـورد نیـاز بـا یـک داده لاک هسـتند. عـلاوه بـر ایـن، مـدیریت یـک داده لاک و مـدیریت یـک RDW مجموعـه هـای مهـارت متفـاوتی هسـتند، و اسـتخدام بـرای هـر دو می توانـد هزینـه هـای اضـافی داشـته باشد.

حاکمیت داده

داشـتن دو نسـخه از داده هـا در دو محـیط ذخـیره سـازی مختلـف، احتمـالاً بـا دو نـوع مختلـف امنیـت، افـزایش می دهـد کـه کسی- داده هـایی را ببینـد کـه نبایـد ببینـد. همچنـین، تضـمین اینکـه هـر دو سیسـتم بـه همـان قـوانین بـرای کیفیـت داده هـا و تبـدیلات داده هـا پایبنـد باشـند، چـالش بـرانگیز اسـت. بـا یـک داده لاک خانـه و یـک نسـخه از داده هـا، ایـن مشـکلات وجود ندارد.

پیچیدگی

مـدیریت هـر دو یـک داده لاک و یـک RDW می توانـد پیچیـده باشـد و نیـاز بـه مهـارت هـای تخصصی- و منـابع خـاص دارد. از آنجا که یک داده لاک خانه دارای RDW نیست، تعداد کمتری از مهارت های تخصصی مورد نیاز است.

اگر از انبار داده های رابطه ای رد شوید چه؟

در اینجـا مـا یـک مسـئله پیچیـده را بـاز می کنـیم. نـه اخـیرا، مـن می گفـتم کـه اسـکیپ کـردن یـک RDW در معمـاری شـما یـک ایـده بسـیار بـدی اسـت. امـا حـالا کـه Lake Delta در حـال اضـافه کـردن ویـژگی هـای مشـابه RDW بـه داده لاک اسـت، مـن دیـده ام کـه مـوارد اسـتفاده بیشـتری وجـود دارد کـه داده لاک خانـه بهـترین معمـاری اسـت. مـورد اسـتفاده از یـک داده لاک خانـه بـه خصـوص بـا مجموعـه داده هـای کـوچکتر جـذاب اسـت. زیـرا بیشـتر فروشـندگان ابـری دارای محاسـبات بـدون سرور (serverless compute) هسـتند کـه می توانیـد بـر روی یـک Lake Delta اسـتفاده کنیـد، شـما می توانیـد هزینـه هـا را بـا پرداخـت بـرای هـر پـرس و جـو و نـه بـرای کـپی داده هـا در ذخـیره سـازی رابـط داده ای کـه گـران تـر و اسـتفاده از محاسـبات رابـط داده ای گـران قیمـت تـر اسـت، صرفـه جـویی کنیـد. همچنـین هزینـه هـا را بـا نداشـتن نیـاز بـه کـپی داده هـا در ذخـیره سـازی رابـط داده ای کـه گـران قیمـت تـر اسـت، و اسـتفاده از محاسـبات مخصـوص بـرای یـک RDW بـه جـای رفـتن بـه سرور بـدون کـپی، شـما هزینـه می کنیـد، حـتی اگـر از آن محاسـبات اسـتفاده نـمی کنیـد. همـانطور کـه بـا هـر معمـاری، معـاملات و نگـرانی هـایی وجـود دارد و بسـیار مهـم اسـت کـه اگـر تصـمیم بگیریـد کـه در معمـاری خـود یـک RDW نداشته باشید، آنها را بشناسید.

معاملـه اول ایـن اسـت کـه پـرس و جوهـای پایگـاه داده رابـطی سریـع تـر از پـرس و جوهـای در برابـر یـک Lake Delta هسـتند، بـه خصـوص زمـانی کـه RDW شـما از تکنولـوژی MPP اسـتفاده می کنـد (ببینیـد فصـل ۷). RDW هـا ویـژگی هـایی برای بهبود عملکرد پرس و جو دارند که در یک Delta Lake در دسترس نیستند، از جمله:

- فهرست بندی پیشرفته (مانند فهرست بندی های ستونی گروه بندی شده و فهرست بندی های متن کامل)

- آمار پیشرفته

- کش (مگر اینکه از Spark استفاده می کنید)

- بهینه سازی برنامه پرس و جو پیشرفته

- نماهای مواد موجود

- بهینه سازی اندوه پیشرفته

برخی از ویژگی های عملکرد Delta Lake، مانند Z-order، می توانند برخی از این ویژگی های گم شده را کاهش دهند.

Delta Lake نیز برخی از موارد اصلی امنیت مخزن داده‌های رابطه‌ای (RDW) را ندارد، مانند امنیت سطری، امنیت ستونی، رمزنگاری داده‌های در حال استراحت (data-at-rest encryption)، رمزنگاری ستونی، رمزنگاری داده‌های شفاف (transparent data encryption یا TDE) و ماسک داده‌های پویا (dynamic data masking که بخش‌هایی از داده‌ها را به طور خودکار جایگزین یا ماسک می‌کند تا کاربران غیرمجاز نسخه‌ای ماسک شده از داده‌ها را ببینند به جای اطلاعات حساس واقعی). همچنین ارائه نماهای SQL، یکپارچگی مرجع، مدیریت بار کاری یا ویژگی‌های پیشرفته برای نظارت و مطابقت مانند سوابق نظارت، سیاست‌های حفظ داده‌ها و مجوزهای مطابقت را نمی‌دهد. مخازن داده‌های رابطه‌ای نیز پشتیبانی برای همزمانی بیشتری نسبت به Delta Lake ارائه می‌دهند، زیرا ویژگی‌های پیشرفته‌ای مانند قفل‌سازی پیشرفته، سطوح جداسازی و مدیریت تراکنش را فراهم می‌کنند.

پیچیدگی نیز مسئله‌ای است. با یک مخزن داده‌های رابطه‌ای، شما یک لایه فراوانیت اجباری دارید - شما باید یک پایگاه داده، یک طرح و یک جدول با فیلدهایی که نوع داده را توصیف می‌کنند ایجاد کنید و سپس داده‌ها را بارگذاری کنید (schema-on-write). این بدین معنی است که شما همیشه فراوانیت روی داده‌های واقعی قرار دارد. این نیاز به کار اولیه دارد، اما مزیت بزرگ آن این است که فراوانیت و داده‌ها همیشه با هم قفل شده‌اند، بنابراین آسان است که از فراوانیت برای پیدا کردن داده‌ها استفاده کنید. شما هرگز "فراوانیت را از دست نمی‌دهید" و همیشه به طور دقیق داده‌ها را توصیف می‌کند. این بسیار متفاوت از Delta Lake است که در یک جهان پوشه و فایل است. این دلیل اصلی است که کاربران نهایی که به یک مخزن داده‌های رابطه‌ای عادت دارند، اغلب با مشکل مواجه می‌شوند.

در Delta Lake، فراوانیت لازم نیست که با داده‌ها وجود داشته باشد. شما ممکن است آن را در یک یا چند فایل جداگانه، درون فایلی که داده‌ها را حاوی می‌شود، در همان پوشه، در یک پوشه جداگانه، یا در همه چیز نداشته باشید. برای اینکه به بالا برسیم، فراوانیت ممکن است بسیار نادرست باشد، زیرا نسبت یک به یک با داده‌ها که در یک مخزن داده‌های رابطه‌ای دارد، را ندارد. می‌توانید ببینید که چقدر این برای کاربران نهایی می‌تواند گیج کننده باشد، که ممکن است با پیدا کردن فراوانیت مبارزه کنند، به طور اشتباه تصمیم بگیرند که فراوانیتی وجود ندارد، یا حتی از فراوانیت اشتباه استفاده کنند. در نهایت، فراوانیت ممکن است با داده‌ها ناهماهنگ شود وقتی تغییراتی خارج از Delta Lake ایجاد می‌شود.

استفاده از برخی از ویژگی‌های معماری Delta Lake ممکن است شما را مجبور کند تا از Spark استفاده کنید. همچنین، اگر از یک محصول که از نسخه‌ای SQL غیر از Spark SQL استفاده می‌کند مهاجرت کنید، آماده باشید تا روندهای ذخیره شده خود، نماها، گزارشات، داشبوردها و غیره را بازنویسی کنید. اگر نیاز به استفاده از Spark SQL دارید، این ممکن است به معنی آموزش مجدد کاربران نهایی باشد که قبلاً با تعامل با مخازن داده‌های رابطه‌ای آشنا بوده‌اند و با ابزارهای موجود برای این کار آشنا هستند. آن‌ها ممکن است از SQL مطابق با ANSI استفاده می‌کنند یا یک محصول مخزن داده‌های رابطه‌ای مانند T-SQL را که با محصولات مایکروسافت استفاده می‌شود، استفاده می‌کنند. آن‌ها همچنین با مدل رابطه‌ای آشنا هستند و می‌دانند چگونه به سرعت از آن پرس و جو و گزارش بگیرند. تغییر به یک جهان پوشه-فایل احتمالاً باعث می‌شود آن‌ها برای یادگیری استفاده از ابزارهای جدید، آشنایی با schema-on-read و یادگیری چگونگی مدیریت مشکلات با سرعت، امنیت، ویژگی‌های گم شده، پیچیدگی و تغییرات SQL توصیف شده در بالا باشند. این مقدار زیادی آموزش است.

فناوری‌های موجود می‌توانند برخی از این نگرانی‌ها را کاهش دهند و در نهایت ممکن است آن‌ها را نامعنی کنند. اما تا زمانی که این اتفاق بیفتد، به آن‌ها با دقت در ارتباط با نیازهای خود برای تصمیم‌گیری درباره استفاده از معماری مخزن داده‌های خانه‌ای در نظر بگیرید. برای مثال، اگر پرس و جوها به داده‌ها در معماری مخزن داده‌های خانه‌ای میانگین پنج ثانیه طول می‌کشند، آیا این مشکلی است؟ اگر کاربران نهایی با این موضوع موافق هستند، می‌توانید این

نگرانی را نادیده بگیرید، اما اگر آن‌ها از داشبوردها استفاده می‌کنند و نیاز به زمان پاسخ پرس و جو در میلی‌ثانیه دارند، باید داده‌های استفاده شده برای داشبوردها را به یک مخزن داده‌های رابطه‌ای (RDW) کپی کنید (به این معنی که در نهایت باید از یک معماری MDW استفاده کنید). شما می‌توانید به جای آن از یک محصول گزارشگیری استفاده کنید که به شما اجازه می‌دهد داده‌ها را به حافظه خود وارد کنید تا به زمان پاسخ پرس و جو در میلی‌ثانیه داشته باشید، اما داده‌ها به اندازه کافی به‌روز نخواهد بود و شما باید داده‌ها را در حافظه ابزار گزارشگیری در بازه‌های خاص تازه کنید.

دوباره در مورد معاملات صحبت می‌کنیم. اگر شما یک معمار هستید، بخش عمده‌ای از نقش شما شناسایی محصولاتی است که می‌توانند در هر معماری استفاده شوند و تعیین و تحلیل معاملات آن‌ها را برای انتخاب بهترین معماری و محصولات برای مورد استفاده خاص خود انجام دهید.

هیچ یک از نگرانی‌های فوق به تنهایی نمی‌تواند یک ماهیت متوقف کننده باشد، اما در ترکیب با هم، می‌توانند دلیل کافی برای استفاده از یک مخزن داده‌های رابطه‌ای فراهم کنند. بسیاری از شرکت‌ها با برخی اثبات مفهومی شروع می‌کنند تا تصمیم بگیرند که آیا هر یک از معاملات پرکردن مخزن داده‌های رابطه‌ای مشکلی ایجاد می‌کند یا خیر. اگر نه، آن‌ها با یک مخزن داده‌های خانه‌ای ادامه می‌دهند.

لایه سرویس دهی رابطه ای

به دلیل اینکه Lake Delta بر اساس schema-on-read عمل می‌کند (مشاهده کنید در فصل ۲)، schema به داده‌ها زمانی اعمال می‌شود که خوانده می‌شوند، نه قبل از آن. Lake Delta یک سیستم فایل-پوشه است، بنابراین زمینه‌ای برای توصیف داده‌ها فراهم نمی‌کند. (مقایسه کنید با لایه نمایش فراوانیت RDW که روی و به طور مستقیم به داده‌ها مرتبط است.) روابط تعریف شده در Lake Delta وجود ندارند. هر فایل در یک جزیره جداگانه است، بنابراین برای ایجاد یک "لایه خدمت‌رسانی" روی داده‌ها در Lake Delta نیاز دارید تا فراوانیت را به طور مستقیم به داده‌ها مرتبط کنید. برای کمک به کاربران نهایی در درک داده‌ها، احتمالاً می‌خواهید آن‌ها را در یک مدل داده‌ای رابطه‌ای ارائه دهید، بنابراین آنچه که ساخته‌اید یک "لایه خدمت‌رسانی رابطه‌ای" است.

با این لایه روی داده‌ها، اگر نیاز به ادغام بیش از یک فایل دارید، می‌توانید روابط بین آن‌ها را تعریف کنید. لایه خدمت‌رسانی رابطه‌ای می‌تواند به صورت‌های مختلفی باشد: یک نماهای SQL، یک مجموعه داده در یک ابزار گزارشگیری، یک جدول Hive Apache، یا در یک پرس و جو SQL تصادفی. اگر به درستی انجام شود، کاربر نهایی فکر نخواهد کرد که واقعاً داده‌ها را از یک Delta Lake بکش می‌کند - او فکر می‌کند که از RDW است.

بسیاری از شرکت‌ها نماهای SQL روی فایل‌ها در Lake Delta ایجاد می‌کنند، سپس از یک ابزار گزارشگیری برای فراخوانی این نماها استفاده می‌کنند. این برای کاربران نهایی آسان می‌کند تا گزارشات و داشبوردها را ایجاد کنند.

حتی با یک لایه خدمت‌رسانی رابطه‌ای، Lake Delta هنوز چالش‌هایی را ارائه می‌دهد. لایه خدمت‌رسانی رابطه‌ای ممکن است داده‌ها را به طور نادرست نمایش دهد، برای مثال، یا شما ممکن است دو لایه داشته باشید که به داده‌های یکسان اشاره می‌کنند اما فراوانیت متفاوتی دارند. فراوانیت به داده‌ها مرتبط نبودن یک مشکل بنیادی با Lake Delta است. RDW این مشکل را با داشتن یک مدل داده جهانی که همه می‌توانند از آن استفاده کنند، اجتناب می‌کند.

خلاصه

این فصل مفهوم مخزن داده‌های خانه‌ای را بررسی کرد، با تمرکز بر نقش Lake Delta به عنوان یک لایه ذخیره‌سازی تراکنشی- که قابلیت اطمینان، امنیت و عملکرد مخازن داده‌های موجود را به طور قابل توجهی افزایش می‌دهد.

شما درباره مشکلات احتمالی مرتبط با اجتناب از یک RDW سنتی یاد گرفتید، با تأکید بر چالش‌های مرتبط با سرعت، امنیت و همزمانی. فقط با آگاهی از معاملات، و اگر نگرانی‌های بزرگ فوری وجود نداشته باشد، از یک مخزن داده‌های خانه‌ای استفاده کنید تا زمانی که نتوانید. اگر یکی از این معاملات برای یک مجموعه داده خاص بسیار زیاد برای غلبه بر آن بود، می‌توانید آن داده را به یک RDW کپی کنید (نیازی به کپی کردن تمام داده‌ها نیست). با پیشرفت فناوری برای Lake Delta، برای فناوری‌های مشابه مانند Iceberg Apache و Hudi Apache، و برای ذخیره‌سازی و محاسبات، تعداد کمتر و کمتر دلیل برای حفظ یک RDW خواهد بود. بیشتر معماری‌های داده جدید مخزن داده‌های خانه‌ای خواهند بود.

شما رویکرد منحصر به فرد schema-on-read در Lake Delta را مشاهده کردید، که در آن تفسیر داده‌ها در نقطه خواندن اتفاق می‌افتد، نه قبل از آن. شما همچنین درباره ساختار فایل-پوشه آن یاد گرفتید، که بدون زمینه است و بنابراین به طور قابل توجهی از لایه نمایش فراوانیت ساختاری RDW متفاوت است. این نیازمند یک "لایه سرور رابطه‌ای" برای برقراری یک لینک مستقیم مبتنی بر زمینه به داده‌ها در Delta Lake است.

پیشرفت‌های فناوری سریع ادامه دارد و استراتژی‌های معماری داده را تحت تأثیر قرار می‌دهد. چند سال پیش، اجتناب از یک RDW در معماری شما به عنوان یک خطای قابل توجه در نظر گرفته می‌شد، اما روندهای کنونی نشان می‌دهند که تعداد افزایش یافته است که موارد استفاده در آن‌ها مخزن داده‌های خانه‌ای به عنوان معماری بهینه ظاهر می‌شود.

تمام معماری‌هایی که تا کنون بحث کردم، راه‌حل‌های متمرکز هستند، به این معنی که داده‌های منبع به یک مکان مرکزی که توسط IT متعلق است کپی می‌شوند. شما در فصل بعدی تفاوت عمده‌ای خواهید دید، زیرا ما در مورد راه‌حل غیرمتمرکز مشبک داده صحبت می‌کنیم.

فصل سیزدهم: مبانی دیتا مش

یک شبکه داده یک معماری داده‌ای غیرمتمرکز با چهار ویژگی خاص است. اول، آن نیاز به تیم‌های مستقل در دامنه‌های مشخص شده برای داشتن داده‌های تحلیلی خود دارد. دوم، در یک شبکه داده، داده به عنوان یک محصول در نظر گرفته می‌شود تا به مصرف کننده داده کمک کند تا آن را کشف، اعتماد به آن بیاورد و برای هر منظوری که می‌خواهد از آن استفاده کند. سوم، بر پایه تهیه کردن زیرساخت خودکار است. و چهارم، از حکومت برای اطمینان از اینکه تمام محصولات داده مستقل امن هستند و قوانین جهانی را رعایت می‌کنند، استفاده می‌کند. اگرچه مفاهیمی که یک شبکه داده را تشکیل می‌دهند جدید نیستند، اما زحمک دهقانی، مدیر عامل و بنیانگذار Nextdata، برای اختراع اصطلاح شبکه داده و ترکیب این مفاهیم اعتبار دارد. اگرچه دیگران نظرات مختلفی در مورد آنچه یک شبکه داده است داشته‌اند، اما این کتاب تعریف خود را بر اساس کار دهقانی است، به ویژه چهار اصل شبکه داده اش که در این فصل من در مورد آن صحبت می‌کنم. بسیار مهم است که درک کنید که شبکه داده یک مفهوم است، نه یک فناوری. همه چیز در مورد تغییر سازمانی و فرهنگی در شرکت‌ها است. فناوری استفاده شده برای ساخت یک شبکه داده می‌تواند به معماری انبار داده مدرن، فابریک داده یا انبار داده لاک جدید بپیوندد - یا حتی دامنه‌ها می‌توانند به معماری‌های مختلف بپیوندند.

همانطور که در این فصل و فصل بعدی به دنیای پیچیده معماری‌های شبکه داده وارد می‌شویم و به تفصیل به آن نگاه می‌کنیم، یک نقشه راه را برای پیمایش در پایداری پیچیده شبکه داده ارائه می‌دهم که بر اساس اصل غیرمتمرکز است. در این فصل، معماری داده‌ای غیرمتمرکز را برش می‌دهم و میزان حرکت در مورد شبکه داده را توضیح می‌دهم و چهار اصل پایه‌ای آن را توضیح می‌دهم. ما به عمق به جوهر شبکه داده خالص و سفر را در پیچیدگی‌های دامنه‌های داده انجام می‌دهیم، سپس معماری منطقی و توپولوژی‌های متنوعی که زیرساخت را پشتیبانی می‌کنند را کشف می‌کنیم و میان شبکه داده و فابریک داده مقایسه می‌کنیم. در نهایت، من استفاده مناسب شبکه داده را بحث می‌کنم.

در فصل ۱۴، تمرکز خود را به چالش‌های پتانسیلی در پیاده‌سازی یک شبکه داده منتقل می‌کنم، متوجه شدم که برخی از این میت‌ها را برطرف کرده‌ام تا به سازمان‌ها کمک کنم تا با اطلاعات لازم تصمیم‌گیری منطقی در مورد پذیرش آن انجام دهند. سپس من در مورد انجام یک ارزیابی سازمانی برای تعیین اینکه آیا پذیرش یک شبکه داده با نیازهای سازمان شما همخوانی دارد صحبت خواهم کرد. از آنجا که به این نقطه رسیدیم، به توصیه‌هایی برای پیاده‌سازی یک شبکه داده موفق می‌پردازم، تضمین یک انتقال بی‌درز و بهره‌های حداکثری. سپس به آینده نگاه خواهم کرد و مسیر پیش‌بینی شده شبکه داده در چشم‌انداز داده‌هایی که همیشه در حال تکامل هستند را مشاهده خواهم کرد و با یک بحث در مورد زمان مناسب برای استفاده از هر یک از چهار معماری: انبار داده مدرن، فابریک داده، انبار داده لاک و شبکه داده ختم خواهم کرد.

به من بپیوندید که پیچیدگی‌های معماری شبکه داده را برطرف کنم، به شما بینش عمیق اما مختصری در مورد چارچوب غیرمتمرکز آن ارائه می‌دهم.

یک معماری غیر متمرکز

معماری‌های داده‌ای که در این کتاب تا به حال بحث کردم - انبار داده مدرن، فابریک داده و انبار داده لاک - همه معماری‌های متمرکز هستند که در آن‌ها تمام داده‌های تحلیلی توسط تیم IT ایجاد و مالکیت دارد. تفاوت بزرگی بین

شبکه داده و این معماری‌های داده دیگر این است که شبکه داده غیرمتمرکز است. این بدین معنی است که داده، همراه با همه چیزی که برای مدیریت آن استفاده می‌شود، توسط تیم‌های فردی یا "دامنه‌ها" در یک سازمان مالکیت و مدیریت دارد و بر اساس دامنه تجاری گروه‌بندی می‌شود، در مقابل یک نیروی مرکزی یا تیم تک در IT. این تیم‌های دامنه مسئول جمع‌آوری، پردازش و مدیریت داده‌های خود هستند و اتونومی برای تصمیم‌گیری در مورد چگونگی استفاده و به اشتراک‌گذاری داده‌های خود را دارند. داده در دامنه نگهداری می‌شود؛ کاربران نهایی می‌توانند داده را در مکانی که قرار دارد بازیابی و پرس و جو کنند، بدون نیاز به کپی آن به مکان مرکزی. این منجر به پایداری بیشتر و اطمینان می‌آورد که داده توسط کسانی که بیشترین تخصص در مورد آن دارند مدیریت و نگهداری شود.

این مشکل اصلی معماری‌های متمرکز را حل می‌کند، که این است که مسئولیت سازمانی و فناوری. همانطور که یک سازمان رشد می‌کند و بیشتر داده باید به سرعت متمرکز شود، اغلب IT بار می‌شود و یک گلوگاه می‌شود.

وقتی در مورد تمام معماری‌های داده دیگر در این کتاب به معماری‌های متمرکز اشاره می‌کنم، من به مجموعه گسترده‌ای از جنبه‌های این معماری‌ها اشاره می‌کنم. نه تنها تیم IT مرکزی مالکیت و کنترل دارد که تیم، بلکه مسئول ادغام داده‌ها از منابع مختلف، نگهداری داده در مکان متمرکزی که آن را مالکیت دارد، و ساخت مدل‌های داده و نگهداری آن‌ها در یک مخزن مرکزی است.

- ادغام داده‌ها از منابع مختلف
- نگهداری داده در مکان متمرکزی که آن را مالکیت دارد
- ساخت مدل‌های داده و نگهداری آن‌ها در یک مخزن مرکزی
- حکومت و اطمینان به عنوان تعریف و اجرای استانداردهای کیفیت داده، امنیت، و حریم خصوصی
- ایجاد و نگهداری خطوط داده و منطق تبدیل
- اجرای کیفیت داده از طریق پروفایلینگ، اعتبارسنجی، تمیز کردن، و فرآیندهای دیگر
- بهینه‌سازی عملکرد
- مدیریت تکنولوژی محاسباتی
- مدیریت ابزارهای تحلیل و گزارش
- نگهداری و مدیریت متادیتا
- بازیابی از حوادث و پشتیبان‌گیری

علاوه بر این، در یک معماری متمرکز، سخت‌افزار و قدرت پردازشی برای مقیاس‌بندی عمودی طراحی شده‌اند.

دیتامش هایپ

شبکه داده از زمانی که برای اولین بار در سال ۲۰۱۹ معرفی شد، بسیاری از توجه را به خود جلب کرده است. Gartner یک "چرخه انتظار" معروف برای مدیریت داده دارد که میزان بلوغ فناوری‌ها را بر اساس سطح فعلی پذیرش و تعداد سال‌ها تا پذیرش میان‌انبوه اندازه‌گیری می‌کند. در چرخه انتظار مدیریت داده ۲۰۲۳ خود، شبکه داده را نزدیک به بالای چرخه انتظار (نزدیک به "بالای انتظارات منحرف") قرار داده و "متوسط" را در رتبه بنفیت قرار داده است. شبکه داده دارای تراکم بازار ۵ تا ۲۰ درصد است و به طور جالب، Gartner پیش‌بینی می‌کند که "شبکه داده قبل از نقطه استوار منقضی‌ـ خواهد شد. روش و فناوری پشتیبانی به سمت فابریک داده حرکت می‌کند زیرا سازمان‌ها شروع به جمع‌آوری متادیتا بی‌فعال می‌کنند."

بسیاری که شبکه داده را تحسین می‌کنند، ادعا می‌کنند که پروژه‌های انبار داده با نرخ بسیار بالایی شکست می‌خورند، قابل پایداری نیستند و دیگر نمی‌توانند با داده‌های بزرگ مقابله کنند زیرا نمی‌توانند مقیاس بندی کنند. با تمام این تحسین، شما ممکن است فکر کنید که ساخت یک شبکه داده پاسخ به تمام این "مشکلات" با انبار داده است. حقیقت این است که در حالی که پروژه‌های انبار داده شکست می‌خورند، این به دلیل مشکلات با افراد و/یا فرآیند

است و یا اینکه سازمان فناوری کاملاً اشتباه انتخاب کرده است. گاهی اوقات احساس می‌کنم که شبکه داده یک راه حل است که به دنبال یک مشکل است و افرادی که شبکه داده را تحسین می‌کنند، تمام چالش مدیریت و یکپارچگی کردن داده را به سادگی کاهش می‌دهند.

من دیدم که راه‌حل‌های بزرگ داده برای سال‌ها بدون شبکه داده بسیار خوب کار می‌کنند، از دهه ۱۹۸۰ تا امروز که یک ترابایت به عنوان "داده بزرگ" در نظر گرفته می‌شد تا پتابایت‌های داده امروز. فناوری همیشه پیشرفت نکرده است. معماری‌های داده متمرکز و محصولات ابری بسیار قادر به مدیریت تقریباً هر مقدار داده دسته‌ای و جریانی هستند.

این راه‌حل‌ها برای داده بزرگ سال‌ها کار کرده‌اند. چرا باید فرض کنیم که آن‌ها در آینده نمی‌کنند؟

البته، این همه تخمین است. شبکه داده هنوز جدید است، اما به طور واضح همه نمی‌بایست به تحسین آن اعتماد کنند. من در مخالفان تحسین قرار دارم. من بیش از ۳۵ سال با پایگاه‌های داده کار کرده‌ام و برای ۱۵ سال با انبار داده کار کرده‌ام و تعداد زیادی از فناوری‌های تحسین شده را می‌بینم که آمده و رفته‌اند. در حالی که من فکر نمی‌کنم شبکه داده منقضی‌ـ شود، من فکر نمی‌کنم هیچ شرکتی یک "شبکه داده خالص" بسازد. تعداد کمی از شرکت‌ها یک شبکه داده تغییر یافته را بسازند و بقیه از یک معماری دیگر استفاده خواهند کرد.

من به شرکت‌های بسیار کمی که یک شبکه داده تغییر یافته را به تولید می‌برند مشاهده کردم. بسیاری ادعا می‌کنند که آن‌ها را دارند، به دنبال به نظر برسند کاتلی، اما بیشتر از زمان واقعاً یک فابریک داده یا انبار داده لاک است. اما حتی با تفسیر بسیار آزادی "شبکه داده"، درصد شرکت‌هایی که یکی را ساخته‌اند بسیار کم است. وقتی در مورد معماری داده صحبت می‌کنم، همیشه مخاطبان را می‌پرسم، "چه تعدادی از شما شنیده‌اند از شبکه داده؟" معمولاً حداقل ۷۵ درصد دست خود را برازش می‌کنند. سپس با می‌گویم، "چه تعدادی در حال ساخت یک شبکه داده هستند؟"

ممکن است دست یا دو در یک مخاطبان بزرگ برازش شوند. سپس می‌پرسم، "چه تعدادی یک شبکه داده را در تولید دارند؟" تا به حال دست یک در مخاطبان برازش نشده است. همکاران دیگر تجربیات مشابهی گزارش می‌کنند. در حالی که این تجربیات شخصی‌ـ است، به نظر می‌رسد که نشان می‌دهد شبکه داده با تحسین مقابله نمی‌کند. با این حال، معماری‌های شبکه داده جنبه‌های مثبت زیادی دارند که می‌توانند هر یک از معماری‌های داده دیگر را بهبود بخشند.این فصل و فصل بعدی به طور دقیق‌تر توضیح خواهند داد.

چهار اصل دهقانی دیتا مش

شبکه داده سعی می‌کند چهار چالش بزرگتر معماری‌های داده متمرکز را حل کند: کمبود مالکیت، کیفیت پایین داده، مقیاس‌بندی فنی و مقیاس‌بندی سازمانی. چهار اصل که دهغانی در آن‌ها توضیح می‌دهد، این چالش‌ها را مواجه می‌کنند. بیایید هر کدام را به ترتیب بررسی کنیم.

اصل #۱: مالکیت دامنه

اصل #۱ توصیه می‌کند که مسئولیت را به افرادی که نزدیک‌ترین به داده‌ها هستند توزیع کنید تا به تغییرات مداوم و قابلیت مقیاس‌بندی پشتیبانی کنید (به عنوان مثال، تولید، فروش، تامین‌کننده).

در یک معماری متمرکز، معمولاً در مورد کسی‌ـ که "مالک" داده‌های تحلیلی است ابهام وجود دارد (یا در برخی از جلسات، اختلافات شدید).

بیشتر داده‌های منبع توسط سیستم‌های عملیاتی خانگی، ابزارهای CRM مانند Salesforce و Microsoft Dynamics، و ابزارهای برنامه‌ریزی منابع سازمانی (ERP) مانند SAP و Dynamics Microsoft تولید می‌شوند. هر یک از این سیستم‌ها و برنامه‌های مخصوص دامنه و کاربرد توسط افراد عملیاتی متخصص اجرا می‌شوند که معمولاً با

تـیم پلتفـرم داده IT مرکـزی ارتبـاط کـمی یـا هـیچ ارتبـاطی ندارنـد. اغلـب، افـراد عملیـاتی آگـاه نیسـتند کـه "داده‌هـای آن‌هـا" به یک پلتفرم داده ارسال می‌شود.

وقـتی داده در یـک پایگـاه داده عملیـاتی قـرار دارد، آن بـه دامنـه عملیـاتی کـه پایگـاه داده را مالکیـت دارد تعلـق دارد. وقـتی کـپی شـده بـه مخـزن داده متمرکـز (معمـولاً یـک دریاچـه داده) منتقـل می‌شـود، معمـولاً IT آن را مالکیـت می‌گـیرد—امـا برخی معتقدند که هنوز باید به دامنه تعلق داشته باشد.

از سـوی خـود، تـیم پلتفـرم داده IT مرکـزی بـه نـدرت درک قـوی از چگـونگی کـارکرد سیسـتم‌های عملیـاتی یـا برنامـه‌های کسـب و کـار دارد. آن‌هـا فقـط داده‌هـا را خـوب فهمیـده ندارنـد و زمینـه‌ای بـرای تولیـد یـا معنـای واقـعی آن‌هـا ندارنـد. آن‌هـا فقـط نـام جـدول و فیلـد یـا فایل‌هـا را می‌بیننـد. در نتیجـه، آن‌هـا در تولیـد داده بـا کیفیـت و تحلیل‌هـای قابـل اسـتفاده مبارزه می‌کنند.

ایـن چـالش توسـط اولـین اصـل شـبکه داده حـل می‌شـود - مالکیـت دامنـه. افـرادی کـه در بـر پایگاه‌هـای داده عملیـاتی مسئولیت دارند، همیشه مالک داده‌های تحلیلی هستند و آن‌ها مسئول مدیریت و نگهداری داده هستند.

مسـئولیت و مالکیـت بـه صـورت متمرکـز و توزیـع شـده‌اند بـه افـرادی کـه نزدیـک‌ترین بـه داده‌هـا هسـتند، تـا بـه تغییـرات مـداوم و قابلیـت مقیاس‌بنـدی پشـتیبانی کننـد. ایـن افـراد بـر اسـاس دامنـه کسـب و کـار گروه‌بنـدی می‌شـوند، ماننـد تولیـد، فـروش یـا تامین‌کننـده؛ تعریـف دامنـه‌های شرکـت می‌توانـد چـالش باشـد. هـر دامنـه داده‌هـای عملیـاتی و تحلـیلی خـود را مالکیت دارد.

اصل #۲: داده به عنوان یک محصول

اصـل #۲ پیشـنهاد می‌کنـد کـه داده‌هـای تحلـیلی ارائـه شـده توسـط دامنـه‌ها را بـه عنـوان یـک محصـول در نظـر بگـیریم و مصـرف‌کنندگان آن داده را بـه عنـوان مشـتریان در نظـر بگـیریم. هـر دامنـه تیم‌هـای دامنـه، کـد API، داده و متادیتـا، و زیرساخت را دارد.

در یـک معمـاری متمرکـز، تـیم IT مالـک داده‌هـای تحلـیلی اسـت و بنـابراین مسـئول کیفیـت داده اسـت - امـا آن‌هـا معمـولاً داده را خـوب نمی‌فهمنـد و ممکـن اسـت در تبـدیل آن اشـتباه کننـد. از سـوی دیگـر، تـیم دامنـه دارای درک عمـیقی از داده و زمینـه آن درون دامنـه‌های کسـب و کـار خـود اسـت. ایـن دانـش بـه آن‌هـا اجـازه می‌دهـد تـا مشـکلات کیفیـت داده را شناسـایی و مقابلـه کننـد کـه ممکـن اسـت توسـط تیم‌هـای IT مرکـزی کـه کمـتر زمینـه دارنـد بـرای درک نیازهـای کسـب و کـار کمک کنند، نادیده گرفته شود.

اصـل دوم شـبکه داده ایـن اسـت کـه بـه جـای فکـر کـردن بـه داده‌هـای تحلـیلی بـه عنـوان ورودی، یـک دارایی، یـا یـک بارنـده فرآینـدی کـه دیگـران مـدیریت می‌کننـد، مالکـان داده داده را بـه عنـوان یـک محصـول کـاملاً محافظـت شـده در نظـر می‌گیرنـد کـه آن‌هـا مسـئول آن هسـتند. آن‌هـا مصـرف‌کنندگان آن داده (کـاربران داده، تحلیلگـران داده، دانشـمندان داده) را بـه عنـوان مشـتریان در نظـر می‌گیرنـد. ایـن بـدین معناسـت کـه تیم‌هـای دامنـه بـه داده‌هـای خـود نگـاهی بـه محصـول می‌اندازنـد تـا مطمـئن شـوند کـه آن‌هـا قابـل کشـف هسـتند و اطمینـان حاصـل کننـد کـه آن‌هـا قابـل اعتمـاد، قابـل دسـترسی، امن و قابل فهم هستند.

اولـین قـدم شـما در در نظـر گـرفتن داده ماننـد یـک محصـول ایـن اسـت کـه دقیقـاً مشـخص کنیـد کـه چـه چـیزی یـک محصـول داده در سـازمان شـما اسـت. یـک محصـول داده یـک واحـد مسـتقل، قابـل نصـب مسـتقل از داده اسـت کـه ارزش تجـاری را ارائـه می‌دهـد. ایـن شـامل داده‌هـای تحلـیلی اسـت کـه از برنامـه‌های عملیـاتی یـا معـاملاتی منبـع گرفتـه شـده اسـت. آن طـراحی، سـاخته و مـدیریت شـده توسـط یـک تـیم محصـول تـکی. هـر محصـول داده مسـتقل اسـت و کـاملاً مسـتقل از سـایر محصـولات داده مـدیریت می‌شـود. ایـن یـک مشـکل تجـاری خـاص را حـل می‌کنـد و بایـد بـا درک واضـحی از نیازهـای کـاربران هـدف طـراحی شـود. یـک محصـول داده بـا یـک دامنـه داده مـرتبط اسـت، بنـابراین یـک زمینـه خـاص تجـاری را خـدمت می‌کنـد و بایـد بـا اهـداف تجـاری آن زمینـه هماهنـگ باشـد. معمـولاً در هـر دامنـه تعـداد زیـادی محصـول وجود دارد.

توجه داشته باشید که محصول داده همان چیزی نیست که داده به عنوان یک محصول. داده به عنوان یک محصول ایده‌ای را توصیف می‌کند که مالکان داده داده را به عنوان یک محصول کاملاً محافظت شده که آن‌ها مسئول آن هستند، به جای یک بارنده فرآیندی که دیگران مدیریت می‌کنند، و باید داده را برای سایر دامنه‌ها و مصرف‌کنندگان در دسترس قرار دهند. محصول داده به معماری پیاده‌سازی داده به عنوان یک محصول اشاره دارد. برخی در صنعت IT مجموعه‌های داده، مدل‌های تحلیلی و گزارشات داشبورد را به عنوان محصولات داده می‌بینند، بر روی نمایش فیزیکی داده تمرکز دارند و هیچ متادیتا، کد یا زیرساخت را به عنوان محصولات داده در نظر نمی‌گیرند.

توجه داشته باشید که یک محصول داده می‌تواند از زیرمحصولات تشکیل شده باشد، هر کدام مسئول ارائه یک زیرمجموعه خاص از داده یا قابلیت که دور یک زمینه موضوعی سازماندهی شده است. به عنوان مثال، یک محصول داده برای داده‌های مشتری می‌تواند زیرمحصولاتی برای داده‌های معاملاتی، دموگرافی و رفتاری داشته باشد، هر کدام توسط یک تیم جداگانه مدیریت می‌شوند.

محصولات داده از طریق قراردادهای داده مصرف می‌شوند، که قراردادهایی بین دامنه‌های داده یا محصولات داده هستند که فرمت و ساختار تبادل داده بین یک تولید کننده و یک مصرف‌کننده را تعریف می‌کنند. قرارداد داده یک سند بسیار غنی است که معمولاً شامل متادیتا، طرح‌های داده، اطلاعات تبدیل داده و قوانین دسترسی به داده است. همچنین اطلاعاتی در مورد چگونگی دسترسی سایر دامنه‌ها، محصولات یا مصرف‌کنندگان به محصول داده را نیز دارد، معمولاً از طریق APIها، یک پایگاه داده، یک فرمت فایل استاندارد، یک جریان رویداد یا یک گراف.

از آنجا که تیم‌های دامنه مسئول مستقیم داده‌ها هستند، آن‌ها می‌توانند به سرعت هر مشکل کیفیتی را شناسایی و مقابله کنند. این منجر به زمان‌های حل سریع‌تر می‌شود و خطر استفاده از داده‌های نادرست در تصمیمات تجاری را کاهش می‌دهد. در نهایت، تیم‌های دامنه درون کسب و کار جاسازی شده‌اند، بنابراین درک بهتری از نیازهای خاص داده برای دامنه‌های خود دارند. این اطمینان می‌دهد که داده به گونه‌ای مدیریت و نگهداری می‌شود که با نیازهای کسب و کار هماهنگ باشد، که منجر به داده‌های تحلیلی با کیفیت بالاتر و مرتبط بیشتر می‌شود.

یک مورد دیگر درباره اصل #۲: هر تیم دامنه مسئول بهره‌برداری و مدیریت خود زیرساخت و تخصیص منابع خود است، همچنین استخدام افراد برای ساختن معماری و مدیریت داده. این یک تغییر بزرگ است؛ به جای پاک کردن و پردازش داده‌ها توسط IT برای مصرف (با کپی داده‌های عملیاتی به داده‌های تحلیلی)، این مسئولیت‌ها حالا بر عهده تیم دامنه است. تیم‌های دامنه باید نیازهای تجاری برای داده‌های خود را تعیین کنند. این بدین معناست که هر تیم می‌تواند نیازهای منابع خود را بر اساس نیازهای دامنه و حجم داده تعیین کند. هر تیم همچنین می‌تواند به صورت مستقل منابع را تأمین و مقیاس‌بندی کند تا بتواند تغییرات در تقاضا را مدیریت کند، بنابراین تیم‌های دامنه متعدد در رقابت برای منابع زیرساخت از یک تیم IT مرکزی نیستند.

اصل #۳: زیرساخت داده Self Service به عنوان یک پلتفرم

اصل #۳ پیشنهاد می‌کند که شما ایجاد و مدیریت محصولات داده را با اتوماسیون تأمین زیرساخت (مانند ذخیره‌سازی، محاسبات، خط تحلیل داده، و کنترل دسترسی) ساده‌تر کنید.

همانطور که در ابتدای این فصل اشاره شد، در معماری‌های متمرکز، تیم IT مرکزی می‌تواند یک گلوگاه شود، که این باعث می‌شود مقیاس‌بندی موثر دشوار باشد. این تیم همه معماری را ایجاد و نظارت می‌کند، از جمله ذخیره‌سازی، محاسبات، خطوط تحلیل داده، و سیاست‌های دسترسی به داده. اغلب آن‌ها فقط ندارند کافی افراد تا داده را به موقع در دسترس قرار دهند.

در یک شبکه داده، چون هر تیم دامنه داده خود را مالکیت می‌گیرد و داده خود را به عنوان یک محصول در نظر می‌گیرد، برای ساخت، ارائه، نظارت، و فراهم کردن دسترسی به محصول داده خود نیاز به تیم‌های مهندسی و

زیرساخت خود دارد. این مسئولیت تیم‌های دامنه را از ساخت برنامه‌های عملیاتی گسترش می‌دهد تا شامل ساخت تحلیل‌ها و ایجاد یک راه حل برای به اشتراک گذاری داده باشد. به طور کلی، دو اصل اول شبکه داده تلاش اضافی بر روی تیم‌های مهندسی دامنه ایجاد می‌کند. این اصل درباره کاهش آن تلاش است.

تیم‌های دامنه ممکن است با فکر به مدیریت، تامین کارکنان، نظارت، و پرداخت برای توسعه و مدیریت محصول داده خود احساس ناراحتی داشته باشند. برای جلوگیری از ناراحتی و مخالفت، شما باید به آن‌ها یک "مسیر کوتاه" برای غلبه بر برخی از این کارهای اضافی بدهید. شما نمی‌خواهید هر دامنه به طور مستقل زیرساخت خود را از صفر بسازد؛ این کار باعث می‌شود "دوباره ابزار را اختراع کند" تلاش‌ها را در هر دامنه تکرار کند، که این باعث تاخیر قابل توجه در ارائه شبکه داده به تولید می‌شود، هزینه‌ها را افزایش می‌دهد، و خطر ناهمسانی و ناسازگاری در مقیاس بزرگ را افزایش می‌دهد.

برای جلوگیری از این مشکلات، یک پلتفرم شبکه داده باید توسط یک تیم پلتفرم متخصص و مرکزی ساخته شود. همه چیز در یک شبکه داده متمرکز نیست. پلتفرم باید تمام ابزارها و رابط‌هایی که یک تیم مهندسی دامنه برای ساده‌سازی چرخه زندگی ساختن، تست کردن، ارائه دادن، امنیت دادن، نگهداری، و به اشتراک گذاری یک محصول داده با یک مصرف‌کننده یا با سایر دامنه‌های داده نیاز دارد، پیاده‌سازی کند. تیم‌های دامنه نباید نگران تأمین منابع زیرساخت باشند. شما می‌توانید این کار را با ارائه یک مجموعه از APIها یا اسکریپت‌های پلتفرم استاندارد شده انجام دهید که توسعه‌دهنده محصول داده می‌تواند برای اعلام نیازهای زیرساخت خود از آن‌ها استفاده کند. سپس می‌توانید به پلتفرم اجازه دهید که ایجاد و مدیریت ذخیره‌سازی، محاسبات، خطوط تحلیل داده، و کنترل دسترسی را بپذیرد. این منجر به یک روش استاندارد شده برای ایجاد محصولات داده، امنیت آن‌ها، پیدا کردن آن‌ها، اتصال به آن‌ها، و خواندن داده از آن‌ها می‌شود. در یک شبکه داده، تیم‌های مهندسی دامنه از تکنسین‌های عمومی تشکیل شده‌اند به جای تخصصیان، بنابراین کاهش پیچیدگی از طریق یک پلتفرم شبکه داده می‌تواند آن‌ها را آزاد کند تا با داده‌های خود ابداع کنند.

هدف از این اصل کاهش هزینه‌ها، کاهش پیچیدگی، سبک کردن بار تیم‌های دامنه، کاهش نیاز به تخصص فنی، و اتوماسیون سیاست‌های حاکمیت است.

اصل #٤: حاکمیت محاسباتی متقاطع

اصل #٤ می‌گوید که یک حاکمیت داده همکاری بین دامنه‌ها و یک تیم داده مرکزی وجود دارد تا قوانین جهانی (مانند قابلیت تعامل، کیفیت داده، امنیت داده، قوانین و مدل‌سازی داده) را تعریف، پیاده‌سازی، و نظارت کند.

حاکمیت داده در یک معماری متمرکز بسیار آسان‌تر از در یک شبکه داده است، زیرا با رویکرد معماری متمرکز، تنها یک تیم داده مرکزی تمام داده‌ها را مالکیت می‌گیرد و تمام حاکمیت داده را تعریف، پیاده‌سازی، و نظارت می‌کند. با این حال، این رویکرد متمرکز اغلب منجر به یک گلوگاه می‌شود، که فرآیند تصمیم‌گیری را کند کرده و توانایی دامنه‌های فردی برای پاسخ سریع به شرایط تغییر کننده و شرایط خاصی که با آن‌ها روبرو هستند، را محدود می‌کند. این همچنین می‌تواند منجر به کمبود دانش محلی و درک متناظر با شرایط محلی شود، زیرا تیم مرکزی باید تعداد زیادی از داده‌ها را بدون دانش عمیق از جزئیات هر دامنه حاکمیت کند.

در یک شبکه داده، یک تیم داده مرکزی تمام استانداردها و سیاست‌های حاکمیت داده را تعریف و نظارت می‌کند، از جمله چیزهایی مانند کیفیت داده، امنیت داده، قوانین، و مدل‌سازی داده. تیم داده مرکزی تشکیل شده از نمایندگان دامنه و کارشناسان موضوع (در مورد امضاء قانونی، حقوقی، امنیت و غیره) است. با این حال، پیاده‌سازی و نظارت حاکمیت داده به دامنه‌ها باقی می‌ماند، زیرا آن‌ها داده را مالکیت دارند و آن را بهتر می‌دانند.

چالش با توزیع مسئولیت بین چندین تیم دامنه احتمال ناهمسانی‌ها و روش‌های متناقض است، که منجر به ناکارآمدی‌های عملیاتی و افزایش پیچیدگی می‌شود. دامنه‌ها ممکن است استانداردهای مختلفی برای کیفیت داده را پیاده‌سازی کنند (به عنوان مثال، استفاده از نام‌های کامل ایالات متحده در مقابل اختصارات پستی دو حرفی). آن‌ها ممکن است سیستم‌های امنیتی مختلفی یا تکنیک‌های مدل‌سازی داده را استفاده کنند. اما همه قوانین حاکمیت نمی‌توانند به صورت جهانی تنظیم شوند؛ برخی از آن‌ها متناظر با شرایط خاص هستند و واقعاً باید توسط دامنه تنظیم شوند. به عنوان مثال، تیم داده مرکزی می‌تواند مشخص کند که کاربران باید تأیید شوند، در حالی که تیم‌های دامنه تعیین می‌کنند که هر کاربر دسترسی به چه محصولات داده‌ای دارد.

این نوع ناهمسانی می‌تواند منجر به داده‌های بد شود، افرادی که باید داده‌ها را نمی‌بینند می‌بینند، و مشکلات دیگری. سیستم‌های متمرکز اغلب با این جزئیات مبارزه می‌کنند زیرا آن‌ها سعی می‌کنند یک سیاست که برای همه مناسب است را اجرا کنند، که می‌تواند منجر به ناهمسانی‌ها و عدم توانایی در برآورده کردن نیازها و تقاضای خاص واحدها یا شعب مختلف شود.

این دلیل این است که مفهوم حاکمیت داده متقاطع، که نظارت مرکزی را با استقلال خاص دامنه ترکیب می‌کند، برای حفظ یکپارچگی و امنیت در سراسر سازمان حیاتی است. امیدواری است که یک تعادل بین استقلال دامنه و نظارت تیم داده مرکزی برقرار شود. هر زمان که امکان پذیر است، کمک می‌کند تا پیاده‌سازی و نظارت سیاست‌ها را اتوماتیک کند و داده‌ها را برای شناسایی آسان‌تر برچسب‌گذاری کند.

The “Pure” Data Mesh

شبکه داده "اصلی" یا "Mesh Data Pure" معماری بالایی است که از چهار اصل استفاده می‌کند. اول دو اصل متمرکز هستند، با این معنی که هر دامنه با استفاده از تیم IT خود و زیرساخت خود داده‌های تحلیلی را ایجاد می‌کند. اما دو اصل دیگر نیاز به تیم‌های مرکزی دارند—یکی برای پیاده‌سازی یک پلتفرم زیرساخت داده و یکی برای پیاده‌سازی حاکمیت مشترک برای تمام دامنه‌ها برای استفاده. این معماری بالایی شبکه داده نشان می‌دهد که چگونه از اصل‌های مختلف برای ساختار کلی سازمان استفاده می‌شود. این معماری به ما نشان می‌دهد که چگونه می‌توان با ترکیب از مراکزیت و استقلال دامنه، یک سیستم مدیریت داده متقاطع را ایجاد کرد که هم امکان مدیریت مرکزی را فراهم می‌کند و هم امکان استقلال دامنه را حفظ می‌کند. این ترکیب به ما امکان می‌دهد تا با حفظ یکپارچگی و امنیت در سراسر سازمان، همچنین امکان ابداع و انتقال داده‌ها را در دامنه‌های مختلف فراهم کنیم.

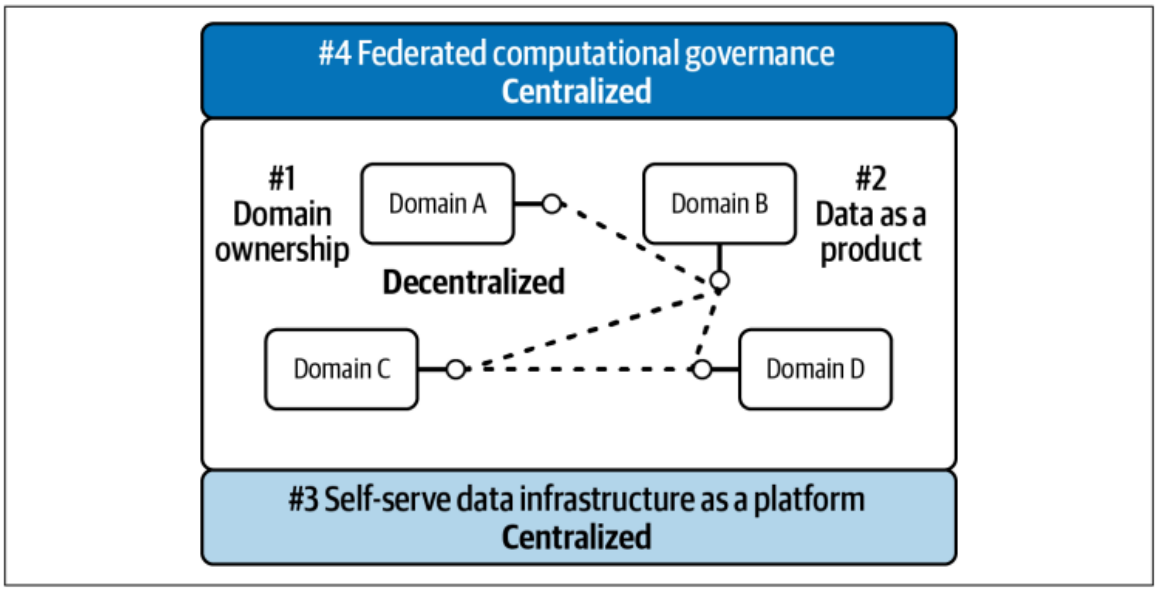


Figure 13-1. Data mesh architecture

تعریف یک "شبکه داده اصلی" یا "Mesh Data Pure" بدون قوانین یکپارچه و مشخص است. اگر یک شرکت یک شبکه داده ایجاد کند که کاملاً همه چهار اصل را دنبال کند، ما همه از آن را به عنوان یک "شبکه داده اصلی" مطمئن خواهیم شد. اما اگر راه حل فقط سه از این اصل را استفاده کند؟ می‌توانیم هنوز آن را یک شبکه داده نامید؟ و اگر راه حل همه چهار اصل را استفاده کند، اما به گونه‌ای که همه نیازهای اصل را برآورده نمی‌کند؟ به عنوان مثال، شما می‌توانید اصل #۲ را دنبال کنید، اما دامنه‌ها را با استفاده از یک دریاچه داده فیزیکی که به صورت منطقی به پوشه‌های مخصوص دامنه تقسیم شده است که فقط توسط دامنه‌های مربوطه قابل دسترسی هستند، یا شاید با اختصاص چندین دامنه به یکی از چندین خوشه محاسباتی به هم متصل شوند، به اشتراک بگذارید. به عنوان مثال دیگر، با استفاده از اصل #۳، IT مرکزی ممکن است اسکریپت‌هایی را ارائه دهد که به ساخت ذخیره‌سازی و مدیریت دامنه کمک کند، اما دامنه‌ها را به ساخت بقیه زیرساخت خود به تنهایی به آن‌ها بگذارد. و در

نظر بگیرید یک مثال دیگر: برای اصل #٤، IT مرکزی ممکن است استانداردهایی برای امنیت داده و قوانین را برقرار کند، اما دامنه‌ها را به خودمان نظارت برای امضاء قانونی به آن‌ها بگذارد و آن‌ها را آزاد به انجام کیفیت داده و مدل‌سازی به دلخواه خود بگذارد.

بنابراین، تعریف یک "شبکه داده اصلی" یا "Mesh Data Pure" به میزان پیاده‌سازی و استفاده از اصل‌ها بستگی دارد. اگر یک شبکه داده کاملاً همه چهار اصل را پیاده‌سازی کرده و به طور کامل نیازهای هر اصل را برآورده کرده باشد، می‌توان آن را به عنوان یک "شبکه داده اصلی" نامید. اما اگر فقط بخشی- از این اصل‌ها را پیاده‌سازی کرده باشد یا اگر به طور کامل نیازهای هر اصل را برآورده نکرده باشد، ممکن است نتوان آن را به عنوان یک شبکه داده کامل نامید. در نهایت، تعریف یک "شبکه داده اصلی" یا "Mesh Data Pure" به میزان پیاده‌سازی و استفاده از اصل‌ها بستگی دارد و ممکن است بسته به شرایط و نیازهای خاص سازمان متفاوت باشد.

برخی از شرکت‌ها یک راه حل با دو یا سه اصل به نیمه پیاده‌سازی شده ایجاد می‌کنند و آن را یک شبکه داده می‌نامند. دیگران دریاچه‌های داده جداگانه برای هر دامنه ایجاد می‌کنند و می‌گویند که آن‌ها یک شبکه داده ساخته‌اند. با این حال، فقط گروه‌بندی داده بر اساس دپارتمان‌های تجاری یا سازماندهی داده به دامنه‌ها یک شبکه داده نیست. حتی فرض کنید که آن‌ها دامنه‌های خود را به درستی طراحی کرده‌اند (موضوع بخش بعدی), این رویکرد فقط اصل #۱ را اعمال می‌کند و هیچ یک از دیگر اصل‌ها را اعمال نمی‌کند.

برخی از فروشندگان ممکن است بگویند که اگر از محصول آن‌ها استفاده کنید، شما یک شبکه داده خواهید داشت، حتی اگر شبکه داده در واقع در مورد تغییر سازمانی و فرهنگی است و نه در مورد فناوری. من حتی شنیدم که برخی افراد می‌گویند که همه چیزی که برای ایجاد یک شبکه داده نیاز دارید، تمام دریاچه‌های داده و انبارهای داده خود را با استفاده از نرم‌افزار ویرتوالایزاسیون متصل کردن است!

به عنوان مثال، این روش هیچ یک از چهار اصل را استفاده نمی‌کند. به نظر می‌رسد که می‌توان در هر اصل برای یک شبکه داده اصلی تعداد زیادی "استثناء" داشت. اگر چهار اصل به طور کامل پذیرفته نشوند، نمی‌توانیم حتی سعی کنیم به این سوال پاسخ دهیم. و آیا ما باید ۱۰۰٪ به تعریف‌های Dehghani پایبند باشیم یا آن‌ها را بر اساس بازخورد دیگران در صنعت تنظیم کنیم؟ آیا نیاز به یک کمیته صنعتی داریم تا یک تعریف بازنگری شده از شبکه داده را ارائه دهیم؟ و چگونه می‌توانیم بازخورد مبتنی بر خودمان را جلوگیری کنیم؟

من از آن خوشحال نیستم که همه پاسخ‌ها را نداشته باشم. امیدوارم کتاب این بحث سالم و سازگار را با ارائه روشنایی در مفاهیم معماری داده مشترک و معماری‌هایی که قبل از شبکه داده ایجاد شده‌اند، شروع کند.

دامنه های داده

فرآیند طراحی یک معماری محور به دامنه، که به عنوان طراحی محور به دامنه (DDD) شناخته می‌شود، بسیار دشوار و زمان‌بر است. DDD از اصول طراحی نرم‌افزار می‌آید، اما هنگام کار با داده‌ها بسیار چالش‌برانگیزتر می‌شود. قدم اول تعریف معنی "دامنه" در زمینه شرکت شما است.

بر اساس اصل اول دی‌غانی، داده‌های تحلیلی باید توسط دامنه‌های کسب و کاری که به آن‌ها نزدیک‌تر هستند - یا منبع داده یا مصرف‌کنندگان اصلی آن‌ها - مالکیت داشته باشند. او سه نوع داده تحلیلی محور به دامنه را تعریف می‌کند:

۱. **داده دامنه مطابقت منبع**: این نوع داده تحلیلی به منابع عملیاتی که داده از آن‌ها منشأ می‌شود، بسیار نزدیک است. داده از پایگاه‌های داده عملیاتی برنامه منبع به دامنه کپی می‌شود و تبدیل می‌شود تا به عنوان داده تحلیلی همکار شود. این داده برای مصرف‌کننده خاصی طراحی یا مدل‌سازی نشده است. دامنه‌های مطابقت می‌توانند یک یا چند منبع داشته باشند: داده‌ها معمولاً از سیستم‌های عملیاتی دامنه منتقل می‌شوند اما می‌توانند داده‌های عملیاتی یا تحلیلی از دامنه‌های دیگر باشند. به عنوان مثال، یک شرکت که کتاب‌ها را آنلاین فروش می‌دهد، می‌تواند داده‌های خود را بر اساس سفر خرید مشتری به دامنه‌ها تقسیم کند: جستجوی محصول، مرور، پرداخت و پرداخت. چالش ایجاد دامنه‌ها مالکیت است؛ به عنوان مثال، اگر یک کسب و کار در حال حاضر به دامنه‌های برنامه (دامنه‌هایی که فقط برای پشتیبانی از یک برنامه خاص ایجاد شده‌اند) تقسیم شده باشد اما به جای آن نیاز به

تقسیم به دامنه‌های کسب و کار (بر اساس توانایی‌های کسب و کار) دارد، یک دامنه کسب و کار می‌تواند بیش از یک برنامه را مالکیت داشته باشد.

داده‌های دامنه تجمعی

داده‌های دامنه تجمعی داده‌های تحلیلی هستند که داده‌ها را از دامنه‌های دیگر ترکیب و/یا تجمع می‌کنند، معمولاً برای کمک به عملکرد پرس و جو. به عنوان مثال، می‌توانید داده‌ها را از دامنه تولید و دامنه فروش ترکیب کنید تا بتوانید به راحتی و سریع گزارش سود و زیان را ایجاد کنید یا یک مدل داده ایجاد کنید که برای مصرف‌کنندگان بسیار راحت‌تر است برای استفاده.

داده‌های دامنه مطابقت مصرف‌کننده

داده‌های دامنه مطابقت مصرف‌کننده داده‌های تحلیلی هستند که تبدیل شده‌اند تا نیازهای یک یا چند دپارتمان خاص یا موارد استفاده را برآورده کنند. دامنه‌های مطابقت مصرف‌کننده تقریباً همیشه داده‌های خود را از دامنه‌های مطابقت منبع دریافت می‌کنند. به عنوان مثال، داده‌های دامنه برای یک دامنه تولید می‌تواند در دو روش مختلف مدل‌سازی و توصیف شود: یکی برای کسانی که تولید را می‌دانند (مطابقت منبع) و یکی برای کسانی که خارج از تولید هستند (مطابقت مصرف‌کننده). مثال دیگری تبدیل داده‌های مطابقت منبع به داده‌های مطابقت مصرف‌کننده به عنوان راهی برای ساده‌سازی آموزش مدل‌های یادگیری ماشین است.

برای جزئیات بیشتر در مورد دامنه‌های داده و طراحی محور به دامنه، من بسیار توصیه می‌کنم کتاب " Data Management at Scale"، نسخه دوم (O'Reilly, ۲۰۲۳) را توسط Piethein Strengholt بخوانید.

معماری منطقی دیتامش

شکل ۱۳-۲ نشان می‌دهد که چگونه معماری منطقی داده می‌تواند برای یک شرکت با چندین دامنه به نظر برسد. هر یک از این دامنه‌ها دارای یک یا چند محصول خواهد بود.

در شکل ۱۳-۲، تولید، فروش و تامین‌کننده همگی دامنه‌های مطابقت منبع هستند. هر دامنه دارای داده‌های عملیاتی و تحلیلی خود است. به عنوان مثال، دامنه فروش می‌تواند Salesforce را برای ردیابی تمام اطلاعات درباره مشتریان خود (داده‌های عملیاتی) استفاده کند. فروش سپس می‌تواند داده‌های خود را در یک معماری دی‌لاک خانه (مشاهده کنید فصل ۱۲) قرار دهد و آن را با منابع داده دیگری که داده‌های عملیاتی را دارند و با داده‌های تحلیلی از دامنه تامین‌کننده ترکیب کند. این ایجاد داده‌های تحلیلی فروش خواهد شد که سپس می‌تواند در دی‌لاک خانه قرار گیرد. فروش همچنین APIها را ایجاد می‌کند تا داده‌های تحلیلی و متادیتای مرتبط با آن را با دامنه‌ها و مصرف‌کنندگان دیگر به اشتراک بگذارد.

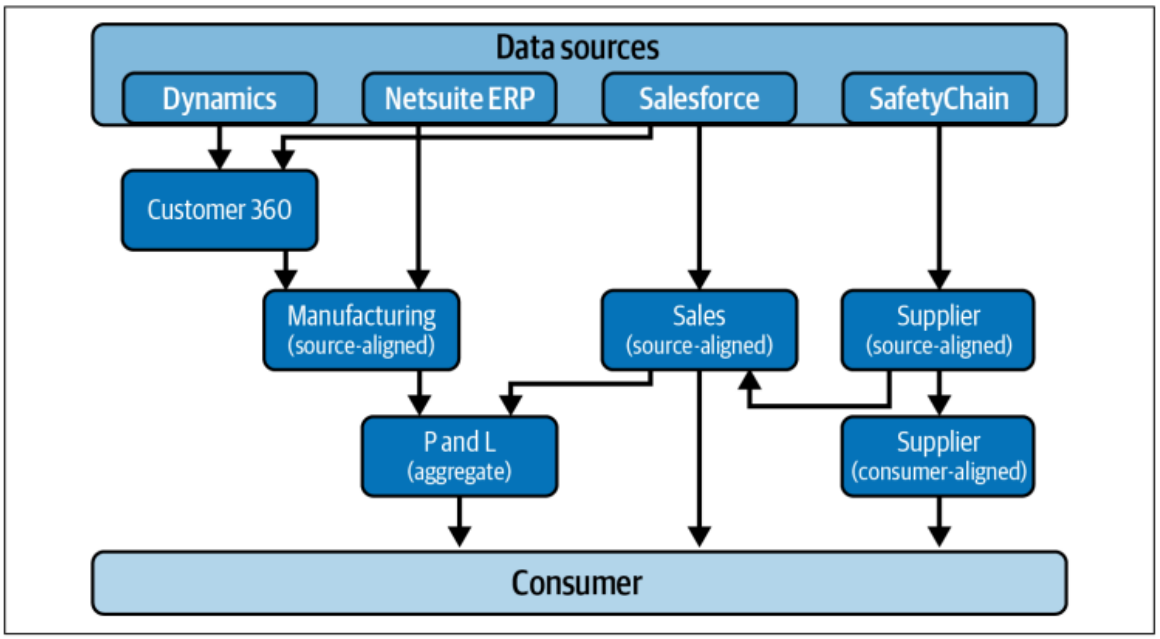


Figure 13-2. Data mesh logical architecture

توجه داشته باشید که دامنه مطابقت مصرف‌کننده برای تامین‌کننده وجود دارد. این به این دلیل ایجاد شد که داده‌های تحلیلی در دامنه مطابقت منبع تامین‌کننده برای افراد خارج از دامنه تامین‌کننده خیلی پیچیده بود. در دامنه مطابقت مصرف‌کننده تامین‌کننده، مدل داده ساده‌سازی شده و اصطلاحات تامین‌کننده آسان‌تر برای فهم شده است.

همچنین یک دامنه تجمعی سود و زیان (P&L) وجود دارد که با استفاده از داده‌های دامنه مطابقت منبع تولید و دامنه مطابقت منبع فروش ساخته شده است. در بخش زیادی از این، این برای دلایل عملکرد انجام شده است. اجرای یک پرس و جو یا گزارش که داده‌ها را از چندین دامنه ترکیب می‌کند، خیلی آهسته است اگر این دامنه‌ها دی‌لاک‌های داده جداگانه‌ای داشته باشند که در مکان‌های جغرافیایی بسیار دور از هم قرار دارند.

در نهایت، یک دامنه ۳۶۰ مشتری وجود دارد. این دامنه داده‌های مشتری (مانند داده‌های توصیفی، رفتاری، معاملاتی و بازخورد مشتری) را از منابع مختلف می‌گیرد، آن‌ها را تمیز و مستر می‌کند (مشاهده فصل ٦) و آن‌ها را ترکیب می‌کند تا یک تصویر کامل از مشتری بدست آورد. همه دامنه‌هایی که نیاز به داده‌های مشتری دارند، از این دامنه ۳۶۰ مشتری می‌گیرند.

این راه حل بسیار بهتر از داشتن هر دامنه خود داده‌های مشتری را بگیرد، تمیز و مستر کند، است. همانطور که شما مشاهده کردید، داشتن بسیاری از دامنه‌ها می‌تواند منجر به یک مخلوط پیچیده شود. تصویر شکل ۱۳-۲ را با ده‌ها یا حتی صدها دامنه در نظر بگیرید. این دلیلی است که طراحی دامنه بسیار مهم است. من در این فصل قبلاً به آن پرداختم، اما من تشویق می‌کنم که به منابع دیگری که به عمق بر این موضوع وارد می‌شوند، نگاهی بیندازید. شما نیاز دارید که زمان زیادی را در ابتدا صرف طراحی دامنه‌ها و محصولات کنید تا اطمینان حاصل کنید که نیازی به بازگشت و طراحی مجدد آن در ماه‌های آینده نخواهید داشت.

قراردادهای متفاوت

شکل ۱۳-۳ سه نوع مختلف معماری داده می‌شود که من در آن‌ها بحث خواهم کرد:

نوع شبکه ۱

در این معماری، همه دامنه‌ها از همان فناوری استفاده می‌کنند. به عنوان مثال، آن‌ها ممکن است همگی از یک ارائه‌دهنده ابری یکسان استفاده کنند و به محصولات این ارائه‌دهنده محدود شوند، بنابراین هر دامنه از همان محصولات برای ذخیره‌سازی، خطوط داده، امنیت، انبارهای داده رابطه‌ای، گزارشات، کاتالوگ متادیتا، MDM، مجازی‌سازی داده‌ها، APIها، ابزارهای ML و غیره استفاده می‌کند. ممکن است چندین گزینه در برخی از محصولات وجود داشته باشد، اما این انتخاب‌ها بسیار کم خواهند بود. هر دامنه دارای ساختار داده‌ای خود است، بنابراین همه چیز در مقیاس متمرکز است - به جز ذخیره‌سازی. به جای اینکه هر دامنه دامنه خود را داشته باشد، یک دی‌لاک داده کارآمد برای تمام دامنه‌ها وجود دارد، با هر دامنه دارای یک کانتینر یا پوشه در دی‌لاک که فقط به آن دسترسی دارد.

نوع شبکه ۲

در این معماری، دامنه‌ها از همان فناوری استفاده می‌کنند که در نوع شبکه ۱ استفاده شده است. به جای یک دی‌لاک داده کارآمد برای تمام داده‌های دامنه، هر دامنه دامنه خود را دارد. تمام دی‌لاک‌های داده از همان فناوری استفاده می‌کنند. این باعث می‌شود ساختار فنی واقعاً متمرکز شود، اما با هزینه: چالش فنی پیوند کردن تمام دی‌لاک‌های داده با هم و دریافت عملکرد قابل قبول هنگام ترکیب داده‌ها از چندین دامنه. این دلیلی است که من بیشتر شرکت‌ها را در نوع شبکه ۱ می‌بینم.

نوع شبکه ۳

در این معماری، هر دامنه می‌تواند از هر فناوری و ارائه‌دهنده ابری که می‌خواهد استفاده کند و دامنه خود را داشته باشد (که همچنین می‌تواند از هر فناوری استفاده کند). ممکن است دامنه‌های DomainA و DomainD از Azure استفاده کنند، DomainB از AWS استفاده کند و DomainC از GCP استفاده کند، در حالی که DomainA برای انبار داده رابطه‌ای خود Azure Synapse را انتخاب می‌کند و DomainD برای انبار داده خود Server SQL را در یک VM انتخاب می‌کند. چالش‌ها شامل مقابله با انواع مختلف امنیت برای هر دامنه، پیدا کردن و پشتیبانی از کارشناسان برای ده‌ها محصول، ایجاد استانداردهای حاکمیت برای بسیاری از محصولات در ابرهای مختلف (اصل ٤#)، ایجاد ساختار فنی خودکار که می‌تواند این تنوع را اجازه دهد (اصل #۳)، و سعی در ترکیب داده‌ها از چندین محصول در ابرهای مختلف. این در دسته‌بندی داده مشکل ساخته شده قرار می‌گیرد که قبلاً بحث شده است. این لیست قدرتمند چرا من احساس می‌کنم نوع شبکه ۳ هرگز پذیرفته نخواهد شد.

خلاصه - زهرا کشاورز از معماری داده

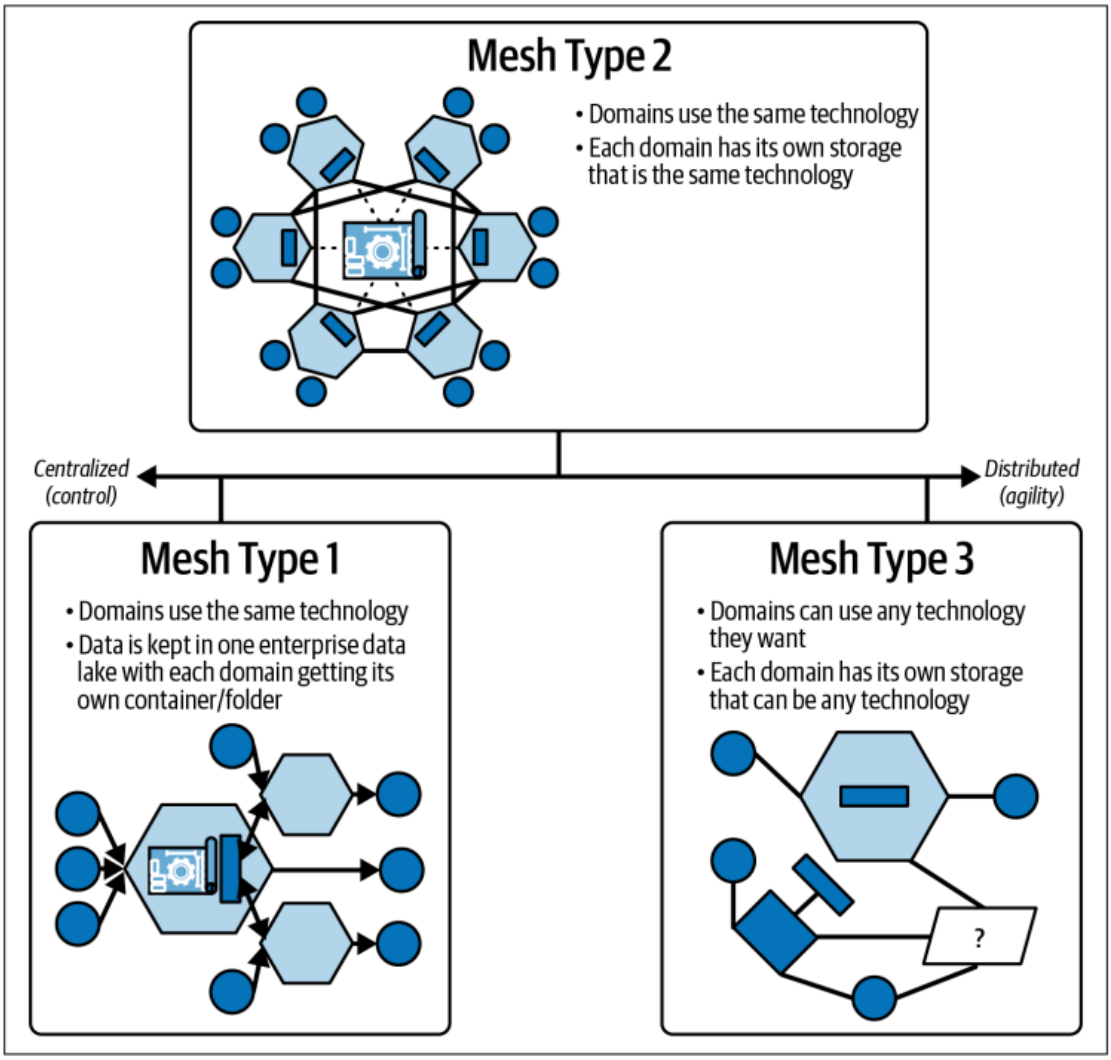


Figure 13-3. Three data mesh topologies on a spectrum from control to agility

در سمت چپ شکل ۱۳-۳، معماری‌ها بیشترین مرکزیت را دارند و با حرکت به سمت راست، معماری‌ها به صورت توزیع‌یافته‌تر می‌شوند.

مش دیتا در مقابل دیتا فابریک

داده‌فابریک و داده‌مشهور هر دو مفاهیم مهم در چشم‌انداز داده‌های امروز هستند، اما نقش‌های متفاوتی ایفا می‌کنند و باید با هم اشتباه گرفته نشوند. در هسته خود، داده‌فابریک یک چارچوب معماری است که برای استفاده در یک یا چند حوزه درون داده‌مشهور طراحی شده است. داده‌مشهور با این حال، یک مفهوم جامع است که تکنولوژی، استراتژی‌ها و روش‌های مختلف را در بر می‌گیرد. در واقع، تعریف آن به این قدر جامع است که من حتی می‌خواهم آن را به عنوان یک معماری برچسب بزنم.

از سوی دیگر چرخه، پایه‌های تکنولوژیکی و معماری داده‌فابریک می‌توانند برای پشتیبانی از جزئیات و اجزای مختلف درون داده‌مشهور سفارشی شوند. روابط آن‌ها نه یک رقابت یا قابلیت تعویض است بلکه یک همکاری است. یک داده‌مشهور ممکن است شامل چندین داده‌فابریک باشد، هر کدام برای نیازهای منحصر به فرد حوزه خود سفارشی شده باشند.

در این زمینه، می‌توانید هر حوزه در داده‌مشهور را به عنوان یک اکوسیستم خود در نظر بگیرید. در این اکوسیستم، حوزه می‌تواند داده‌فابریک خود را بسازد، بهینه‌سازی کند و عملیات کند، اطمینان حاصل کند که با نیازها و اهداف خاص خود هماهنگ است. به عبارت دیگر، در حالی که داده‌مشهور یک طرح برای رویکرد غیرمتمرکز و محور حوزه به داده‌ها فراهم می‌کند، داده‌فابریک‌ها زیرساخت معماری و تکنولوژیکی را فراهم می‌کنند که این حوزه‌ها را تحت تأثیر قرار می‌دهد. استفاده از آن‌ها به صورت همزمان می‌تواند منجر به یک زیرساخت داده بیشتر یکپارچه و کارآمد شود. این موضوع همچنین برای ترکیب داده‌مشهور با معماری انبار داده‌های مدرن یا داده‌لاک‌هاوس اعمال می‌شود.

موارد استفاده

امیدوارم که تا حالا درک خوبی از داده‌مشهور در توصیف داده شده توسط ده‌غانی داشته باشید، که من آن را "داده‌مشهور خالص" می‌نامم. در حالی که داده‌مشهور خالص در نظریه خوب به نظر می‌رسد، اما بسیار ایده‌آل است و تکنولوژی محدودی برای پشتیبانی از آن دارد. بنابراین، راه‌حل‌های عملی داده‌مشهور شامل بسیاری از استثناها نسبت به دیدگاه ده‌غانی خواهد بود.

با بیشتر راه‌حل‌های ساخته شده، بسیاری از استثناها نسبت به داده‌مشهور خالص وجود دارد که این پیاده‌سازی‌ها "داده‌مشهور" باید حتی به عنوان یک داده‌مشهور نامیده نشوند.

سوال اصلی که باید بپرسید این است: آیا شما در حال تلاش برای فقط ادغام داده‌ها هستید - یعنی ادغام داده‌ها از منابع متفاوت و جداگانه بدون ادغام یا ذخیره‌سازی فیزیکی داده در یک مخزن متناسب توسط IT - یا واقعاً در حال ساخت یک داده‌مشهور هستید؟ شما می‌توانید ادغام داده‌ها را بدون ساخت داده‌مشهور به دست آورید؛ ادغام داده‌های ابتدایی می‌تواند به عنوان برآورد کردن اصل #۱ از داده‌مشهور و نه سه اصل دیگر تفسیر شود. اگر این مسیر را انتخاب کنید، آیا باید حل شما هنوز به عنوان داده‌مشهور نامیده شود؟ من فکر می‌کنم که می‌توانید آن را هر چه می‌خواهید نام بزنید، اما ممکن است دقیق‌تر باشد که آن را یک معماری داده‌لاک‌هاوس یا داده‌فابریک کسب و کار نامیده شود، به خصوص زمانی که هر حوزه خود فضای کاری را دارد؛ توصیف دقیق‌ترین از آنچه ساخته‌اید این است که یک معماری داده‌لاک‌هاوس یا داده‌فابریک کسب و کار با چندین فضای کاری (یک فضای کاری برای هر حوزه) است. به یاد داشته باشید که می‌توانید اصل #۱ از یک داده‌مشهور را برآورد کنید اگر هر حوزه داده‌های تحلیلی مشتق شده از داده‌های عملیاتی خود را مالکیت داشته باشد، حتی اگر IT داده‌های تحلیلی را ایجاد کند. با این حال، شما نمی‌توانید اصل #۲ را برآورد کنید مگر اینکه هر حوزه خود داده‌های تحلیلی را ایجاد کند و رابط را برای به اشتراک گذاری این داده‌ها ایجاد کند.

همچنین به یاد داشته باشید که برای برآورد کردن اصل #۱، سازمان شما باید از فرآیند بازسازی شرکت خود با استفاده از حوزه‌های تجاری عبور کرده باشد. من می‌پرسم که آیا انجام کار ایجاد حوزه‌ها ارزش دارد اگر شما همچنین اصل #۲ را برآورد نکنید (داده به عنوان یک محصول); اگر IT نه حوزه‌ها، داده‌ها را پاک کند، پس شما یک نقطه بزرگ از داشتن یک داده‌مشهور را از دست خواهید داد.

به عنوان مثال، فرض کنید شرکت شما تعداد زیادی شرکت دیگر را خریداری کرده است. شما هر یک از این خریداری‌ها را مالکیت داده‌های خود دارند و IT از نرم‌افزار مجازی‌سازی برای دسترسی به آن‌ها استفاده می‌کند. در این سناریو، شما ادغام داده‌ها را به دست آورده‌اید، اما قطعاً داده‌مشهور ندارید؛ شما سازمان خود را بر اساس حوزه‌های داده تقسیم نکرده‌اید (مگر اینکه حوزه‌ها را بر اساس عملکرد تجاری تعریف کنید و تصمیم بگیرید که کسب و کارهای خریداری شده حوزه‌های خود هستند، که منطقی نیست زیرا در این صورت، به عنوان مثال، شما چندین حوزه HR با محصولات HR تکراری خواهید داشت). حتی با این حال، دوباره، سازماندهی حوزه‌ها بر اساس خریداری‌ها فقط اصل #۱ را برآورد می‌کند. من معتقدم که برای نامگذاری معماری به عنوان داده‌مشهور، باید اصل #۲ را برآورد کرد زیرا این دو بزرگترین منفعت از داده‌مشهور را به واقعیت می‌آورد: مقیاس‌پذیری سازمانی و فنی.

نکته آخر: در جستجوی یک تعادل بین مرکزیت و توزیع در یک داده‌مشهور، درک توزیع مسئولیت‌ها بسیار حیاتی است. شکل ۱۳-٤ یک مقیاس است که نشان می‌دهد که هر منطقه مسئولیتی ممکن است برای یک مورد خیالی کجا قرار بگیرد.

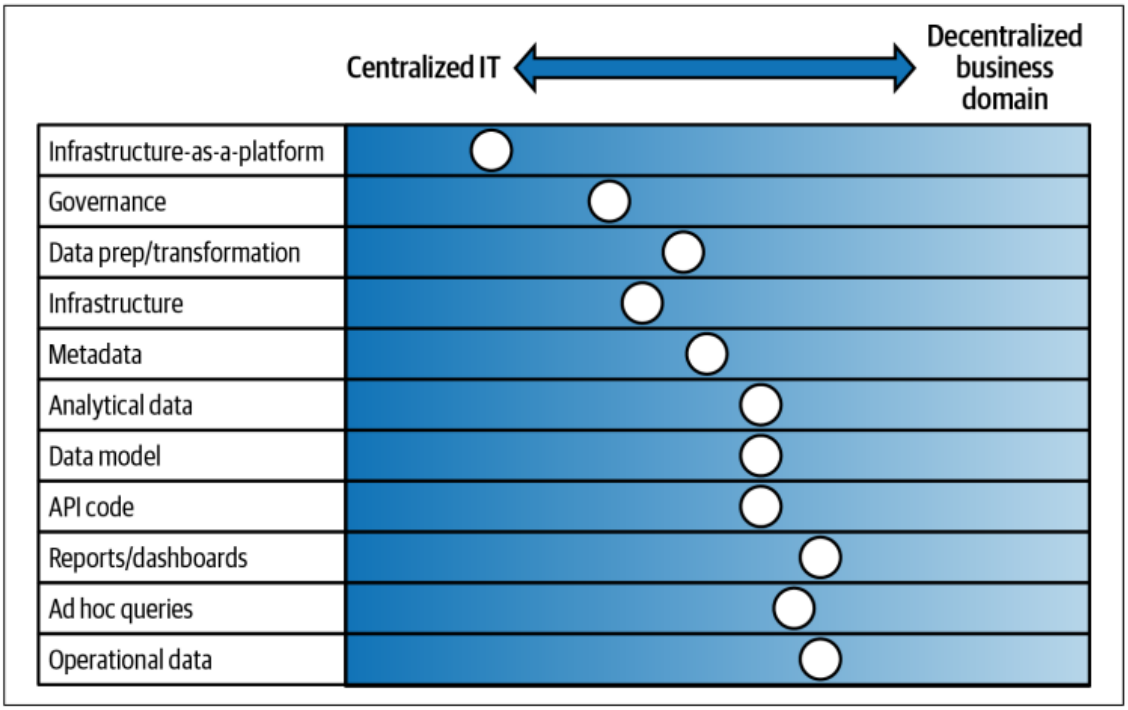


Figure 13-4. Division of responsibilities and ownership between a centralized IT department and decentralized business domains (fictional example)

در سمت چپ شکل ۱۳-٤، IT مرکزی قرار دارد و در سمت راست، حوزه‌های تجاری توزیع‌شده هستند. یک نقطه در سمت چپ بسیار نشان می‌دهد که IT مرکزی همه موارد خاص را انجام می‌دهد، و یک نقطه در سمت راست نشان می‌دهد که هر حوزه تجاری توزیع‌شده همه موارد خاص را انجام می‌دهد. نقاط در میان نشان می‌دهند که هر منطقه مسئولیتی در این طیف قرار دارد.

اگر بیشتر نقاط شما در سمت چپ بسیار قرار داشته باشند، به این معنی است که مرکزیت بسیار زیادی در حل شما وجود دارد که نامگذاری آن به عنوان داده‌مشهور مناسب نیست. ممکن است بهتر باشد آن را به عنوان یک داده‌فابریک کسب و کار یا داده‌لاک‌هاوس طبقه‌بندی کنید.

خلاصه

این فصل یک بررسی جامع از مفهوم داده‌مشهور و اصول اساسی آن ارائه داده است. با شروع با یک تحلیل مقایسه‌ای از مرکزیت و توزیع، تاکید بر تغییر از یک معماری یکپارچه به یک چشم‌انداز داده‌های توزیع‌شده که در آن داده به عنوان یک محصول ارزشمندی که توسط تیم‌های چند منظوره مالکیت دارد، شروع شد. سپس چهار اصل داده‌مشهور را توسط زهمک دغانی توضیح دادم: مالکیت داده توزیع‌شده محور بر حوزه، داده به عنوان یک محصول، زیرساخت داده Self Service به عنوان یک پلتفرم، و حکمرانی محاسباتی توزیع‌شده. این مفاهیم اهمیت درک داده به عنوان یک موجودیت جامع را که توسط تیم‌های خاص حوزه مالکیت دارد، می‌رسانند. ایده این است که هر واحد داده را به عنوان یک محصول فردی با چرخه زندگی خود در نظر بگیریم، که توسط تیمی که بهترین درکی از ارزش و استفاده از آن دارد، مدیریت می‌شود.

پس از توضیح اصول اساسی، فصل به داده‌مشهور خالص پرداخت، تاکید بر فرم خالص آن. سپس به حوزه‌های داده پرداختیم، که نقش آن‌ها در ساختاردهی محیط توزیع‌شده را برجسته کردیم. معماری منطقی داده‌مشهور و توپولوژی‌های مختلف که پشتیبانی از داده‌مشهور را فراهم می‌کنند، بحث شدند، نشان دادن تأثیر آن‌ها بر جریان داده و تعامل. یک مقایسه بین داده‌مشهور و داده‌فابریک ویژگی‌ها و کاربردهای متفاوت آن‌ها را وضح کرد. در نهایت، موارد استفاده از داده‌مشهور بحث شد.

با وجود اینکه رویکرد داده‌مشهور می‌تواند در نحوه مدیریت داده‌ها یک تغییر کامل باشد، اما با مشکلات و دشواری‌هایی همراه است، که در فصل بعدی به آن‌ها خواهم پرداخت. همچنین برخی از اشتباهات رایج در مورد داده‌مشهور را مطرح خواهم کرد و به شما کمک خواهم کرد تا بفهمید آیا اعتماد به یک داده‌مشهور انتخاب مناسبی برای سازمان شما است یا خیر.

فصل چهاردهم؛؛

در صفحات بعدی، من اشتباهات مربوط به داده‌مشهور را تجزیه و تحلیل خواهم کرد، به نگرانی‌های واقعی که اغلب بی‌گفته می‌مانند پاسخ خواهم داد، به شما کمک خواهم کرد تا میزان مناسب بودن آن در ساختار سازمانی خود را ارزیابی کنید و توصیه‌های قابل اجرا برای پیاده‌سازی موفق آن ارائه خواهم داد. در نهایت، به چشم‌انداز بالقوه سفر داده‌مشهور نگاهی خواهم انداز و با بحث درباره بهترین مورد استفاده برای هر یک از چهار معماری داده ختم خواهم کرد.

افسانه

با افزایش محبوبیت مفهوم داده‌مشهور در جامعه فناوری، مجموعه‌ای از اشتباهات مختلف ظهور کرده است، که اغلب درک واقعی از دامنه کامل، منافع و چالش‌های داده‌مشهور را مبهم می‌کنند. در این بخش، هدف من شروع به شکستن این اشتباهات است، با ارائه یک دیدگاه متعدد موارد بر این که داده‌مشهور واقعاً شامل چه چیزی است و چگونه در چشم‌انداز گسترده‌تر معماری داده قرار می‌گیرد.

افسانه: استفاده از مش داده ها یک گلوله نقره ای است، همه چالش های داده را به سرعت حل می کند

در واقع، واقعیت دقیقاً معکوس این است. ساخت یک داده‌مشهور زمان بیشتری نسبت به ساخت معماری‌های دیگری که در این کتاب ذکر شده‌اند، می‌برد. دهغانی ادعایی نمی‌کند که داده‌مشهور یک لوله نقره‌ای است، اما در هیپ رسانه‌ای، ایده‌های او اغلب به این شکل تلقی می‌شوند. این دلیل است که مهم است که در مورد پیچیدگی و چالش‌های استفاده از داده‌مشهور صحبت کنیم.

افسانه: یک دیتا مش جایگزین دریاچه داده و انبار داده شما خواهد شد

همانطور که در فصل ۱۳ یاد گرفتید، هر حوزه در یک داده‌مشهور هنوز نیاز به یک داده‌لاک و احتمالاً همچنین یک انبار داده‌های رابطه‌ای دارد. تغییر این است که دیگر نیازی به یک "مرکزی" داده‌لاک و RDW که توسط یک تیم IT مرکزی مالکیت دارد، نیست. به جای اینکه تمام داده‌ها برای تمام حوزه‌ها در یک داده‌لاک و یک RDW قرار بگیرند، هر حوزه داده‌های خود را نگه می‌دارد. بنابراین، با یک داده‌مشهور، در واقع شما داده‌لاک‌ها و انبارهای داده‌های رابطه‌ای بیشتری دارید، با که حوزه‌ها باید به همکاری کنند.

افسانه: پروژه های انبار داده همه شکست می خورند، و یک Data Mesh این مشکل را حل خواهد کرد

من اغلب می‌شنوم که افراد معتقد می‌شوند که راه‌حل‌های بزرگ داده فعلی مقیاس‌پذیر نیستند و داده‌مشهور را به عنوان راه حل پیشنهاد می‌کنند. من باید با این ادعا مخالفت کنم. در حالی که صدها هزار سازمان راه‌حل‌های موفقیت‌آمیز بزرگ داده را پیاده‌سازی کرده‌اند، تعداد بسیار کمی از داده‌مشهورها در تولید هستند (هیچ یک از آن‌ها یک "داده‌مشهور خالص" نیستند).

در ۱۵ سال کار در انبار داده، من می‌بینم که بسیاری از "معماری‌های یکپارچه" تکنولوژی و افراد IT خود را بسیار خوب مقیاس‌پذیر کرده‌اند، حتی حمایت از پتابایت‌های داده.

البته، بسیاری از پروژه‌های بزرگ داده شکست می‌خورند. با این حال، بیشتر شکست به دلایلی است (معمولاً مرتبط با افراد و فرآیندها) که همچنین می‌تواند باعث شکست یک داده‌مشهور شود. تکنولوژی موجود برای مرکزیت داده‌ها به شدت بهبود یافته است و ادامه خواهد داشت. گزینه‌های بدون سرور اکنون نیازهای بزرگ داده بیشتر سازمان‌ها را برآورده می‌کنند، اجازه می‌دهند راه‌حل‌ها همراه با صرفه‌جویی در هزینه‌ها مقیاس پذیر شوند.

افسانه: ایجاد یک دیتامش ابزار تمرکززدایی مطلق از همه چیز

این احتمالاً یکی از اشتباهات رایج است. واقعیت این است که حتی با یک داده‌مشهور خالص، یک تیم IT مرکزی هنوز مسئول ایجاد سرویس زیرساخت (اصل #۳) برای شروع مجدد زیرساخت هر حوزه است. یک تیم IT مرکزی همچنین مسئول ایجاد و نظارت بر سیاست‌های حاکمیت داده که هر حوزه را دنبال می‌کند (اصل #٤) است. منطقی

نیست که هر حوزه خود زیرساخت خود را از صفر بسازد. همچنین منطقی نیست که هر حوزه خود سیاست‌های حاکمیت داده را ایجاد کند - شما با تعداد بسیاری از امنیت و تفسیرات استانداردها و قوانین مواجه خواهید شد که برابر با تعداد حوزه‌ها است.

افسانه: شما می توانید از مجازی سازی داده برای ایجاد مش داده استفاده کنید

من بسیاری از معتقدان داده مجازی‌سازی شنیدم که اگر از داده مجازی‌سازی برای اتصال تمام داده‌لاک‌ها و انبارهای داده استفاده کنید، شما داده‌مشهور خواهید داشت! اما این رویکرد کاملاً اصول چهار داده‌مشهور (مشاهده فصل ۱۳) را نادیده می‌گیرد.

این یک اشتباه است که می‌توانید یک داده‌فابریک تک سازمانی (فصل ۱۱) را استفاده کنید، داده مجازی‌سازی را به آن اعمال کنید و آن را به یک داده‌مشهور تبدیل کنید. می‌توانید سعی کنید این کار را انجام دهید، اما نتیجه بسیار از یک داده‌مشهور دور است - این داده توزیع است، که به معنای ادغام داده‌ها از منابع متفاوت و متفاوت بدون اینکه آن‌ها را در یک مخزن مرکزی می‌ترانهاد یا ذخیره کنید. توزیع‌شدگی یکی از قطعات کوچک است که تشکیل می‌دهد یک داده‌مشهور - فقط اضافه کردن مجازی‌سازی هیچ یک از اصول چهار داده‌مشهور را برآورده نمی‌کند!

به عنوان مثال، فرض کنید یک شرکت دارد که یک داده‌فابریک تک سازمانی استفاده می‌کند که از داده مجازی‌سازی استفاده می‌کند. هر حوزه خود داده‌های تحلیلی را تولید می‌کند و داده‌فابریک به این داده از طریق نرم‌افزار داده مجازی‌سازی متصل می‌شود. این یک داده‌مشهور است؟ بیایید چهار اصل را برای بررسی اعمال کنیم:

- اصل #۱ درباره مالکیت حوزه است. آیا این حوزه‌های واقعی داده هستند یا فقط سازمان‌های جداگانه در شرکت؟

- اصل #۲ درباره داده را به عنوان یک محصول در نظر می‌گیرد. آیا هر حوزه تیم خود را برای ساخت راه حل تحلیلی دارد، با استفاده از زیرساخت خود حوزه؟ این تیم‌ها آیا یک قرارداد داده را دنبال می‌کنند که چگونگی ارائه داده و متادیتا به دیگران را مدیریت می‌کند؟ و این سازمان آیا از داده مجازی‌سازی برای اتصال به تمام حوزه‌های خود استفاده می‌کند یا فقط برخی از آن‌ها؟

به طور مهم، اگر هر حوزه خود تیم حوزه خود و زیرساخت نداشته باشد، یک تیم مهندسی داده مجازی‌سازی هنوز به عنوان یک گلوگاه IT عمل می‌کند. به طور مشابه، اگر هر حوزه‌ای از داده مجازی‌سازی استفاده نکند، داده‌های آن نیاز به جمع‌آوری و مرکزیت دارد. کمک به مقیاس‌پذیری زیرساخت و سازمان دو مزیت اصلی یک داده‌مشهور است و اگر حوزه‌ها از اصل #۲ پیروی نکنند، آن‌ها از این مزایا بهره‌مند نمی‌شوند.

- برای اصل #۳، می‌توانیم بپرسیم: آیا یک تیم جداگانه زیرساخت داده Self Service به عنوان یک پلتفرم را ایجاد می‌کند؟

- و برای اصل ٤: آیا یک تیم جداگانه حاکمیت محاسباتی توزیع‌شده را ایجاد می‌کند.

به طور واضح، تفاوت بزرگی بین یک راه حل داده‌مشهور واقعی و یک راه حل داده‌فابریک که از داده مجازی‌سازی استفاده می‌کند، وجود دارد.

چالش ها

به نظر می‌رسد که برای داده‌مشهور بسیار هیپ وجود دارد، اما تا به حال بسیار کمی درباره ناکارایی‌ها و چالش‌های آن منتشر شده است. این بخش سعی می‌کند این چالش‌ها را به طور جامع پوشش دهد، آن‌ها را بر اساس حوزه موضوعی گروه‌بندی کند و راه‌حل‌هایی را ارائه دهد تا بتوان به سرعت با نگرانی‌های ممکن مواجه شد. اگر پس از خواندن تمام این نگرانی‌ها، مطمئن هستید که می‌توانید هر کدام از آن‌ها را که مشکل ایجاد می‌کند، برطرف کنید، عالی! یک داده‌مشهور ممکن است برای شما مناسب باشد. و وقتی با هر یک از این نگرانی‌ها در آینده مواجه می‌شوید، نگران نخواهید شد—شما آماده خواهید بود. امیدوارم که با آگاهی از این مسائل، شما قبل از شروع به آن‌ها می‌توانید به آن‌ها پاسخ دهید، که احتمال موفقیت شما را به طور قابل توجهی افزایش می‌دهد.

در حالی که این لیست بر روی داده‌مشهور تمرکز دارد، بسیاری از این نگرانی‌ها برای سایر معماری‌های داده نیز اعمال می‌شود، بنابراین خوب است که آن‌ها را همه بررسی کنید. حتی اگر از یکی از سایر معماری‌های داده استفاده می‌کنید، من به طور شخصی- مشاهده کردم که یادگیری درباره داده‌مشهور می‌تواند مکالمات زنده را شروع کند که می‌تواند به شما کمک کند تا هر معماری داده‌ای که استفاده می‌کنید را بهبود بخشید.

مسائل فلسفی و مفهومی

بلاگ‌ها و کتاب اصلی زمانک دهقانی در مورد داده‌مشهور استاندارد را تنظیم کرده‌اند، اما افراد نگرش‌های متفاوتی به نوشته‌های او داشته‌اند. برخی از آن‌ها با استثناها به داده‌مشهور خالص و ایده‌های دیگری درباره اینکه چه چیزی باید داده‌مشهور باشد، به دست آمده‌اند. بنابراین، اگر می‌خواهید یک داده‌مشهور بسازید، کدام تعریف را استفاده خواهید کرد؟ فقط دریافت همه در ساخت داده‌مشهور برای رسیدن به یک مشاوره می‌تواند ماه‌ها طول بکشد.

این به خصوص چالش برای افرادی است که محصولات متعددی را برای ساخت یک داده‌مشهور استفاده می‌کنند و همه این محصولات ایده‌های متفاوتی درباره اینکه چه چیزی باید داده‌مشهور باشد، دارند.

برای هماهنگی تیم شما در یک درک مشترک از داده‌مشهور، در نظر بگیرید تشکیل یک کمیته حاکمیت متقاطع برای تعریف اساسی و استانداردها. از کارگاه‌های ذینفعان و تعریف‌های مستند استفاده کنید تا مطمئن شوید همه در همان صفحه هستند. پروژه‌های آزمایشی- می‌توانند به عنوان آزمایش‌های عملی برای این تعریف‌ها عمل کنند، به شما اجازه می‌دهد هر نیاز به تطبیق را داشته باشید. ارتباط باز و بازبینی‌های دوره‌ای به حفظ این هماهنگی کمک خواهد کرد هنگام پیاده‌سازی داده‌مشهور شما.

داشتن یک منبع واحد حقیقت کمک می‌کند تا سازمان تصمیمات خود را بر اساس داده‌های قابل اعتماد بگیرد، بدون هرگونه ابهام یا اختلاف در مورد اینکه کدام داده درست است.

همه کسانی که از یک مجموعه اطلاعات یکسان کار می‌کنند، عملیات را کارآمدتر و کارا تر می‌کنند.

در یک داده‌مشهور، داده می‌تواند توسط یک حوزه خوانده شود و توسط حوزه دیگری تبدیل و ذخیره شود. این توپولوژی پویا می‌تواند بسیار سخت باشد تا یک منبع واحد حقیقت را حفظ کنید. در واقع، دهقانی ایده یک منبع واحد حقیقت را یک "اشتباه" نامیده است.

من به شدت مخالف هستم؛ شخصاً دیده‌ام دهزده شرکت که یک منبع واحد حقیقت را در معماری‌های داده مختلف پیاده‌سازی کرده‌اند و درباره صدها شرکت دیگر خوانده‌ام. این می‌تواند با یک داده‌مشهور انجام شود اگر سازمان حاکمیت داده رابطه‌ای را برقرار کند و از یک کاتالوگ داده مرکزی استفاده کند (مشاهده فصل ٦). با متریک‌های کیفیت داده متقاطع حوزه و همکاری قوی بین حوزه‌های داده، نه تنها منبع واحد حقیقت قابل انجام است، بلکه می‌تواند نیز انعطاف‌پذیری و قابلیت مقیاس‌پذیری مشهور را افزایش دهد.

پیاده‌سازی یک داده‌مشهور خطرات بزرگی دارد، به خصوص نسبت به موفقیت ثابت انبار داده. این نیازمند تغییر سازمانی شدید و یک معماری کاملاً جدید است. علاوه بر این، داده‌مشهور فرض می‌کند که هر سیستم منبع می‌تواند به طور پویا مقیاس بزند تا با تقاضای مصرف‌کننده مقابله کند. این می‌تواند به خصوص زمانی چالش‌برانگیز باشد که برخی از دارایی‌های داده در اکوسیستم به عنوان "نقاط گرم" شناخته شوند، با افزایش سوء استفاده یا پرس و جوها.

ترکیب داده ها در یک محیط غیرمتمرکز

همیشه نیاز خواهید داشت که داده‌ها را از چندین حوزه برای پرس و جوها یا گزارشات ترکیب کنید. گاهی اوقات ممکن است نیاز به ترکیب داده‌ها از دهزده حوزه داشته باشید، که ممکن است در چندین انبار داده جداگانه قرار داشته باشند که جغرافیایی دور هستند. شما باید به طور دقیق درباره چگونگی غلبه بر مشکلات عملکرد فکر کنید زیرا راه‌حل‌هایی مانند داده مجازی‌سازی (مشاهده فصل ٦) و حوزه‌های تجمعی خود معاملات خود را دارند.

یک معامله دیگر از داشتن حوزه‌های مستقل این است که هر حوزه معمولاً فقط بر روی محصولات داده برای نیازهای تحلیلی خود تمرکز دارد. اغلب، مالکان فراموش می‌کنند که چگونه داده‌های خود می‌توانند با داده‌های حوزه‌های دیگر ترکیب شوند و کمتری انجام می‌دهند تا برای ترکیب مدل داده خود با دیگران آسان‌تر شود. وقتی کسی- نیاز دارد داده‌ها را از چندین حوزه ترکیب کند، ممکن است با بسیاری از مشکلات مواجه شود.

اصل #٤ می‌تواند به شما کمک کند تا مجموعه‌ای از قوانین مدل‌سازی داده را برای تمام حوزه‌ها تعریف کنید، اما این نیازمند داشتن کسی- است که تمام حوزه‌ها را درک کند - یک ویژگی از معماری‌های متمرکز که داده‌مشهور سعی دارد از

آن فاصله بگیرد. با تمرکز داخلی حوزه‌ها در یک داده‌مشهور، شرکت ممکن است کسی‌ـ نداشته باشد که راه‌هایی را برای استخراج ارزش از داده‌های ترکیب شده بررسی کند. بنابراین، مهم است که اطمینان حاصل شود که چنین کسی‌ـ وجود دارد.

اگر از یک مدل داده مشترک (CDM; مشاهده فصل ۸) استفاده می‌کنید، هر حوزه نیاز دارد که از آن استفاده کند. شما باید هماهنگی کنید تا مطمئن شوید که هر حوزه مجموعه خاصی از شناسه‌های یکتا برای رکوردها با انواع داده مشابه با حوزه‌های دیگر (مانند مشتریان یا محصولات) دارد. در غیر این صورت، وقتی داده‌ها را از چندین حوزه ترکیب می‌کنید، ممکن است با شناسه‌های تکراری مواجه شوید. این می‌تواند خطاهای شمارش، خطاهای تجمع و ناهماهنگی داده را در پرس و جوها و گزارشات ایجاد کند. برای جلوگیری از شناسه‌های تکراری در CDM در چندین حوزه، می‌توانید از یک شناسه یکتا جهانی (GUID) استفاده کنید یا یک سیستم مدیریت شناسه مرکزی را برای تخصیص شناسه‌های یکتا برقرار کنید. به عبارت دیگر، می‌توانید فضای نام (namespacing) را اعمال کنید، که در آن هر شناسه با یک کد خاص به حوزه پیشوند یا پسوند می‌شود.

علاوه بر این، هنگامی که حوزه‌ها محصولات و مدل‌های داده خود را ساخته و توسعه می‌دهند، حوزه‌ها و مصرف‌کنندگان دیگر داده‌های آن‌ها را مصرف می‌کنند و داده‌ها را از چندین حوزه بر اساس این مدل‌های داده ترکیب می‌کنند. وقتی یک حوزه مدل داده یا رابط خود را تغییر می‌دهد، وابستگی‌ها می‌توانند برای حوزه‌ها و مصرف‌کنندگان دیگر که با مدل‌های داده اصلی یا پرس و جوهایی که مدل‌های داده حوزه را ادغام می‌کنند، مشکلاتی ایجاد کنند. شما باید توانایی همه حوزه‌ها را برای هماهنگی در هر زمانی که هر حوزه مدل داده خود را تغییر می‌دهد، آماده کنید.

در نهایت، تیم‌های حوزه ممکن است ایده‌های متفاوتی درباره اینکه چه چیزی به عنوان "داده‌های تمیز" یا "استاندارد" شناخته می‌شود داشته باشند. اگر حوزه‌ها در مورد سوالاتی مانند آیا باید استانها را با اختصارات یا نام کامل استان تعریف کنند، متفاوت نظر داشته باشند، ترکیب داده‌ها از چندین حوزه می‌تواند باعث آشفتگی شود. و اگر حوزه‌ها زمان لازم برای ساختن کد برای تمیز کردن داده‌ها را نداشته باشند؟ برای مقابله با ناهماهنگی‌ها در تمیز بودن داده‌ها در حوزه‌ها، یک چارچوب حاکمیت مرکزی را برای تعریف استاندارد برای داده‌های تمیز یا استاندارد برقرار کنید. قالب‌های تمیز کردن داده یا کتابخانه‌های مشترک را ارائه دهید تا زمان را صرفه‌جویی کنید و یکنواختی را تضمین کنید. یک کمیته حاکمیت یا مربی داده می‌تواند در اجرای این استانداردها کمک کند، که باعث می‌شود ترکیب داده‌ها از چندین حوزه بدون مشکلات آسان‌تر شود.

سایر مسائل تمرکززدایی

حوزه‌های تجمعی و مطابق با مصرف‌کننده (مشاهده فصل ۱۳) متمرکز نیستند. برای ایجاد یک حوزه تجمعی، شما داده‌ها را از چندین حوزه می‌گیرید و آن‌ها را مرکزی می‌کنید. با حوزه‌های تجمعی و مطابق با مصرف‌کننده، باید تصمیم بگیرید که کی این حوزه‌ها را می‌سازد و نگهداری می‌کند - مشکلی که اصل #۱ قصد دارد آن را جلوگیری کند. ساخت چنین حوزه‌هایی نیز منجر به تکرار داده‌ها می‌شود. همه این به نظر می‌رسد با روح یک داده‌مشهور متضاد است.

برای هماهنگی حوزه‌های تجمعی و مطابق با مصرف‌کننده با اصول یک داده‌مشهور، به حاکمیت مشترک و مربیان داده مشخص شده برای مدیریت این حوزه‌ها فکر کنید. از داده مجازی‌سازی برای کاهش تکرار استفاده کنید و یک لایه API برای انتزاع استفاده کنید. از طریق متادیتا قوی و نقاط ردیابی شفافیت را حفظ کنید.

امنیت نیز مسئله‌ای است. در یک معماری متمرکز، یک تیم مسئول امنیت تمام داده‌ها است. در یک داده‌مشهور، این مسئولیت به دزدیز ها یا حتی صدها حوزه منتقل می‌شود، که باعث می‌شود بسترهای بیشتری برای نفوذ یک هکر ایجاد شود. این باعث افزایش خطر نقض امنیت می‌شود.

شما باید کسی‌ـ را تعیین کنید که برای جستجو برای اطلاعات شناسایی شخصی‌ـ (PII) اسکن کند و ببیند کی آن را دسترسی دارد، و همچنین کسی‌ـ را تعیین کنید که مشکل را حل کند اگر کسی‌ـ به PII که نباید ببیند دسترسی داشته باشد. کسی باید مسئول اطمینان از این باشد که تمام حوزه‌ها امنیت کافی دارند.

همچنین سخت است تکنولوژی مناسب را در حوزه‌ها انتخاب و پیاده‌سازی کنید. اگرچه اصل #۳ (سرویس به عنوان زیرساخت) قصد دارد این موضوع را جلوگیری کند، اما اگر شما دهزده حوزه دارید که افراد آن‌ها مهارت‌ها و تجربیات متفاوتی دارند، احتمالاً تکنولوژی‌های مختلفی را انتخاب می‌کنند، زیرساخت خود را به شکل‌های مختلفی

راه‌اندازی می‌کنند و تنظیمات محصولات ابری خود را به شکل‌های مختلفی پیکربندی می‌کنند. این می‌تواند منجر به مشکلات اصلی در مصرف داده شود.

شما می‌توانید محدود کنید که چه تکنولوژی‌هایی برای هر حوزه به طور خودکار ارائه می‌شوند، اما این با نقض اصل #۱ مواجه می‌شود زیرا حریم آزادی هر حوزه را برای انتخاب تکنولوژی برای مورد استفاده خود محدود می‌کند. پشتیبانی از تکنولوژی‌های متفاوت می‌تواند چالش‌برانگیز باشد، همانطور که ایجاد یک رابط مشترک در سراسر تمام حوزه‌ها می‌تواند. به عنوان یک تساوی، می‌توانید یک کاتالوگ بزرگ از تکنولوژی‌های تأیید شده و بررسی شده ایجاد کنید - حوزه‌ها را یک انتخاب به ارائه می‌دهد، اما در یک چارچوبی که اطمینان از ادغام آسان‌تر را تضمین می‌کند.

علاوه بر این، اگرچه اصل #٤ (حاکمیت متحد) سیاست‌های داده‌مشهور را ایجاد می‌کند، اما هر حوزه باید آن‌ها را پیاده‌سازی کند. این می‌تواند منجر به پیاده‌سازی‌های ناهماهنگ شود، که ممکن است منجر به کاهش عملکرد و پروفایل‌های هزینه متفاوت شود.

پیچیدگی

ایده‌های پشت ساختار داده‌مشهور پیچیده هستند. من بدون تردید به این اعتراف می‌کنم که برای درک کامل این معماری باید به بلاگ‌های Dehghani دهزده بار بخوانم. حتی سال‌ها بعد، گاهی اوقات هنوز متوجه می‌شوم که بخشی- از آن را به درستی درک نکرده‌ام. من تنها نفری نیستم؛ تقریباً هر کسی- که با آن صحبت می‌کنم در مورد داده‌مشهور گیج شده است. این پیچیدگی، با عدم وجود تعریف استانداردی برای داده‌مشهور، بدین معنی است که بیشتر تیم‌هایی که قصد ساختن یک داده‌مشهور دارند، باید زمان زیادی را در ابتدا در اطمینان از اینکه همه آن را درک می‌کنند (و آن را به درستی درک می‌کنند) سرمایه‌گذاری کنند. اگر زمان زیادی لازم است تا شروع شود و به سرعت رشد کند، مطمئن شوید که داده‌مشهور به اندازه کافی منفعت برای آن زمان و تلاش را به دست می‌آورد.

این پیچیدگی پیچیده به پیچیدگی سازمانی گسترش می‌یابد، اگرچه پیشنهادگران داده‌مشهور معکوس این موضوع را ادعا می‌کنند. تیم‌های توزیع شده شامل بسیاری از مردم بیشتری هستند و آن‌ها همیشه به طور موثر ارتباط نمی‌برند.

تکراری شدن

Dehghani معتقد است که تکرار داده‌های منبع در داده‌مشهور مشکلی نیست، با نوشتن اینکه برای هر حوزه، "احتمالاً نیازی به یک داده‌لاک نخواهیم داشت زیرا لاگ‌ها و ذخیره‌سازی توزیع شده که داده‌های اصلی را نگهداری می‌کنند، برای کاوش از مجموعه‌های داده‌ای غیرقابل تغییر و قابل آدرس به عنوان محصولات در دسترس هستند." اما شما همیشه نمی‌توانید به طور مستقیم به داده‌های اصلی از این مجموعه‌های داده‌ای غیرقابل تغییر دسترسی پیدا کنید، زیرا اغلب مشکلاتی در زمینه امنیت یا بازیابی فوری وجود دارد. (من هنوز بسیاری از مشتریان دارم که به مین‌فریم‌هایی نمی‌توانند متصل شوند.) بنابراین، برای انجام هر کاری با این داده‌های منبع، هنوز باید یک کپی از آن را ایجاد کرده و در یک داده‌لاک ذخیره کنید. در واقع، یک داده‌مشهور ممکن است نتیجه بیشتر کپی‌های داده تر از سایر معماری‌ها باشد، زیرا حوزه‌ها ممکن است نیاز به کپی داده از حوزه‌های دیگر برای تکمیل محصولات داده خود یا برای ایجاد حوزه‌های تجمعی یا محور به مشتری داشته باشند. نگهداری همه این کپی‌ها در هماهنگی هزینه و تلاش اضافی ایجاد می‌کند.

به طور مشابه، یک بُعد مطابق، که در یک ساختار ستاره استفاده می‌شود، همان معنایی را با هر حقیقتی که با آن مرتبط است دارد. بُعدهای مطابق به شما اجازه می‌دهند تا حقیقت‌ها و اندازه‌گیری‌ها (مانند یک بُعد تاریخ یا بُعد مشتری) را به همان شکل دسته‌بندی و توصیف کنید، اطمینان حاصل کردن گزارشات یکنواخت در سراسر شرکت. بُعدهای مطابق بسیار محبوب در یک راه حل ساختار ستاره هستند. در یک داده‌مشهور، این بُعدهای مطابق باید در هر حوزه‌ای که جداول حقیقت مرتبط دارد، تکرار شوند. این یک مقدار زیادی از داده‌های تکرار شده است که نیاز به نگهداری هماهنگ دارد.

احتمالاً شرکت شما داده‌های سوم شخص را خریداری می‌کند، مانند آب و هوا، سهام، یا داده‌های رقبا. در یک معماری متمرکز، این مورد توسط یک شخص یا گروه مدیریت می‌شود، بنابراین احتمال دو تیم در یک شرکت خرید داده‌های یکسان کم است. با این حال، در یک داده‌مشهور، هر حوزه خود داده‌های خود را خریداری می‌کند. چگونه می‌توان جلوگیری کرد تا دو حوزه با خرید مجموعه‌های داده یکسان پول ضایع کنند؟ هیچ یک از اصول داده‌مشهور به این زمینه نمی‌پردازد، اما می‌توانید اصل #٤ را با تنظیم یک تیم مرکزی که تمام درخواست‌ها برای داده‌های سوم شخص را مدیریت می‌کند، بهبود بخشید.

با وجود اصل #٣، برخی از حوزه‌ها احتمالاً پلتفرم‌های دریافت داده یا تحلیلی مشابه را ساخته خواهند کرد. با یک رویکرد متمرکز، فقط یک تیم یک پلتفرم را ساخته است، اما در یک داده‌مشهور، صدها حوزه ممکن است این کار را انجام دهند. ایده ترویج این راه حل‌ها برای اشتراک گذاری آن‌ها خوب است تا حوزه‌ها نتوانند تلاش یکدیگر را تکرار کنند یا از راه حل‌های مفیدی که حوزه‌های دیگر ایجاد می‌کنند از دست بدهند.

امکان سنجی

انتقال به یک داده‌مشهور یک کار بزرگ است که نیاز به سرمایه‌گذاری زیادی در تغییر سازمانی و پیاده‌سازی فنی دارد، بسیار بیشتر از سایر معماری‌های داده. این منجر به هزینه بسیار بالاتر و یک زمان‌بندی بسیار طولانی‌تر می‌شود - چندین ماه، حتی امکان دارد چندین سال طول بکشد قبل از اینکه بتوانید آن را در تولید قرار دهید. برای اینکه حوزه‌ها بتوانند گزارشات و داشبوردهای خود را به اندازه کافی به خوبی استفاده کنند تا تکنولوژی مرکزی IT بتواند آن‌ها را تامین کند، بسیاری از چیزها لازم است. در حالی که شما یک شرکت جدید نیستید، شما باید پلتفرم موجود خود را ادامه دهید. این می‌تواند بسیار چالش‌برانگیز باشد، به خصوص زمانی که حوزه‌های جدید از تامین داده‌های عملیاتی به تکنولوژی مرکزی IT و نگهداری آن در حوزه خود می‌پردازند.

دوباره، داده‌مشهور یک مفهوم است، نه یک تکنولوژی. همه در مورد ایجاد تغییرات سازمانی و فرهنگی در یک شرکت است. کتاب Dehghani نمی‌گوید که چگونه از تکنولوژی موجود برای ساخت یک داده‌مشهور استفاده کنید. احتمالاً به این دلیل است که در زمینه‌هایی مانند کوانتوم داده، خودکارسازی، اشتراک محصول، تنظیم زیرساخت حوزه، و حاکمیت مشترک داده، شکاف بزرگی در تکنولوژی موجود وجود دارد.

اصل #٢ می‌خواهد که تکنولوژی برای انتشار و اشتراک‌گذاری حوزه‌ها در دسترس باشد، اما تکنولوژی فعلی برای این منظور در مرحله نوزادی است. اصل #٣ نیاز به راه‌حل‌های بسیار پیچیده برای خودکارسازی تأمین زیرساخت دارد، اما راه‌حل‌های فعلی فقط یک قطعه کوچک از این کار را به بهترین شکل انجام می‌دهند. و ایجاد و مدیریت قوانین جهانی اصل #٤ - در همان زمانی که حوزه‌ها قوانین خود را ایجاد و مدیریت می‌کنند - به وابستگی سنگین به خودکارسازی و راه‌حل‌های کدنویسی‌ـ برای مدیریت چرخه عمر محصولات داده، استانداردها به عنوان کد، سیاست‌ها به عنوان کد، آزمایش‌های خودکار، نظارت خودکار، و جاسازی اجرای سیاست در هر محصول داده می‌رسد. این نیاز به نرم‌افزار بسیار پیچیده دارد که که در هر جایی که وجود دارد، بسیار محدود است. شرکت‌ها باید خودشان آن را ساخته باشند یا این مرحله را حتی انجام ندهند، بنابراین اصول داده‌مشهور را نقض می‌کنند.

در کتاب خود، Dehghani محصولات داده را با پورت‌های خروجی داده (رابط‌ها) با قراردادها و APIهای به صورت صریح تعریف شده معرفی می‌کند. این APIها برای دسترسی چندمدلی، داده‌های دو باره‌ای، و دسترسی فقط خواندنی غیرقابل تغییر می‌سازند و می‌توانند مقادیر بزرگی از داده‌ها را انتقال دهند و به تعداد بزرگی از درخواست‌ها مقیاس بزنند. علاوه بر این، این APIها بر روی یک یا چند زیرساخت فیزیکی متعدد، چندین ابر، و محیط‌های میزبانی در محل می‌گذرند. تا به حال، تکنولوژی برای پشتیبانی از همه این موارد وجود ندارد. احتمالاً هرگز وجود ندارد.

علاوه بر این، APIها به ندرت با داده‌ها استفاده می‌شوند؛ تعامل با داده‌ها معمولاً به سوالات SQL بستگی دارد. ایجاد APIها برای تمام تعاملات با داده‌ها یک تغییر فرهنگی بزرگ است که زمان بیشتری نسبت به استفاده از سوالات SQL نیاز دارد.

در کتاب خود، Dehghani یک واحد جدید از معماری منطقی را به نام کوانتوم داده معرفی می‌کند که کنترل و کپسولاسیون تمام کامپوننت‌های ساختاری مورد نیاز برای اشتراک‌گذاری داده به صورت خودکار به عنوان یک محصول داده که دیگران می‌توانند آن را مصرف کنند را انجام می‌دهد. این شامل داده‌ها، متادیتا، کد (محاسبات)، سیاست، و وابستگی‌های زیرساخت می‌شود. داده و محاسبات به صورت منطقی به عنوان یک واحد متصل هستند،

اما زیرساخت‌های فیزیکی پشتیبانی کننده کد و داده به صورت جداگانه نگهداری می‌شوند. در حال حاضر تکنولوژی برای این مورد وجود ندارد.

من فکر می‌کنم که کوانتوم داده یک مفهوم خیلی خطرناک است. شما قطعاً می‌توانید یک داده‌مشهور بدون استفاده از آن بسازید. به خاطر داده خروجی با استفاده از یک کوانتوم داده فکر کنید. اگر متادیتا هر حوزه فقط در خود محصول داده خود در دسترس باشد، برای دیدن متادیتا، خروجی، یا اطلاعات کیفیت داده باید سوالات متقاطع پیچیده را در برابر تمام محصولات داده اجرا کنید. شما می‌توانید تمام متادیتا را به یک مکان مرکزی کپی کنید، اما این نیازمند همزمان سینک دائمی است و معماری کلی را بسیار پیچیده‌تر می‌کند. سپس مسئله امنیت وجود دارد؛ شما باید متادیتایی مانند برچسب‌های حساسیت، طبقه‌بندی‌های حریم خصوصی، و اطلاعات مالکیت را برای داده‌هایی که در چندین مکان فیزیکی قرار دارند، به‌روز کنید.

مشکلات مربوط به کپی کردن داده‌ها به محصولات داده نیز گسترش می‌یابد. اگر محصولات داده کپی شوند، متادیتای آن‌ها نیز کپی می‌شود، که منجر به چندین کپی از همان متادیتا می‌شود که نیاز به نگهداری همزمان دارند. هر به‌روزرسانی به متادیتا باید به صورت همزمان در سراسر چندین کانتینر محصول داده انجام شود، که این کانتینرها باید همیشه در دسترس و قابل دسترسی باشند. شما خطر دارید که دارای مالکان متفاوت برای همان متادیتا شوید.

در نهایت، اجرای صدها یا حتی هزاران محصول داده کوچک کنار هم به طور خودکار پیچیدگی را افزایش می‌دهد، استفاده از منابع زیرساخت و شبکه‌ها را افزایش می‌دهد، مدیریت پیکربندی را پیچیده می‌کند و چالش‌هایی در مورد عملکرد و یکپارچگی داده‌های مرجع می‌ایجاد.

Dehghani معتقد است که از آنجا که هنوز هیچ تکنولوژی به طور محلی داده‌مشهور ایجاد نشده است، "شما می‌توانید از تکنولوژی‌های انبار داده به عنوان حافظه پایه و حالت دسترسی SQL استفاده کنید اما یک ساختار جداگانه را به هر محصول داده اختصاص دهید،" بنابراین محاسبات را به اشتراک می‌گذارد. اما این منجر به یک زیرساخت متمرکز می‌شود، که با نقض اصل #۱ مواجه می‌شود و بزرگترین مزیت استفاده از یک داده‌مشهور را از دست می‌دهد: مقیاس‌پذیری فنی.

نفرات

برای ایجاد یک داده‌مشهور، شما نیاز به پیدا کردن و استخدام افراد مهارتمند مهندسی کیفیت و کارگران مهارتمند دیگر برای هر حوزه خواهید داشت. پیدا کردن افراد خوب برای یک تیم IT مرکزی سخت است، اما در یک داده‌مشهور، هر حوزه نه تنها باید یاد بگیرد تا یک زیرساخت و پلتفرم تحلیلی بسازد، بلکه باید خود تیم کوچک IT خود را ایجاد و استخدام کند. اگر شما تعداد زیادی از حوزه‌ها دارید، این به معنای داشتن بسیاری از افراد بیشتر از آنچه که در یک معماری متمرکز نیاز دارید است—یک معامله بزرگ زمانی که شما در حال مقیاس‌پذیری سازمانی افقی هستید.

اصل #۳ طراحی شده است تا نیاز به مهارت‌های فنی عمیق را بردارد و به جای آن از مهارت‌های متقاطع بالاتر استفاده کند. این اغلب سازمان‌ها را ترغیب می‌کند تا افراد از حوزه‌های کسب و کار، به جای افراد با مهارت‌های فنی قوی و تجربه، پیاده‌سازی را در هر گروه محصول انجام دهند، به خصوص زیرا پیدا کردن کافی افراد با مهارت‌های فنی کیفیت دشوار است. افراد نامتجربه معمولاً محصولات داده گران قیمت، پیچیده و عملکرد بد را ساخته می‌کنند.

داده‌مشهور فرض می‌کند که مهندسان داده IT مرکزی ندارند دانش کسب و کار و حوزه و بنابراین بهتر است از افرادی استفاده کنیم که با حوزه‌هایی که داده‌ها را مالکیت می‌گیرند آشنا هستند. این همچنین فرض می‌کند که هر تیم حوزه مهارت‌های لازم برای ساخت محصولات داده قوی را دارد یا می‌تواند این مهارت‌ها را به دست آورد.

این همیشه درست نیست. به طور شگفت‌انگیز، من مردم مرکزی IT را با دانش بیشتری در حوزه مورد نظر تر از کارشناسان حوزه مشاهده کردم! افراد IT همچنین می‌فهمند که چگونه داده‌ها را از حوزه‌های مختلف ترکیب کنند. گاهی اوقات افرادی که سیستم‌های عملیاتی را اجرا می‌کنند، درک کمی از داده‌های خود دارند، به خصوص از دیدگاه تحلیلی؛ آن‌ها فقط به دنبال نگهداری روشنایی هستند. اگر مهندسان داده IT مرکزی دانش حوزه مورد نیاز خود را ندارند، چرا آن‌ها را برای به دست آوردن این دانش و بهبود ارتباط بین IT و حوزه‌ها ترغیب نکنیم؟ این می‌تواند یک راه حل بهتر باشد تا روش کار جدیدی را معرفی کنیم.

پیشگویان داده‌مشهور می‌گویند که به جای داشتن کارشناسان داده با تخصص عمیق در یک زمینه، داده‌مشهور ترغیب به عمومی‌سازی می‌کند. هیچ کس در یک گام جداگانه کار نمی‌کند. داده‌مشهور تیم‌های متقاطع از افرادی را تصور می‌کند که مهارت‌های میانی در بسیاری از زمینه‌ها دارند: به عنوان مثال، جریان، پردازش دسته‌ای ETL، طراحی انبار داده، و تصویرسازی داده، همراه با روش‌های مهندسی نرم‌افزاری مانند کدنویسی، تست، و تحویل مداوم (DataOps). فکر کنید "یک کارگر همه چیزها، ماستر هیچ چیز". دوباره، اگر شما تعداد زیادی از حوزه‌ها دارید، این به معنای تعداد زیادی از نقش‌هایی است که باید پر کنید و حقوق برای آن‌ها پرداخت کنید. پیدا کردن و استخدام افراد با چنین مجموعه ای از مهارت‌ها سخت است، حتی بیشتر از افراد. مناسب بودن این افراد چندمهارتی نسبت به افرادی که سال‌ها تخصص عمیق در یک زمینه دارند، مورد بحث است.

موانع سطح دامنه

فهمیده باشید که دامنه شما در نهایت اجازه داده شده است تا تیم IT مرکزی ویژگی‌های حیاتی که برای داده‌های تحلیلی دامنه نیاز دارد را اضافه کند تا از داده‌های خود ارزش واقعی ببرید. شما با تیم IT مرکزی یک برنامه را برای ایجاد تحلیل‌ها در چند ماه آینده طراحی کرده‌اید. سپس متوجه می‌شوید که شرکت قصد دارد یک شبکه داده ایجاد کند و برای ایجاد ویژگی‌های مورد نیاز و APIهای مرتبط با آن‌ها، باید یک تیم کاملاً جدید استخدام کنید. سپس متوجه می‌شوید که باید پروژه را برای چند ماه تاخیر دهید در حالی که شرکت اصول #۳ و #٤ را پیاده‌سازی می‌کند، دامنه‌های داده را طراحی می‌کند و قراردادی را می‌نویسد که به روزرسانی هر دامنه را راهنمایی می‌کند.

چگونه احساس می‌کنید؟ و اگر افراد در دهها دامنه به همین ترتیب احساس می‌کنند؟ شرکت نمی‌تواند دامنه‌ای بی‌قرار داشته باشد؛ همچنین مصرف‌کنندگان و سایر دامنه‌ها به داشتن داده‌های موجود از هر دامنه وابسته‌اند، بنابراین حیاتی است که هر دامنه در پروژه شبکه داده مشارکت کند. شما باید:

- انگیزه‌های جذاب را برای جبران کار اضافی، آسیب‌پذیری و تاخیر ارائه دهید.

- کافی نیست تا انگیزه بگویید که آن‌ها را برای "بهره‌ی بزرگتر" انجام می‌دهند.

- مشارکت همه دامنه‌ها در پیاده‌سازی یک شبکه داده را بدست آورید.

- حمایت از سطح اجرایی بالا را بدست آورید.

- یک دیدگاه واضح را ارائه دهید که مزایایی را برای هر دامنه نشان دهد.

- برنامه‌های آزمایشی را برای نشان دادن کارایی رویکرد جدید اجرا کنید.

- انگیزه‌ها، هم مالی و هم غیرمالی، را برای تشویق مشارکت ارائه دهید.

- کانال‌های شفاف برای ارتباط مداوم برقرار کنید.

- از طریق مراکز تخصصی یا مراکز برتر پشتیبانی ارائه دهید.

برنامه ریزی کنید که اگر یک دامنه تمایل نداشته باشد قوانین را رعایت کند، چه کاری انجام دهید. این ممکن است برای استانداردهای امنیت داده و کیفیت داده بسیار مشکل‌ساز باشد. اگر تصمیم بگیرید که دامنه‌هایی که قوانین را رعایت نمی‌کنند نمی‌توانند به شبکه داده بپیوندند، شما سیلوهای داده خواهید داشت. همچنین ممکن است دامنه‌هایی وجود داشته باشد که نمی‌توانند به شبکه داده بپیوندند، حتی اگر تمایل داشته باشند. شاید آن‌ها بودجه‌ای برای ساخت تیمی برای ایجاد یک پلتفرم تحلیلی نداشته باشند، یا شرایط آن‌ها را مجبور کند تا از فناوری قدیمی‌تری استفاده کنند که نمی‌تواند پلتفرم تحلیلی را پشتیبانی کند. برای ادغام داده‌های آن‌ها در شبکه داده، شما ممکن است نیاز به یک معماری متمرکز کننده کنار شبکه داده داشته باشید.

به اشتراک گذاری محصولات موثر برای موفقیت شبکه داده شما حیاتی است. به اشتراک گذاری محصولات درون و خارج هر دامنه در یک شبکه داده بسیار متفاوت از به اشتراک گذاری داده است. شما نیاز به یک راه برای دامنه‌ها یا مصرف‌کنندگان دیگر دارید تا تمام دامنه‌های شرکت خود و محصولات داده آن‌ها را پیگیری کنند. این نیازمند

کاتالوگ‌بندی تمام متادیتای دامنه در جایی با یک مکانیزم جستجو و روشی برای درخواست دسترسی است: یک نوع بازار داده، همانطور که در فصل ٦ توضیح داده شده است.

ارائه دهندگان اصلی ابر بسیاری از راه حل‌های نرم‌افزاری برای به اشتراک گذاری داده ارائه می‌دهند، اما بسیار کمی برای به اشتراک گذاری محصولات داده و این همه تازه ارائه شده‌اند. شما باید تصمیم بگیرید: آیا می‌خواهید یکی را خریداری کنید یا خودتان یکی ایجاد کنید؟

یک شبکه داده منجر به تغییر نسبت به یک مدل خدمات خودمصرف برای درخواست‌های داده می‌شود، که چالش‌هایی را ایجاد می‌کند. سنتیاً، دامنه‌ها بر روی دپارتمان‌های مرکزی IT برای مدیریت تمام نیازهای تحلیلی خود اعتماد داشته‌اند، که شامل همه چیز از مدیریت داده تا پاسخگویی به درخواست‌های خاص مانند پرس و جوها و گزارشات بود. با شبکه داده، این مسئولیت به دامنه‌های فردی منتقل می‌شود و بر عهده آن‌ها قرار می‌گیرد تا این درخواست‌های داده را برآورده کنند. خطر احتمالی این است که برخی از دامنه‌ها ممکن است دانش لازم برای مدیریت این وظایف به طور کارآمد را نداشته باشند. علاوه بر این، آن‌ها نمی‌توانند فقط راه حل‌های پیاده‌سازی شده توسط IT مرکزی را تکرار کنند، باید آن‌ها را بهبود بخشند تا ارزش یک شبکه داده را نشان دهند. هنگامی که آن‌ها راه حل‌های منحصر به فرد خود را طراحی می‌کنند، احتمال وجود این است که این راه حل‌ها حتی به اندازه کارآمد یا قوی تر از سیستم‌های متمرکز شده که آن‌ها را جایگزین می‌کنند، نباشند.

ارزیابی سازمانی: آیا باید یک Data Mesh را بپذیرید؟

با این حال، شما فهمیده‌اید که چیست یک شبکه داده و مشکلات پتانسیلی که با آن همراه است. اگر هنوز احساس می‌کنید که یک شبکه داده گزینه‌ای است، مرحله بعدی انجام یک ارزیابی جامع است تا ببینید آیا سازمان شما آماده ساختن یک شبکه داده موفق است یا خیر.

شما می‌توانید از معیارهای موجود در جدول ١٤-١ استفاده کنید، که از کتاب Dehghani گرفته شده‌اند. Dehghani توصیه می‌کند که شرکت‌ها فقط در صورتی که تمام معیارها به اندازه‌ای قابل اجرا یا بسیار قابل اجرا باشند، به ساخت یک شبکه داده فکر کنند.

طراحی و پیاده‌سازی از معماری داده

Table 14-1. Self-assessment of your company's readiness for data mesh

Criterion	Rating (select one)		
We have too much organizational complexity for our current system and our existing data architecture has become a blocker; it can't scale quickly enough to get value from data from many parts of the business.	Not applicable	Somewhat applicable	Highly applicable
Our company has a data-oriented strategy and a strategic vision for using and getting value out of data.	Not applicable	Somewhat applicable	Highly applicable
Our executives support the culture and work changes needed for a data mesh and are committed to motivating the entire organization to change how people work.	Not applicable	Somewhat applicable	Highly applicable
We see data as a valuable asset and regularly collect it, analyze it, and use it to inform business decisions. We have built our operations, decision-making processes, culture, and strategies around data. We invest heavily in a data technology infrastructure, analytics platforms, and machine learning. Our employees are data literate.	Not applicable	Somewhat applicable	Highly applicable
Our company likes to be on the "bleeding edge" and is always one of the first to adopt and implement new technologies and innovations, even when it involves some risk.	Not applicable	Somewhat applicable	Highly applicable
Our company employs modern engineering practices and technologies, including data-driven decision making and user-centered design, that enable many smaller teams instead of one large centralized team.	Not applicable	Somewhat applicable	Highly applicable
Our organization is structured around specific business areas or data domains, each with dedicated technical teams who build and support the products that serve that domain. Every domain has (or will soon have) a budget for creating analytical data from its operational data and for building its own data mesh infrastructure.	Not applicable	Somewhat applicable	Highly applicable
Even if it takes a long time, our company is committed to using machine learning and analytics to gain a competitive advantage over our competitors. We are committed to a process of culture change and to designing and building the technology necessary for data mesh, as well as a network of domain-oriented teams.	Not applicable	Somewhat applicable	Highly applicable

واقعیت بر اساس تجربه من، این است که فقط یک درصد کمی از شرکت‌ها— شاید ۱ درصد— تمام این دسته‌بندی‌ها را به اندازه‌ای قابل اجرا یا بسیار قابل اجرا می‌یابند. امتیازات بالا در دسته‌بندی‌های مبتدیان، متمرکز بر دامنه و مهندسی مدرن به خصوص نادر هستند.

توصیه هایی برای پیاده سازی یک مش داده های موفق

اگر شما یک راه حل داده فعلی دارید و تصمیم بگیرید یک شبکه داده بسازید، من توصیه می‌کنم که یک مدل هوب و اسپوک را استفاده کنید، در که شبکه داده را به عنوان یک گسترش از یک راه حل داده متمرکز ساخته می‌شود (شکل ۱-۱۴). شما ممکن است با استفاده از داده‌های جدید دامنه‌های جدیدی برای شبکه داده ایجاد کنید، اما این دامنه‌ها را با استفاده از راه حل داده متمرکز فعلی خود تکمیل کنید که با نیازهای کسب و کار هماهنگ است. به تدریج داده‌های متمرکز خود را در دامنه‌های شبکه داده منتقل کنید هر زمان که دامنه‌های جدیدی از داده‌های جدید اضافه می‌کنید— نتوانید تمام داده‌های متمرکز خود را در یک حالت ناگهانی به دامنه‌های شبکه داده تبدیل کنید.

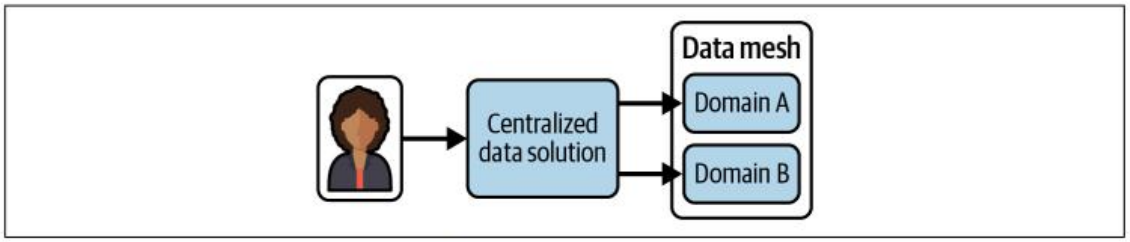


Figure 14-1. A hub-and-spoke model, with a data mesh supplementing and extending a centralized data solution

پیاده‌سازی شبکه داده به تدریج به شما کمک می‌کند تا با ریسک‌های مرتبط با یک بازسازی کامل از معماری داده موجود خود مدیریت کنید. اگر در یک بخش از شبکه خرابی وجود داشته باشد، آن محدود می‌شود و تاثیری بر سیستم کلی ندارد. شما کنترل بهتری بر روی اقدامات مطابقت و امنیت دارید و زمان برای برقراری پروتکل‌های مناسب مدیریت و امنیت دارید. این رویکرد به تدریج همچنین هزینه‌ها و نیازهای نیروی کار را توزیع می‌کند. فرآیندهای کسب و کار موجودی که بر روی داده‌های متمرکز اعتماد دارند، می‌توانند در طول انتقال به طور صاف عمل کنند، کاهش مخالفت با کسب و کار را حداقل می‌کند.

آنچه من پیشنهاد می‌کنم، یک رویکرد تکراری است. پیاده‌سازی دامنه‌های جدید به تدریج به شما اجازه می‌دهد تا از هر مرحله یاد بگیرید، تنظیمات لازم را انجام دهید و این درس‌ها را به دامنه‌های بعدی اعمال کنید، سیستمی قوی‌تر در طول زمان ایجاد می‌کند. علاوه بر این، به شما اجازه می‌دهد تا سرمایه‌گذاری فناوری موجود خود را در حال حرکت به ساختارهای مدرن‌تر استفاده کنید.

این رویکرد همچنین یک رویکرد انعطاف‌پذیر است. به شما انعطاف‌پذیری می‌دهد تا برخی از دامنه‌ها را که ارزش فوری بیشتری ارائه می‌دهند یا با اهداف کسب و کار استراتژیک همخوانی دارند، اولویت بدهید. این امکان می‌دهد تا با نیازهای کسب و کار همخوانی کنید به جای یک انتقال مجبور در یک زمان. شما همچنین فرصت دارید تا عملکرد هر دامنه را بهینه‌سازی کنید و آن را به نیازها و محدودیت‌های خاص دامنه تنظیم کنید.

در نهایت، رویکرد هوب و اسپوک راهی برای تغییر فرهنگی در طول زمان فراهم می‌کند، که تشویق به همکاری متقاطع و متمرکز بر دامنه بین مالکان محصولات داده، مهندسان و ذی‌نفعان کسب و کار می‌کند.

پیاده‌سازی شبکه داده به قطعات یک رویکرد استراتژیک است که اجازه می‌دهد تا با دقت برنامه‌ریزی، مدیریت ریسک و هماهنگی با اهداف کسب و کار انجام شود. این از یک معماری داده متمرکز به یک معماری متمرکز شده انتقال را پشتیبانی می‌کند، استفاده هم از سیستم‌های قدیمی و هم از سیستم‌های جدید در دوره انتقال انجام می‌دهد. این رویکرد انعطاف‌پذیرتر و مقاوم‌تر از تلاش برای انتقال کامل در یک زمان است، که با ریسک و پیچیدگی مواجه است.

آینده دیتا مش

با بسیاری از مردم صحبت کردم که باور دارند که ایده یک تیم مرکزی که تمام معماری داده را می‌سازد، در حال مرگ است و در نهایت هر شرکت به یک مدل متمرکز شده منتقل خواهد شد. به هر حال، مدل‌های متمرکز خوب نمی‌شوند.

این موضوع در زمان مناقشات Kimball در مقابل Inmon (مشاهده کنید کتاب ۸) مطرح شد. در کمپ Inmon، IT همه چیز را متمرکز کرد؛ اما در کمپ Kimball، روش ساخت مارت‌های داده متمرکز به طور آزاد بود و آن‌ها را با استفاده از بُعدهای مطابق و معماری باس با یکدیگر به هم متصل می‌کردند. در عمل، با این حال، شما دامنه‌های زیادی داشتید که مارت‌های داده خود را در جدایی با استفاده از فناوری‌های مختلف ساختند، که منجر به یک معماری متمرکز شده شد. وقتی نیاز بود که افراد داده‌ها را از چندین دامنه با هم ترکیب و تجمع کنند، مارت‌های داده اضافی و/یا کوب‌های OLAP ایجاد کردند.

گاهی اوقات این استراتژی خوب کار می‌کرد، اما اغلب منجر به تلاش‌های تکراری شد و استفاده از تکنولوژی‌ها و محصولات مختلف. این نیز دشوار بود و معمولاً هزینه‌ها را به سمت بیش از حد منتقل می‌کرد. سپس شرکت به مدل متمرکز بازگشت می‌کرد تا این مشکلات را جلوگیری کند.

آیا شبکه داده منجر به همان چرخه‌ای از مشکلات خواهد شد؟ چه چیزی این بار متفاوت است؟ با وجود پیشرفت‌های فناوری از زمان مارت‌های داده، مشکل واقعاً یک مسئله فناوری نیست - این یک مسئله انسانی و فرآیند است و ما تا حالا به حد زیادی در حل این مسائل پیشرفت نکرده‌ایم. هر گونه کلمات جدید و روش‌های خنده‌داری که دوست دارید بدانید، اما تغییر دادن افراد بسیار سخت است - و شبکه داده نیاز به تغییر زیادی دارد.

من پیش‌بینی می‌کنم که بسیار کمی از شرکت‌ها معماری‌های شبکه داده را پذیرفته خواهند شد و من باور ندارم که هیچ یک از این راه‌حل‌ها یک شبکه داده خالص باشد. برای کسانی که آن را شبکه داده می‌نامند، تخمین می‌کنم که بیشتر - شاید حدود ۹۰ درصد - مالکیت دامنه (اصل #۱) را پذیرفته خواهند شد، در حالی که کمتر - شاید حدود ۷۰ درصد - داده را به عنوان یک محصول پذیرفته خواهند شد (اصل #۲). شاید یک درصد بسیار کم از آن‌ها آن را به سطح یک کوانتوم داده ببرند. حدود یک سوم شرکت‌ها از برخی از فرم‌های زیرساخت خودکار برای هر دامنه استفاده خواهند کرد (اصل #۳: زیرساخت داده Self Service به عنوان یک پلتفرم)، و شاید نیمی از آن‌ها قرار بگیرند که حکومت محاسباتی متحد را پذیرفته باشند (اصل #٤). من انتظار دارم که یک درصد بسیار زیادی از راه‌حل‌ها محدودیت‌هایی را بر روی فناوری‌هایی که هر دامنه می‌تواند استفاده کند، قرار دهد.

به جای تلاش برای پیاده‌سازی هر عنصری از شبکه داده، من توصیه می‌کنم که بر روی چگونگی توانایی تیم‌های داده شما برای ارائه ارزش بیشتر و با دوره‌های بیشتر برای مشتریان خود تمرکز کنید. از آنجا که شروع کنید، می‌توانید به سمت عقب حرکت کنید تا عناصر خاصی از شبکه داده را شناسایی و پذیرفته شود که به شما کمک می‌کند تا این هدف را دست یابید.

در نهایت، یک شبکه داده یک مبارزه بین متمرکز شدن و متمرکز نشدن است. همیشه ترکیبی از هر دو در هر راه حل شبکه داده وجود خواهد داشت. مسئله در یافتن تعادل مناسب برای مورد استفاده شما است.

بزرگنمایی: درک معماری داده ها و کاربردهای آنها

در دنیای مدرن پیشرفت فناوری سریع، کسب و کارها در یک میانگین قرار دارند، احاطه شده توسط یک منابع غنی از داده‌ها. این داده‌ها، اگر به درستی مدیریت شوند، بازهای نظریه‌ای را باز می‌کنند، عملیات را بهینه‌سازی می‌کنند و یک مزیت رقابتی فراهم می‌کنند. کلید برای باز کردن این پتانسیل در معماری است - چارچوبی که تصمیم می‌گیرد که چگونه داده‌ها ذخیره شده، پردازش شده و دسترسی پیدا خواهند کرد.

در طول این کتاب، به عمق در پیچیدگی‌های مختلف معماری‌های داده، تکامل آن‌ها و کاربرد عملی آن‌ها پرداخته‌ام. حالا که در حال آماده شدن برای بستن این فصل هستم، مهم است که یک قدم برگشته باشید و منظره را از نقطه نمایش بالاتر ببینید.

شما بسیاری از انتخاب‌ها دارید. چگونه می‌توانید بدانید کدام یک برای سازمان شما مناسب است؟ برای پاسخ به این موضوع، بیایید به برخی از موارد استفاده بالاتر برای هر معماری نگاه کنیم، که بر اساس هزینه و پیچیدگی دسته‌بندی شده‌اند:

مخزن داده مدرن (MDW)

MDW برای کسب و کارهایی که مقدار کمی از داده را مدیریت می‌کنند و به مخازن داده رابطه‌ای عادت دارند، مناسب ترین گزینه است، که برای یک انتقال ملایم است.

پارچه داده (Data fabric)

هرچه کسب و کارها رشد و تنوع پیدا می‌کنند، اغلب با داده‌های متنوع از منابع مختلف روبرو می‌شوند، هر کدام در اندازه، سرعت و نوع متفاوت هستند. پارچه داده گزینه معماری برای کسانی است که نیاز به یک راه بی‌درز برای یکپارچگی این جریان‌های داده متنوع دارند.

خانه دیتا لاک (Data lakehouse)

این می‌تواند به عنوان یک راه حل میانه تصور شود. شما باید با یک خانه دیتا لاک به عنوان راه حل اصلی خود روبرو شوید تا به محدودیت‌های آن برخورد کنید.

خلاصه - رمز گشایی از معماری داده

شبکه داده (Data mesh)

برای شرکت‌های بزرگ و متمرکز بر دامنه، شبکه داده برای سازمان‌هایی است که با چالش‌های مقیاس‌پذیری بزرگ مواجه هستند و منابع و زمان لازم برای سرمایه‌گذاری در یک راه حل قوی دارند.

این معماری‌ها به طور متقابل انتخاب نیستند. دنیای واقعی نادر از زمان با مقادیر مطلق مواجه می‌شود. بیشتر سازمان‌ها متوجه خواهند شد که نیازهای منحصر به فرد آن‌ها نیاز به یک مجموعه از ویژگی‌ها از چندین معماری را دارد. چنین راه حل سفارشی می‌تواند جنبه‌های هر یا همه از این معماری‌ها را ترکیب کند، که به موارد استفاده خاص داده سازمان و توانایی‌های کسب و کار در حال تکامل خود تنظیم شده است.

خلاصه

این فصل به بررسی ایده‌های غلط در مورد شبکه داده پرداخت، نگرانی‌های معتبر را توضیح داد و مناسب بودن آن در ساختارهای سازمانی مختلف را ارزیابی کرد. من توصیه‌های کلیدی برای پیاده‌سازی موفق ارائه دادم، که بر اساس روش‌های عملی و مطالعات موردی استوار است. ما آینده شبکه داده را در زمینه فناوری‌ها و روندهای کسب و کار در حال تکامل بررسی کردیم. در نهایت، من تمام معماری‌های داده را خلاصه کردم تا به شما در انتخاب معماری داده مناسب برای نیازهای کسب و کار خاص شما کمک کند. امیدوارم که حالا هم درک نظری و بینش‌های عملی داشته باشید که به شما اجازه می‌دهد تصمیمات آگاهانه در مورد پذیرش شبکه داده بگیرید.

آینده کسب و کار به طور ناپذیر با آینده داده مرتبط است. همانطور که پیش می‌رویم، معماری‌هایی که انتخاب می‌کنیم نه تنها چگونگی ذخیره و دسترسی به داده‌های ما را تعیین می‌کنند بلکه چگونگی رشد، نوآوری و رقابت کسب و کارهای ما را نیز تعیین می‌کنند. انتخاب این گزینه باید با درک عمیق، قابلیت تطبیق و دیدگاهی برای آینده انجام شود. به یاد داشته باشید، معماری که امروز انتخاب می‌کنید مسیری را برای کسب و کار شما در آینده به شما می‌سازد. به درستی انتخاب کنید و به داده‌های خود اجازه دهید راه را برای شما باز کنند.

فصل‌های قبلی این کتاب به جنبه‌های فناوری معماری‌های داده پرداختند. در فصل‌های ۱۵ و ۱۶، ما موضوعات حیاتی در مورد انسان‌ها و فرآیندها را بررسی خواهیم کرد، با بررسی سازماندهی تیم، علل شکست پروژه‌ها و کلیدهای موفقیت پروژه.

بخش سوم: نفرات، فرآیندها و تکنولوژی

تبریک می‌گویم! شما از بخش‌های پایه، مفاهیم معماری داده رایج و چهار معماری داده عبور کرده‌اید. اما هنوز تمام نشده‌اید.

فصل ۱۵ به عوامل بزرگترین تعیین‌کننده موفقیت هر راه حلی که ساخته می‌شود می‌پردازد: انسان‌ها و فرآیندها. فصل ۱۶ به فناوری‌های معماری داده می‌پردازد. برای اطمینان از اینکه برای سال‌ها آینده مرتبط باقی بماند، این بحث از دیدگاه‌های بی‌نسبت به فروشنده، تاریخی و سطح بالا پیروی می‌کند.

فصل پانزدهم: نفرات و فرایندها

در دنیای پیچیده معماری‌های داده، در حالی که فناوری و ابزارها ممکن است برجسته باشند، انسان‌ها و فرآیندها قلب و روح آن را تشکیل می‌دهند. همانطور که سازمان‌ها از مدل‌های داده سنتی به ساختارهای نوآورانه مانند MDW، پارچه داده، یا خانه دیتا لاک و حتی به سمت پارادیگم‌های متمرکز شده مانند شبکه داده می‌روند، عنصر انسانی همچنان مرکزی است. اما چرا؟

در سال‌های زیادی که در ساخت مخازن داده، برنامه‌ها و پروژه‌های گزارش کار کرده‌ام، من می‌بینم که چه چیزی به موفقیت پروژه‌ها کمک می‌کند و چه چیزی آن‌ها را می‌بیند. تصور کنید که یک شهربانی معماری را ساخته‌اید - یک برج با امکانات پیشرفته. طرح آن بدون خطا است، مواد اولیه برتر هستند. اما اگر سازندگان، مهندسان و مدیران در دیدگاه خود هماهنگ نباشند، یا اگر فرآیندها وجود نداشته باشند تا اقدامات هماهنگ را تضمین کنند، پروژه می‌تواند قبل از شروع شکست بخورد.

به طور مشابه، تبدیل یک معماری داده نیازمند نیست فقط بازنگری فناوری بلکه تغییرات عمیق در فرهنگ سازمانی است. تغییر در نحوه مدیریت، پردازش و ارزش‌دهی داده به معنای تنظیم مجدد نحوه فکر کردن و تعامل تیم‌ها با داده است. به معنای چالش برانده کردن استانداردهای موجود و پذیرش روش‌های جدید است. چنین انتقالاتی هرگز فقط فنی نیستند؛ آن‌ها عمیقاً انسانی هستند.

مرکزی که به این عنصر انسانی اختصاص دارد، تعریف نقش‌ها است. کی چه کاری انجام می‌دهد؟ کی مسئولیت تأمین این است که داده به طور صاف جریان یابد، به طور صحیح پردازش شود و به طور اخلاق و کارآمد استفاده شود؟ اگر این نقش‌ها را نادرست درک یا کمتر ارزیابی کنید، شما در حال حاضر در اعتماد به ناکارآمدی یا بدتر، شکست پروژه هستید. برعکس، دریافت نقش‌ها درست - اطمینان از اینکه هر عضو تیم درک مسئولیت‌های خود را دارد و منابعی برای برآورده کردن آن‌ها دارد - مانند قرار دادن سنگ‌های پایه‌ای محکم برای شهربانی معماری شما است.

با این حال، مانند هر پیشاندازی بزرگ، پروژه‌های داده دارای داستان‌های موفقیت و داستان‌های هشدار هستند. خط بین یک انتقال داده موفق و یک راه حل شکسته می‌تواند اغلب به نحوه مدیریت انسان‌ها و طراحی فرآیندها برگردد. یادگیری از هر دو پیروزی‌ها و معیارها باعث می‌شود تا بینش‌های بی‌ارزشی در مورد چالش‌ها و پیچیدگی‌های تبدیل معماری داده ارائه دهد.

هنگامی که به این فصل عبور می‌کنیم، ما پیچیدگی‌های سازماندهی تیم، نوانگی‌های نقش‌ها در چارچوب‌های داده مختلف و عوامل حیاتی که تفاوت بین موفقیت و شکست پروژه‌های داده را مشخص می‌کند را بررسی خواهیم کرد. در پایان این سفر، شما تشخیص خواهید داد که در حالی که ابزار و فناوری‌ها تکامل می‌یابند، انسان‌ها و فرآیندها تعیین می‌کنند که آیا برای موفقیت استفاده می‌شوند یا به معادن فرصت‌های گم شده منتقل می‌شوند.

سازمان تیم: نقش ها و مسئولیت ها

ساخت یک معماری داده نیازمند یک تیم (یا چندین تیم) با مجموعه‌ای از مهارت‌ها و نقش‌ها است. این بخش برخی از نقش‌های کلیدی، مسئولیت‌های آن‌ها و چگونگی همکاری آن‌ها برای ساخت یک مخزن داده مدرن، پارچه داده یا خانه دیتا لاک را شرح می‌دهد. (برای شبکه داده، نقش‌های اضافی نیاز دارید؛ در بخش بعدی در مورد آن‌ها صحبت خواهم کرد.) در شرکت‌های کوچکتر، یک شخص ممکن است چندین نقش را بازی کند؛ به عنوان مثال، همان شخص ممکن است هم مدیر حفظ داده و هم مدیر کیفیت داده باشد. در شرکت‌های بزرگتر، چندین شخص ممکن است نقش یکسانی را بازی کنند؛ به عنوان مثال، شما ممکن است دوزان مهندس داده داشته باشید.

نقش های MDW، Data Fabric یا Data Lake House

بیایید ابتدا به نقش‌های کلی نگاه کنیم:

- **معمار داده (Data Architect)**: ساختار سطح بالای معماری داده (MDW، پارچه داده یا خانه دیتا لاک) را طراحی می‌کند و تصمیم می‌گیرد که چه فناوری‌ها و سیاست‌های حفظ داده پروژه باید استفاده کند.

- **مهندس داده (Data Engineer)**: مسئول ساخت، تست و نگهداری معماری داده است. او اسکریپت‌هایی را برای استخراج، بارگذاری و تبدیل داده (ELT) از منابع مختلف به راه حل داده می‌نویسد و با معمار داده برای پیاده‌سازی معماری طراحی شده کار می‌کند.

- **مربی داده (Data Steward)**: مسئول کیفیت و حفظ داده است. او قوانین و استانداردهایی را برای استفاده از داده تعریف و پیاده‌سازی می‌کند، نظارت بر تمیز کردن داده را انجام می‌دهد، حقوق دسترسی به داده را مدیریت می‌کند و با کاربران برای حل هر مشکل کیفیت یا دسترسی به داده کار می‌کند.

- **مدیر پایگاه داده (DBA - Database Administrator)**: مسئول عملیات، نگهداری و عملکرد سیستم داده است. او برنامه‌های پشتیبان‌گیری و بازیابی را پیاده‌سازی می‌کند، پرس و جوها را برای عملکرد بهینه‌سازی می‌کند و سیستم را به طور صاف اجرا می‌کند.

- **تحلیلگر داده (Data Analyst)**: داده‌ها را جمع‌آوری، پردازش و تحلیل توصیفی و تشخیصی روی مجموعه‌های داده بزرگ انجام می‌دهد، الگوها و روندها را شناسایی می‌کند، این یافته‌های پیچیده را به دیدگاه‌های قابل فهم تبدیل می‌کند و آن‌ها را به ذی‌نفعان ارتباط می‌دهد. او گزارشات جامع و داشبوردها را برای ردیابی معیارهای کسب و کار ایجاد می‌کند.

- **دانشمند داده (Data Scientist)**: از تکنیک‌های آماری پیشرفته، مدل‌سازی پیش‌بینی و یادگیری ماشین برای استخراج بینش‌های ارزشمند از داده و حل مسائل پیچیده استفاده می‌کند. او مدل‌های یادگیری ماشین را طراحی و پیاده‌سازی می‌کند، تحلیل پیش‌بینی و توصیه‌گرا را انجام می‌دهد، آزمایش‌های A/B را انجام می‌دهد و داده‌ها را استخراج می‌کند. او همچنین به ارائه بینش‌های مبتنی بر داده و توصیه‌ها کمک می‌کند تا تصمیمات استراتژیک و عملیاتی را تأثیر بگذارد.

- **تحلیلگر کسب و کار (Business Analyst)**: بین تیم داده و کاربران کسب و کار ارتباط می‌کند و به کمک آن‌ها نیازمندی‌های کسب و کار برای معماری داده را تعریف می‌کند، سپس آن‌ها را به نیازمندی‌های فنی ترجمه می‌کند. او همچنین بینش‌ها و تحلیل‌های تحلیلگران داده و دانشمندان را تأیید می‌کند و آن‌ها را به کاربران کسب و کار ارتباط می‌دهد.

- **مدیر پروژه/Master Scrum (Project Manager/Scrum Master)**: کار تیم را هماهنگ می‌کند، اطمینان می‌آورد که آن‌ها بر روی رسیدن به اهداف خود در حال پیشرفت هستند و هر موانعی را حل می‌کند. او همچنین وضعیت پروژه را به ذی‌نفعان ارتباط می‌دهد و هر خطرات را مدیریت می‌کند.

- **مربی حفظ حریم خصوصی داده (Data Privacy Officer)**: مطمئن می‌شود که با قوانین حفظ حریم خصوصی داده مطابقت دارد، با مربی داده کار می‌کند تا سیاست‌های استفاده را تنظیم کند و به هر حادثه حفظ حریم خصوصی داده پاسخ می‌دهد. در برخی از بخش‌ها یا مناطق، قوانین این نقش را مجبور می‌کنند.

- **مربی حفظ داده (Data Governance Manager)**: مطمئن می‌شود که داده‌های سازمان در دسترس، قابل استفاده، یکپارچگی و امن هستند و سیاست‌ها و روش‌های مدیریت داده را طراحی و اجرا می‌کند تا بهترین روش‌ها را در تمام واحدهای کسب و کار تضمین کند.

- **مربی کیفیت داده (Data Quality Manager)**: کامل بودن، یکپارچگی و دقت داده را نظارت می‌کند، از جمله پیاده‌سازی قوانین کیفیت داده، نظارت بر معیارهای کیفیت داده و همکاری با تیم‌های دیگر برای حل هر مشکل کیفیت داده انجام می‌دهد.

نقش های دیتا مش

همانطور که در فصل‌های ۱۳ و ۱٤ یاد گرفتید، رویکرد شبکه داده (Data Mesh) داده را به عنوان یک دارایی مشترک در میان دامنه‌های مختلف در یک سازمان می‌بینند. به جای داشتن یک تیم داده مرکزی که همه جنبه‌های داده را مدیریت می‌کند، رویکرد شبکه داده مسئولیت‌ها را به تیم‌های دامنه‌ای متقاطع توزیع می‌کند. هر تیم مسئول داده‌ای است که تولید می‌کند، که شامل ارائه داده به عنوان یک محصول، تضمین کیفیت آن و اطمینان از دسترسی به آن برای دیگران است. این تغییرات می‌تواند تیم‌ها، نقش‌ها و مسئولیت‌ها را به طور قابل توجهی تغییر دهد. بیایید به تفکیک تیم به تیم بپردازیم:

تیم‌های دامنه

هر دامنه یک تیم دامنه دارد. اعضای آن مسئول ساخت راه حل تحلیلی دامنه خود و پیاده‌سازی سیاست‌های حفظ و اصول شبکه داده (که در فصل ۱۳ توضیح داده شده است) هستند. بیشتر نقش‌های موجود در تیم دامنه، مانند مهندسان داده دامنه، تحلیلگران داده و مربیان داده دامنه، نقش‌های مشابهی با نقش‌های فراهم شده برای MDW، پارچه داده یا خانه دیتا لاک دارند، به جز اینکه دامنه خاص آن‌ها نسبت به سازمان کلی است. از آنجا که هر دامنه خود راه حل خاص دارد، بیشتر، اگر نه همه این نقش‌ها در هر دامنه وجود دارد. یک نقش اضافی مربوط به شبکه داده نقش مالک محصول داده است.

یک مالک محصول داده دامنه مسئولیت داده‌ای را که دامنه آن تولید می‌کند بر عهده می‌گیرد. این شامل مدیریت چرخه عمر آن، تضمین کیفیت آن، اطمینان از دسترسی به آن و هماهنگ آن با نیازهای کسب و کار است. مالک محصول باید هر دو جنبه فنی و کسب و کاری داده را درک کند.

تیم پلتفرم زیرساخت ساختار داده

یک تیم پلتفرم که بر روی زیر سروی ساختار داده تمرکز دارد، با هماهنگی با اصل شبکه داده #۳، می‌تواند به تیم‌های دامنه زیرساخت فناوری، ابزارها و راهنمایی‌هایی را برای ساخت و مدیریت محصولات داده ارائه دهد. نقش‌های موجود در این تیم عبارتند از:

- معمار شبکه داده (Data Mesh Architect)

- معماری کلی ساختار و راهنمایی‌ها را طراحی می‌کند، تلاش‌های تیم‌های دامنه را هماهنگ می‌کند و ارتباط و یادگیری را تسهیل می‌کند. در حالی که هر تیم دامنه داده خود را مدیریت می‌کند، این نقش معمار اطمینان می‌دهد که کلیت شبکه داده به طور کلی هماهنگ باشد.

- معمار پلتفرم (Platform Architect)

- معماری کلی پلتفرم داده را طراحی می‌کند تا نیازهای دامنه‌ها را پشتیبانی کند و انواع مختلف داده‌ها، بارهای کار و ابزارهای تحلیلی را در بر بگیرد.

مهندسان/توسعه‌دهندگان پلتفرم

ساخت و نگهداری زیرساخت پلتفرم، از جمله پایگاه‌های داده، APIها، هماهنگی کانتینرها و جنبه‌های دیگر را انجام می‌دهند.

مهندس داده

بر روی نقل‌هایی که داده را در پلتفرم حرکت و تبدیل می‌دهند، کار می‌کند تا اطمینان حاصل شود که داده تمیز، یکپارچگی و برای همه کسانی که نیاز دارند در دسترس است.

مهندس DevOps

اتوماسیون نصب، مقیاس‌بندی و مدیریت پلتفرم را انجام می‌دهد، تا اطمینان حاصل شود که پلتفرم پایدار و قابل اعتماد است و می‌تواند نیازهای تیم‌های داده را برطرف کند.

مهندس DataOps

خلاصه - درختایی از معماری داده

چرخه عمر داده و مدل‌های علم داده را از ابتدا تا انتها مدیریت و بهینه‌سازی می‌کند، از جمله با نظارت بر کیفیت داده و اطمینان از اینکه نیازهای حفظ حریم خصوصی و مطابقت با مقررات رعایت می‌شوند.

مربی امنیت و مطابقت

اطمینان حاصل می‌کند که پلتفرم با تمام مقررات و بهترین روش‌های لازم برای امنیت و حفظ حریم خصوصی داده مطابقت دارد.

مالک/مدیر محصول

به عنوان پیوند بین تیم فنی و ذی‌نفعان کسب و کار عمل می‌کند، اولویت‌بندی و مدیریت ویژگی‌ها و بهبودهای پلتفرم را انجام می‌دهد تا با نیازهای کاربران و اهداف کسب و کار هماهنگ شود.

طراح تجربه کاربری (UX)

رابط‌هایی را که تیم‌های دامنه برای تعامل با پلتفرم استفاده می‌کنند، طراحی می‌کند، با هدف سادگی استفاده و کارایی.

کارکنان پشتیبانی و آموزش

به تیم‌های دامنه در استفاده از پلتفرم کمک می‌کند، مشکلات را حل می‌کند و کاربران جدید را آموزش می‌دهند.

تیم پلت فرم حاکمیت محاسباتی فدرال

با مطابقت با اصل شبکه داده #٤، یک تیم پلتفرم دیگر به طور خاص بر روی حکومت محاسباتی متحد تمرکز دارد، به هر دامنه طرح‌هایی را ارائه می‌دهد تا بتواند با مقررات امنیتی و حفظ حریم خصوصی مطابقت داشته باشد. بیشتر تیم‌های دامنه این طرح‌ها را خودشان پیاده‌سازی می‌کنند، اما این تیم پلتفرم گاهی اوقات سیاست‌های جهانی را پیاده‌سازی می‌کند. این تیم راهنمایی لازم را برای کمک به هر تیم دامنه در مدیریت داده خود با هماهنگی با سیاست‌ها و استانداردهای سازمان ارائه می‌دهد. این تیم باید درک عمیقی از نیازهای خاص دامنه به داده داشته باشد و کار کند تا این نیازها را با نیازهای کلی سازمان به حکومت داده تعادل بزند. نقش‌های موجود در این تیم عبارتند از:

رهبر حکومت داده

راهنمایی در استراتژی کلی حکومت داده و اطمینان از اینکه ابتکارات تیم با اهداف استراتژیک سازمان هماهنگ هستند. آن‌ها با نمایندگان مختلف دامنه و کارشناسان موضوع مشترک همکاری می‌کنند تا استانداردها و سیاست‌های حکومت داده را تعریف کنند.

کارشناس حکومت داده

استانداردها و سیاست‌های داده را تأسیس و نظارت می‌کند، با تیم‌های دامنه کار می‌کند تا اطمینان حاصل شود که دستورالعمل‌ها را به درستی اعمال می‌کنند.

مربی حفظ حریم خصوصی و مطابقت

اطمینان حاصل می‌کند که تیم‌های دامنه با مقررات حفظ حریم خصوصی داده و سیاست‌های مطابقت داخلی مطابقت دارند، آن‌ها را در مورد موضوعات قانونی و مقرراتی راهنمایی می‌کند و با کارشناسان قانونی و مطابقت خارجی همکاری می‌کند.

تحلیلگر امنیت داده

استانداردها و روش‌های امنیتی را تعریف می‌کند که تیم‌های دامنه باید رعایت کنند، آن‌ها را راهنمایی می‌کند و اطمینان حاصل می‌کند که آن‌ها به جنبه‌های امنیتی سیاست‌های حکومت داده رعایت می‌کنند.

کارشناس کیفیت داده

استانداردها و روش‌های کیفیت داده را تعریف می‌کند تا کیفیت داده‌ای که در سراسر سازمان تولید می‌شود را تضمین کند. با تیم‌های دامنه کار می‌کند تا معیارها و سیستم‌های نظارت بر کیفیت داده را تنظیم کنند.

نماینده دامنه

به عنوان نماینده از تیم‌های دامنه خود عمل می‌کند، دانش خاص دامنه را در تیم پلتفرم حکومت داده به آن می‌برد و اطمینان حاصل می‌کند که سیاست‌های حکومت داده با نیازهای دامنه خود هماهنگ هستند.

معمار داده

استانداردهای مدل‌سازی داده را تعریف می‌کند، از جمله طراحی و مدیریت مدل‌های داده منطقی و فیزیکی.

مهندس DataOps

اتوماسیون پیاده‌سازی و نظارت بر سیاست‌های حکومت داده را انجام می‌دهد. ممکن است همچنین سیستم‌ها و ابزارهایی را که به برچسب‌زنی داده و سایر مکانیزم‌های شناسایی و مدیریت داده امکان می‌دهند، نگهداری کند.

نقش‌ها و مسئولیت‌های دقیق هر تیم بسته به جزئیات خاص سازمان و شبکه داده آن متفاوت خواهد بود. مهم‌ترین این است که هر تیم دامنه مسئولیت برای داده آن دامنه به عنوان یک محصول را بر عهده بگیرد.

این نقش‌ها باید به طور نزدیک همکاری کنند تا اطمینان حاصل شود که معماری داده نیازهای کسب و کار و کاربران را برآورده می‌کند. معماری باید انعطاف‌پذیر باشد تا تغییرات در منظره کسب و کار و فناوری را مدیریت کند و به اندازه کافی قوی باشد تا کیفیت و در دسترس بودن داده را تضمین کند.

همانطور که به عمق درک پویایی تیم‌ها می‌پردازیم، ضروری است که توجه داشته باشیم که حتی تیم‌های بهتر سازماندهی شده نیز می‌توانند با چالش‌های غیرمنتظره روبرو شوند.

بیایید تمرکز خود را از نقش‌هایی که موفقیت را راه‌اندازی می‌کنند به معیارهای رایج که می‌توانند یک پروژه را مسدود کنند و چگونه می‌توانید اقدامات جلوگیری انجام دهید، تغییر دهیم.

چرا پروژه ها شکست می خورند: دام ها و پیشگیری

حالا بیایید در مورد دلایل رایج شکست پروژه‌های بزرگ داده صحبت کنیم. برخی از مطالعات نشان می‌دهند که نرخ شکست بالایی دارند؛ به عنوان مثال، در سال ۲۰۱۹، VentureBeat گزارش داد که "۸۷ درصد پروژه‌های علم داده

هرگـز بـه مرحلـه تولیـد نمی‌رسـند." ایـن بخـش بـرخی از دلایـل رایـج شکسـت پروژه‌هـای بـزرگ داده را کـه مـن دیـده‌ام، را بـه همراه راه‌های جلوگیری از این معیارها معرفی می‌کند.

خلاصه - رمز گشایی از معماری داده

معیار: اجازه دادن به اجرایی‌ها فکر کنند که تحلیل و گزارش گیری باشند "آسان"

من با تعدادی از اجرایی‌ها مواجه شده‌ام که ایده‌های بسیار غیرواقع‌بینانه‌ای در مورد زمان لازم برای ساخت یک معماری داده دارند. آن‌ها اغلب فشار می‌آورند تا مدیران پروژه به پذیرش زمان‌بندی‌ها و نقاط مهم غیرواقع‌بینانه متوجه شوند، بنابراین پروژه را برای شکست آماده می‌کنند.

برای جلوگیری از این معیار، باید اجرایی‌ها را آموزش دهید. به آن‌ها کمک کنید تا فرآیند را درک کنند و زمان لازم برای ساخت و آزمایش یک راه حل مناسب. حتی دیدم که اجرایی‌ها در یک جلسه کدنویسی- یک روزه شرکت کرده‌اند تا بتوانند به طور مستقیم ببینند چقدر سخت است ساخت یک راه حل داده ای.

خطای اصلی: استفاده از فناوری‌های نادرست

وقتی شرکت‌ها نتوانند خود را در مورد تمام محصولات موجود برای حل مسائل داده و موارد استفاده آن‌ها آموزش دهند، ممکن است به اشتباه محصول مناسب برای کار استفاده کنند. (به عنوان مثال، من یک شرکت را دیدم که از SQL Server به عنوان یک دریاچه داده استفاده می‌کند!) افراد نمی‌دانند که چه چیزی نمی‌دانند، بنابراین فقط از محصولاتی استفاده می‌کنند که آن‌ها را شنیده‌اند. همانطور که گفته می‌شود، وقتی شما یک چاقو دارید، همه چیز یک نیل است.

تصمیم‌گیران باید وقت کنار بگذارند تا تمام ابزارهای ممکن را درک کنند، سپس ابزار مناسب را برای کار انتخاب کنند تا از این خطا جلوگیری کنند. من توصیه می‌کنم که چندین کمیته را تشکیل دهید، هر کدام بر روی تحقیق گروه خاصی از محصولات، مانند ابزارهای ETL، تجزیه و تحلیلی یا ابزارهای گزارش‌گیری تمرکز کنند.

خطای اصلی: جمع‌آوری بیش از حد نیازمندی‌های تجاری

وقتی من یک توسعه‌دهنده بودم، برای یک شرکت کار می‌کردم که بیش از یک سال زمان صرف جمع‌آوری نیازمندی‌های تجاری برای یک راه حل داده استفاده کرد در حالی که ما در حال انتظار برای اتمام آن‌ها بودیم. آن‌ها در نهایت صدها صفحه نیازمندی تولید کردند، که همه چیز را به تاخیر انداخت. هیچ کس تمام سند را نخواند؛ مرحله نیازمندی‌ها به اندازه‌ای طول کشیده بود که ما فقط چند بخش را قبل از شروع توسعه خواندیم.

برای جلوگیری از این خطا، فقط چند هفته را برای نیازمندی‌ها صرف کنید، به اندازه‌ای کافی جمع‌آوری کنید تا به توسعه‌دهندگان اجازه شروع به کدنویسی- دهید، سپس جمع‌آوری نیازمندی‌ها را به طور همزمان با کدنویسی- ادامه دهید.

خطای اصلی: جمع‌آوری بسیار کم نیازمندی‌های تجاری

برخی از تیم‌های پروژه به ندرت وقتی با کاربران نهایی می‌گذرانند نیازمندی‌ها را جمع‌آوری می‌کنند؛ برخی از آن‌ها این کار را به توسعه‌دهندگان بستند تا تخمین کنند که کاربران نهایی چه چیزی می‌خواهند. آن‌ها اغلب ویژگی‌های اشتباه را توسعه می‌دهند، کاربران را در همه چیز از آنچه نیاز دارند یا می‌خواهند ارائه نمی‌دهند.

اگر شما روش تکراری را برای توسعه پروژه پذیرفته باشید، می‌توانید از این مشکل جلوگیری کنید. این رویکرد شامل ساخت بخش‌های راه حل، سپس بازبینی مراحل طراحی و برنامه‌ریزی، اجازه می‌دهد تا بر اساس بازخورد و نیازمندی‌های تکاملی در یک چرخه مداوم تنظیمات را انجام دهید.

خطای اصلی: ارائه گزارشات بدون حصول اطمینان از محتوای آن‌ها

اگر شما یک گزارش از راه حل جدیدی که در حال ساخت هستید را به یک کاربر نهایی ارائه کنید و این گزارش شامل یک عدد نادرست باشد، اولین تصور او خیلی بد خواهد بود. وقتی کاربران اعتماد خود را از گزارشات از دست می‌دهند، بازیابی اعتماد آن‌ها بسیار سخت است. شرکت‌ها همیشه وقت لازم را برای اطمینان از صحت همه چیز در گزارش ندارند و نتیجه می‌تواند یک عقب‌العمل اصلی باشد.

یک فرآیند QA دقیق داشته باشید که شامل مقایسه کل گزارشات در راه حل جدید با آن‌ها در گزارش تولید که می‌دانیم صحیح است. اگر ممکن است، از کاربران نهایی کمک بگیرید؛ نه تنها می‌توانند تأیید کنند که گزارشات صحیح هستند، بلکه همچنین احساس مالکیت از بخش شدن در ساخت راه حل جدید خواهند داشت.

خطای اصلی: استخدام یک شرکت مشاوره‌ای بدون تجربه

بیشتر شرکت‌ها وقت یا تخصص لازم برای ساخت یک راه حل داده وارهاوس خودشان ندارند، بنابراین آن‌ها را برای ساخت آن استخدام می‌کنند. این یک ایده عالی است، تا زمانی که شما شرکت مشاوره‌ای مناسب را انتخاب کنید. آن‌ها می‌توانند کارشناسان باشند - اما در نوع کاملاً متفاوتی از پروژه. شرکت‌های مشاوره‌ای گاهی اوقات در تلاش برای برنده شدن کسب و کار شما، کارشناسان با تجربه مرتبط را درگیر می‌کنند، اما سپس با استفاده از افراد کمتر تجربه‌دار جایگزین می‌کنند وقتی پروژه شروع می‌شود.

برای جلوگیری از این خطا، سوالات بپرسید. آیا مشاوران تجربه ساختن راه حل خاصی که می‌خواهید را دارند؟ اگر بله، آیا این افراد در پروژه شما کار خواهند کرد.

خطای اصلی: استخدام یک شرکت مشاوره‌ای که توسعه را به کارگران خارجی انتقال می‌دهد

شرکت‌های مشاوره‌ای از توسعه‌دهندگان خارجی برای کاهش هزینه‌ها استفاده می‌کنند، اما این می‌تواند منجر به مشکلات شود. تفاوت‌های منطقه‌ای می‌تواند باعث شود تا با دشواری ملاقات برقرار شود، مانع از ارتباط صحیح نیازمندی‌ها می‌شود، و کیفیت کار ممکن است پایین‌تر باشد.

سوالات مطرح کنید تا اطمینان حاصل کنید که شرکت مشاوره‌ای پروژه شما را به یک تیم خارجی انتقال نمی‌دهد تا از این مشکل جلوگیری کند.

خطای اصلی: انتقال مالکیت پروژه به شرکت‌های مشاوره‌ای

ممکن است تمایل داشته باشید که "کلیدهای" پروژه را به شرکت مشاوره‌ای بدهید، به طوری که آن‌ها نه تنها پروژه را توسعه دهند بلکه آن را به طور کامل مدیریت کنند. این می‌تواند منجر به کمبود شفافیت شود؛ پروژه می‌تواند به یک "جعبه سیاه" تبدیل شود که شما هیچ ایده‌ای ندارید که مشاوران در کار چه چیزی هستند یا چگونه کار را انجام می‌دهند.

مطمئن شوید که داستان واقعی و کامل پیشرفت پروژه را دریافت می‌کنید. گزارشات وضعیت کافی نیستند؛ مدیران پروژه را قرار دهید تا به نزدیک مشاهده کنند.

خطای اصلی: نادیده گرفتن نیاز به انتقال دانش به سازمان

اگر از یک شرکت مشاوره‌ای استفاده می‌کنید، نمی‌خواهید آن‌ها بیایند، پروژه را انجام دهند و بدون آموزش در مورد آنچه ساخته‌اند بیرون بروند. شما نیاز دارید تا بتوانید باگ‌ها را رفع کنید و ویژگی‌ها را اضافه کنید بدون اینکه نیاز به استخدام دوباره شرکت مشاوره‌ای داشته باشید.

برخی از مردم شما را کنار مشاوران بگذارید هنگام ساخت راه حل، حتی اگر فقط برای چند هفته، تا آشنا شوند. وقتی نزدیک به پایان پروژه باشید، برای انتقال دانش زمان تعیین کنید و از شرکت مشاوره‌ای بخواهید که جلسات آموزش و بازبینی برگزار کند تا سازمان شما در مورد راه حل جدید آموزش ببیند.

خطای اصلی: کاهش بودجه در میان پروژه

پروژه‌های بد برنامه‌ریزی شده گاهی اوقات بودجه خود را به حداکثر می‌رسانند، به طوری که هنگام نزدیک شدن به خط پایان برای پول به اندازه کافی ندارند. اگر می‌خواهید شکست را تضمین کنید، شروع به کاهش بودجه کنید. این شامل اقداماتی مانند اخراج توسعه‌دهندگان، آزمایشگران، مدیران پروژه، یا DBAs؛ کاهش سخت‌افزار راه حل؛ یا کاهش آموزش کاربران نهایی در سیستم جدید است. در این زمینه‌ها کاهش انجام ندهید.

یا بودجه را افزایش دهید یا در زمینه‌های کمتر حیاتی کاهش انجام دهید تا از این خطا جلوگیری کنید. به عنوان مثال، ممکن است تعداد منابع داده را که در ابتدا باید مصرف شوند، کاهش دهید، به طوری که آن‌ها را برای فاز بعدی پروژه بگذارید.

خطای اصلی: شروع با یک تاریخ پایان و کار به عقب

تمرکز بر روی یک مهلت سخت گیرانه می‌تواند کیفیت پروژه را متضرر کند. ابتدا، استفاده از مراحل پیچیده منابع داده، ادغام، برقراری استانداردهای کیفیت، ساخت فرآیندهای ETL، طراحی مدل‌های داده، و ایجاد رابط‌های

کاربری می‌تواند منجر به خطاها و ناکارآمدی‌ها شود. دوم، یک زمان‌بندی سخت گیرانه همچنین می‌تواند مراحل کشف و کاوش اساسی پروژه را محدود کند، که ممکن است منجر به یک سیستمی شود که نیازهای واقعی سازمان را برآورده نمی‌کند. سوم، غیرممکن است فرض کنید که با درک کامل از حوزه پروژه شروع می‌کنید. یک تاریخ پایان سخت گیرانه باعث می‌شود تا دشوار باشد تا تغییرات غیرمنتظره یا نیازمندی‌های جدیدی که هنگام پیشرفت پروژه ظهور می‌کنند، را جذب کنید، که می‌تواند منجر به تاخیر یا محصولات ناقص شود. در نهایت، یک تاریخ پایان سخت گیرانه می‌تواند همکاری ضروری ذینفعان را محدود کند، که منجر به یک محصول می‌شود که به کاربران به طور کافی خدمت نمی‌کند.

توجه ندهید که مهلت‌ها و برنامه‌ها سخت گیرانه نباشند. پذیرش کنید که برنامه‌ها ممکن است نیاز به تغییر داشته باشند هنگامی که شرایط جدیدی ظهور می‌کنند.

خطای اصلی: ساختاردهی داده وارهاوس برای نشان دادن داده‌های منبع به جای نیازهای تجاری

داده‌های منبع از سیستم‌های عملیاتی گرفته می‌شوند که معمولاً برای تحلیل داده‌های جامع یا وظایف هوش مصنوعی طراحی نشده‌اند، که می‌تواند قابلیت استفاده از MDW (یا داده لاک‌هاوس) شما را محدود کند. این سیستم‌ها با اهداف استراتژیک و KPIهای تجاری همخوانی ندارند، که می‌تواند منجر به از دیدگاه‌های ارزشمندی که می‌تواند تاثیر منفی بر تصمیم‌گیری داشته باشد، شود. همچنین، مدل‌سازی سیستم شما بر اساس داده‌های منبع به معنای این است که تغییرات در سیستم‌های عملیاتی، که معمول هنگام تکامل تجاری رخ می‌دهد، نیاز به تغییرات عمده در وارهاوس شما خواهد داشت، که منجر به ناپایداری و افزایش نگهداری می‌شود.

ساختار MDW (یا داده لاک‌هاوس) خود را بر اساس نیازهای تجاری خود ساخته و نه تنها بر اساس ساختار داده‌های منبع را بلافاصله نشان دهید.

خطای اصلی: ارائه گزارشات با زمان پاسخگویی کند یا سایر مشکلات عملکردی به کاربران نهایی

راه اطمینان دار برای دریافت شکایات مستقیم از کاربران نهایی این است که به آن‌ها یک راه حل بدهید که برای تولید گزارشات، داشبوردها یا پرس و جوهایی که نیاز دارند کند باشد - به خصوص اگر راه حل قبلی آن را سریع‌تر انجام داده بود. همانطور که با داده‌های نادرست، اگر زمان‌های پاسخگویی کند به کاربران اولین تصور بد بدهد، آن‌ها اعتماد و اعتماد به شما و راه حل را از دست می‌دهند، که بازیابی آن بسیار سخت خواهد بود.

نتوانید منتظر شکایات قبل از اینکه بهبود عملکرد را انجام دهید. زمان‌های پاسخگویی را از گزارشات مهاجرت شده اندازه‌گیری کنید و گزارشات جدید را تنظیم کنید تا حداقل برابر باشند. اگر این گزارشات مهاجرت نشده‌اند، از کاربران بخواهید که سطح قابل قبولی برای عملکرد ایجاد و تعامل با گزارش را تعیین کنند.

خطای اصلی: طراحی بیش از حد (یا کم از حد) معماری داده‌های شما

اگر زمان زیادی را برای طراحی معماری داده‌ها برای راه حل خود صرف نکنید، خطر ساختن چیزی وجود دارد که نمی‌تواند منابع داده از اندازه، نوع یا سرعت خاصی را مدیریت کند؛ نسبت قابل مقیاس‌پذیری ندارد؛ عملکرد ضعیفی دارد؛ یا ریسک‌های امنیتی را معرفی می‌کند. از سوی دیگر، طراحی بیش از حد معماری می‌تواند منجر به پیچیدگی بیش از حد، کمبود انعطاف‌پذیری، تحلیل پارالیزی، و افزایش هزینه‌ها شود.

ارزش دارد که هنگام ایجاد معماری داده‌ها توازن مناسبی را برقرار کنید. طراحی باید کافی باشد تا یک ساختار قوی، قابل مقیاس‌پذیری و امن را فراهم کند اما کافی انعطاف‌پذیر و ساده باشد تا به راحتی فهمیده، پیاده‌سازی و تطبیق با نیازهای تغییر کننده شود.

خطای اصلی: ارتباط ضعیف بین IT و حوزه‌های تجاری

این دو گروه حیاتی اغلب از زبان‌های مختلفی استفاده می‌کنند، اهداف مختلفی دارند و از دیدگاه‌های مختلفی نگاه می‌کنند، بنابراین شکاف‌های بین آن‌ها غیرعادی نیستند. کارکنان IT معمولاً بر جنبه‌های فنی و پیاده‌سازی سیستم تمرکز دارند، در حالی که حرفه‌ایان تجاری بر روی اهداف استراتژیک و درآمدزایی تمرکز دارند. ارتباط ضعیف می‌تواند منجر به مشکلات حیاتی مانند هماهنگ نبودن اهداف، استفاده ناکارآمد از منابع، فرصت‌های از دست رفته و کاهش انرژی شود.

موفقیت یک پروژه داده به شدت به این دو گروه بستگی دارد که به طور همکاری کار کنند. IT باید نیازهای تجاری را درک کند، در حالی که تیم تجاری باید احترام به پتانسیل و محدودیت‌های زیرساخت تکنولوژی را درک کند. مهم است که فرهنگی از گفتگوی باز و احترام متقابل را ترویج کنید. نقش‌هایی مانند تحلیلگر تجاری می‌تواند با ترجمه نیازهای تجاری به مشخصات فنی و برعکس، این شکاف را پر کند.

دو گروه باید به طور منظم با هم ملاقات کنند، بحث‌های شفاف را برگزار کنند و نیازها را به طور واضح مستند کنند. همکاری موثر می‌تواند تفاوت زیادی در موفقیت یا شکست پروژه ایجاد کند.

راهنمایی‌های موفقیت:

پس از گذر از میدان معلق از چالش‌ها و خطاهای احتمالی که می‌توانند حتی پروژه‌های با پتانسیل بالا را به طور ناگهانی به موقعیت خارج شوند، ضروری است که دیدگاه خود را متعادل کنید. به اندازه اینکه حیاتی است که با آگاهی از آنچه می‌تواند اشتباه شود، به طور مساوی حیاتی است که درک کنید چه چیزی یک پروژه را به سمت اهداف خود هدایت می‌کند. این راهنمایی‌ها به شما کمک می‌کنند تا استراتژی‌های اثبات شده، بهترین روش‌ها، و دانش با دیدگاه باشید، شما را از مراحل احتمالی خطا دور کند و به سمت موفقیت هدایت کند.

نکته مهم:

گاهی اوقات بهتر است دو برابر زمان بگذرید و دو برابر پول صرف کنید تا یک راه حل را بسازید که ۵۰ برابر ارزش راه حل قدیمی شما را فراهم کند. ارزش بلندمدت سرمایه‌گذاری در زمان و منابع را در ساخت راه حل‌های داده با کیفیت بالا، مقاوم و قابل مقیاس‌پذیری که قابلیت‌های تحلیل پیشرفته را پشتیبانی می‌کنند، تأکید کنید. این برای تشویق فرهنگ مبتنی بر داده که نوآوری و کارایی عملیاتی را تشویق می‌کند، حیاتی است.

استفاده سریع از پروژه‌ها یا کمبود منابع می‌تواند منجر به عیوب سیستم، نتایج نادرست و مسائل مطابقت شود. در نهایت، مقابله با نگهداری مداوم یا بازسازی یک سیستم خراب می‌تواند بیشتر از سرمایه‌گذاری در ابزارهای بهترین در ابتدا باشد. سرمایه‌گذاری بیشتر در زمان و منابع در ابتدا اجازه می‌دهد تا برنامه‌ریزی بهتر، افراد بهتر و ابزارهای بهتر را امکان‌پذیر کند، همه این عوامل به ایجاد سیستم‌های قابل اعتماد، دقیق و قابل مقیاس‌پذیری کمک می‌کنند.

دریافت دانش و مهارت فنی مناسب در تیم ممکن است به معنای پرداخت بیشتر در حقوق باشد، اما ساخت یک راه حل داده نیازمند ترکیبی بی‌ارزش از درک کسب و کار، مهارت‌های فنی و قابلیت‌های مدیریتی است. علاوه بر این، برای پروژه حداقل یک نفر نیاز دارید که با معماری داده‌ها آشنا باشد. او باید فرآیند تصمیم‌گیری در مورد استفاده از کدام معماری داده را رهبری کند. این یک تصمیم بزرگ است. اگر آن را به عادلان نامتخصص بگذارید، راه حل نتیجه‌گیری شده احتمالاً در مسیر درست نخواهد بود، منجر به مشکلات عملکرد، ناتوانی در دریافت داده‌ها از منابع خاص، نتایج نادرست و سایر مسائل می‌شود. افرادی را درگیر کنید که می‌دانند چه کاری انجام می‌دهند.

در کل، هزینه‌های اولیه و سرمایه‌گذاری زمانی توسط بازده بلندمدت قابل تعویض معنادار تعویض خواهد شد، که شامل کاهش هزینه‌های عملیاتی بلندمدت و مزایا استراتژیک و بینش‌های ارزشمند است.

کاربران نهایی را درگیر کند ، نتایج را به آنها نشان دهید و آنها را به سمت تحریک بکشید

کاربران نهایی را در ابتدا درگیر کنید تا آنها با شما همکاری کنند نه در عقب. در ابتدای پروژه، آنها را دعوت کنید تا در تصمیم‌گیری درباره سوالاتی مانند انتخاب منابع داده و چگونگی تمیز کردن، تجمیع و اعتبارسنجی داده‌ها کمک کنند. اگر آنها احساس مشارکت داشته باشند، پروژه را تشویق خواهند کرد و خواهند خواست که هر مشکلی که به وجود می‌آید را حل کنند نه آن را شکایت کنند. اگر آنها را درگیر نکنید، آنها احساس خواهند کرد که تغییرات به آنها اجباری می‌شود. آنها سریعاً درباره راه حل شکایت خواهند کرد و در برابر استفاده از آن مخالفت خواهند کرد.

اگر راه حل شما نیاز دارد که کاربران با تکنولوژی‌های جدید ناشناخته کار کنند (مانند ابزارهای گزارشگیری)، آنها را آموزش دهید! شرکت در آموزش راهی عالی برای کاربران برای آماده‌سازی برای راه حل جدید و حتی یادگیری روش‌های بهتر و سریع‌تر برای انجام کارهای خود است. به آنها کمک کنید تا هر ویژگی و عملکرد جدید که ابزار جدید ارائه می‌دهد را درک کنند تا آنها به سمت اجرای موفق راه حل جدید تمایل داشته باشند.

به همین منظور، نتایج را سریع و مکرراً نشان دهید. نگران نباشید که ماه‌های کار پروژه بدون نشان دادن هرگونه نتیجه به کاربران و ذی‌نفعان برسد. از رویکرد تعاملی و آگاهانه برای تحویل پروژه استفاده کنید، با انتشارات مکرر.

پـروژه را بـه فازهـا تقسـیم کنیـد و بـرای یـک پـیروزی سریـع کـه نشـان می‌دهـد چـه مزایـایی را می‌توانیـد تـرویج کنیـد، برنامـه‌ریزی کنیـد. بـرای مثـال، وقـتی شروع بـه دریافـت منـابع می‌کنیـد، بـا یـکی کـه اولویـت بـالا دارد امـا همچنـین کوچـک و آسـان شروع کنیـد. سـپس فرآینـد را تکـرار کنیـد. ایـن یـک پایـه قـوی ایجـاد می‌کنـد و بـه توسـعه‌دهندگان کمـک می‌کنـد تـا بهترین روش‌ها را برای ساخت راه حل ارائه دهند.

افزودن ارزش به گزارشات و داشبوردهای جدید

اگـر در حـال انتقـال از یـک محـیط بـه محـیط دیگـر هسـتید - بـه عنـوان مثـال، از سرور محـلی بـه ابـر - نکنیـد فقـط گزارشـات موجـود خـود را دوبـاره بسـازید؛ بـا ارائـه ویژگی‌هـا و عملکردهـای جدیـد بـه کـاربران، ارزش اضـافه کنیـد. داشـبورد هـایی بـا شـاخص‌های کلیـدی عملکـرد (KPIs) کـه قـبلاً ندیده‌انـد ایجـاد کنیـد؛ بـه آنهـا هـنر امکان‌پـذیری را نشـان دهیـد و راه‌هـای جدیـد بـرای دریافـت بینش‌هـا از داده‌هـای خـود نشـان دهیـد. راه‌هـایی را پیـدا کنیـد تـا بـه آنهـا کمـک کنیـد کارهـای خـود را سریـع‌تر و آسـان‌تر انجـام دهنـد. اگـر راه حـل جدیـد بهـتر از راه حـل قـدیمی نباشـد، آنهـا می‌پرسـند، "چـرا پـول زیـادی را صرف دوبـاره سـاختن آنچـه کـه قـبلاً داشـتیم کـردیم؟" آنهـا را متقاعـد کنیـد کـه سـاخت یـک راه حـل جدیـد پـول خـوبی را صرف کرده است.

از کاربران نهایی بخواهید که یک نمونه سازی بسازند

تنهـا بخشی_ از راه حـل داده کـه بیشـتر کـاربران نهـایی آن را می‌بیننـد و اسـتفاده می‌کننـد، ابـزار گزارشـگیری اسـت. شـما می‌خواهید مطمئن شوید که آنها با بیشتر، اگر نه همه، ویژگی‌های آن آشنا هستند.

بسـیاری از کاربران حـتی از ویژگی‌هـایی کـه می‌توانـد بـه آنهـا کمـک کنـد و بـه آنهـا بینـش بدهـد، آگاه نیسـتند. ایـن ماننـد رانـندگی لامبـورگینی در گـرد اول اسـت زیـرا می‌دانیـد کـه گردهـای دیگـری دارد! آنهـا را در ابـزار گزارشـگیری آمـوزش دهیـد و یـک کارشناس را با استفاده از داده‌های خودشان نشان دهید.

درگیر کـردن کاربران نهـایی در ایجـاد نمونـه سـازی نقـش کلیـدی در ایـن درک ایفـا می‌کنـد. هنگـامی کـه نیازمنـدی‌های کسـب و کـار پـروژه را تعیـین می‌کنیـد و مـردم را در ابـزار گزارشـگیری آمـوزش می‌دهیـد، کـاربران نهـایی را بـرای سـاخت یـک نمونـه سـازی دعـوت کنیـد. ایـن نمونـه سـازی بایـد شـامل گزارشـات نمونـه باشـد کـه همـه اطلاعـاتی کـه بـه آنهـا نیـاز دارنـد را شـامل می‌شـود. بـا سـاخت ایـن نمـایش عمـلی، کـاربران می‌تواننـد بهـتر قابلیت‌هـای بهبـود یافتـه گزارشـات و داشـبوردهای جدیـد را درک کننـد. ابزارهـای گزارشـگیری پیشـرفته امـروز بـه ایـن قـدر سـاده شـده‌اند کـه فقـط نیـاز بـه مهارت‌هـای فـنی کمینـه بـرای سـاخت یـک نمونـه سـازی داریـد. می‌گوینـد یـک تصـویر برابـر بـا هـزار کلمـه اسـت؛ داشـتن یـک نمونـه سـازی کار کـردن از کـاربران نهـایی را یـک راه عـالی اسـت تـا بـه طـور مسـتقیم ارزش اضـافه و بینش‌هـایی کـه راه حـل جدیـد ارائـه می‌دهـد را ببینند.

یافتن یک پشتیبان/مدیر پروژه

یـک پشـتیبان پـروژه یـک اجـرای سـنیور یـا فـرد تاثیرگـذار اسـت کـه پشـتیبانی و تحریـک بـرای پـروژه ارائـه می‌دهـد. نقـش ایشـان ضروری و چنـد جانبـه اسـت. ابتـدا، او بـه عنـوان یـک مـدعی داخـلی عمـل می‌کنـد، ارتبـاط برقـرار می‌کنـد و مزایـای بلندمـدت یـک معمـاری داده خـوب سـاخته شـده را ارائـه می‌دهـد، از جملـه تصـمیم‌گیری بهبـود یافتـه، کـارایی عملیـاتی و برنامـه‌ریزی اسـتراتژیک. دوم، او در تـأمین منـابع و سرمایه‌گـذاری لازم نقـش مهـمی دارد. بـا توجـه بـه اینکـه پروژه‌هـای داده نیـاز بـه سرمایه‌گـذاری‌های قابـل تـوجهی در فنـاوری، منـابع انسـانی و زمـان دارنـد، یـک پشـتیبان پـروژه کـه می‌توانـد ایـن منابع را جمع‌آوری کند، می‌تواند موثر باشد.

سـوم، پشـتیبان پـروژه نقـش حیـاتی در تسـهیل همکـاری بـین توابـع بـین بخش‌هـایی بـا علاقه‌منـدی‌های مختلـف بـازی می‌کنـد، همـه چـیز را بـه سـمت هـدف مشـترک پـروژه همسـو می‌کنـد. در نهایـت، او بـا کمـک بـه مشـارکت کارکنـان و مقابلـه بـا اعتراضـات آنهـا در مـورد سیسـتم‌ها و فرآینـدهای جدیـد، مقاومـت بـه تغییـر را مـدیریت می‌کنـد - یـک چـالش قابـل توجـه در بیشـتر پروژه‌هـای داده اسـت. ایـن کمـک می‌کنـد تـا انتقـالات روان‌تـر انجـام شـود و احتمـال موفقیـت پـروژه را افـزایش می‌دهد.

برنامه‌ریزی پروژه‌ای که به دنبال کارایی ۸۰٪ است

شخص مسئول مدیریت روزانه و اجرای پروژه - که اغلب به عنوان مدیر پروژه یا رهبر پروژه شناخته می‌شود - باید اطمینان حاصل کند که همه به طور بهینه درگیر شده‌اند. این نقش متفاوت از نقش پشتیبان پروژه یا اسپانسور است که از سطح بالاتر پشتیبانی استراتژیک و تحریک ارائه می‌دهد. مدیر پروژه بیشتر "در نیمه‌های مختلف" است، وظایف، برنامه‌های زمانی و منابع را هماهنگ می‌کند. به طور ایده‌آل، هر عضو تیم باید حدود ۸۰٪ ساعت کاری خود را بر روی پروژه استفاده کند. (۲۰٪ دیگر را برای مواردی مانند استراحت یا بیماری، جلسات غیر پروژه، یادگیری و غیره نگه دارید.) شما نیاز به یک برنامه پروژه خوب و با در نظر گرفتن کاهش زمان انتظار هر فرد برای اتمام یک وظیفه وابسته به دیگران دارید. این همچنین به معنای اطمینان از این است که افراد مناسب وظایف مناسب را انجام می‌دهند و همه قبل از شروع پروژه آموزش داده شده‌اند. هر چیزی کمتر از ۸۰٪ کارایی می‌تواند منجر به از دست دادن مهلت‌ها و افزایش هزینه‌ها شود.

خلاصه

در این فصل، ما در مسیر نقش انسانی در معماری‌های داده سفر کرده‌ایم. در حالی که ابزارها و فناوری ممکن است چشم‌انداز و تحریک بزرگی را ایجاد کنند، آنها فقط به عنوان ابزارهایی در دست کاربران انسانی خود عمل می‌کنند. چگونگی سازمان‌ها در استفاده از این ابزارها و چگونگی مقابله با تغییرات فرهنگی در فرآیند می‌تواند موفقیت یا شکست معماری‌های آنها را تحریک کند. همانطور که یاد گرفتید، تعیین ساختارهای تیم مناسب، ترویج وضوح در نقش‌ها و مسئولیت‌ها، و درک عمیقی از معیارهای موفقیت و دنباله‌های خطر ضروری برای موفقیت پروژه است. لحظه‌ای صبر کنید تا درک کنید که هر تکنولوژی پیچیده و پیشرفته‌ای فقط به اندازه‌ای قدرتمند است که توسط افرادی که آن را استفاده می‌کنند و فرآیندهایی که استفاده از آن را تنظیم می‌کنند. یک داده‌کانال خوب طراحی شده یا یک شبکه داده قوی تاثیر فقط زمانی دارد که توسط یک تیم سازمان‌یافت، آگاهانه و توانمند تکمیل شود. واقعیت موفقیت تبدیل داده‌ها در این تعادل حساس بین انسان و فناوری است.

با این حال، در حالی که این دیدگاه مرکزی انسان اساسی است، ما نمی‌توانیم تکنولوژی‌های شگفت‌انگیزی را که در سال‌های اخیر چشم‌انداز معماری‌های داده را تغییر داده‌اند، نادیده بگیریم. فصل بعدی عمیق به عرصه تکنولوژیک می‌پردازد، با بررسی نوآوری‌ها و پیچیدگی‌های حرکت‌های منبع باز، قابلیت‌های شکاف‌آور هداپ، منافع تبدیل‌کننده ابر، و پیشنهادات ارائه‌دهندگان اصلی ابر. علاوه بر این، من شما را به دنیای استراتژی‌های چند ابری می‌برم و شما را با تغییرکنندگان مانند Databricks و Snowflake آشنا می‌کنم.

انسان و فناوری گزینه‌ای نیستند یا یا. آنها باید یک ترکیب هماهنگ باشند، با هر کدام دیگری را ارتقاء دهند. بیایید بررسی خود را ادامه دهیم، درک کنید که تلاش‌های تکنولوژیک ما واقعاً فقط زمانی تحقیق می‌شوند که توسط افراد مناسب و فرآیندها تکمیل می‌شوند.

فصل شانزدهم:تکنولوژی ها

هنگام ورود به این فصل، تصمیم اولیه مهم شما انتخاب بین راه‌حل‌های منبع باز و پیشنهادات ارائه‌دهندگان ابر است. با این حال، چیزهای بیشتری وجود دارد. من شما را در فکر کردن در مورد مقیاس نیازهای داده شما و نیازهای انعطاف‌پذیری سازمان شما هدایت خواهم کرد.

بعد از این، ما در منظره مدل‌های خدمات ابری پرداخته‌ایم، از ارائه‌دهندگان اصلی تا پیچیدگی راه‌حل‌های چند ابری. فصل با معرفی سه چارچوب نرم‌افزاری اساسی: Databricks، Hadoop، و Snowflake پایان می‌یابد.

پاسخ‌های شما به سوالات در این فصل به شکل عمیقی معماری داده شما را شکل خواهد داد. این فقط مسائل منبع باز در مقابل مالکیت محصول نیست؛ این در مورد ترکیب بهینه فناوری برای نیازهای منحصر به فرد شما در جهان معماری داده است.

انتخاب یک پلتفرم

تصمیم برای استفاده از نرم‌افزار منبع باز یا نرم‌افزار از یک ارائه‌دهنده خدمات ابری (CSP) بستگی به عوامل مختلفی دارد، از جمله نیازهای خاص، بودجه، تخصص، حساسیت داده‌ها، و ترجیحات کنترل. نیازهای سازمان خود را با

دقت بررسی کنید و در نظر بگیرید که معاملات بین مزایای نرم‌افزار CSP و انعطاف‌پذیری و سفارشی‌سازی ارائه شده توسط نرم‌افزار منبع باز (OSS) چگونه است.

راه‌حل‌های منبع باز

نرم‌افزار منبع باز به نرم‌افزاری اشاره دارد که به صورت رایگان توزیع می‌شود و کد منبع آن برای هر کسی- برای مشاهده، تغییر، و توزیع باز است. این معمولاً توسط یک جامعه از برنامه‌نویسان که دانش و تخصص خود را برای ایجاد و بهبود نرم‌افزار ارائه می‌دهند، به صورت همکاری توسعه می‌یابد.

تاریخچه غنی OSS چند دهه پیش برمی‌گردد. این از حرکت نرم‌افزار آزاد در دهه ۱۹۸۰ شروع شد، که توسط Richard Stallman رهبری شد و برای آزادی نرم‌افزار مورد تحریک قرار گرفت. موسسه منبع باز (OSI)، که در سال ۱۹۹۸ تأسیس شد، تعریف منبع باز را ایجاد کرد و معیارهایی برای مجوزهای OSS تعیین کرد که اطمینان می‌دهد نرم‌افزار برای هر کسی- آزاد و باز باقی می‌ماند. Linux نقش مهمی در محبوب‌سازی OSS داشت. استفاده از راه‌حل‌های منبع باز در دهه ۲۰۱۰ به دلیل انعطاف‌پذیری و هزینه پایین محبوب شد. پلتفرم‌های همکاری مانند GitHub توسعه OSS را تسهیل کردند و حرکت به سمت داده‌ها و سخت‌افزار باز گسترش یافته است. برخی از برنامه‌های منبع باز برتر عبارتند از MySQL، PostgreSQL، MongoDB، Apache Hadoop، Apache Spark، Presto، Apache Kafka، و Apache Airflow.

بسیاری از معماری‌های داده از OSS استفاده می‌کنند، اما بسیاری کمتر کاملاً با آن ساخته شده‌اند. بیشتر معماری‌ها از OSS به همراه محصولات ارائه‌دهنده ابر استفاده می‌کنند. شامل کردن OSS برای معماری داده شما بسیار ارزشمند است.

برخی از دلایل اصلی که ممکن است شما را منجر به استفاده از نرم‌افزار منبع باز (OSS) به جای محصولات ارائه‌دهنده ابر کند، عبارتند از:

- صرفه‌جویی در هزینه

OSS - معمولاً رایگان است و این آن را به یک گزینه جذاب برای سازمان‌هایی که به دنبال بهینه‌سازی بودجه خود هستند، تبدیل می‌کند.

- انعطاف‌پذیری و سفارشی‌سازی

OSS - اجازه می‌دهد تا کد منبع را برای تنظیم و سفارشی‌سازی به نیازهای خاص تغییر دهید. این به شما اجازه می‌دهد تا نرم‌افزار را بر اساس نیازهای سازمان خود سفارشی کنید، وظایف را به دلخواه اضافه یا حذف کنید.

- شفافیت و اعتماد

- کد منبع OSS به صورت باز در دسترس است و هر کسی- می‌تواند آن را برای بررسی آسیب‌پذیری‌های امنیتی، اشکالات، و درب‌های پشتیبانی شفاف کند. این شفافیت اعتماد را ترویج می‌کند و محیطی برای بررسی و بهبود کد ایجاد می‌کند.

- نوآوری و همکاری سریع

OSS - از یک جامعه جهانی از توسعه‌دهندگان بهره می‌برد که تخصص و بهبودهای خود را ارائه می‌دهند. این محیط همکاری نوآوری سریع را ترویج می‌کند، چرخه‌های توسعه نرم‌افزار را تسریع می‌کند، و منجر به راه‌حل‌های قابل اعتماد و غنی از ویژگی‌ها می‌شود.

- استقلال از تامین‌کننده

- با OSS، سازمان‌ها به یک تامین‌کننده خاص یا به نرم‌افزار مالکیتی محدود نیستند. آنها آزاد هستند تا تامین‌کننده خدمات را تغییر دهند یا نرم‌افزار را به دلخواه سفارشی کنند.

همچنین چندین معایب پتانسیلی وجود دارد که باید در نظر گرفته شود:

- پشتیبانی محدود

- متفاوت از نرم‌افزار ارائه‌دهنده ابر، OSS ممکن است پشتیبانی مشتری مستقل یا قراردادهای سطح خدمات نداشته باشد. اعتماد به پشتیبانی جامعه می‌تواند باعث شود زمان‌های پاسخگویی متفاوت باشند، بنابراین مسائل حیاتی ممکن است زمان بیشتری برای حل شوند. برای جلوگیری از این مشکل، شرکت‌ها اغلب یک قرارداد پشتیبانی از یک فروشنده خریداری می‌کنند. علاوه بر این، برخی از تامین‌کنندگان سخت‌افزار و نرم‌افزار اولویت می‌دهند پشتیبانی و سازگاری با راه‌حل‌های ارائه‌دهنده ابر بر روی OSS. این می‌تواند منجر به در دسترس بودن محدود درایورهای سخت‌افزار یا عدم پشتیبانی از برخی برنامه‌های نرم‌افزاری شود.

- منحنی یادگیری و تجربه کاربر

- کار با راه‌حل‌های OSS اغلب نیاز به توسعه‌دهندگان ماهر یا کارکنان فنی برای پیاده‌سازی، سفارشی‌سازی، و نگهداری آنها دارد. این می‌تواند منجر به هزینه‌های آموزش یا استخدام اضافی شود. پروژه‌های OSS نیز در مورد کیفیت مستندات، دسترسی کاربر به رابط‌ها، و سادگی نصب متفاوت هستند. برخی از پروژه‌ها ممکن است مستندات جامع نداشته باشند یا رابط‌های کاربری شفاف نباشند، که منجر به سختی‌های بیشتر در راه‌اندازی و پیکربندی اولیه می‌شود.

- تقسیم‌بندی، استانداردسازی، و سازگاری

- به دلیل طبیعت توزیع‌شده توسعه OSS و تعداد زیاد پروژه‌ها و نسخه‌های OSS، تقسیم‌بندی در سبک‌های برنامه‌نویسی، اصول، یا رویکردها وجود دارد. این می‌تواند منجر به تقسیم‌بندی شود، که انتخاب درست راه‌حل یا اطمینان از سازگاری بین اجزای مختلف OSS را دشوار می‌کند. ادغام با سیستم‌های موجود ممکن است نیاز به تلاش اضافی داشته باشد. (برای یادگیری بیشتر، به بخش "Hadoop" در صفحه ۲۳۵ مراجعه کنید.)

- امنیت و مسئولیت

- حتی در زیر نظر جامعه جهانی، آسیب‌پذیری‌ها و اشکالات هنوز در OSS وجود دارند. اگر از آن استفاده می‌کنید، باید در اعمال تصحیحات امنیتی و به‌روزرسانی‌ها با دقت باشید. علاوه بر این، اگر سازمان شما کد OSS را تغییر دهد، مسئولیت نگهداری امنیت و یکپارچگی نسخه سفارشی را بر عهده می‌گیرد.

- نامعلوم بودن نگهداری پروژه

- تمام پروژه‌های OSS به طور فعال توسعه یافته یا نگهداری طولانی مدت را ارائه نمی‌دهند. گاهی اوقات پروژه‌ها می‌توانند به حالت استراحت برسند، با به‌روزرسانی‌ها یا تصحیحات امنیتی کم. سطح فعالیت و پشتیبانی جامعه را قبل از اعمال یک راه‌حل OSS ارزیابی کنید.

- در نظر گرفتن مسئولیت مالکیت فکری

- هنگام استفاده یا کمک به OSS، سازمان‌ها باید مسیرهای مالکیت فکری و مسئولیت‌های مجوزی را مدیریت کنند. بسیار حیاتی است که شرایط و ضوابط مجوزهای OSS را درک کنید تا مطابقت را تضمین کنید و از مسائل قانونی جلوگیری کنید.

راه‌حل‌های داخلی

اگر شرکت شما یک راه‌حل انبار داده داخلی دارد، ممکن است به دنبال مهاجرت آن به ابر باشید یا حداقل پروژه‌های جدید در ابر را ساختن. با این حال، هنوز چندین موقعیتی وجود دارد که ممکن است بخواهید راه‌حل انبار داده خود را داخلی نگه دارید:

- اگر اتصال اینترنت شما کند یا وجود ندارد، مانند در یک معدن عمیق یا روی یک کشتی زیرزمینی، یک نفتخانه دریایی خارجی، یا یک کشتی مسافر،

- اگر نیاز به عملکرد میلی‌ثانیه‌ای دارید، مانند برای سرورها در یک تاسیس پردازش نامه با حجم بالا،

- اگر انتقال برنامه‌های خود به ابر منجر به شکست قرارداد پشتیبانی شده از طرف سوم شود،

- اگر شما یک اجاره قطع طولانی مدت با مرکز داده خود دارید یا تا حد زیادی خرید تجهیزات انجام داده‌اید،

- اگر مقدار زیادی از داده‌های داخلی دارید که باید به ابر منتقل شوند و می‌دانید که کانال شما برای برآورده کردن نیازهای گزارش‌دهی کاربران محدود است. گزارشات شما ممکن است داده‌های بروز را نشان ندهند، که ممکن است تصمیم‌گیری را تاخیر بزند.

- اگر قصد دارید برنامه‌ها و پایگاه‌های داده خود را در نزدیک آینده جایگزین یا بازنشستگی کنید،

- اگر داده‌های شما بسیار حساس هستند (اگرچه این دیگر بسیاری از سازمان‌ها را از قرار دادن داده در ابر منع نمی‌کند—اکثر CSPها اکنون ابرهای سراسری با سطح سرد دارند).

اگر هر یک از این موارد برای پایگاه‌های داده در شرکت شما اعمال می‌شود، شما باید با محدودیت‌ها آگاه باشید:

قابلیت مقیاس‌پذیری محدود

- توانایی شما برای مقیاس بندی زیرساخت شما محدود است بر اساس سخت‌افزاری که می‌توانید خریداری کنید. سخت‌افزار ممکن است محدودیت باشد اگر باید از برخی از تامین‌کنندگان استفاده کنید، اگر با سخت‌افزار وجود مشکلات سازگاری وجود داشته باشد، یا اگر تامین‌کنندگان سخت‌افزار با تاخیر در تولید مواجه شوند. به همین ترتیب، مراکز داده فقط فضای فیزیکی محدودی دارند. وقتی که فضای اضافی برای اضافه کردن سرورهای بیشتر به اندازه کافی نیست، شما با دو گزینه بسیار گران قرار دارید: گسترش مرکز داده خود یا ساخت یکی دیگر. هر دو می‌توانند چند ماه طول بکشد.

هزینه‌های پیش‌فرض

خرید سخت‌افزار برای راه‌حل‌های داخلی باعث می‌شود تا یک هزینه سرمایه‌گذاری پیش‌فرض بسیار زیاد را برای شما داشته باشید. سخت‌افزار شامل سخت‌افزار سرور، فضای مرکز داده برای نگهداری سخت‌افزار، دستگاه‌های ذخیره‌سازی اضافی، درایه‌های آتش‌نشان، سوئیچ‌های شبکه، یک شبکه با سرعت بالا (با تکرار) برای دسترسی به داده‌ها، تامینات برق و تامینات برق تکراری مورد نیاز برای نگهداری سیستم در حال کار و هزینه‌های امنیتی و پشتیبان‌گیری داده‌ها است.

اگر انبار شما برای شما حیاتی است، پس شما همچنین نیاز به تنظیم یک سایت بازیابی بعد از حوادث دارید، که به طور موثر هزینه‌ها را دو برابر می‌کند.

من اغلب به شرکت‌ها می‌گویم که اگر شرکت خود مرکز داده داخلی خود دارد، آنها در کسب و کار تهویه هستند، زیرا باید تمام این سخت‌افزار را خنک نگه دارند.

آیا نمی‌خواهید تمام تلاش خود را بر روی تجزیه و تحلیل داده‌ها متمرکز کنید؟ بیشتر شرکت‌ها ترجیح می‌دهند هزینه عملیاتی سالانه کار با یک ارائه‌دهنده ابر را داشته باشند.

هزینه‌های کارکنان

شما باید برای پرداخت به کارکنان یا مشاوران با تخصص مشاوره برای راه‌اندازی، مدیریت، اداره و پشتیبانی از سخت‌افزار و نرم‌افزار، تنظیم محصولات برای عملکرد، و ارائه محصولات و راه‌حل‌ها به تولید اضافه کنید. این می‌تواند یک ماهیت بندی برای مواجهه با مشکلات ایجاد کند و مسئولیت سیستم را با مشتری نه با فروشنده نگه دارد.

هزینه‌های نرم‌افزار

سازمان‌ها اغلب صدها هزار دلار در هزینه‌های مجوز نرم‌افزار برای نرم‌افزار انبار داده و بسته‌های اضافی، به علاوه هزینه‌های مجوز برای اعطای دسترسی به کاربران نهایی بیشتر، مانند مشتریان و تامین‌کنندگان، به داده‌ها می‌پردازند. قراردادهای پشتیبانی سالانه برای چنین نرم‌افزاری می‌تواند به سرعت به ۲۰٪ از هزینه اولیه مجوز برسد.

حالا بیایید به ابر نگاه کنیم.

راه‌حل‌های ارائه‌دهنده ابر

ابر مانند یک انبار ذخیره‌سازی بزرگ و غیرقابل مشاهده است که در جایی در اتر وجود دارد. این به ما اجازه می‌دهد تا داده‌ها را از هر جایی ذخیره، دسترسی پیدا کنیم و به اشتراک بگذاریم، در حالی که از فضای خالی شدن یا از دست دادن داده‌های ارزشمند جلوگیری می‌کند. تأثیر آن در تمام معماری‌های توصیف شده در این کتاب است و روش کار با داده‌ها را تغییر می‌دهد. محاسبات ابری دسترسی در زمان دهی به منابع محاسباتی مشترک مانند سرورها، ذخیره‌سازی و برنامه‌ها از طریق اینترنت را فراهم می‌کند، بنابراین دیگر نیازی به خرید، املاک و مدیریت مراکز داده و سرورهای فیزیکی ندارید. این برای محاسبات قابل مقیاس‌پذیری، انعطاف‌پذیری و کارآمد هزینه مناسب است.

بسیاری از مزایای داشتن یک راه‌حل انبار داده در ابر وجود دارد:

- قابلیت مقیاس‌پذیری و انعطاف‌پذیری

- نرم‌افزار CSP طراحی شده است تا به طور بی‌درز مقیاس شود. شما می‌توانید در طول کاربردهای متفاوت و در زمان‌های بار و سایر نیازهای کاری منابع ذخیره‌سازی و محاسباتی را پیش‌فرض کنید. ظرفیت هر چه شما نیاز دارید و هر زمانی که نیاز دارید.

- هزینه

- در سیستم ابری، پیچیدگی‌ها و هزینه‌های برنامه‌ریزی ظرفیت و اداره (مانند اندازه‌گیری، تعادل‌بندی و تنظیم سیستم) درون ساخته شده، خودکار و توسط هزینه اشتراک ابر پوشش داده می‌شوند. CSP همچنین مدل‌های قیمت‌گذاری انعطاف‌پذیری ارائه می‌دهند، بنابراین می‌توانید هزینه‌ها را با پرداخت فقط برای منابعی که واقعاً استفاده می‌کنید بهینه کنید. شما می‌توانید سخت‌افزار را کاهش دهید که هنگامی که تقاضا کاهش می‌یابد یا از گزینه‌های بدون سرور استفاده کنید.

- این به این معنی است که شما می‌توانید یک راه‌حل را به سرعت بسازید، و اگر کار نکرد، می‌توانید به سادگی تمام منابع ابری که پروژه استفاده کرده است را حذف کنید. هزینه شکست بسیار کاهش یافته است، که به این معنی است که می‌توانید ریسک‌ها را بگیرید و چیزهای جدید را امتحان کنید.

- همچنین مهم است که به حفظ هزینه‌های غیرمستقیم نگاه کنید: شما برای سخت‌افزار، برق، کارکنانی که مرکز داده را ساخته و نگهداری می‌کنند، یا توسعه به‌روزرسانی‌های خود پرداخت نمی‌کنید.

- راحتی در راه‌اندازی و مدیریت

- نرم‌افزار CSP اغلب با ابزارهای راه‌اندازی و مدیریت یکپارچه آمده است که فرآیندهای راه‌اندازی و پیکربندی را ساده می‌کنند. این ابزارها فرایند تامین منابع و وظایف مقیاس‌پذیری، نظارت و نگهداری را ساده می‌کنند، کاهش بار کار عملیاتی. شما می‌توانید تمام منابع ابری که نیاز دارید (مانند محاسبات، ذخیره‌سازی و شبکه) را در ساعات، اگر نه دقایق ایجاد کنید - در حالی که شروع یک راه‌حل در محل است می‌تواند ماه‌ها یا حتی ماه‌ها طول بکشد.

- پشتیبانی از فروشنده و قراردادهای سطح خدمات (SLAs)

CSP - ها پشتیبانی اختصاصی و SLAs را ارائه می‌دهند، کمک، رفع مشکلات و دسترسی خدمات گارانتی شده.

- اکوسیستم‌های یکپارچه

- معمولاً، نرم‌افزار CSP

خدمات مدیریت شده

CSP ها اغلب جنبه‌های خاصی از زیرساخت نرم‌افزار، مانند پایگاه‌های داده مدیریت شده، خدمات یادگیری ماشین یا محاسبات بدون سرور را مدیریت می‌کنند. این خدمات مدیریت شده می‌توانند پیچیدگی‌های عملیاتی را به عنوان بار بر برده و به سازمان‌ها اجازه دهند تا بر روی کسب و کار اصلی خود تمرکز کنند به جای مدیریت زیرساخت موجود. این همچنین شما را از پیدا کردن و پرداخت خدمات از تامین‌کنندگان سوم شرکت دار.

نوآوری مداوم و به‌روزرسانی ویژگی‌ها

CSP ها به طور منظم ویژگی‌های جدید، خدمات و به‌روزرسانی‌ها را به پیشنهادات نرم‌افزاری خود معرفی می‌کنند. شما از نوآوری آن‌ها بهره‌مند می‌شوید بدون اینکه نیاز به سرمایه‌گذاری در به‌روزرسانی یا ادغام ویژگی‌های جدید خود داشته باشید.

مدل‌های خدمات ابری

سه مدل اصلی برای ارائه و مدیریت منابع IT در ابر وجود دارد:

۱. سرویس به عنوان زیرساخت (IaaS)

در این مدل، ارائه دهنده خدمات ابری (CSP) زیرساخت سخت افزاری را میزبانی و مدیریت می‌کند. این مدل منابع محاسباتی مجازی مانند سرورها، ذخیره‌سازی و شبکه را ارائه می‌دهد که می‌توانند به راحتی افزایش یا کاهش یابند؛ مشتری سیستم عامل، برنامه‌ها و داده‌ها را مدیریت می‌کند. بیشتر خدمات درون یک ماشین مجازی (VM) قرار دارند که کاربر نهایی به صورت از راه دور به آن دسترسی خواهد داشت. این مدل انعطاف‌پذیری و قابلیت مقیاس‌پذیری بیشتری نسبت به سیستم‌های محلی ارائه می‌دهد، اما مدیریت سیستم عامل، برنامه‌ها و معماری داده‌ها هنوز نیاز به سطح متوسطی از دانش فناوری اطلاعات دارد.

۲. سرویس به عنوان پلتفرم (PaaS)

در این مدل، CSP یک پلتفرم را برای توسعه‌دهندگان ارائه می‌دهد تا بتوانند برنامه‌ها و پایگاه‌های داده را بسازند، آزمایش کنند و ارائه دهند، همچنین زیرساخت پایه‌ای (مشابه IaaS) را فراهم می‌کند. این مدل شامل ابزارهای توسعه، سیستم‌های مدیریت پایگاه داده و خدمات هوش مصنوعی است. CSP کنترل محیط اجرایی، میان‌سیستم‌ها و سیستم‌های عامل را دارد - هیچ VM وجود ندارد. این امکان را به توسعه‌دهندگان می‌دهد تا بر روی کدنویسی و مدیریت برنامه‌ها تمرکز کنند بدون نگرانی از مدیریت پلتفرم و زیرساخت پایه‌ای. این مدل نسبت به IaaS حتی بیشتر انعطاف‌پذیر و آسان‌تر برای استفاده است، اما در قابلیت سفارشی‌سازی و انعطاف‌پذیری محدودیت‌هایی دارد.

۳. سرویس به عنوان نرم‌افزار (SaaS)

در این مدل، CSP نرم‌افزار و زیرساخت را میزبانی و کاملاً مدیریت می‌کند؛ مشتریان به نرم‌افزار از طریق اینترنت دسترسی دارند و نیازی به مدیریت سخت افزار یا نرم‌افزار خود ندارند. این مدل ساده‌ترین و آسان‌ترین برای استفاده است، اما ممکن است در زمینه سفارشی‌سازی و کنترل محدودیت‌هایی داشته باشد. برخی از CSPها ابزارهای معماری داده‌هایی را ارائه می‌دهند که "مشابه SaaS" هستند: یعنی کاملاً مدیریت شده نیستند مانند یک ابزار SaaS واقعی مانند Salesforce، نیاز فقط به مدیریت برنامه کمی دارند.

با هر یک از مدل‌های خدمات ابری، شما آخرین نسخه محصول پایگاه داده‌ای که استفاده می‌کنید را دریافت می‌کنید. به عنوان مثال، پایگاه داده ابری SQL Database ،Azure، نسخه‌ای جدیدتر از یک محصول پایگاه داده

خلاصه - رمز گشایی از معمای داده

محلی به نام SQL Server ۲۰۲۲ است. SQL Database را می‌توان به عنوان SQL Server ۲۰۲۲+ در نظر گرفت. ویژگی‌های جدید هر چند هفته اضافه می‌شوند - شما می‌توانید انتخاب کنید که آن‌ها را استفاده کنید یا نه. هر چند سال، این ویژگی‌های جدید جمع‌شوند و به نسخه بسته‌شده SQL Server اضافه می‌شوند، اما شما با استفاده از SQL Database آن‌ها را فورا دریافت می‌کنید. و کد شما هنگام به‌روزرسانی شکست نخواهد خورد. تعداد زیادی از مردم از محصولات CSP استفاده می‌کنند؛ OSS می‌تواند داشته باشد که دارای یک پایگاه کاربری کمتر باشد. بنابراین، می‌توان برای پیدا کردن مراجعین کار که با محصولات ابری که سازمان شما استفاده می‌کند آشنا هستند، ساده‌تر بود.

در دسترسی جهانی و در دسترسی بالا

داده‌سازی‌های CSP در سراسر جهان توزیع شده‌اند، که به شما اجازه می‌دهد نرم‌افزار خود را نزدیک‌تر به مخاطب هدف خود راه‌اندازی کنید یا از تکرار چند منطقه‌ای برای در دسترسی بالا استفاده کنید. CSP ها از تعادل‌بندی بار برای توزیع بار بر روی چندین سرور استفاده می‌کنند، اطمینان حاصل می‌کنند که هیچ سرور واحدی بار زیادی را بپذیرد و باعث تعطیل شدن زمان نشود.

این زیرساخت جهانی خدمات با کمترین تاخیر را تضمین می‌کند و مقاوم در برابر خطا است. در صورت خرابی سخت‌افزار یا نرم‌افزار، خدمات به طور خودکار به یک نمونه ثانویه فشار می‌آیند، اطمینان حاصل می‌کند که دسترسی بدون وقفه و قابل اعتماد حفظ شود. همچنین، شما می‌توانید برای بازیابی بعد از حوادث برای ذخیره‌سازی در ماده چند دقیقه راه‌اندازی کنید.

امنیت و مطابقت با قوانین

CSP ها در مواردی مانند رمزگذاری، کنترل دسترسی و بازرسی‌های منظم سرمایه‌گذاری می‌کنند تا داده‌ها را محافظت کنند و مطابقت با استانداردها و قوانین صنعت را تضمین کنند.

با هر یک از مدل‌های خدمات ابری، شما آخرین نسخه محصول پایگاه داده‌ای که استفاده می‌کنید را دریافت می‌کنید. به عنوان مثال، پایگاه داده ابری SQL Database، Azure، نسخه‌ای جدیدتر از یک محصول پایگاه داده محلی به نام SQL Server ۲۰۲۲ است. SQL Database را می‌توان به عنوان SQL Server ۲۰۲۲+ در نظر گرفت. ویژگی‌های جدید هر چند هفته اضافه می‌شوند - شما می‌توانید انتخاب کنید که آن‌ها را استفاده کنید یا نه. هر چند سال، این ویژگی‌های جدید جمع‌شوند و به نسخه بسته‌شده SQL Server اضافه می‌شوند، اما شما با استفاده از SQL Database آن‌ها را فورا دریافت می‌کنید. و کد شما هنگام به‌روزرسانی شکست نخواهد خورد.

همه CSPها تشخیص پیشرفته تهدیدات داخلی را با استفاده از نظارت برای تشخیص فعالیت‌های غیرمعمول که ممکن است نشان‌دهنده تلاش‌های غیرمعمول و احتمالاً مضر برای دسترسی یا استفاده از پایگاه‌های داده باشند، ارائه می‌دهند؛ ارزیابی‌هایی که به شما کمک می‌کنند تا آسیب‌پذیری‌های احتمالی پایگاه داده را کشف، ردیابی و رفع کنید؛ و قابلیت‌های پیشرفته برای کشف، طبقه‌بندی، برچسب‌زنی و گزارش داده‌های حساس در پایگاه‌های داده شما.

در کل، همانطور که شکل ۱۶-۱ نشان می‌دهد، این چهار مدل میزان‌های مختلفی از انعطاف‌پذیری و کنترل بر روی زیرساخت فناوری اطلاعات و پشته نرم‌افزار را نشان می‌دهند. سیستم‌های محلی کنترل کامل را ارائه می‌دهند اما نیاز به سرمایه‌گذاری قابل توجه و دانش متخصص دارند، در حالی که PaaS، IaaS و SaaS سطوح افزایشی از انعطاف‌پذیری و آسانی استفاده را ارائه می‌دهند - با هزینه کنترل و سفارشی‌سازی.

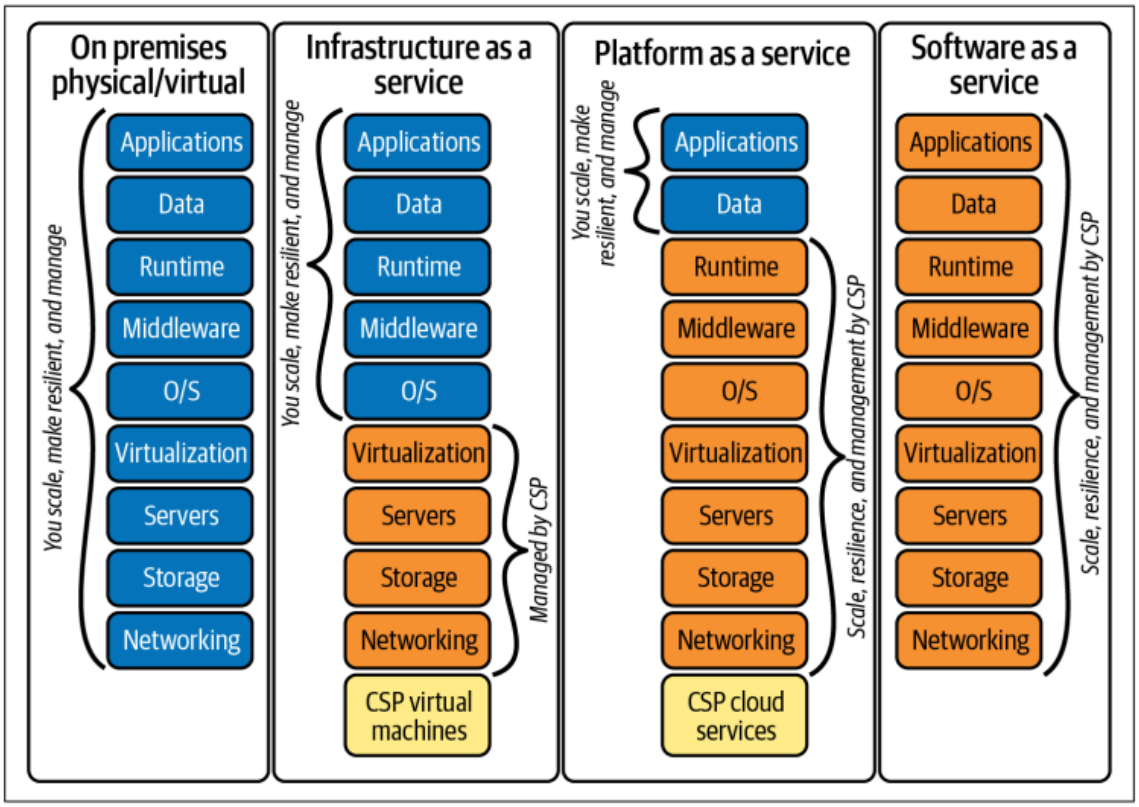


Figure 16-1. Comparison of service models

انتخاب مدل خدماتی که شما انتخاب می‌کنید، بستگی به نیازها و اهداف سازمان شما دارد. با این حال، شخصاً ترجیح می‌دهم PaaS برای انبارهای داده که امکان پذیر است، برای چندین دلیل. اولاً، شما نیازی به مقابله با یک ماشین مجازی (VM) ندارید. این VM در جایی وجود دارد و پایگاه‌های داده را میزبانی می‌کند، اما شما هرگز نیازی به ورود به سرور یا مدیریت آن در هر روشی ندارید - همه چیز از طریق پنل CSP یا ابزارهای شخص ثالث انجام می‌شود. همچنین شما مسئول تعمیر یا به‌روزرسانی نیستید: بیشتر پایگاه‌های داده CSP با هیچ دوره تعطیلی تعمیر می‌شوند. شما پشتیبان‌گیری پایگاه داده را به طور خودکار دریافت می‌کنید - بیشتر تنظیم و نظارت نیازی نیست. در نهایت، PaaS باعث می‌شود بازیابی از حوادث (DR) بسیار ساده‌تر شود. بیشتر راه‌حل‌های DR برای IaaS برای تنظیم و نظارت دشوار هستند، اما با PaaS، شما به طور کلی فقط باید کلیک کنید و منطقه‌ای از کشور را انتخاب کنید که می‌خواهید پایگاه داده DR در آن قرار گیرد. CSP مسئول تنظیم آن و نگهداری آن است.

ارائه‌دهندگان اصلی ابری

سه ارائه‌دهنده اصلی خدمات ابری (CSP)، Amazon Web Services (AWS)، Microsoft Azure و Google Cloud Platform (GCP)، اکثریت بازار را در نیمه دوم سال ۲۰۲۳ با تخمین ۶۵ درصد دارند. برای اطلاعات بیشتر و به‌روز درباره پیشنهادات هر ارائه‌دهنده، توصیه می‌کنم به وب‌سایت‌های مربوطه مراجعه کنید. جدول ۱۶-۱ ویژگی‌های اصلی آن‌ها را مقایسه می‌کند تا به شما یک بررسی جامع از آن‌ها ارائه دهد.

Table 16-1. Comparison of the big three cloud service providers

Criteria	Azure	AWS	GCP
Year announced	2008	2006	2008
Data center regions	60	31	24
Global market share	23%	32%	10%
Services offered	200+ services across various categories	Around 200+ services across various categories	100+ services, known for data analytics and machine learning
Pricing	Cost benefits for Microsoft users	More options for instance types	Straightforward and customer friendly
Integration/open source	Strong integration with Microsoft products	Wide variety of open source integrations	Good integrations with various open source platforms
Learning curve	Easier for those familiar with other Microsoft products	Wide range of services leading to steeper learning curve	User-friendly interface, easier learning curve
Analytic products	Azure Synapse Analytics, Microsoft Fabric	Amazon Redshift	Google BigQuery

هر یک از این سه ارائه‌دهنده خدمات ابری اصلی قوت‌های منحصر به فرد خود را دارند، و انتخاب بین آن‌ها اغلب به نیازهای خاص و زیرساخت موجود شما بستگی دارد. یک کسب و کار که به شدت در اکوسیستم Microsoft جذب شده است، ممکن است به سمت Azure گرایش داشته باشد، یک کسب و کار که بر روی پردازش داده‌های پیشرفته تمرکز دارد ممکن است برای GCP انتخاب کند، و یک کسب و کار که نیاز به مجموعه‌ای از خدمات گسترده دارد ممکن است به سمت AWS گرایش داشته باشد. در نهایت، تصمیم شامل تعادل بین نیازهای فنی، هزینه و جهت‌گیری استراتژیک کسب و کار است.

راه‌حل‌های چند ابری

بسیاری از سازمان‌ها به دنبال تصمیم‌گیری بین پذیرش یکی از سه ارائه‌دهنده خدمات ابری بزرگ (CSP) یا استفاده از خدمات از بیش از یک مورد، که اغلب به عنوان رویکرد چند ابری شناخته می‌شود، هستند. برخی از شرکت‌ها احساس می‌کنند که اعتماد به تنها یک CSP معادل "قرار دادن تمام تخم‌های خود در یک سطل" است یا از اینکه CSP ممکن است به تنافس کننده کسب و کار آن‌ها تبدیل شود، می‌خواهند. (به عنوان مثال، زمانی که Amazon Whole Foods را خریداری کرد و به یک رقیب در بازار سوپرمارکت‌ها تبدیل شد.)

این رویکرد چندین مزیت بزرگی دارد، عمدتاً زیرا از قوت‌ها و ضعف‌های هر CSP استفاده می‌کند. اگر یک CSP دارای کمترین تاخیر باشد و یکی دیگر دارای بهترین دردسر یا بازیابی از حوادث باشد، استفاده از هر دو می‌تواند توانایی سازمان را برای برآورده کردن معیارهای خدمات (SLA) خود بهبود بخشد. می‌توانید مقایسات مشابهی برای هزینه، ظرفیت و ویژگی‌ها و محصولات مختلف انجام دهید؛ اگر یک CSP همه چیزی که می‌خواهید را نداشته باشد، می‌توانید بر روی یکی دیگر برای تکمیل آن اعتماد کنید. با این حال، این کمتر مرتبط است که قبلاً بود؛ امروزه، همه سه CSP پیشنهادات بسیار مشابهی دارند، با چندین ویژگی یا محصول منحصر به فرد.

همچنین قضاوت داده‌ها یک عامل است. تمام CSPها دارای مناطق ابری مشابهی نیستند، که می‌تواند برای رعایت قوانین و مقررات بسیار مهم باشد. به عنوان مثال، اگر شما در Azure از MongoDB استفاده می‌کنید اما Azure منطقه‌ای در تایوان ندارد، عملیات شما در تایوان می‌توانند از MongoDB در AWS استفاده کنند.

تصور کنید یک عظیم سرمایه‌گذار تجارت الکترونیکی خیالی که ما آن را Tech Treasures Inc. می‌نامیم. Tech Treasures در ایالات متحده - به طور خاص در Ponte Vedra، فلوریدا - مستقر است اما در سراسر جهان عمل می‌کند، از جمله آمریکای شمالی، اروپا و آسیا.

برای تضمین عملکرد بهینه، قابلیت اطمینان و مطابقت با قوانین محلی داده، Tech Treasures از رویکرد چند ابری استفاده می‌کند. این از Azure Microsoft برای عملیات آمریکای شمالی خود استفاده می‌کند به دلیل شبکه و مراکز داده با کمترین تاخیر قوی آن در منطقه و در اروپا، از GCP برای قابلیت‌های پیشرفته تجزیه و تحلیل داده و حضور قوی آن در مناطق ابری اروپایی استفاده می‌کند. برای عملیات آسیایی خود، Tech Treasures به AWS اعتماد دارد، که مناطق ابری در بازارهای آسیایی کلیدی از جمله تایوان را ارائه می‌دهد.

این استراتژی چند ابری به TechTreasures امکان می‌دهد تا خدمات سریع و قابل اطمینان را به مشتریان جهانی خود ارائه دهد در حالی که مطابقت با قوانین قضاوت داده را تضمین می‌کند. این یک انتخاب استراتژیک است که به کسب و کار اجازه می‌دهد تا قوت‌های منحصر به فرد هر یک از سه ارائه‌دهنده خدمات ابری بزرگ را برای بهینه‌سازی عملیات کسب و کار استفاده کند.

اما همچنان بسیاری از معایب در چند ابری وجود دارد. از آنجا که این ابرها برای کار کردن با هم طراحی نشده‌اند، حرکت داده و محاسبات به سمت یکدیگر می‌توانند به آرامی و سنگین باشد. همچنین قابلیت تعامل یک مشکل است؛ محصولات یک CSP همیشه با محصولات از CSPهای مختلف خوب کار نمی‌کنند. حرکت داده و برنامه‌ها از یک CSP به یکی دیگر و راه‌اندازی محصول دیگری می‌تواند بسیار گران باشد، به خصوص اگر باید برنامه‌ها را برای پشتیبانی از چند ابر تعمیر کنید. CSPها هزینه‌های خروجی را در صورت حرکت داده از ابر آن‌ها به ابر یک رقیب اعمال می‌کنند و آن را آسان نمی‌کنند که به یک CSP دیگر مهاجرت کنید. و اگر شما در یک پایپ اینترنت بزرگتر، مانند Azure که Route Express نام دارد، سرمایه‌گذاری کرده‌اید، باید با هر CSP دیگری که استفاده می‌کنید سرمایه‌گذاری مشابهی انجام دهید. علاوه بر این، ذخیره داده‌های شما در چند ابر باعث افزایش در معرض تهدیدات امنیتی می‌شود.

اگر می‌خواهید انتقال از یک ابر به یک ابر دیگر را آسان کنید، باید از یک مدل خدمات IaaS به جای PaaS استفاده کنید، که بسیاری از خدمات با ارزش بالاتری که هر یک از CSPها ارائه می‌دهد را برداشته می‌کند - یک هزینه ارزش بالا.

در مورد افراد کار، شما باید افراد را استخدام و/یا آموزش دهید تا دو یا چند ابر را درک کنند، که هزینه‌ها را به شدت افزایش می‌دهد. مدیریت یک CSP اضافی؛ مدیریت امنیت؛ و ایجاد سیاست‌ها، استانداردها و روش‌هایی برای آن نیز نیاز به افراد بیشتر و مجموعه‌ای از مهارت‌ها دارد. شما همچنین باید انواع مختلفی از امنیت را برای کار کردن با هم فعال کنید، و دو مکان برای اضافه کردن، حذف یا به‌روزرسانی کاربران؛ دو مکان برای نظارت؛ و دو مجموعه از گزارشات قبض دارید، که پیچیدگی و هزینه مدیریتی قابل توجهی اضافه می‌کند.

در نهایت، داشتن دو ارائه‌دهنده ابروی احتمالاً شما را از یک قطعی اطلاع نمی‌دهد. CSPها شبکه‌های خود را به طور خاص طراحی کرده‌اند تا یک قطعی در یک منطقه تاثیری بر سایر مناطق نداشته باشد. راه حل بازیابی از حوادث شما باید به یک منطقه دیگر در همان CSP فرار کنید، نه به یک CSP دیگر.

بسیاری از افراد باور دارند که استفاده از رویکرد چند ابری منجر به قرار دادن CSPها در یک موقعیت رقابتی می‌شود، که به شما اجازه می‌دهد تا یک معامله بهتر را منجر کنید. با این حال، این بیشتر یک افسانه است.

برای قراردادهای مجوز کسب و کار (CSP ،(ELAsها تخفیفات بیشتری را بر اساس مقدار مصرفی که می‌توانید در ابر آن‌ها تعهد کنید ارائه می‌دهند - هر چه بیشتر استفاده کنید، تخفیفات بزرگتری دارید. وقتی شما قیمت‌گذاری مبتنی بر سطح و تخفیفات استفاده مداوم را اضافه کنید، احتمالاً با باقی ماندن فقط یک CSP کمتر می‌توانید وفادار شوید. با این حال، برخی از شرکت‌ها تهدید می‌کنند که به یک CSP دیگر منتقل شوند تا قیمت‌گذاری خود را کنترل کنند. باقی ماندن فقط یک CSP همچنین می‌تواند برای شرکت‌های بزرگ کسب و کار امکانات همکاری را فراهم کند. به عنوان مثال، Microsoft با همکاری با Young & Ernst برای رشد ۱۵ میلیارد دلاری و فرصت نوآوری فناوری در سراسر صنایع اقدام کرده است.

من انتظار دارم که رویکرد چند ابری به طور گسترده‌تری پذیرفته نشود. در طول زمان، فکر می‌کنم که بیشتر شرکت‌ها یک CSP را انتخاب و با آن باقی می‌مانند، تا به طور عمیق‌تری و به طور کامل خدمات آن، PaaS و SaaS، و اکوسیستم را استفاده کنند. برخی از شرکت‌های بسیار بزرگ ممکن است در صورتی که CSPها چیزهای مختلفی را ارائه دهند که به بار کار مفید هستند، بر روی دو ابر تعیین کنند. این منطقی است اگر شرکت بزرگ به اندازه‌ای باشد که زمان و افراد را برای رفتن عمیق بر هر دو و استفاده از خدمات PaaS هر ارائه‌دهنده را سرمایه‌گذاری کند.

حالا از مدل‌های خدمات ابری به چارچوب‌های نرم‌افزاری منتقل می‌شویم، از ایجاد پایه معماری داده به بررسی ابزارهایی که این پایه را بهره‌برداری می‌کنند. در بخش بعدی، به چارچوب‌های نرم‌افزاری کلیدی که زیرساخت ابری را برای باز کردن پتانسیل کامل داده‌های شما استفاده می‌کنند نگاه خواهیم کرد.

چارچوب‌های نرم‌افزاری

در دنیای معماری داده، انتخاب شما از چارچوب‌های نرم‌افزاری می‌تواند تفاوت بزرگی در این کنترل کند که چگونه به طور موثری داده‌های خود را پردازش، تجزیه و تحلیل کنید و از آن‌ها بینش بگیرید. این چارچوب‌ها به عنوان موتورهایی عمل می‌کنند که عملیات داده‌های شما را قدرت می‌دهند، به شما اجازه می‌دهند اطلاعات ارزشمندی استخراج کنید و تصمیمات مبتنی بر داده را هدایت کنید. در این بخش، ما به دنیای چارچوب‌های نرم‌افزاری منتقل می‌شویم، با بررسی سه نماینده برجسته: Databricks، Hadoop، و Snowflake. هر یک از این چارچوب‌ها قوت‌های منحصر به فردی را به میز می‌آورد و درک قابلیت‌های آن‌ها برای ساخت یک معماری داده که با نیازها و اهداف خاص شما مطابقت دارد، ضروری است.

Hadoop

Hadoop یک چارچوب نرم‌افزاری متن‌باز (OSS) است که در سال ۲۰۰۵ توسط فناوری Apache ایجاد شده است. این چارچوب طراحی شده است تا قدرت محاسبات توزیع‌شده را بهره‌برداری کند. ابزارها و کتابخانه‌های مختلفی قابلیت‌های Hadoop را گسترش می‌دهند، که آن را به یک انتخاب محبوب برای سازمان‌هایی که با حجم‌های بزرگی از داده‌ها مواجه هستند و به دنبال قابلیت مقیاس‌پذیری با هزینه کم هستند، تبدیل می‌کند.

در هسته‌ای، Hadoop شامل دو بخش اصلی است. اول، سیستم فایل توزیع‌شده (HDFS Hadoop) می‌تواند با تقسیم داده‌ها به بلوک‌های کوچک‌تر و توزیع آن‌ها در سراسر گره‌ها و خوشه‌های متعدد کامپیوتر، به طور کارآمد برای پردازش کاربردهای بزرگ داده استفاده کند، همانطور که در شکل ۱۶-۲ نشان داده شده است. این امکان مقاومت بالا در برابر خطاها و وظایفی مانند پردازش دسته‌ای، یکپارچگی داده‌ها و تجزیه و تحلیل را فراهم می‌کند. HDFS دسترسی بالا به داده‌ها را فراهم می‌کند و حتی در صورت شکست سخت‌افزار، قابلیت اطمینان را فراهم می‌کند.

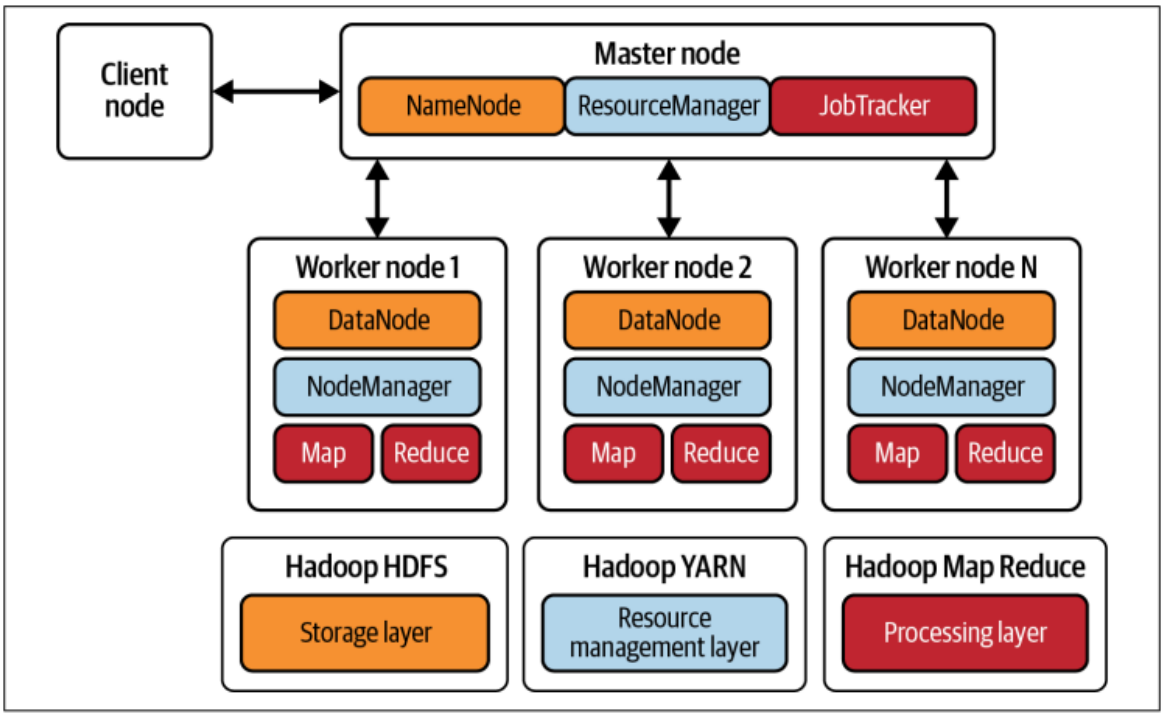


Figure 16-2. Hadoop's cluster architecture

دوم، چارچوب MapReduce یک مدل برنامه‌نویسی است که برای پردازش و تجزیه و تحلیل مجموعه‌های داده بزرگ در پارالل در سراسر یک خوشه استفاده می‌شود، با تقسیم بار کار به وظایف کوچک‌تر و توزیع آن‌ها در سراسر گره‌های متعدد در خوشه. این چارچوب محاسبات بزرگ و پیچیده را به چندین وظیفه تقسیم می‌کند و این وظایف را به گره‌های کارگر فردی اختصاص می‌دهد که نتایج را هماهنگ و ترکیب می‌کنند. چارچوب MapReduce به طور خودکار مقاومت در برابر خطاها، پارالل‌سازی و توزیع داده را مدیریت می‌کند. یک برنامه MapReduce شامل یک

رویه Map() است که داده‌ها را فیلتر و مرتب می‌کند (مانند مرتب‌سازی دانش‌آموزان بر اساس نام اول به صف‌ها، یک صف برای هر نام) و یک رویه Reduce() که داده‌ها را خلاصه می‌کند (مانند شمارش تعداد دانش‌آموزان در هر صف، که فرکانس نام‌ها را تولید می‌کند). کتابخانه‌های MapReduce در زبان‌های برنامه‌نویسی- مختلفی (معمولاً جاوا) نوشته شده‌اند، با سطوح مختلف بهینه‌سازی.

Hadoop چندین مزیت ارائه می‌دهد که آن را به یک انتخاب محبوب برای مدیریت داده‌های بزرگ تبدیل می‌کند:

قابلیت مقیاس‌پذیری

Hadoop یک معماری قابل مقیاس‌پذیری ارائه می‌دهد که قادر به پردازش حجم‌های بزرگی از داده است. این به شما اجازه می‌دهد تا قابلیت‌های ذخیره‌سازی و پردازش خود را به راحتی با افزودن سخت‌افزار متداول به خوشه مقیاس بزرگ کنید، که این کار هزینه‌محاسبه و انعطاف‌پذیری را افزایش می‌دهد.

محاسبات توزیع‌شده

Hadoop داده‌ها و وظایف پردازش را در سراسر یک خوشه از ماشین‌ها توزیع می‌کند، که پردازش موازی را فراهم می‌کند. تقسیم کار بین چندین گره اجازه می‌دهد تا به سرعت بزرگ‌تر مجموعه‌های داده را پردازش کند.

مقاومت در برابر خطا

Hadoop داده‌ها را در چندین گره در خوشه تکرار می‌کند، اطمینان حاصل می‌کند که داده‌ها باقی می‌مانند و حتی در صورت شکست سخت‌افزار در دسترس هستند. اگر یک گره شکست خورد، Hadoop به طور خودکار وظایف را به گره‌های دیگر موجود تغییر می‌دهد، که اطمینان حاصل می‌کند پردازش بدون وقفه ادامه دارد.

ذخیره‌سازی هزینه‌محاسبه

HDFS به شما اجازه می‌دهد تا مقادیر زیادی از داده را بر روی سخت‌افزار استاندارد و قابل خرید که می‌توانید از طریق فروشگاه خریداری کنید، به جای سیستم‌های ذخیره‌سازی متخصص و گران‌قیمت، ذخیره کنید.

محلیت داده

Hadoop با جابجایی محاسبات نزدیک به محل ذخیره‌سازی داده، به جای انتقال داده در شبکه، پردازش داده را بهینه‌سازی می‌کند، کاهش بار شبکه و بهبود کلی کارایی را فراهم می‌کند.

انعطاف‌پذیری و سازگاری

Hadoop با فرمت‌های داده مختلف سازگار است و قادر به پردازش داده‌های ساختاری، نیمه ساختاری و غیرساختاری است. این پشتیبانی از محدوده گسترده‌ای از زبان‌های برنامه‌نویسی- و ابزارها را فراهم می‌کند، که به توسعه‌دهندگان اجازه می‌دهد با زبان‌های مورد علاقه خود کار کنند و اجزای مختلف را در خود کانال‌های داده خود ادغام کنند.

اکوسیستم ابزار

Hadoop یک اکوسیستم غنی از ابزارها و چارچوب‌ها دارد که قابلیت‌های آن را گسترش می‌دهند و انتزاعات سطح بالاتر، زبان‌های پردازش داده و تجزیه و تحلیل را فراهم می‌کنند. این شامل:

خلاصه - رمزگشایی از معماری داده

YARN** -**، برای مدیریت منابع و برنامه‌ریزی کار

Hive** -**، برای انبار داده

Pig** -**، برای تجزیه و تحلیل داده

HBase** -**، یک پایگاه داده NoSQL

Spark** -**، برای پردازش در حافظه

Presto** -**، برای SQL توزیع‌شده

فروشندگان Hadoop پلتفرم‌هایی را فروش می‌دهند که چندین محصول متن‌باز (OSS) را در بر می‌گیرند، آزمایش می‌کنند و به صورت پیش‌از ادغام می‌کنند، که به شما وقت و تلاش ادغام و آزمایش مختلف اجزای خود را نشان می‌دهد.

پشتیبانی از جامعه

Hadoop یک جامعه متن‌باز بزرگ و فعال دارد که به توسعه، بهبود و پشتیبانی از آن کمک می‌کند و منابع غنی، مستندات و دانش را به کاربران ارائه می‌دهد.

ناقصیت‌های پتانسیلی Hadoop

- **پیچیدگی**: Hadoop یک منحنی یادگیری شدید دارد و ممکن است پیچیده باشد تا راه‌اندازی و مدیریت آن. این نیاز به دانش و تخصص در سیستم‌های توزیع‌شده دارد که ممکن است برای سازمان‌هایی بدون تجربه قبلی یا منابع اختصاصی چالش برانگیز باشد.

- **تاخیر**: MapReduce، چارچوب پردازش اصلی Hadoop، بر پایه پردازش دسته‌ای است که تاخیر را معرفی می‌کند، بنابراین ممکن است برای سناریوهای پردازش داده در زمان واقعی یا تعاملی مناسب نباشد. Hadoop برای وظایف پردازش داده در مقیاس بزرگ، مانند تجزیه و تحلیل دسته‌ای و پردازش مجموعه‌های داده بزرگ، بسیار موثر است. اگر نیاز به بینش‌های فوری یا نزدیک به زمان واقعی دارید، راه‌حل‌ها یا چارچوب‌های جایگزین مانند Apache Kafka یا Apache Flink ممکن است مناسب‌تر باشند.

بار اضافی مدیریت داده

در حالی که Hadoop با HDFS مقاومت در برابر خطا و تکرار را فراهم می‌کند، تکرار داده در گره‌ها فضای ذخیره‌سازی را مصرف می‌کند. این باعث می‌شود بار اضافی در ذخیره‌سازی داده ایجاد شود.

اکوسیستم پیچیده

در حالی که اکوسیستم Hadoop غنی و گسترده است، همچنین پیچیده است، بنابراین مدیریت و ادغام اجزای آن می‌تواند چالش‌برانگیز باشد. این نیاز به درک قابلیت‌ها و سازگاری هر ابزار و حفظ پیکربندی‌ها و وابستگی‌های ابزارها دارد.

امنیت داده و حاکمیت

طبیعت توزیع‌شده Hadoop و تکرار داده می‌تواند چالش‌های منحصر به فرد در زمینه امنیت داده، حریم خصوصی و حاکمیت را معرفی کند. برخلاف سیستم‌های ذخیره‌سازی داده سنتی، طبیعت توزیع‌شده Hadoop نیازمند این است که سازمان‌ها فراتر از روش‌های امنیتی استاندارد بروند و اقدامات امنیتی و کنترل‌های دسترسی مخصوص به آن را بپذیرند. این اقدامات ضروری هستند تا حفظ حفاظت از داده‌ها و حفظ مطابقت با نیازها و بهترین روش‌های خاص Hadoop را اطمینان حاصل کنند.

رشد و تغییر در استفاده از Hadoop

Hadoop در دهه ۲۰۱۰ اولیه مقدار قابل توجهی از مصرف کرد، اما با توسعه چشم‌انداز داده‌های بزرگ و محاسبات ابری، تکنولوژی‌هایی مانند محاسبات بدون سرور، انبارهای داده ابری و پلتفرم‌های جریان داده، استفاده از Hadoop کاهش یافته است. امروزه، زیرساخت ابری می‌تواند جایگزین خوشه‌های Hadoop باشد و ذخیره‌سازی ابری می‌تواند جایگزین HDFS باشد. اصول اصلی آن، مانند ذخیره‌سازی و پردازش توزیع‌شده، همچنان مرتبط هستند، اما اجزا و ابزارهای خاص و مشخص آن در میزان مختلفی از مصرف را داشته‌اند.

به عنوان مثال، تکنولوژی‌هایی مانند Spark Apache به عنوان جایگزین‌ها یا مکمل‌های MapReduce محبوبیت کسب کرده‌اند.

Databricks

Databricks یک پلتفرم مبتنی بر ابر است که برای اجرای بارهای کاری در Spark Apache، یک موتور پردازش تحلیل داده‌های متن‌باز است. Spark در ابتدا در سال ۲۰۰۹ در MPLab دانشگاه Berkeley UC توسعه یافت تا با محدودیت‌های MapReduce مقابله کند. این برنامه برای انجام کارهای پردازش دسته‌ای و جریان داده در زمان واقعی طراحی شده بود.

برخی از سازندگان اصلی Spark در سال ۲۰۱۳ به عنوان بنیانگذاران Databricks شرکت کردند. Databricks ادامه داده است تا Spark را تجاری کند، یک پلتفرم تحلیل داده یکپارچه را ارائه می‌دهد، یک سرویس کاملاً مدیریت شده که یک پلتفرم ساده شده برای تحلیل داده‌های بزرگ را ارائه می‌دهد، که علم داده، مهندسی و کسب و کار را یکپارچه می‌کند. این طراحی شده است تا برای کسب و کارها آسان‌تر باشد از Spark استفاده کنند بدون اینکه نگران راه‌اندازی و نگهداری ساختار خود باشند، و تحلیل‌های پیشرفته و یادگیری ماشین را به مجموعه‌ای گسترده‌تر از کاربران قابل دسترسی کند.

مشارکت در توسعه Apache Spark

از زمان تأسیس، Databricks یک مشارکت مهم در Spark Apache بوده است، کار کرده است تا سرعت پردازش آن را از طریق پروژه Tungsten، سری بهینه‌سازی‌هایی که با هدف استفاده از معماری‌های CPU مدرن برای افزایش کارایی طراحی شده‌اند، بهبود ببخشد. همچنین یک سیستم اتوماتیک بهینه‌سازی شده را توسعه داده است که به طور دینامیکی تعداد ماشین‌های اختصاص یافته به یک کار را بر اساس نیازهای محاسباتی تنظیم می‌کند، که هر دو هزینه‌پذیری و عملکرد را بهبود می‌بخشد.

کتابخانه‌های گسترش قابلیت‌های Spark

Databricks کتابخانه‌های مختلفی را معرفی کرده است تا قابلیت‌های Spark را گسترش دهد، مانند MLlib (برای یادگیری ماشین)، GraphX (برای محاسبات گراف) و Streaming Spark (برای داده‌های زمان واقعی). همچنین نقش حیاتی در آموزش و پشتیبانی از جامعه Spark ایفا کرده است، که موجب شده استفاده و پذیرش سریع‌تر از قابلیت‌های Spark را فراهم کند.

ساده‌سازی استفاده از Spark

Databricks بر روی ساده‌سازی استفاده از Spark تمرکز داشته است، واژه‌نامه‌های سطح بالا در Java، Python و Scala را معرفی کرده است تا نوشتن برنامه‌های Spark را قابل دسترسی‌تر کند. همچنین نوتبوک‌های Databricks را توسعه داده است، محیط‌های تعاملی که در آن علم‌دان‌های داده و مهندسان می‌توانند کد Spark را بنویسند، نتایج خود را بصری کنند و مدل‌ها را بسازند. این پلتفرم اجازه داده علم‌دان‌ها و مهندسان داده برای کار به صورت موثرتر با هم به یکپارچگی علم داده، مهندسی داده و تحلیل کسب و کار کردند.

بهبود محلات داده و افزایش قابلیت استفاده از Delta Lake

برای مقابله با محدودیت‌های دیتا لیک‌ها و بهبود قابلیت اطمینان و کیفیت داده، Databricks همچنین Delta Lake را معرفی کرده است، یک لایه ذخیره‌سازی متن‌باز که بر روی دیتا لیک‌های موجود اجرا می‌شود. این بهبود قابلیت‌های بزرگ داده Spark را با تراکنش‌های ACID، مدیریت متادیتای قابل مقیاس‌پذیری و پردازش داده‌های در حال جریان و دسته‌ای یکپارچه می‌کند. Delta Lake به چالش‌های دیتا لیک‌های سنتی با کیفیت داده، قابلیت اطمینان و عملکرد پاسخ می‌دهد. Databricks دیتا لیک را در سال ۲۰۱۹ متن‌باز کرده و پروژه را به یاد فردین

لینوکس ارائه داده است تا جامعه گسترده‌تری بتواند به آن کمک کند. امروزه، Delta Lake به طور گسترده‌ای پذیرفته شده است.

معرفی معماری داده‌های جدید: دیتا لایک‌هاوس

در سال ۲۰۲۰، Databricks یک معماری داده جدید را معرفی کرد، دیتا لایک‌هاوس (همچنین در فصل ۱۲ پوشش داده شده است). این نوآوری نشان‌دهنده یک تغییر قابل توجه در معماری‌های مدیریت داده بود. به جای اینکه نیاز به انتخاب بین دیتا لیک‌ها و انبارهای داده داشته باشند، سازمان‌ها می‌توانند از یک معماری ترکیبی استفاده کنند که مقیاس‌پذیری و هزینه‌پذیری دیتا لیک‌ها را با ساختار قوی، قابلیت اطمینان و عملکرد انبارهای داده ترکیب می‌کند.

از دیدگاه Databricks، معماری‌های داده مدرن باید باز، همکاری‌پذیر و قادر به مدیریت همه انواع داده و موارد استفاده از تحلیل‌های پیشرفته باشند. بنابراین، شرکت معماری‌هایی را پشتیبانی می‌کند که به کسب و کارها اجازه می‌دهد هنگام رشد و تغییر در طول زمان مقیاس بزنند.

با پلتفرم تحلیل داده یکپارچه و ابزارهایی مانند Databricks، Delta Lake می‌تواند زیربنای زیرساخت را برای هر محصول یا حوزه داده در یک شبکه داده (مشاهده فصل‌های ۱۳ و ۱٤) فراهم کند. این پلتفرم قادر است با مدیریت انواع مختلف داده و پشتیبانی از پردازش داده‌های دسته‌ای و زمان واقعی با انواع مختلف داده کار کند، که به اندازه کافی متنوع است تا نیازهای حوزه‌های داده مختلف را برآورده کند. ویژگی‌های کلیدی Delta Lake، مانند تراکنش‌های ACID، اجرای اسکیما و نسخه‌بندی، می‌تواند کیفیت و یکپارچگی داده را در هر حوزه داده تضمین کند. علاوه بر این، امکان "سفر در زمان" (دسترسی به نسخه‌های قبلی داده) را فراهم می‌کند. این می‌تواند به خصوص در یک شبکه داده که تیم‌ها ممکن است نیاز به بازبینی یا بازگشت تغییرات در حوزه‌های خود داشته باشند، مفید باشد. محیط همکاری Databricks همچنین می‌تواند به تیم‌های چند تخصصی- در یک شبکه داده کمک کند تا با هم کار کنند و به طور کارآمدتر همکاری کنند، که با اصل فعال‌سازی تیم‌های متمرکز بر حوزه مطابقت دارد.

خلاصه‌ای از کتابی از معماری داده

Snowflake

Snowflake یک راه حل کاملاً مدیریت شده و مبتنی بر ابر برای ذخیره، سازماندهی و تحلیل مقادیر زیادی از داده است و به طور منحصر به فرد می‌تواند به عنوان یک دیتا لیک عمل کند علاوه بر ارائه قابلیت‌های سنتی انبار داده. شرکت در سال ۲۰۱۲ برای مقابله با محدودیت‌های راه حل‌های سنتی انبار داده تاسیس شد. بنیانگذاران آن یک معماری منحصر به فرد را توسعه داده‌اند که محاسبه و ذخیره‌سازی را از هم جدا می‌کند، اجازه می‌دهد تا کاربردهای تحلیل داده که می‌توانند به طور مستقل و خودکار بر اساس تقاضا مقیاس بزنند. پلتفرم، که در سال ۲۰۱٤ راه‌اندازی شد، برای توانایی در مدیریت انواع مختلف داده و توانایی ذخیره و پردازش فرمت‌های گسترده‌ای از داده‌ها، از جمله Parquet، JSON، CSV و Avro، شناخته شد. رویکرد به صورت ابری و معماری آن مشتریان گسترده‌ای را از صنایع مختلف جذب کرده است. این ارائه قیمت بر اساس استفاده، امنیت قوی، انعطاف‌پذیری و قابلیت اطمینان را می‌دهد.

شما می‌توانید داده‌ها را در فرمت‌های مختلف مستقیماً به Snowflake بارگذاری کنید مانند یک دیتا لیک، که نیازی به تبدیل یا پیش‌پردازش داده قبل از ورود وجود ندارد. Snowflake فرآیند را با استنباط خودکار به اسکیمای داده‌های وارد شده ساده‌تر می‌کند. پس از بارگذاری داده، Snowflake محیطی ساختاریافته و قابل مقیاس برای سازماندهی و کاتالوگ کردن آن را فراهم می‌کند، که به کاربران اجازه می‌دهد جداول ایجاد کنند، اسکیماها را تعریف کنند و برچسب‌های متا را اعمال کنند. این اطمینان می‌دهد که سازماندهی کارا و دسترسی آسان ایجاد می‌شود.

استفاده از Snowflake به عنوان یک دیتا لیک مزایای یک پلتفرم داده یکپارچه را فراهم می‌کند، که نیازی به یک زیرساخت دیتا لیک جداگانه و کاهش پیچیدگی را حذف می‌کند. این ارائه محیطی قابل مقیاس و عملکرد بالا برای ذخیره، مدیریت و تحلیل انواع مختلف داده را می‌دهد، که آن را گزینه‌ای جذاب برای سازمان‌هایی که می‌خواهند قدرت یک دیتا لیک را در یک اکوسیستم یکپارچه بهره‌برداری کنند، می‌کند.

قدرت اصلی Snowflake در قابلیت‌های بسیار بهینه‌سازی شده انبار داده و پرس و جوی کارا و تحلیل درون پلتفرم آن است. با این حال، اگر نیاز به محاسبات خاص یا پردازش تخصصی دارید که می‌تواند بهتر توسط موتورهای محاسباتی دیگر خدمت شود، صادر کردن داده از Snowflake و استفاده از این موتورها رویکرد متداول است. در حالی که Snowflake قابلیت‌های صادرات داده بدون درز را فراهم می‌کند، اما به طور مستقیم پشتیبانی از اجرای موتورهای محاسباتی خارجی یا ابزارها روی داده‌های ذخیره شده در پلتفرم آن را نمی‌دهد.

Snowflake به خوبی با اصول شبکه داده (که در فصل ۱۳ پوشش داده شده است) هماهنگ است. شرکت اهمیت تقویت توانایی متخصصان حوزه و تیم‌ها را برای استفاده از داده‌های خود به عنوان مالکین شناخته است. با Snowflake، واحدهای تجاری یا حوزه‌های مختلف می‌توانند حساب‌های جداگانه یا انبارهای مجازی داشته باشند، که به آن‌ها اجازه می‌دهد تا دارایی‌های داده خود را مدیریت و کنترل کنند. این رویکرد متمرکز باعث می‌شود تا انعطاف‌پذیری و اتحادیه را ترویج کند، اجازه می‌دهد تا تیم‌های حوزه تصمیمات مبتنی بر داده بگیرند و مسئولیت کیفیت و قابلیت استفاده از داده‌های خود را بپذیرند.

قابلیت‌های مدیریت داده قوی Snowflake، از جمله مدیریت اسکیما، برچسب‌زنی متا و کاتالوگ کردن، به شما اجازه می‌دهد تا داده‌های خود را به عنوان یک دارایی ارزشمند در نظر بگیرید که می‌تواند توسط تیم‌های دیگر در سازمان دسترسی و مصرف شود، که تفکر محصول داده را ترویج می‌کند. Snowflake همچنین با جنبه‌های قابلیت مقیاس و انعطاف‌پذیری مفهوم شبکه داده هماهنگ است: معماری به صورت ابری آن امکان مقیاس مستقل منابع محاسباتی را فراهم می‌کند، که به تیم‌ها کمک می‌کند تا به نیازهای تغییر کننده سازگار شوند و به طور کارآمد پردازش داده و تحلیل را مدیریت کنند. در خلاصه، Snowflake به سازمان‌ها امکان می‌دهد تا یک رویکرد شبکه داده را بپذیرند که همکاری، اتحادیه و انعطاف‌پذیری را در اکوسیستم‌های داده آن‌ها ترویج می‌کند.

خلاصه

تصمیماتی که در مورد فناوری‌های معماری داده می‌گیرید، برای موفقیت شما در پروژه‌های داده از اهمیت بالایی برخوردار خواهد بود. این فصل شما را از طریق انتخاب‌های اساسی که پایه زیرساخت داده شما را تشکیل می‌دهد، هدایت کرد.

ما سه چارچوب نرم‌افزار اساسی را بررسی کردیم: Databricks، Hadoop و Snowflake، هر کدام رویکرد منحصر به فردی برای پردازش و تحلیل داده ارائه می‌دهند. چه اگر برای راه‌حل‌های منبع باز انتخاب کنید، چه اگر ارائه‌های

ارائه‌دهنـدگان ابـر را بپذیریـد، یـا هـر دو را ترکیـب کنیـد، در نظـر بگیریـد کـه مقیـاس نیازهـای داده شـما و نیازهـای انعطاف‌پذیری سازمان شما چقدر است.

عـلاوه بـر ایـن، بـه یـاد داشـته باشـید کـه دنیـای فنـاوری داده دودویی نیسـت. ایـن بـه منظـور مـدل‌های خـدمات ابـری می‌رسـد، جـایی کـه ارائه‌دهنـدگان اصـلی یـک مجموعـه متنـوع از خـدمات را ارائـه می‌دهنـد و راه‌حل‌هـای چنـد ابـری بـه تصمیم‌گیری شما یک لایه پیچیدگی اضافه می‌کنند.

هنگامی کـه در سـفر معمـاری داده خـود ادامـه می‌دهیـد، بـه یـاد داشـته باشـید چـه تصـمیماتی کـه در ایـن فصـل گرفته‌ایـد جداگانـه نیسـتند؛ آن‌هـا بـه هـم مـرتبط هسـتند و منظـره زیرسـاخت داده شـما را شـکل خواهنـد داد. ایـن فقـط در مـورد انتخـاب راه‌حل‌هـای منبـع بـاز یـا خصـوصی نیسـت؛ ایـن در مـورد ترکیـب هماهنـگ فناوری‌هـا اسـت کـه بـا اهـداف و چالش‌هـای سـازمان شـما همخـوانی دارد. بـا ایـن در نظـر گـرفتن، شـما بـه خـوبی مجهـز بـرای پیمـایش در دنیـای همیشـه تغییر کردن فناوری‌های معماری داده هستید.

با خوش شنوایی در تفسیر!

حارصه - رمز گشایی از معماری داده